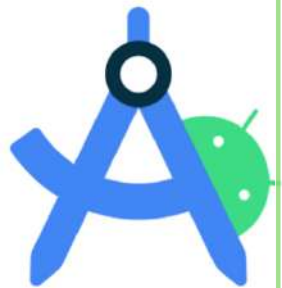


2024

BIBBIA DI PWM

android
studio



**Traduzione e rielaborazione
delle slide del professore
Federico Concone**

Marco Marino

1. KOTLIN

Kotlin è un linguaggio di programmazione moderno, sviluppato da JetBrains, progettato per interoperare con Java e correre sulla Java Virtual Machine (JVM). È stato rilasciato per la prima volta nel 2011 e ha guadagnato popolarità grazie alla sua sintassi concisa e le sue caratteristiche avanzate.

1.1. Dichiarare una funzione

```
fun max(a: Int, b: Int): Int{  
    return if (a > b) a else b  
}
```

fun è la parola chiave usata per dichiarare una funzione, ed è seguita dal nome della funzione (max), dai parametri che la funzione accetta in input (in questo caso due interi) e da un tipo di ritorno (in questo caso sempre intero). Come si può evincere dal suo corpo, questa funzione ritorna il primo intero passato in input (a) se e solo se $a > b$, mentre b altrimenti. In questo caso il corpo della funzione è scritto tra delle parentesi graffe, quindi diciamo che tale funzione ha un **block body**. Un altro modo di dichiarare la stessa funzione potrebbe essere il seguente:

```
fun max(a: Int, b: Int): Int = if (a > b) a else b
```

In questo caso la funzione ritorna direttamente un'espressione, quindi si dice che ha un **expression body**.

1.2. Dichiarare una variabile

In Kotlin è possibile dichiarare una variabile tramite due parole chiave:

- **val** (value): indica che la variabile dichiarata è immutabile e il suo valore non può essere modificato dopo la sua inizializzazione. È buona pratica generalmente dichiarare le variabili con **val** per avere un controllo più accurato sugli errori.
- **var** (variable): indica che la variabile dichiarata è mutabile e il suo valore può essere modificato anche dopo la sua inizializzazione.

Il vantaggio di Kotlin è che il tipo di una variabile non deve essere sempre esplicitamente dichiarato, infatti il compilatore ha la capacità di valutare il tipo dell'espressione inizializzatrice e usarlo come tipo della variabile che si sta dichiarando:

```
val question = "Quanto fa 2 + 2?"  
  
val answer = 3
```

In questo caso l'inizializzatore, ovvero il numero 3, è di tipo intero, quindi la variabile avrà tipo intero. Però, se una variabile non ha un inizializzatore, allora dovremo specificare il suo tipo in modo esplicito (il compilatore non legge la nostra mente e non può dedurre il tipo se non diamo alcuna informazione sui valori che possono essere assegnati alla variabile).

Una variabile **val** deve essere inizializzata esattamente una volta durante l'esecuzione del blocco all'interno del quale è dichiarata. Tuttavia, in alcuni casi, è possibile inizializzarla con valori

differenti se il compilatore può garantire che solo una delle istruzioni di inizializzazione venga eseguita, come nel caso seguente:

```
val message: String
if(canPerformOperation()){
    message = "Success"
    //... Perform operation
}else{
    message = "Failed"
}
```

Warning #1: anche se una variabile `val` è di per sé immutabile e non può essere cambiata, l'oggetto a cui punta potrebbe essere mutabile, come nel seguente caso:

```
val languages = arrayListOf("Java")
languages.add("Kotlin")
```

In questo caso inizialmente la variabile `languages` contiene un `arrayList` di stringhe contenente l'elemento "Java". L'`arrayList` viene in seguito aggiornata aggiungendo l'elemento "Kotlin".

Warning #2: anche se le variabili `var` permettono a una variabile di cambiare il proprio valore, il loro tipo è fisso. Per esempio il seguente codice non compila:

```
var answer = 42
answer = "no answer"
```

Il tipo non può variare da intero a stringa in questo modo, ma dovrà essere manualmente convertito (come vedremo in seguito).

1.3. String templates

```
fun main(args: Array<String>) {  
    val name = if (args.size > 0) args[0] else "Kotlin"  
    println("Hello, $name!")  
}
```



Prints "Hello, Kotlin", or
"Hello, Bob" if you pass
"Bob" as an argument

Kotlin permette di riferirti alle variabili locali nei letterali di stringa inserendo il carattere \$ davanti al nome della variabile. Questo metodo è equivalente alla concatenazione di stringhe ("Hello " + name + "!"), ma è più compatto ed efficiente e viene controllato staticamente, ovvero il codice non verrà compilato se proviamo a riferirci ad una variabile che non esiste.

Per includere il carattere \$ in una stringa possiamo utilizzare il carattere di escape "\", ad esempio println("\\$x") stampa \$x e non interpreta x come un riferimento alla variabile.

Possiamo utilizzare anche le {} dopo il carattere \$ per espressioni più complesse, ad esempio:

```
fun main(args: Array<String>) {  
    if (args.size > 0) {  
        println("Hello, ${args[0]}!")  
    }  
}
```



Uses the \${} syntax to insert the
first element of the args array

Oppure:

```
fun main(args: Array<String>) {  
    println("Hello, ${if (args.size > 0) args[0] else "someone"}!")  
}
```


1.4. Classi e proprietà

Listing 2.3 Simple Java class Person

```
/* Java */
public class Person {
    private final String name;

    public Person(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

Listing 2.4 Person class converted to Kotlin

```
class Person(val name: String)
```

Classi di questo tipo (che contengono solo dati ma non codice) sono spesso chiamate **value objects** e diversi linguaggi offrono una sintassi concisa per dichiararle. Rispetto a Java, il modificatore `public` è stato omesso in quanto, in Kotlin, corrisponde alla visibilità predefinita.

L'idea di una classe è quella di incapsulare i dati e il codice che lavora su di essi in un'unica entità. In Java, i dati sono memorizzati in campi, solitamente privati. Per questo, se è necessario accedere a tali dati, si ricorre a metodi accessori come i getter e i setter. In Java, la combinazione del campo e dei suoi metodi accessori è spesso definita come **proprietà**.

In Kotlin, le proprietà sono una caratteristica del linguaggio di primo livello che sostituisce completamente i campi e i metodi accessori. Dichiarare una proprietà in una classe è simile a dichiarare una variabile, utilizzando le parole chiave `val` e `var`. Una proprietà dichiarata come `val` è di sola lettura, mentre una proprietà dichiarata come `var` è mutabile e può essere modificata, come si evince dal seguente esempio:

```
class Person(
    val name: String,
    var isMarried: Boolean
)
```

Read-only property: generates a field and a trivial getter

Writable property: a field, a getter, and a setter

In Kotlin è possibile creare anche dei metodi accessori personalizzati. Ad esempio, supponiamo di dichiarare un rettangolo che può indicare se è un quadrato. Non è necessario memorizzare queste informazioni come campo separato, perché possiamo controllare all'istante se l'altezza è uguale alla larghezza:

```
class Rectangle(val height: Int, val width: Int) {
    val isSquare: Boolean
    get() {
        return height == width
    }
}
```

Property getter declaration

In questo esempio, `isSquare` è una proprietà calcolata che verifica se `height` e `width` sono uguali. Il valore di `isSquare` viene calcolato ogni volta che vi si accede. Provando il codice otteniamo:

```
>>> val rectangle = Rectangle(41, 43)
>>> println(rectangle.isSquare)
false
```

Potremmo chiederci se sia meglio dichiarare una funzione senza parametri o una proprietà con un getter personalizzato. Entrambe le opzioni sono simili: non vi è differenza in termini di implementazione o performance; differiscono solo in leggibilità. Generalmente, se descriviamo la caratteristica (la proprietà) di una classe, dovremmo dichiararla come una proprietà.

1.5. Espressione if

In Kotlin, if può essere utilizzato non solo come comando di controllo di flusso, ma anche come un'espressione che ritorna un valore. Vediamo i differenti utilizzi dell'if:

1. Assegnazione condizionale semplice

```
var max = a
if (a < b) max = b
```

In questo esempio, max viene inizialmente impostato ad a. Se a è minore di b, max viene aggiornato a b.

2. Assegnazione condizionale con else

```
if (a > b) {
    max = a
} else {
    max = b
}
```

In questo esempio, max viene impostato ad a se a è maggiore di b, altrimenti viene impostato a b.

3. If usato come espressione

```
max = if (a > b) a else b
```

Questo è un modo più compatto per scrivere l'esempio precedente, utilizzando if come un'espressione che restituisce direttamente il valore di a o b.

4. If else come espressioni

```
val maxLimit = 1
val maxOrLimit = if (maxLimit > a) maxLimit else if (a > b) a else b
```

In questo esempio, viene usata una catena di if per determinare il valore di maxOrLimit, valutando più condizioni in cascata.

5. Ramificazioni in blocchi di codice

Le ramificazioni di un'espressione if possono essere blocchi di codice. In questi casi, l'ultima espressione del blocco è il valore restituito dal blocco:

```
val max = if (a > b) {
    print("Choose a")
    a
} else {
    print("Choose b")
    b
}
```

In questo esempio, se a è maggiore di b, verrà stampato "Choose a" e il valore di a sarà il risultato dell'espressione if. Altrimenti, verrà stampato "Choose b" e il risultato sarà b.

1.6. Espressione when

L'istruzione `when` definisce un'espressione condizionale con molteplici ramificazioni e funziona in modo simile all'istruzione `switch` dei linguaggi simili a C.

```
when (x) {  
    1 -> print("x == 1")  
    2 -> print("x == 2")  
    else -> {  
        print("x is neither 1 nor 2")  
    }  
}
```

L'istruzione `when` confronta il valore della sua espressione (in questo caso `x`) con ogni condizione in modo sequenziale fino a che non trova una condizione che corrisponde. `When` può essere usata sia come **espressione** che come **dichiarazione**. Se usata come espressione, il valore della prima ramificazione corrispondente diventa il valore dell'intera espressione. Se usata come dichiarazione, i valori delle singole ramificazioni sono ignorati. Il ramo `else` viene valutato se nessuna delle altre condizioni delle ramificazioni è soddisfatta, ed è obbligatorio quando `when` è usato come espressione, a meno che il compilatore non sia in grado di verificare che tutti i casi possibili siano coperti.

```
enum class Bit {  
    ZERO, ONE  
}  
  
val numericValue = when (getRandomBit()) {  
    Bit.ZERO -> 0  
    Bit.ONE -> 1  
    // 'else' is not required because all cases are covered  
}
```

In questo esempio, `when` è utilizzato per assegnare un valore numerico basato sul risultato di `getRandomBit()`, che ritorna un valore dell'enumerazione `Bit`. Ogni possibile valore dell'enum è coperto, quindi il ramo `else` non è necessario.

1.7. While loop

Il ciclo while è identico a quello di Java, in questo caso Kotlin non porta nulla di nuovo a questo semplice loop, a differenza del for di cui discuteremo a breve.

```
while (condition) {  
    /*...*/  
}
```

← The body is executed while the condition is true.

```
do {  
    /*...*/  
} while (condition)
```

← The body is executed for the first time unconditionally. After that, it's executed while the condition is true.

1.8. For loop

In Kotlin il loop for esiste in un'unica forma equivalente al loop for-each di Java. Si scrive come:
for <item> in <elements>

Così come in Java, la sua più comune applicazione è quella di iterare sulle collections.

In Kotlin non esiste il classico loop for a cui siamo stati abituati, quindi per sostituire i casi più comuni si ricorre al concetto di **ranges**. Un range è un intervallo chiuso (il secondo valore fa sempre parte del range) tra due valori (solitamente numeri) costituito da un inizio e da una fine e si scrive mediante l'operatore "..". L'applicazione più comune dei ranges è la **progressione**, ovvero iterare su tutti i valori di un intervallo di numeri interi.

```
If i is divisible by 3, returns Fizz  
Else returns the number itself
```

```
fun fizzBuzz(i: Int) = when {  
    i % 15 == 0 -> "FizzBuzz "  
    i % 3 == 0 -> "Fizz "  
    i % 5 == 0 -> "Buzz "  
    else -> "$i "  
}  
  
>>> for (i in 1..100) {  
...     print(fizzBuzz(i))  
... }  
1 2 Fizz 4 Buzz Fizz 7 ...
```

← If i is divisible by 5, returns Buzz

← If i is divisible by 15, returns FizzBuzz. As in Java, % is the modulus operator.

← Iterates over the integer range 1..100

I ranges possono essere utilizzati anche per cicli inversi:

```
>>> for (i in 100 downTo 1 step 2) {  
...     print(fizzBuzz(i))  
... }  
Buzz 98 Fizz 94 92 FizzBuzz 88 ...
```

In questo caso è stato implementato anche un salto tramite l'operatore step per considerare solo i numeri pari.

In alcuni casi conviene invece iterare su ranges mezzî chiusi, ovvero che non includono uno specifico punto di fine. Per farlo, si ricorre alla funzione **until**. Ad esempio il seguente loop:

```
for (x in 0 until size)
```

è equivalente a:

```
for (x in 0..size-1)
```

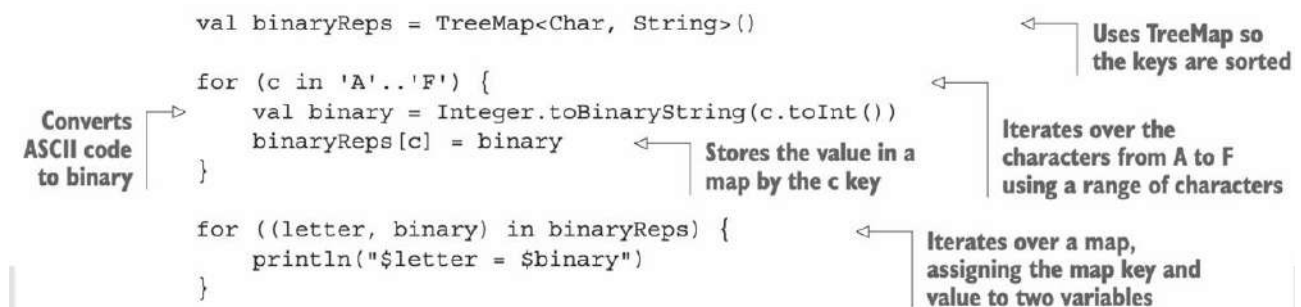
La prima forma in qualche modo esprime l'idea in maniera più chiara.

Come abbiamo detto, il modo più comune di usare un loop for in Kotlin è quello di iterare su una collection, e ciò avviene esattamente come in Java:

```
val binaryReps = TreeMap<Char, String>()

for (c in 'A'..'F') {
    val binary = Integer.toBinaryString(c.toInt())
    binaryReps[c] = binary
}

for ((letter, binary) in binaryReps) {
    println("$letter = $binary")
}
```



Converts ASCII code to binary

Stores the value in a map by the c key

Iterates over the characters from A to F using a range of characters

Uses TreeMap so the keys are sorted

Iterates over a map, assigning the map key and value to two variables

In questo esempio viene creata una variabile `binaryReps` che è una `TreeMap` di tipo `<Char, String>`. `TreeMap` è scelta perché mantiene le chiavi ordinate. In questo caso, le chiavi saranno i caratteri da 'A' ad 'F'. Il ciclo for itera sui caratteri da 'A' a 'F'. Per ogni carattere `c` viene calcolata la sua rappresentazione binaria usando `Integer.toBinaryString(c.toInt())`, che converte prima il carattere in un intero (ASCII) e poi quell'intero in una stringa binaria. La rappresentazione binaria di ogni carattere viene poi memorizzata nella `TreeMap` con `c` come chiave e la rappresentazione binaria come valore. Il secondo ciclo for itera sulla `TreeMap` `binaryReps`. Per ogni coppia chiave-valore, dove `letter` è la chiave e `binary` è il valore, stampa sia la chiave (il carattere) sia il valore (la rappresentazione binaria) in una stringa formattata.

1.9. Operatore in

L'operatore `in` può essere utilizzato per verificare che un certo valore si trovi in un range. Il suo opposto, ovvero l'operatore `!in`, serve invece per verificare che un certo valore non si trovi in un range.

```
fun isLetter(c: Char) = c in 'a'..'z' || c in 'A'..'Z'
fun isNotDigit(c: Char) = c !in '0'..'9'
```

```
>>> println(isLetter('q'))
true
>>> println(isNotDigit('x'))
true
```

`c in 'a'..'z'`

Transforms to
`a <= c && c <= z`

Altri esempi di utilizzo dell'operatore `in`:

```
>>> println("Kotlin" in "Java".."Scala")
true
```

The same as `"Java" <= "Kotlin"`
&& `"Kotlin" <= "Scala"`

```
>>> println("Kotlin" in setOf("Java", "Scala"))
false
```

This set doesn't contain
the string `"Kotlin"`.

1.10. Eccezioni in Kotlin

La gestione delle eccezioni in Kotlin è simile a quella di molti altri linguaggi. Come sappiamo, una funzione può completare la sua esecuzione normalmente oppure lanciare (throw) un'eccezione se si verifica un qualche tipo di errore. Ad esempio:

Example #1

```
if (percentage !in 0..100) {  
    throw IllegalArgumentException(  
        "A percentage value must be between 0 and 100: $percentage")  
}
```

In questo esempio viene lanciata un'eccezione di tipo `IllegalArgumentException` se il valore contenuto nella variabile `percentage` non è contenuto nel range 0-100, con allegato un messaggio customizzato di avviso per l'utente. Un altro modo di scrivere il precedente codice è il seguente:

Example #2

```
val percentage =  
    if (number in 0..100)  
        number  
    else  
        throw IllegalArgumentException(  
            "A percentage value must be between 0 and 100: $number")
```

“throw” is an expression.

Per gestire comportamenti più complessi, si può ricorrere ai noti blocchi `try`, `catch` e `finally`:

```
fun readNumber(reader: BufferedReader): Int? {  
    try {  
        val line = reader.readLine()  
        return Integer.parseInt(line)  
    }  
    catch (e: NumberFormatException) {  
        return null  
    }  
    finally {  
        reader.close()  
    }  
}
```

You don't have to explicitly specify exceptions that can be thrown from this function.

The exception type is on the right.

“finally” works just as it does in Java.

```
>>> val reader = BufferedReader(StringReader("239"))  
>>> println(readNumber(reader))  
239
```

In questo esempio, il blocco `try` tenta di leggere una linea di testo dal reader e di convertirla in un intero. Il blocco `catch` cattura l'eccezione `NumberFormatException`, che viene lanciata se la stringa letta non può essere convertita in un intero, e in questo caso ritorna `null`. Il blocco `finally` viene sempre eseguito indipendentemente dal fatto che si verifichino eccezioni o meno ed è qui usato per chiudere il reader e assicurare che la risorsa venga liberata correttamente. Come possiamo notare, in Kotlin, a differenza di Java, non è necessario dichiarare le eccezioni che possono essere lanciate da una funzione (fatto utilizzando l'operatore `throws` in Java).

1.11. Collections in Kotlin

In Kotlin si possono creare vari tipi di collezioni come set, liste e mappe utilizzando funzioni dedicate che sono direttamente supportate dal linguaggio. Ecco alcuni esempi:

```
val set = hashSetOf(1, 7, 53)
```

Viene creato un set contenente i valori 1, 7, 53.

```
val list = arrayListOf(1, 7, 53)
```

Viene creata una lista contenente gli stessi valori del set precedente.

```
val map = hashMapOf(1 to "one", 7 to "seven", 53 to "fifty-three")
```

Viene creata una HashMap dove ogni chiave intera è associata ad una rappresentazione stringa del numero.

Kotlin usa internamente le classi standard di Java per le collections:

```
>>> println(set.javaClass)
class java.util.HashSet

>>> println(list.javaClass)
class java.util.ArrayList

>>> println(map.javaClass)
class java.util.HashMap
```

← **javaClass is Kotlin's equivalent of Java's getClass().**

In Java le collections possiedono un metodo toString di default, ma la formattazione dell'output è fissa e non sempre corrisponde a ciò di cui abbiamo bisogno:

```
>>> val list = listOf(1, 2, 3)
>>> println(list)
[1, 2, 3]
```

← **Invokes toString()**

Supponiamo di aver bisogno che gli elementi siano separati da un punto e virgola e circondati da parentesi tonde. Per risolvere questo problema Java ricorre a librerie di terze parti o ti obbliga a reimplementare la logica del progetto. Kotlin invece possiede una funzione standard e generica (funziona su collections che contengono elementi di qualsiasi tipo) che permette di risolvere tale limitazione:

```
fun <T> joinToString(
    collection: Collection<T>,
    separator: String,
    prefix: String,
    postfix: String
): String {
    val result = StringBuilder(prefix)

    for ((index, element) in collection.withIndex()) {
        if (index > 0) result.append(separator)
        result.append(element)
    }

    result.append(postfix)
    return result.toString()
}
```

← **Don't append a separator before the first element.**

La funzione `joinToString` prende in input quattro parametri:

1. **collection**: La collezione di elementi di tipo generico `T` che verranno uniti in una stringa.
2. **separator**: Una stringa che viene inserita tra ogni elemento della collezione.
3. **prefix**: Una stringa aggiunta all'inizio del risultato finale.
4. **postfix**: Una stringa aggiunta alla fine del risultato finale.

Si inizia creando un oggetto `StringBuilder`, inizializzato con il prefisso. `StringBuilder` è utilizzato perché permette di costruire stringhe in modo efficiente. La funzione poi itera sulla collezione usando `withIndex()`, che fornisce sia l'indice (`index`) che l'elemento (`element`) corrente. La condizione `if` controlla se l'elemento non è il primo della lista (indice maggiore di 0). Se vero, aggiunge il separatore allo `StringBuilder` prima di aggiungere l'elemento. Questo previene l'aggiunta del separatore prima del primo elemento. Dopo aver terminato l'iterazione su tutti gli elementi, il suffisso viene aggiunto allo `StringBuilder`. Infine, il contenuto di `StringBuilder` viene convertito in una stringa e ritornato. La funzione tornerà quindi la `collection` con la formattazione da noi desiderata:

```
>>> val list = listOf(1, 2, 3)
>>> println(joinToString(list, "; ", "(" , ")"))
(1; 2; 3)
```

Da questo esempio saltano all'occhio due dei maggiori problemi riguardanti le funzioni. Il primo problema riguarda la leggibilità delle chiamate alla funzione. Risulta complicato capire a quali parametri corrispondano queste stringhe:

```
joinToString(collection, " ", " ", ".")
```

Per risolvere questo problema in Kotlin possiamo specificare i nomi dei parametri che passiamo alla funzione. Se si specifica il nome di un parametro è buona pratica specificare anche il nome di tutti gli altri per evitare confusione:

```
joinToString(collection, separator = " ", prefix = " ", postfix = ".")
```

Un secondo problema riguarda la sovrabbondanza di metodi sovraccaricati nelle classi. Kotlin consente di definire valori predefiniti per i parametri di una funzione. Questo significa che, quando invochiamo una funzione, possiamo omettere alcuni dei parametri e verranno usati automaticamente i valori predefiniti. Riferendoci all'esempio precedente, avremo:

```
fun <T> joinToString(
    collection: Collection<T>,
    separator: String = ", ",
    prefix: String = "",
    postfix: String = ""
): String
```

**Parameters with
default values**

Ora potremo invocare la funzione con tutti i suoi argomenti o ometterne alcuni:

```
>>> joinToString(list, " ", " ", "", "")
1, 2, 3
>>> joinToString(list)
1, 2, 3
>>> joinToString(list, "; ")
1; 2; 3
```

1.12. Interfacce in Kotlin

Le interfacce Kotlin sono simili a quelle Java e possono contenere definizioni di metodi astratti. Per dichiarare un'interfaccia in Kotlin si utilizza la parola chiave `interface`:

```
interface Clickable {  
    fun click()  
}
```

In seguito, per implementare l'interfaccia:

```
class Button : Clickable {  
    override fun click() = println("I was clicked")  
}
```

```
>>> Button().click()  
I was clicked
```

In Kotlin le parole chiave `extends` e `implements` sono sostituite dall'operatore `:`.

Così come in Java, il modificatore `override` è usato per individuare quei metodi e proprietà che sovrascrivono quelli della superclasse o dell'interfaccia. Contrariamente a Java, però, in Kotlin è obbligatorio usare il modificatore `override`.

Un'interfaccia Kotlin può possedere metodi che hanno un'implementazione di default:

```
interface Clickable {  
    fun click()  
    fun showOff() = println("I'm clickable!")  
}
```

Regular method declaration →

← Method with a default implementation

Se scegliamo di implementare questa interfaccia dovremo fornire obbligatoriamente un'implementazione per il metodo `click()` e potremo ridefinire il comportamento del metodo `showOff()` utilizzando il modificatore `override`. Ma cosa succede se abbiamo un'altra interfaccia al cui interno è definito un altro metodo `showOff()`? Supponiamo di avere due interfacce che devono essere implementate dalla stessa classe:

```
interface Clickable {  
    fun click()  
    fun showOff() = println("I'm clickable!")  
}
```

Regular method declaration →

← Method with a default implementation

```
interface Focusable {  
    fun setFocus(b: Boolean) =  
        println("I ${if (b) "got" else "lost"} focus.")  
  
    fun showOff() = println("I'm focusable!")  
}
```

In questo caso Kotlin ci costringe a fornire una nostra implementazione del metodo interessato:

```
class Button : Clickable, Focusable {  
    override fun click() = println("I was clicked")  
    override fun showOff() {  
        super<Clickable>.showOff()  
        super<Focusable>.showOff()  
    }  
}
```

**“super” qualified by the supertype
name in angle brackets specifies the
parent whose method you want to call.**

**← You must provide an explicit
implementation if more than
one implementation for the
same member is inherited.**

La classe Button implementa due interfacce. Per avere una corretta implementazione del metodo condiviso showOff() si ricorre alla parola chiave super, che permette alla classe di chiamare l'implementazione del metodo della classe padre o, in questo caso, delle interfacce, e di risolvere l'ambiguità dovuta all'uso dello stesso nome per il metodo showOff(): in questo caso super viene usato per specificare chiaramente quale metodo di quale interfaccia si desidera chiamare:

```
fun main(args: Array<String>) {  
    val button = Button()  
    button.showOff()  
    button.setFocus(true)  
    button.click()  
}
```

**I got
focus.** →

**← I'm clickable!
I'm focusable!**

← I was clicked.

1.13. Modificatori nelle classi Kotlin

Java ci permette di creare sottoclassi di qualsiasi classe e di sovrascrivere qualsiasi metodo a meno che non siano stati esplicitamente dichiarati con la parola chiave `final`. In Kotlin le classi e i metodi sono dichiarati `final` di default. Per permettere la creazione di sottoclassi di una classe, dobbiamo contrassegnare la classe e ogni proprietà o metodo che possono essere sovrascritti con il modificatore **open**:

```
open class RichButton : Clickable {  
    fun disable() {}  
    open fun animate() {}  
    override fun click() {}  
}
```

Annotations explaining the code:

- This class is open: others can inherit from it.** (points to `open class RichButton`)
- This function is final: you can't override it in a subclass.** (points to `fun disable()`)
- This function is open: you may override it in a subclass.** (points to `open fun animate()`)
- This function overrides an open function and is open as well.** (points to `override fun click()`)

Forniamo quindi una panoramica dei modificatori delle classi Kotlin:

Modifier	Corresponding member	Comments
<code>final</code>	Can't be overridden	Used by default for class members
<code>open</code>	Can be overridden	Should be specified explicitly
<code>abstract</code>	Must be overridden	Can be used only in abstract classes; abstract members can't have an implementation
<code>override</code>	Overrides a member in a superclass or interface	Overridden member is open by default, if not marked <code>final</code>

Un altro tipo di modificatore, ovvero quello di visibilità, ci aiuta a controllare l'accesso alle dichiarazioni nel nostro codice. Ciò che differisce dai modificatori di visibilità Java è la visibilità di default, che in Kotlin è `public` e non `private`. Come alternativa, Kotlin offre un modificatore di visibilità **interno**, ovvero visibile dentro ad un modulo, il quale è un insieme di file Kotlin compilati in gruppo. Forniamo adesso una panoramica dei modificatori di visibilità di Kotlin:

Modifier	Class member	Top-level declaration
<code>public</code> (default)	Visible everywhere	Visible everywhere
<code>internal</code>	Visible in a module	Visible in a module
<code>protected</code>	Visible in subclasses	—
<code>private</code>	Visible in a class	Visible in a file

1.14. Classi annidate

Le classi Kotlin possono essere annidate in altre classi:

```
class Outer {  
    private val bar: Int = 1  
    class Nested {  
        fun foo() = 2  
    }  
}  
  
val demo = Outer.Nested().foo() // == 2
```

Tutte le combinazioni sono possibili: interfacce annidate in classi, classi in interfacce e interfacce in interfacce:

```
interface OuterInterface {  
    class InnerClass  
    interface InnerInterface  
}  
  
class OuterClass {  
    class InnerClass  
    interface InnerInterface  
}
```

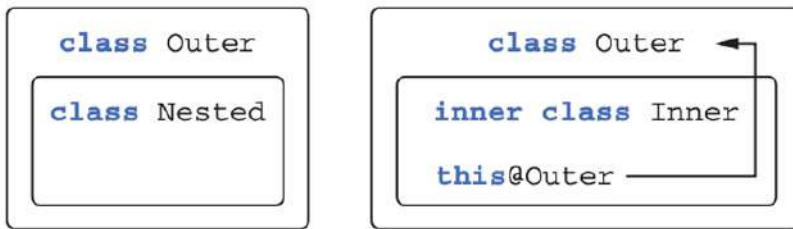
Una classe annidata contrassegnata come interna (inner) può accedere ai membri della sua classe esterna (outer), inclusi quelli privati:

```
class Outer {  
    private val bar: Int = 1  
    inner class Inner {  
        fun foo() = bar  
    }  
}  
  
val demo = Outer().Inner().foo() // == 1
```

In una classe interna possiamo scrivere `this@Outer` per accedere alla classe esterna dalla classe interna:

```
class Outer {  
    inner class Inner {  
        fun getOuterReference(): Outer = this@Outer  
    }  
}
```

Per riassumere i concetti appena spiegati:



Le classi annidate non referenziano le loro classi esterne, mentre quelle interne lo fanno.

1.15. Costruttori non predefiniti e proprietà

Come sappiamo in Java è possibile dichiarare uno o più costruttori. In maniera simile, Kotlin effettua una distinzione più decisa tra il costruttore primario (il principale), il quale è dichiarato al di fuori del corpo della classe, e il costruttore secondario, il quale è dichiarato nel corpo della classe a permette di implementare una logica di inizializzazione aggiuntiva nei blocchi di inizializzazione.

Vediamo un esempio di costruttore primario:

```
class User(val nickname: String)
```

Il costruttore primario è utile a specificare i parametri del costruttore e a definire le proprietà che sono inizializzate da tali parametri. Un altro modo di dichiarare il costruttore primario in maniera più esplicita è il seguente:

```
class User constructor(_nickname: String) {  
    val nickname: String  
  
    init {  
        nickname = _nickname  
    }  
}
```

Primary constructor with one parameter

Initializer block

La parola chiave **constructor** serve a iniziare la dichiarazione di un costruttore primario o secondario. La parola chiave **init** introduce quello che abbiamo chiamato blocco di inizializzazione. Tali blocchi contengono il codice di inizializzazione che viene eseguito quando la classe è creata e sono fatti per essere usati insieme al costruttore primario. Dato che il costruttore primario ha una sintassi così ristretta, non può contenere codice di inizializzazione, ecco perché si usano i blocchi di inizializzazione. È possibile dichiarare molteplici blocchi di inizializzazione in una sola classe. Il trattino basso nel parametro del costruttore serve a distinguere il nome di una proprietà da quello del parametro del costruttore. Un'alternativa per rimuovere l'ambiguità, già comunemente usata in Java, è quella di usare la parola chiave **this**: `this.nickname = nickname`.

Un altro modo in cui possiamo dichiarare la stessa classe è il seguente:

```
class User(_nickname: String) {  
    val nickname = _nickname  
}
```

Primary constructor with one parameter

The property is initialized with the parameter.

Così come abbiamo fatto per le funzioni, possiamo dichiarare dei valori di default per i parametri del costruttore:

```
class User(val nickname: String,  
          val isSubscribed: Boolean = true)
```

Provides a default value for the constructor parameter

Per creare un'istanza della classe creata, si può chiamare il costruttore direttamente, senza utilizzare (come in Java) la parola chiave new:

```
>>> val alice = User("Alice")  
>>> println(alice.isSubscribed)  
true  
>>> val bob = User("Bob", false)  
>>> println(bob.isSubscribed)  
false  
>>> val carol = User("Carol", isSubscribed = false)  
>>> println(carol.isSubscribed)  
false
```

Uses the default value "true" for the isSubscribed parameter

You can specify all parameters according to declaration order.

You can explicitly specify names for some constructor arguments.

Se la nostra classe ha una superclasse, il costruttore primario dovrà inizializzare anche la superclasse. È possibile farlo fornendo i parametri del costruttore della superclasse dopo il riferimento alla superclasse nell'elenco delle classi base:

```
open class User(val nickname: String) { ... }  
  
class TwitterUser(nickname: String) : User(nickname) { ... }
```

Se non si dichiara un costruttore per una classe, verrà generato un costruttore vuoto che non fa nulla:

```
open class Button
```

The default constructor without arguments is generated.

Se ereditiamo la classe Button e non forniamo alcun costruttore, dobbiamo esplicitamente invocare il costruttore della superclasse anche se questo non ha alcun parametro:

```
class RadioButton: Button()
```

In Kotlin è consigliabile specificare valori di default per i parametri direttamente piuttosto che dichiarare molteplici costruttori secondari; tuttavia, in alcune situazioni i costruttori secondari sono necessari. Il seguente esempio mostra una classe che non dichiara costruttori primari, bensì due costruttori secondari:

```
open class View {  
    constructor(ctx: Context) {  
        // some code  
    }  
    constructor(ctx: Context, attr: AttributeSet) {  
        // some code  
    }  
}
```

Secondary constructors

Se vogliamo estendere tale classe ci basterà chiamare la superclasse facendo uso della parola chiave `super`:

```
class MyButton : View {  
    constructor(ctx: Context)  
        : super(ctx) {  
        // ...  
    }  
    constructor(ctx: Context, attr: AttributeSet)  
        : super(ctx, attr) {  
        // ...  
    }  
}
```



Calling superclass constructors

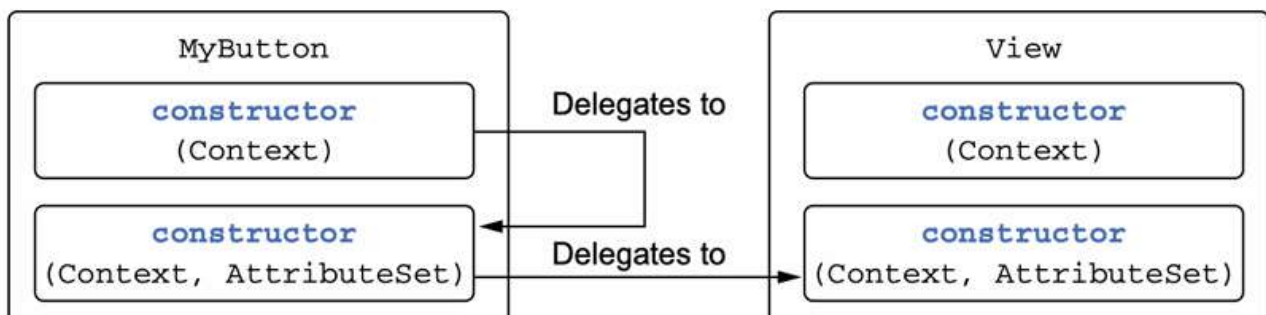
Infine, così come in Java, abbiamo l'opzione di chiamare un altro costruttore della nostra classe da un costruttore, usando la parola chiave `this()`:

```
class MyButton : View {  
    constructor(ctx: Context): this(ctx, MY_STYLE) {  
        // ...  
    }  
}
```



Delegates to another constructor of the class

Ecco una rappresentazione più schematica di come avviene questo gioco di richiami tra costruttori:



1.16. Parola chiave object

La parola chiave `object` è abbastanza ricorrente in Kotlin e può avere svariati significati, i quali però condividono tutti la stessa idea di base: la parola chiave definisce una classe e crea un'istanza di quella classe allo stesso tempo. Le differenti situazioni in cui questa parola chiave è usata sono:

- **Dichiarazione di oggetto:** è un modo per definire un singleton, ovvero un'istanza unica di una classe.
- **Oggetti companion:** possono contenere metodi di fabbrica e altri metodi correlati a questa classe, ma non richiedono l'istanza di una classe per essere chiamati. I loro membri possono essere accessibili tramite il nome della classe.
- **Espressione di oggetto:** viene utilizzata al posto delle classi anonime di Java.

Vedremo adesso ognuna di queste nel dettaglio.

Nella progettazione di sistemi orientati agli oggetti, è abbastanza comune avere una classe di cui è necessaria una sola istanza. Questo concetto è implementato solitamente utilizzando il pattern Singleton. Kotlin fornisce un supporto diretto per questo tramite la funzionalità di **dichiarazione di oggetto** (object declaration), la quale combina la dichiarazione di una classe e la creazione di una singola istanza di quella classe. Per esempio, la dichiarazione di oggetto si potrebbe usare per rappresentare la gestione delle buste paga di un'organizzazione (è probabile che non ci siano più gestioni delle buste paga, quindi usare un oggetto per questo scopo sembra ragionevole):

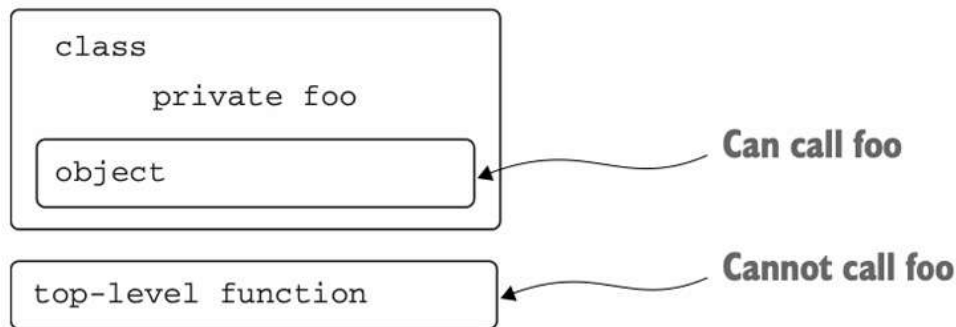
```
object Payroll {  
    val allEmployees = arrayListOf<Person>()  
  
    fun calculateSalary() {  
        for (person in allEmployees) {  
            ...  
        }  
    }  
}
```

Come si evince dall'esempio, le dichiarazioni di oggetto sono introdotte dalla parola chiave `object` e, proprio come una classe, possono contenere dichiarazioni di proprietà, metodi, blocchi di inizializzazione e così via. Ciò che distingue una dichiarazione di oggetto da una classe è che i costruttori (primario o secondari) non sono permessi.

Le dichiarazioni di oggetto possono essere fatte anche all'interno delle classi. Un esempio di applicazione è con le data classes, che in Kotlin sono principalmente utilizzate per contenere dati:

```
data class Person(val name: String) {  
    object NameComparator : Comparator<Person> {  
        override fun compare(p1: Person, p2: Person): Int =  
            p1.name.compareTo(p2.name)  
    }  
}
```

Contrariamente a Java, le classi Kotlin non possono avere membri statici, infatti la parola chiave `static` non fa parte del linguaggio Kotlin. Nella maggior parte dei casi è raccomandato utilizzare funzioni di alto livello che sono funzioni definite al di fuori di qualsiasi classe o oggetto. Tuttavia, tali funzioni non possono accedere ai membri privati di una classe:



Di conseguenza, se abbiamo bisogno di scrivere una funzione che può essere chiamata senza avere un'istanza della classe ma ha bisogno di accedere ai dettagli interni della classe, possiamo scriverla come membro di una dichiarazione di oggetto all'interno di quella classe. Tutto ciò può essere fatto da un cosiddetto **companion object**:

```
class A {  
    companion object {  
        fun bar() {  
            println("Companion object called")  
        }  
    }  
}
```

```
>>> A.bar()  
Companion object called
```

In questo esempio, mediante utilizzo del companion object, saremo in grado di accedere al metodo `bar()` senza usare un costruttore! In un certo senso, i companion object possono essere usati come `Static` in Java.

1.17. Programmazione con espressioni lambda

Nella scrittura di codice è comune dover passare e memorizzare pezzi di comportamento. Ad esempio, potremmo aver bisogno di esprimere idee come "Quando si verifica un evento, esegui questo handler" oppure "Applica questa operazione a tutti gli elementi in una struttura dati."

La **programmazione funzionale** offre un approccio alternativo per risolvere questo problema: la capacità di trattare le funzioni come valori. Invece di dichiarare una classe e passare un'istanza di quella classe a una funzione, possiamo passare una funzione direttamente (usando la classe interna anonima). La sintassi di un'espressione di classe anonima è simile all'invocazione di un costruttore, con la differenza che contiene una definizione di classe all'interno di un blocco di codice:

```
/* Java */
button.setOnClickListener(new OnClickListener() {
    @Override
    public void onClick(View view) {
        /* actions on click */
    }
});
```

In questo esempio viene passata una funzione direttamente. Abbiamo aggiunto un listener responsabile per la gestione del click, il quale implementa la corrispondente interfaccia OnClickListener con un metodo, ovvero onClick.

È chiaro che ripetere molte volte una dichiarazione di classe interna anonima così verbosa può diventare irritante. Ecco perché esiste una notazione, le **espressioni lambda**, che esprime solo il comportamento (cosa dovrebbe avvenire dopo il click) ed aiuta ad eliminare il codice ridondante:

```
button.setOnClickListener { /* actions on click */ }
```

Adesso vediamo come le espressioni lambda siano utili quando si lavora con le collections. Per esempio, usiamo una classe Persona che contiene informazioni sul nome e sull'età di un individuo:

```
data class Person(val name: String, val age: Int)
```

Supponiamo di avere una lista di persone di cui dobbiamo trovare la più anziana. Non avendo esperienza con le espressioni lambda, il nostro primo ingenuo approccio sarebbe quello di implementare la ricerca manualmente:

```
fun findTheOldest(people: List<Person>) {
    Stores the maximum age → var maxAge = 0
    var theOldest: Person? = null
    for (person in people) {
        Stores a person of the maximum age ←
        if (person.age > maxAge) {
            If the next person is older than the current oldest person, changes the maximum ←
            maxAge = person.age
            theOldest = person
        }
    }
    println(theOldest)
}

>>> val people = listOf(Person("Alice", 29), Person("Bob", 31))
>>> findTheOldest(people)
Person(name=Bob, age=31)
```

Ovviamente, Kotlin ci offre un'alternativa migliore mettendo a disposizione la funzione di una certa libreria, come mostrato di seguito:

```
>>> val people = listOf(Person("Alice", 29), Person("Bob", 31))
>>> println(people.maxBy { it.age })
Person(name=Bob, age=31)
```

← Finds the maximum by
comparing the ages

La funzione `maxBy` può essere chiamata su qualsiasi `collection` e prende in input un solo parametro, ovvero la funzione che specifica quali valori devono essere confrontati per trovare il massimo elemento. Nell'esempio sopra riportato, il codice tra parentesi graffe `{it.age}` non è altro che un'espressione lambda che sta implementando la logica utile a trovare l'età massima. Questa lambda riceve un elemento della collezione come argomento, qui rappresentato dal parametro predefinito `it`, e restituisce un valore che deve essere confrontato (in questo caso, l'età della persona).

Abbiamo già parlato abbastanza di espressioni lambda senza però spiegare effettivamente come dichiararle e utilizzarle. Una **lambda** è un piccolo pezzo di comportamento che possiamo passare in giro come un valore. In Kotlin, può essere dichiarata indipendentemente e memorizzata in una variabile. Più frequentemente, viene dichiarata direttamente quando viene passata a una funzione. Possiamo memorizzare un'espressione lambda in una variabile e trattare questa variabile come se fosse una funzione normale.

```
>>> val sum = { x: Int, y: Int -> x + y }
>>> println(sum(1, 2))
3
```

← Calls the lambda
stored in a variable

La sintassi di un'espressione lambda è abbastanza chiara: è sempre circondata da parentesi graffe, non ci sono parentesi che racchiudono gli argomenti e la freccia separa la lista degli argomenti dal corpo della lambda. In questo specifico esempio, la lambda `{x: Int, y: Int → x + y}` è assegnata alla variabile `sum`. Questa lambda prende due parametri `x` e `y` di tipo `Int` e ritorna la loro somma. Successivamente, la variabile `sum` viene chiamata come una funzione normale, passando i valori 1 e 2, e la somma di questi viene stampata.

Esistono vari modi per definire una funzione lambda. Possiamo passare un'espressione lambda come argomento a una funzione mettendola all'interno delle parentesi tonde:

```
people.maxBy({ p: Person -> p.age })
```

Se la lambda è l'unico argomento passato alla funzione, possiamo posizionarla dopo le parentesi:

```
people.maxBy() { p: Person -> p.age }
```

Quando la lambda è l'unico argomento della funzione, possiamo anche rimuovere completamente le parentesi vuote:

```
people.maxBy { p: Person -> p.age }
```

Come abbiamo visto prima, se non si specifica un nome per il parametro, Kotlin utilizza automaticamente il nome `it` come predefinito. Nei tre esempi di dichiarazione della funzione lambda vediamo invece che è stato specificato un nome in maniera esplicita (`p`).

Le espressioni lambda non sono sempre così semplici e contenute, possono infatti contenere molteplici istruzioni:

```
>>> val sum = { x: Int, y: Int ->
...     println("Computing the sum of $x and $y...")
...     x + y
... }
>>> println(sum(1, 2))
Computing the sum of 1 and 2...
3
```

Le lambda in Kotlin possono accedere ai parametri e alle variabili locali della funzione in cui sono definite:

```
fun printMessagesWithPrefix(messages: Collection<String>, prefix: String) {
    messages.forEach {
        println("$prefix $it")
    }
}
```

Accesses the "prefix" parameter in the lambda

Takes as an argument a lambda specifying what to do with each element

```
>>> val errors = listOf("403 Forbidden", "404 Not Found")
>>> printMessagesWithPrefix(errors, "Error:")
Error: 403 Forbidden
Error: 404 Not Found
```

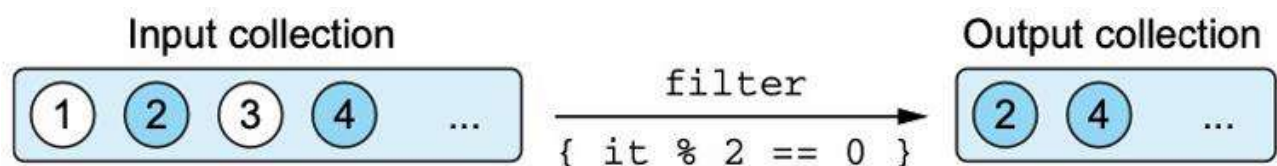
In questo caso la lambda accede al parametro “prefix” dentro il suo corpo. È possibile anche modificare le variabili dall'interno di una lambda, il che suggerisce di fare attenzione quando si utilizzano lambda che potrebbero alterare lo stato delle variabili esterne, in quanto ciò potrebbe introdurre complessità nel codice.

La programmazione funzionale offre molti vantaggi quando si tratta di manipolare collezioni, permettendo di eseguire operazioni complesse in modo chiaro e conciso. A questo proposito, le funzioni **filter** e **map** formano la base per la manipolazione delle collezioni. La funzione filter scorre attraverso una collezione e seleziona gli elementi per i quali la lambda fornita restituisce true:

```
>>> val list = listOf(1, 2, 3, 4)
>>> println(list.filter { it % 2 == 0 })
[2, 4]
```

Only even numbers remain.

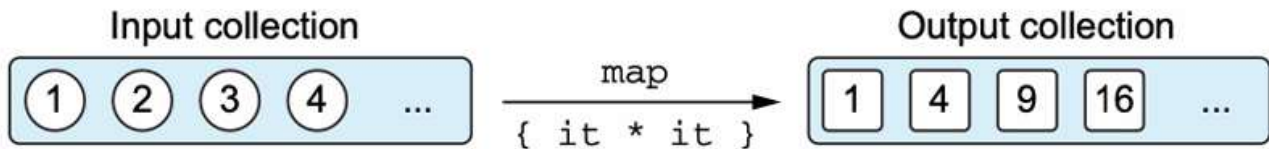
Il risultato è una nuova collection che contiene solo gli elementi presi dalla collection di partenza che soddisfano il predicato:



La funzione filter può rimuovere elementi indesiderati dalla collection, ma non può cambiare gli elementi. La funzione deputata alla trasformazione degli elementi è proprio map.

La funzione `map` applica la lambda fornita a ogni elemento della collezione e raccoglie i risultati in una nuova collezione. Ad esempio, possiamo trasformare una lista di numeri in una lista dei loro quadrati:

```
>>> val list = listOf(1, 2, 3, 4)
>>> println(list.map { it * it })
[1, 4, 9, 16]
```



Adesso vediamo un esempio più completo. Supponiamo di avere la precedente classe `Persona`:

```
data class Person(val name: String, val age: Int)
```

Se vogliamo tenere solo le persone più grandi di 30 anni, possiamo usare la funzione `filter`:

```
>>> val people = listOf(Person("Alice", 29), Person("Bob", 31))
>>> println(people.filter { it.age > 30 })
[Person(name=Bob, age=31)]
```

Se vogliamo stampare una lista di soli nomi, e non una lista di persone, possiamo trasformare la lista usando la funzione `map`:

```
>>> val people = listOf(Person("Alice", 29), Person("Bob", 31))
>>> println(people.map { it.name })
[Alice, Bob]
```

Possiamo anche concatenare vare chiamate di questo tipo, per esempio se vogliamo stampare i nomi delle persone più grandi di 30 anni:

```
>>> people.filter { it.age > 30 }.map(Person::name)
[Bob]
```

It is the same of writing {p: Person -> p.name}

Un altro comune compito è quello di controllare se tutti gli elementi di una `collection` soddisfano una certa condizione. Per farlo si ricorre alle funzioni **all** ed **any**. Per mostrarne il funzionamento, supponiamo di definire il predicato `canBeInClub27` per controllare se una persona ha 27 anni o è più giovane:

```
val canBeInClub27 = { p: Person -> p.age <= 27 }
```

Se siamo interessati a sapere se tutti gli elementi soddisfano il predicato, allora ricorriamo alla funzione `all`:

```
>>> val people = listOf(Person("Alice", 27), Person("Bob", 31))
>>> println(people.all(canBeInClub27))
false
```

Altrimenti, se siamo interessati a sapere se almeno uno degli elementi soddisfa il predicato, allora ricorriamo alla funzione `any`:


```
>>> println(people.any(canBeInClub27))
true
```

Altre funzioni utili messe a disposizione da Kotlin sono **count** e **find**. Rifacendoci al precedente esempio, se vogliamo sapere esattamente quanti elementi soddisfano il predicato, ricorriamo a count:

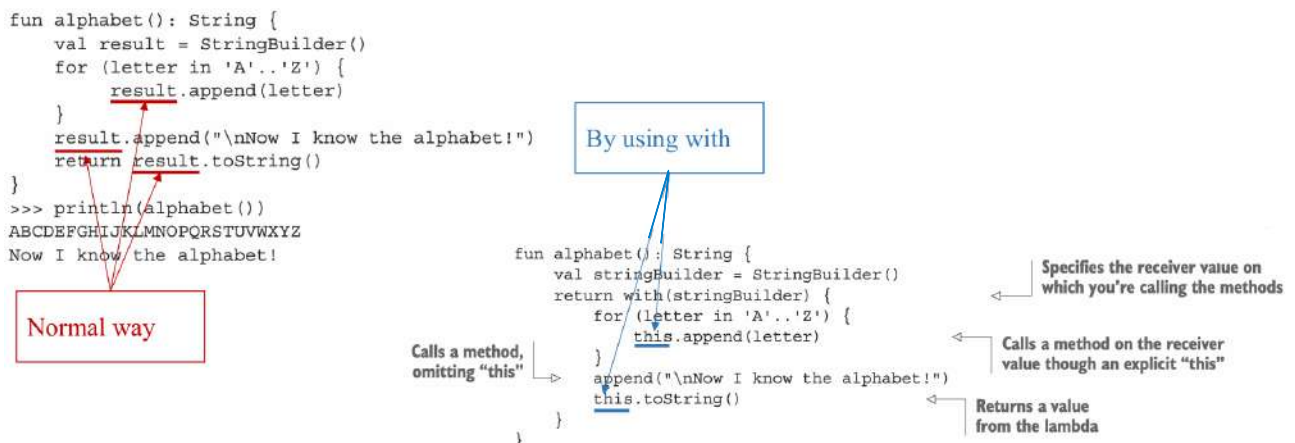
```
>>> val people = listOf(Person("Alice", 27), Person("Bob", 31))
>>> println(people.count { canBeInClub27 })
1
```

Se invece vogliamo trovare un elemento esatto che soddisfa il predicato, ricorriamo alla funzione find:

```
>>> val people = listOf(Person("Alice", 27), Person("Bob", 31))
>>> println(people.find { canBeInClub27 })
Person(name=Alice, age=27)
```

Se diversi elementi soddisfano il predicato, la funzione find ritorna il primo di essi, se nessun elemento soddisfa il predicato ritorna invece null.

Ancora altre due funzioni importanti sono **with** e **apply**. Se vogliamo effettuare molteplici operazioni su un oggetto senza ripetere il suo nome, ricorriamo a with:



La funzione with prende in input due parametri: stringBuilder (in questo caso) e una espressione lambda. In questo caso la convenzione per cui si mette la lambda fuori dalle parentesi è preferibile. La funzione converte il suo primo argomento in un ricevitore della lambda passata come secondo argomento.

1.18. Nullability

La nullability è una feature del linguaggio Kotlin che aiuta ad evitare errori di tipo `NullPointerException`.

- “An error has occurred: java.lang.NullPointerException”
- “Unfortunately, the application X has stopped”.

Kotlin converte questi problemi da errori al tempo di esecuzione a errori al tempo di compilazione. In questo modo il compilatore può individuare diversi possibili errori durante la compilazione e ridurre la possibilità che delle eccezioni siano lanciate al tempo di esecuzione. Per capire concretamente cosa sia un tipo nullable, vediamo un esempio:

```
fun strLen(s: String) = s.length
```

Siamo abituati a pensare che se passiamo un parametro null alla funzione, questa lancerà una `NullPointerException`. Tuttavia, di seguito viene mostrato quello che succede in realtà:

```
>>> strLen(null)
ERROR: Null can not be a value of a non-null type String
```

L'eccezione non viene lanciata perché in Kotlin esiste il tipo nullable. Nella funzione il parametro è dichiarato di tipo stringa, e ciò in Kotlin vuol dire che dovrà sempre contenere un'istanza di una stringa. Poiché il compilatore ti obbliga a seguire questa regola, non potremo passare un argomento di tipo null. Se, però, vogliamo permettere l'uso di questa funzione con tutti i tipi di argomenti, inclusi quelli che possono essere null, allora dobbiamo contrassegnarla esplicitamente ponendo un punto interrogativo dopo il nome del tipo:

```
fun strLenSafe(s: String?) = ...
```

Ovviamente vale anche nel caso di più argomenti.

Una volta che abbiamo un valore di tipo nullable, l'insieme di operazioni che possiamo effettuare su di esso sono alquanto limitate. Non si possono più chiamare metodi su di esso:

```
>>> fun strLenSafe(s: String?) = s.length()
ERROR: only safe (?.) or non-null asserted (!!.) calls are allowed
on a nullable receiver of type kotlin.String?
```

Non si può assegnare il valore nullable a una variabile di tipo non-null:

```
>>> val x: String? = null
>>> var y: String = x
ERROR: Type mismatch: inferred type is String? but String was expected
```

Non puoi passare un valore nullable come argomento a una funzione che ha parametri non-null:

```
>>> strLen(x)
ERROR: Type mismatch: inferred type is String? but String was expected
```

Esistono comunque varie opzioni che permettono di lavorare con i tipi nullable.

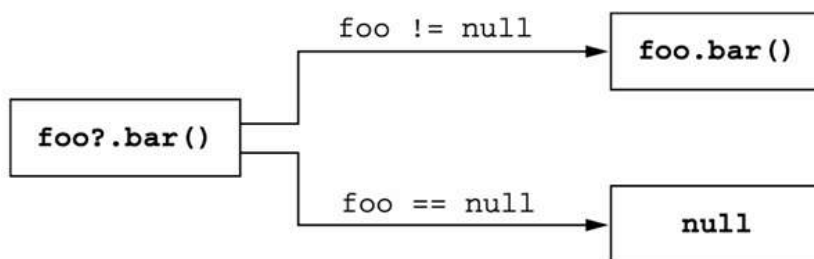
Opzione #1: La prima delle strategie per gestire i valori nullable è confrontare esplicitamente la variabile con null. Dopo aver eseguito questo controllo, il compilatore di Kotlin "ricorda" che la variabile non è null all'interno dell'ambito in cui il controllo è stato effettuato, permettendo così di accedere in sicurezza alle sue proprietà o metodi:

```
fun strLenSafe(s: String?): Int =  
    if (s != null) s.length else 0
```

← By adding the check for null,
the code now compiles.

```
>>> val x: String? = null  
>>> println(strLenSafe(x))  
0  
>>> println(strLenSafe("abc"))  
3
```

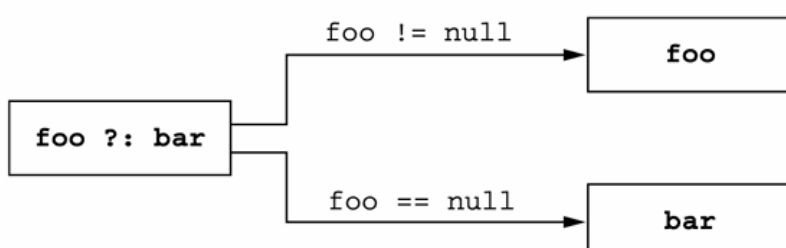
Opzione #2: l'operatore di **chiamata sicura** `"?."` permette di combinare un controllo per null e una chiamata di metodo in una singola operazione. In pratica, se l'oggetto su cui si vuole chiamare un metodo è null, l'intera espressione restituisce null invece di lanciare un'eccezione. Se l'oggetto non è null, il metodo viene chiamato normalmente:



L'operatore `"?."` può essere utilizzato non solo per chiamate di metodi, ma anche per accedere a proprietà di oggetti nullable:

```
class Employee(val name: String, val manager: Employee?)  
  
fun managerName(employee: Employee): String? = employee.manager?.name  
  
>>> val ceo = Employee("Da Boss", null)  
>>> val developer = Employee("Bob Smith", ceo)  
>>> println(managerName(developer))  
Da Boss  
>>> println(managerName(ceo))  
null
```

Opzione #2.1: l'operatore **elvis** `"?:"` prende due valori e restituisce il primo se non è null; altrimenti restituisce il secondo valore:



L'operatore Elvis viene spesso utilizzato insieme all'operatore di chiamata sicura (?) per sostituire un valore diverso da null quando l'oggetto su cui viene chiamato il metodo è null:

```
fun strLenSafe(s: String?): Int = s?.length ?: 0
```

```
>>> println(strLenSafe("abc"))
3
>>> println(strLenSafe(null))
0
```

Vediamo un esempio, quale sarà l'output di questo codice?

```
class Address(val streetAddress: String, val zipCode: Int,
              val city: String, val country: String)

class Company(val name: String, val address: Address?)

class Person(val name: String, val company: Company?)

fun printShippingLabel(person: Person) {
    val address = person.company?.address
    ?: throw IllegalArgumentException("No address")
    with (address) {
        println(streetAddress)
        println("$zipCode $city, $country")
    }
}

>>> val address = Address("Elsestr. 47", 80687, "Munich", "Germany")
>>> val jetbrains = Company("JetBrains", address)
>>> val person = Person("Dmitry", jetbrains)

>>> printShippingLabel(person)
???

>>> printShippingLabel(Person("Alexey", null))
???

```

Throws an exception if the address is absent

"address" is non-null.

L'output sarà:

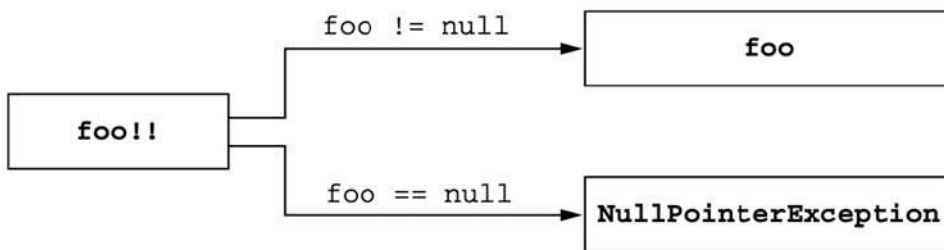
```
>>> val address = Address("Elsestr. 47", 80687, "Munich", "Germany")
>>> val jetbrains = Company("JetBrains", address)
>>> val person = Person("Dmitry", jetbrains)

>>> printShippingLabel(person)
Elsestr. 47
80687 Munich, Germany

>>> printShippingLabel(Person("Alexey", null))
java.lang.IllegalArgumentException: No address

```

Opzione #3: usare l'asserzione not null "!!" è il modo più semplice per gestire un valore di un tipo nullable. Questa converte qualsiasi valore in un tipo non nullable, tuttavia, se il valore è null, viene lanciata un'eccezione:



Un esempio di codice:

```
fun ignoreNulls(s: String?) {  
    val sNotNull: String = s!!  
    println(sNotNull.length)  
}
```

← The exception points to this line.

```
>>> ignoreNulls(null)  
Exception in thread "main" kotlin.KotlinNullPointerException  
    at <...>.ignoreNulls(07_NotnullAssertions.kt:2)
```

Opzione #4: usare la funzione let rende più semplice lavorare con espressioni nullable. In combinazione con l'operatore di chiamata sicura (?.), permette di valutare un'espressione, verificare se il risultato è null e, se non lo è, eseguire una certa logica in un'unica espressione concisa. Vediamo un esempio:

```
fun sendEmailTo(email: String) { /*...*/ }
```

Questa funzione richiede in input un parametro non-null, quindi non le possiamo passare un valore di tipo nullable:

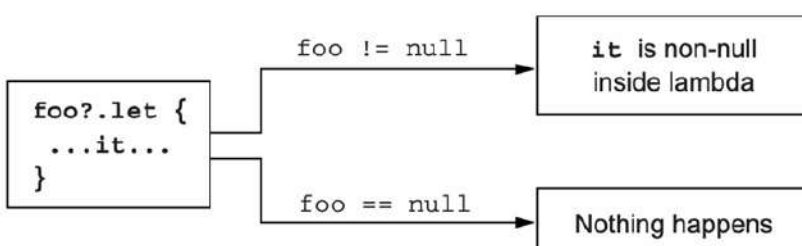
```
>>> val email: String? = ...  
>>> sendEmailTo(email)  
ERROR: Type mismatch: inferred type is String? but String was expected
```

Dobbiamo quindi aggiungere un controllo:

```
if (email != null) sendEmailTo(email)
```

Per bypassare il problema, possiamo ricorrere alla funzione let:

```
email?.let { email -> sendEmailTo(email) }
```



In questo caso, se email non è null, il blocco let viene eseguito e sendEmailTo(email) viene chiamato. Se email è null, il blocco let viene saltato e la funzione non viene chiamata. Vediamo un secondo esempio:

```
fun sendEmailTo(email: String) {
    println("Sending email to $email")
}

>>> var email: String? = "yole@example.com"
>>> email?.let { sendEmailTo(it) }
Sending email to yole@example.com
>>> email = null
>>> email?.let { sendEmailTo(it) }
```

Se abbiamo bisogno di controllare più valori per null, possiamo usare chiamate let annidate, ma il codice può diventare piuttosto verboso e difficile da seguire. In molti casi, è generalmente più semplice utilizzare un'espressione if normale per controllare tutti i valori insieme. In sintesi, la funzione let in Kotlin è molto utile per gestire valori nullable in modo sicuro e conciso, ma è importante non abusarne per evitare codice complesso e difficile da mantenere.

Molti framework, come Android, inizializzano oggetti in metodi specifici che vengono chiamati dopo che l'istanza dell'oggetto è stata creata. Per esempio, in Android, l'inizializzazione di un'attività (activity) avviene nel metodo onCreate. In Kotlin, normalmente siamo obbligati a inizializzare tutte le proprietà nel costruttore. Se una proprietà ha un tipo non-null, dobbiamo fornire un valore iniziale non-null. Tuttavia, questo può essere un problema quando non disponiamo subito del valore da assegnare alla proprietà. In questo caso si utilizza un tipo nullable, il che significa che ogni accesso a quella proprietà richiederà un controllo di null oppure l'uso dell'operatore !! per forzare il tipo. Questo approccio può risultare ingombrante e poco elegante, specialmente se accediamo alla proprietà molte volte. Per risolvere questo problema, possiamo dichiarare la proprietà come **late-initialized** utilizzando il modificatore **lateinit**. Questo ci permette di evitare l'inizializzazione immediata e di inizializzare la proprietà successivamente, ma comunque garantendo che sarà non-null al momento dell'accesso. Vediamo un esempio:

```
class MyService {
    fun performAction(): String = "foo"
}

class MyTest {
    private var myService: MyService? = null

    @Before fun setUp() {
        myService = MyService()
    }

    @Test fun testAction() {
        Assert.assertEquals("foo",
            myService!!.performAction())
    }
}
```

← Declares a property of a nullable type to initialize it with null

← Provides a real initializer in the setUp method

← You have to take care of nullability: use !! or ?.

Questo esempio mostra come gestire le proprietà nullable nei test, dove è comune ritardare l'inizializzazione di una proprietà fino a un metodo di setup. L'uso di `!!` è una soluzione, ma può essere rischioso perché può generare eccezioni se la proprietà è ancora null al momento dell'accesso. Per evitare controlli extra e problemi di questo tipo, possiamo utilizzare `lateinit`:

```
class MyService {
    fun performAction(): String = "foo"
}

class MyTest {
    private lateinit var myService: MyService

    @Before fun setUp() {
        myService = MyService()
    }

    @Test fun testAction() {
        Assert.assertEquals("foo",
            myService.performAction())
    }
}
```

Declares a property of a non-null type without an initializer

Initializes the property in the setUp method as before

Accesses the property without extra null checks

In Java una variabile di tipo primitivo contiene direttamente il suo valore. Ad esempio, una variabile `int` in Java contiene il valore numerico stesso. Una variabile di tipo di riferimento (come `String`), invece, contiene un riferimento alla posizione di memoria che ospita l'oggetto. I valori dei tipi primitivi possono essere memorizzati e passati in modo più efficiente, ma non è possibile chiamare metodi su tali valori o memorizzarli direttamente in collezioni. Per superare questa limitazione, Java fornisce dei **wrapper types** (come `java.lang.Integer`) che incapsulano i tipi primitivi, permettendone l'uso in contesti che richiedono oggetti. Ad esempio, per definire una collezione di interi in Java, si deve usare `Collection<Integer>`.

Kotlin non distingue tra tipi primitivi e tipi wrapper. Questo significa che i tipi primitivi in Kotlin vengono gestiti automaticamente come oggetti quando necessario, senza bisogno di utilizzare esplicitamente un wrapper type:

```
val i: Int = 1
val list: List<Int> = listOf(1, 2, 3)
```

Se Kotlin non distingue tra tipi primitivi e tipi di riferimento, significa che rappresenta tutti i numeri come oggetti? E se così fosse, non sarebbe terribilmente inefficiente? Ovviamente Kotlin non rappresenta tutti i numeri come oggetti. In realtà, a runtime, i tipi numerici in Kotlin sono rappresentati nel modo più efficiente possibile. Nella maggior parte dei casi, come per variabili, proprietà, parametri e tipi di ritorno, il tipo `Int` di Kotlin viene compilato nel tipo primitivo `int` di Java. Questo permette a Kotlin di mantenere l'efficienza di esecuzione tipica dei tipi primitivi, sfruttando la velocità e la ridotta occupazione di memoria che questi offrono. L'unica situazione in cui questa ottimizzazione non è possibile è quando i tipi primitivi sono utilizzati come argomenti di tipo in classi generiche, come le collezioni. In questi casi, un tipo primitivo utilizzato come argomento di tipo di una classe generica viene compilato nel corrispondente wrapper type di Java (ad esempio, `Int` diventa `Integer`).

Ecco un elenco dei tipi primitivi di Kotlin che corrispondono direttamente ai tipi primitivi di Java:

- **Tipi interi:** Byte, Short, Int, Long
- **Tipi a virgola mobile:** Float, Double
- **Tipo carattere:** Char
- **Tipo booleano:** Boolean

Per quanto riguarda i tipi primitivi nullable (Int? , Boolean?, ...) in Kotlin non possono essere rappresentati dai tipi primitivi di Java perché un valore null può essere memorizzato solo in una variabile di un tipo di riferimento di Java. Ogni volta che utilizziamo una versione nullable di un tipo primitivo in Kotlin, questa viene compilata nel corrispondente wrapper type di Java. Ad esempio, Int? viene compilato come Integer.

Infine, bisogna prestare attenzione alle conversioni numeriche:

```
val i = 1
val l: Long = i
```

X

← **Error: type mismatch**


```
val i = 1
val l: Long = i.toLong()
```

V

Esistono poi, in Kotlin, dei tipi di ritorno speciali. Il tipo **Unit** in Kotlin svolge lo stesso ruolo di void in Java. Viene usato come tipo di ritorno per una funzione che non restituisce nulla di significativo:

```
fun f(): Unit { ... }
```

Equivalente a:

```
fun f() { ... }
```

In Kotlin, è possibile omettere la dichiarazione esplicita di Unit perché essa è implicita.

In alcuni casi, in Kotlin, il concetto di "valore di ritorno" non ha senso perché la funzione non completerà mai la sua esecuzione con successo. Un esempio di questo è una funzione che contiene un ciclo infinito o che lancia un'eccezione e non restituisce mai un valore. Quando si analizza il codice che chiama una funzione di questo tipo, è utile sapere che la funzione non terminerà mai normalmente. Per esprimere questo, Kotlin usa un tipo di ritorno speciale chiamato **Nothing**:

```
fun fail(message: String): Nothing {
    throw IllegalStateException(message)
}
```

Passiamo a un altro interrogativo: è possibile in Kotlin usare tipi nullable in una collection?

Kotlin supporta pienamente questa funzione:

```
fun readNumbers(reader: BufferedReader): List<Int?> {  
    val result = ArrayList<Int?>()   
    for (line in reader.lineSequence()) {  
        try {  
            val number = line.toInt()  
            result.add(number)  
        }  
        catch (e: NumberFormatException) {  
            result.add(null)  
        }  
    }  
    return result  
}
```

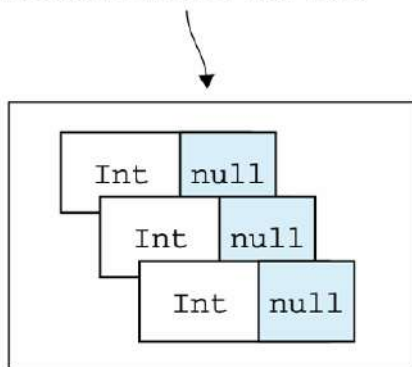
Creates a list of nullable Int values

Adds an integer (a non-null value) to the list

Adds null to the list, because the current line can't be parsed to an integer

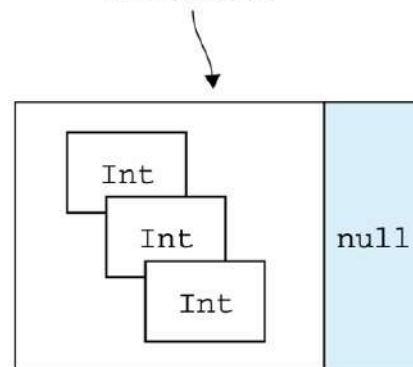
In questo esempio la lista creata può contenere valori Int o null. Ma dobbiamo stare attenti a cosa dichiariamo come nullable: gli elementi della collection o la collection stessa?

Individual values are nullable within the list.



`List<Int?>`

Entire list is nullable.



`List<Int>?`

Nel primo caso la lista sarà sempre non-null, ma ogni suo elemento potrà essere null. Nel secondo caso la variabile può contenere un riferimento null invece di una lista vera e propria, ma se la lista esiste è garantito che tutti i suoi elementi saranno non-null.

Sarebbe anche possibile dichiarare una variabile che contiene una lista nullable di numeri nullable: `List<Int?>?`

In questo caso sia la lista che i suoi elementi possono essere null, può non esistere alcuna lista ma, se esiste, essa può contenere sia valori interi che null.

1.19. Dettagli aggiuntivi sulle collections

Kotlin separa le interfacce per accedere ai dati in una collezione e per modificarli, attraverso due principali interfacce: `Collection` e `MutableCollection`.

`kotlin.collections.Collection`

- **Accesso ai dati:** L'interfaccia `Collection` in Kotlin permette di iterare sugli elementi di una collezione, ottenere la sua dimensione, verificare se contiene un determinato elemento, e altre operazioni che leggono i dati dalla collezione. È un'interfaccia di sola lettura, il che significa che i dati all'interno della collezione non possono essere modificati tramite questa interfaccia.

`kotlin.collections.MutableCollection`

- **Modifica dei dati:** L'interfaccia `MutableCollection` estende `Collection` e aggiunge metodi per modificare la collezione. Oltre ai metodi di `Collection`, `MutableCollection` fornisce metodi come `add()`, `remove()`, e `clear()`, che permettono di aggiungere, rimuovere e cancellare elementi dalla collezione.

Come regola generale, si dovrebbero utilizzare interfacce di sola lettura (`Collection`) ovunque nel codice, a meno che non sia necessario modificare la collezione. Questo approccio rende il codice più chiaro e meno incline a errori accidentali. Bisogna quindi usare le varianti mutabili (`MutableCollection`) solo se il codice deve modificare la collezione. In questo si può osservare un'analogia con le variabili `val` e `var`, le quali definiscono rispettivamente una variabile di sola lettura e una mutabile. Se una funzione accetta un parametro che è una `Collection` ma non una `MutableCollection`, sapremo che quella funzione non modificherà la collezione, ma si limiterà a leggerne i dati. Questo rende il comportamento del codice più prevedibile e comprensibile. Vediamo un esempio:

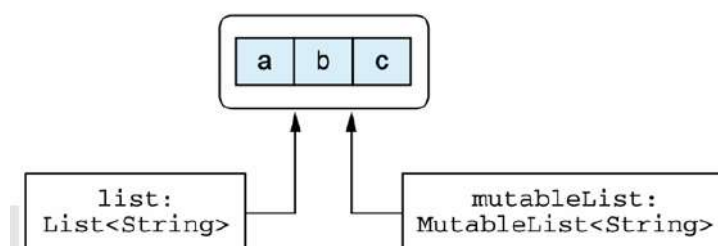
```
fun <T> copyElements(source: Collection<T>,
                    target: MutableCollection<T>) {
    for (item in source) {
        target.add(item)
    }
}
```

← Loops over all items in the source collection

← Adds items to the mutable target collection

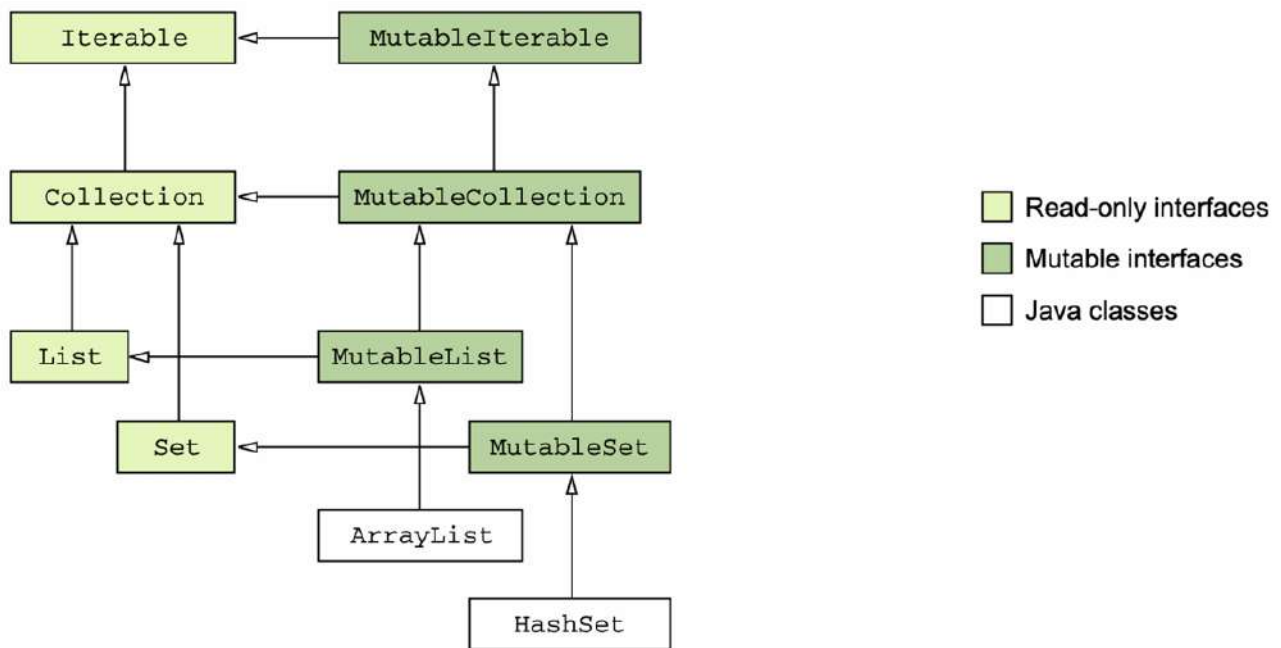
```
>>> val source: Collection<Int> = arrayListOf(3, 5, 7)
>>> val target: MutableCollection<Int> = arrayListOf(1)
>>> copyElements(source, target)
>>> println(target)
[1, 3, 5, 7]
```

Bisogna stare attenti a situazioni in cui si gestiscono collection con riferimenti sia di sola lettura sia mutabili, perché potrebbero portare all'errore `ConcurrentModificationException`:



La figura mostra due riferimenti, uno di sola lettura (`List<String> list`) e uno mutabile (`MutableList<String> mutableList`), che puntano allo stesso oggetto di collezione. Questo scenario può portare a problemi come `ConcurrentModificationException` se si tenta di modificare la collezione attraverso uno dei riferimenti mentre è in corso un'iterazione o un'altra operazione sull'altro riferimento.

La seguente immagine presenta la gerarchia delle interfacce di collezione di Kotlin e la loro relazione con le classi Java comuni come `ArrayList` e `HashSet`:



Collection type	Read-only	Mutable
List	<code>listOf</code>	<code>mutableListOf</code> , <code>arrayListOf</code>
Set	<code>setOf</code>	<code>mutableSetOf</code> , <code>hashSetOf</code> , <code>linkedSetOf</code> , <code>sortedSetOf</code>
Map	<code>mapOf</code>	<code>mutableMapOf</code> , <code>hashMapOf</code> , <code>linkedMapOf</code> , <code>sortedMapOf</code>

In Kotlin, ci sono diverse possibilità per creare un array:

1. **Funzione `arrayOf`:** Questa funzione crea un array contenente gli elementi specificati come argomenti della funzione. Esempio: `val arr = arrayOf(1, 2, 3)` creerà un array contenente i numeri 1, 2, 3.
2. **Funzione `arrayOfNulls`:** Questa funzione crea un array di una dimensione specificata, contenente elementi null. Può essere utilizzata solo per creare array in cui il tipo degli elementi è nullable. Esempio: `val arr = arrayOfNulls<Int>(3)` creerà un array di `Int` nullable di lunghezza 3, con tutti gli elementi inizializzati a null.
3. **Costruttore `Array`:** Il costruttore `Array` prende come parametri la dimensione dell'array e una lambda, utilizzata per inizializzare ogni elemento dell'array. Questo è utile per inizializzare un array con un tipo di elemento non-null senza dover specificare

esplicitamente ogni elemento. Esempio: `val arr = Array(5) { it * 2 }` creerà un array di 5 elementi in cui ogni elemento è il doppio del suo indice (0, 2, 4, 6, 8).

In generale, **arrayOf** è usato quando conosci già gli elementi da includere, **arrayOfNulls** è utile quando vuoi un array di una certa dimensione con elementi inizializzati a null e **il costruttore Array** è potente quando hai bisogno di generare dinamicamente gli elementi dell'array in base a una logica specifica, ad esempio:

```
>>> val letters = Array<String>(26) { i -> ('a' + i).toString() }
>>> println(letters.joinToString(""))
abcdefghijklmnopqrstuvwxyz
```

Kotlin fornisce una serie di classi distinte per rappresentare array di tipi primitivi, una per ogni tipo primitivo. Ad esempio, un array di `Int` è rappresentato dalla classe `IntArray`. Per creare un array di tipo primitivo in Kotlin, abbiamo diverse opzioni:

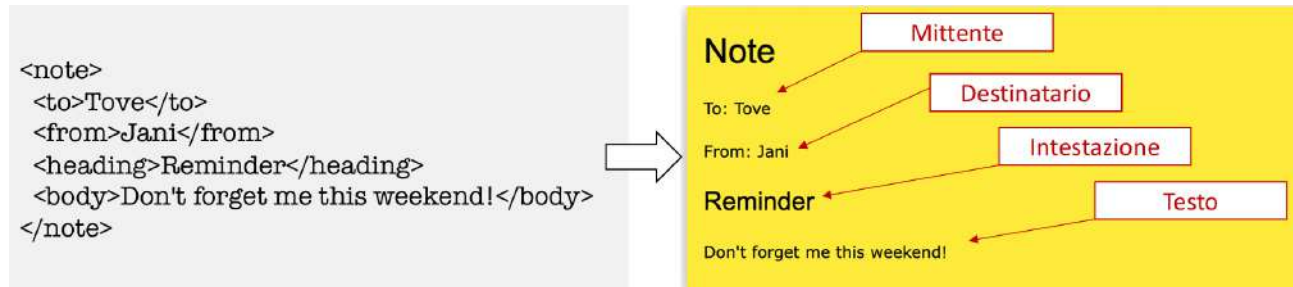
- **Costruttore del tipo:** Il costruttore prende un parametro di dimensione e restituisce un array inizializzato con i valori predefiniti per il tipo primitivo corrispondente (di solito 0 per i numeri). Esempio: `val fiveZeros = IntArray(5)` crea un array di 5 elementi tutti inizializzati a 0.
- **Funzione di fabbrica (`intArrayOf`):** Questa funzione prende un numero variabile di valori come argomenti e crea un array che contiene quei valori. Esempio: `val fiveZerosToo = intArrayOf(0, 0, 0, 0, 0)` crea un array con 5 zeri.
- **Costruttore con lambda:** Un altro costruttore prende una dimensione e una lambda, e utilizza la lambda per inizializzare ogni elemento dell'array. Ad esempio:

```
>>> val squares = IntArray(5) { i -> (i+1) * (i+1) }
>>> println(squares.joinToString())
1, 4, 9, 16, 25
```

2. XML

XML sta per eXtensible Markup Language ed è un linguaggio di markup molto simile ad HTML. XML è stato progettato principalmente per archiviare e trasportare dati, inoltre, per la sua autodescrittività, è una raccomandazione del W3C (World wide web consortium, comunità internazionale che si occupa della definizione di standard web aperti per promuovere l'accessibilità e la compatibilità delle tecnologie in rete).

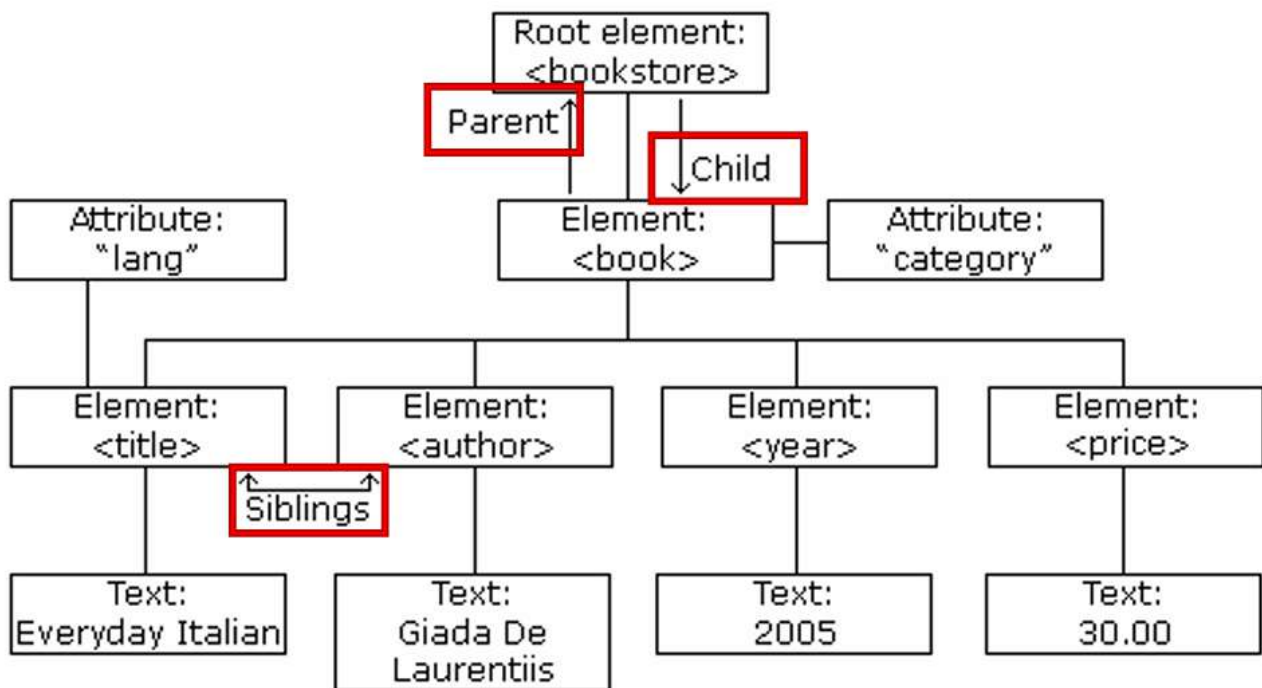
XML si limita a fornire informazioni racchiuse in tag e separa i dati dalla presentazione:



Vediamo la struttura ad albero tipica di XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book category="cooking">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
  </book>
  <book category="web">
    <title lang="en">Learning XML</title>
    <author>Erik T. Ray</author>
    <year>2003</year>
    <price>39.95</price>
  </book>
</bookstore>
```

- <bookstore> apre il documento e </bookstore> lo chiude.
- Sono presenti due <book> all'interno del <bookstore>, i quali contengono <title>, <author>, <year>, <price>. Ogni tag in <book> contiene un valore.



I documenti XML sono quindi strutturati come alberi di elementi. La sintassi XML è semplice e logica:

1. Un albero XML inizia sempre da un elemento radice (root) che è il genitore di tutti gli altri elementi e si ramifica agli elementi figli (child):

```

<root>
  <child>
    <subchild>.....</subchild>
  </child>
</root>

```

2. I documenti XML possono contenere un prologo e, in tal caso, deve essere dichiarato all'inizio:

```

<?xml version="1.0" encoding="UTF-8"?>

```

3. Tutti gli elementi XML devono avere un tag di chiusura:

```

<message>This is correct</message>

```

4. I tag XML sono case-sensitive (fanno distinzione tra maiuscole e minuscole).
5. Gli elementi XML devono essere nidificati correttamente:

```

<tag1> <tag2>This is correct </tag2> </tag1>

```

6. I valori degli attributi XML devono essere sempre quotati (“ ”):

```
<note date="12/11/2007">
  <to>Tove</to>
  <from>Jani</from>
</note>
```

Per ricapitolare: Un elemento XML è tutto il contenuto che va dal tag di inizio (incluso) al tag di fine (incluso). Un elemento può contenere testo, attributi, altri elementi o un mix di quanto detto fino ad ora. Gli elementi XML devono seguire alcune regole di denominazione:

1. I nomi degli elementi sono case-sensitive.
2. I nomi degli elementi devono iniziare con una lettera o un carattere di sottolineatura (“_”).
3. I nomi degli elementi non possono iniziare con la sequenza di lettere “xml” o “XML” o “Xml” e così via.
4. I nomi degli elementi possono contenere lettere, cifre, trattini, caratteri di sottolineatura e punti.
5. I nomi degli elementi non possono contenere spazi.

Gli attributi XML sono concepiti per contenere dati relativi a uno specifico elemento. I valori di tali attributi, come già detto, devono sempre essere quotati tramite doppi apici (“ ”) o apici singoli (‘ ’):

```
<person gender="female">
```

```
<person gender='female'>
```

Ricordiamo che esistono solo due generi: maschio (uomo) e femmina (donna). Inoltre, non ci sono regole su quando usare gli attributi o gli elementi in XML; infatti, i seguenti due esempi forniscono esattamente le stesse informazioni:

```
<person gender="female">
  <firstname>Anna</firstname>
  <lastname>Smith</lastname>
</person>
```

```
<person>
  <gender>female</gender>
  <firstname>Anna</firstname>
  <lastname>Smith</lastname>
</person>
```

È buona pratica assegnare degli identificativi agli elementi a cui è possibile riferirsi:

```
<messages>
  <note id="501">
    <to>Tove</to>
    <from>Jani</from>
    <heading>Reminder</heading>
    <body>Don't forget me this weekend!</body>
  </note>
  <note id="502">
    <to>Jani</to>
    <from>Tove</from>
    <heading>Re: Reminder</heading>
    <body>I will not</body>
  </note>
</messages>
```

Poiché in XML i nomi degli elementi sono definiti dallo sviluppatore, spesso si va incontro a conflitti quando si tenta di combinare documenti XML da diverse applicazioni XML:

```
<table>
  <tr>
    <td>Apples</td>
    <td>Bananas</td>
  </tr>
</table>
```

table.xml

```
<table>
  <name>African Coffee Table</name>
  <width>80</width>
  <length>120</length>
</table>
```

forniture.xml

Ecco perché i namespaces XML aiutano ad evitare conflitti di questo tipo, utilizzando un prefisso per il nome:

```
<h:table>
  <h:tr>
    <h:td>Apples</h:td>
    <h:td>Bananas</h:td>
  </h:tr>
</h:table>

<f:table>
  <f:name>African Coffee Table</f:name>
  <f:width>80</f:width>
  <f:length>120</f:length>
</f:table>
```

Chiaramente per utilizzare tali prefissi è necessario definire un namespace per il prefisso, e ciò può essere fatto da un attributo xmlns nel tag iniziale di un elemento:

xmlns:prefix="URI"

L'URI (Uniform Resource Identifier) è una stringa di caratteri che identifica una risorsa Internet.

```
<root>

<h:table xmlns:h="http://www.w3.org/TR/html4/">
  <h:tr>
    <h:td>Apples</h:td>
    <h:td>Bananas</h:td>
  </h:tr>
</h:table>

<f:table
xmlns:f="https://www.w3schools.com/furniture">
  <f:name>African Coffee Table</f:name>
  <f:width>80</f:width>
  <f:length>120</f:length>
</f:table>

</root>
```

Oppure:

```
<root xmlns:h="http://www.w3.org/TR/html4/"
xmlns:f="https://www.w3schools.com/furniture">

<h:table>
  <h:tr>
    <h:td>Apples</h:td>
    <h:td>Bananas</h:td>
  </h:tr>
</h:table>

<f:table>
  <f:name>African Coffee Table</f:name>
  <f:width>80</f:width>
  <f:length>120</f:length>
</f:table>

</root>
```

L'URI del namespace non viene utilizzato dal parser per cercare informazioni. Lo scopo di utilizzo di un URI è assegnare al namespace un nome univoco.

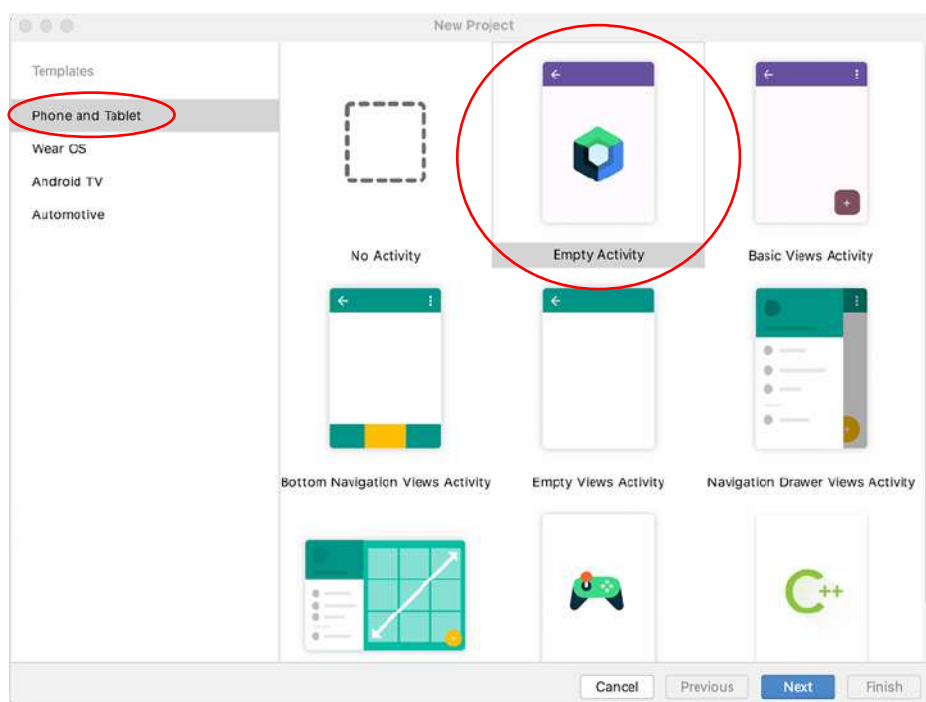
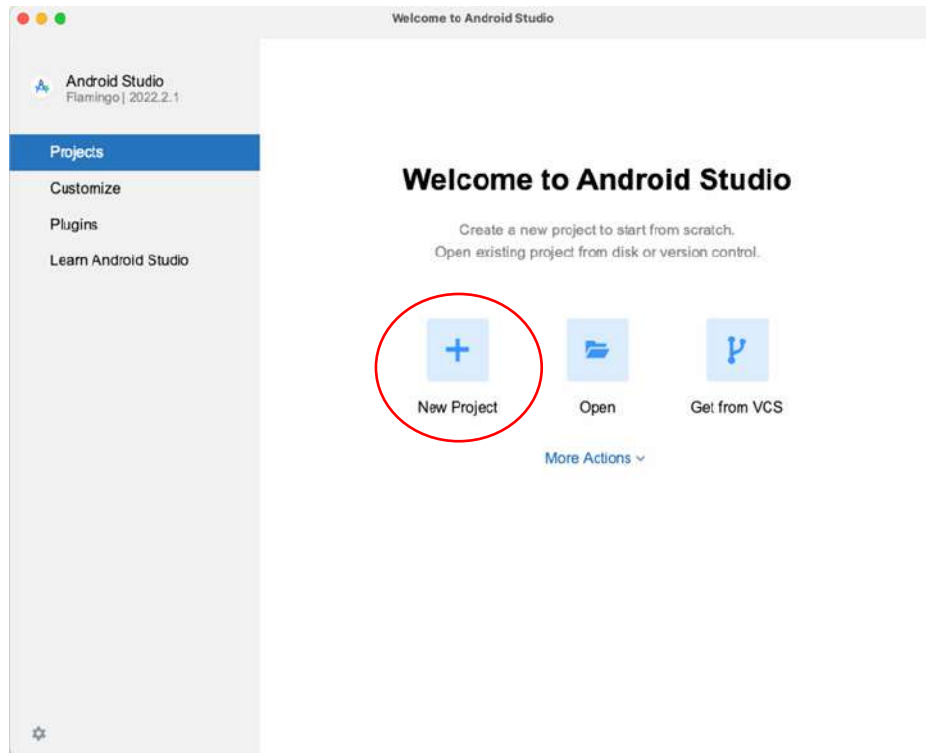
Le aziende usano spesso il namespace come puntatore a una pagina web contenente informazioni sul namespace.

3. Concetti base di app Android

Android studio è il software ufficiale del sistema operativo Android per la creazione di applicazioni.

3.1. Impostare un progetto

Dopo aver scaricato l'ultima versione del software, per creare un nuovo progetto bisogna cliccare su “New project” → “Phone and tablet” → “Empty activity”.



Bisogna inoltre impostare il cosiddetto “Minimum SDK” (che approfondiremo più avanti) a API 31 (“S”; Android 12.0).

New Project

Empty Activity

Create a new empty activity with Jetpack Compose

Name: My Application

Package name: com.example.myapplication

Save location: pa\Terzo anno\Secondo semestre\Programmazione web e mobile\MyApplication

Minimum SDK: API 31 ("S", Android 12.0)

ⓘ Your app will run on approximately 48.6% of devices.
[Help me choose](#)

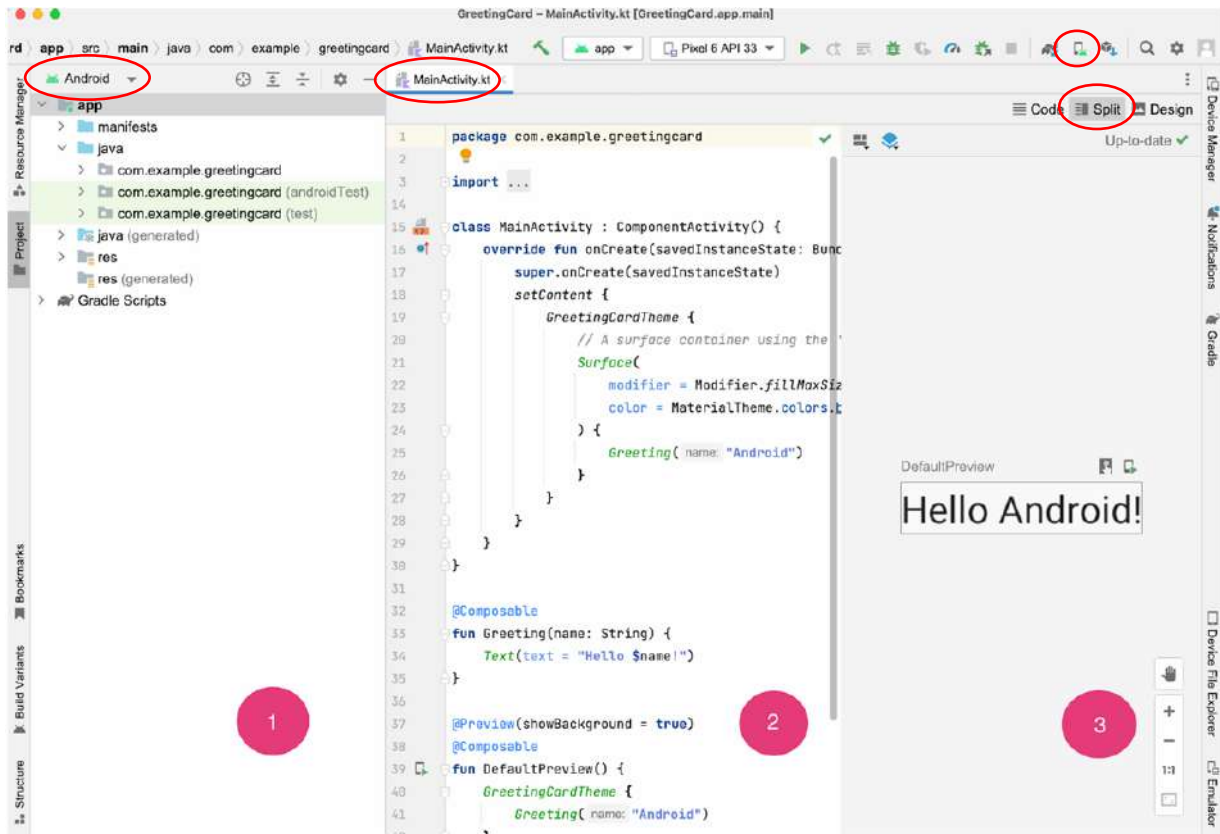
Build configuration language: Kotlin DSL (build.gradle.kts) [Recommended]

⚠ Save location should not contain whitespace, as this can cause problems with the NDK tools.

Previous Next Cancel Finish

3.2. Primo impatto ad Android studio

Una volta terminato il setup del progetto, inizierà il processo di sincronizzazione del gradle (che approfondiremo a breve), al termine del quale potrete usufruire di tutti i file base costruiti dal software. La schermata si presenterà in questo modo:



1. È la porzione di schermo deputata all'organizzazione di tutti i tipi di file che faranno parte del progetto (assicurarsi di aver selezionato la voce "Android").
2. È la porzione di schermo deputata alla scrittura del codice in linguaggio Kotlin.
3. È la porzione di schermo deputata alla visualizzazione del prodotto del codice scritto (se corretto).

Per testare la nostra app sarà possibile emulare una grande varietà di dispositivi android:

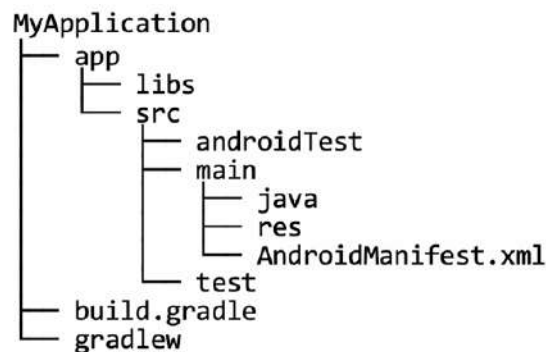


3.3. Anatomia di un'app Android

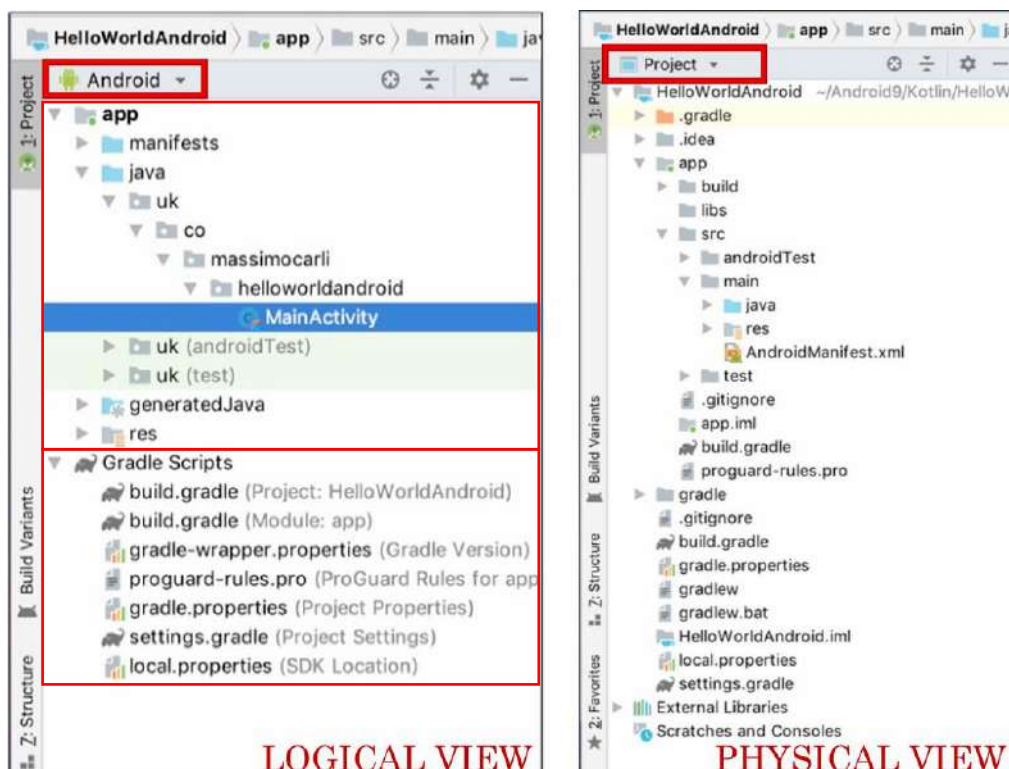
Ogni app android è costituita da 3 tipi di componenti:

- **Activity:** è una componente che gestisce l'input dell'utente e che crea e mostra le schermate dell'app.
- **Risorse:** sono tutti i file aggiuntivi che vengono utilizzati dal codice dell'app, come ad esempio immagini, file audio, file XML, ecc...
- **File gradle:** sono degli script che controllano come compilare il codice dell'app e generare quindi il file APK da installare sui dispositivi Android.

Quando creiamo la nostra prima app tramite template, Android studio genera una struttura di progetto simile alla seguente:



Questa struttura può essere osservata dal menù del software, dove sarà possibile visualizzare il contenuto del progetto secondo due possibili modalità, dette “**Android View**” e “**Project View**”. La prima fornisce una vista logica sul progetto, mentre la seconda mostra i file e le cartelle da cui è fisicamente composto il progetto. La scelta di una delle due modalità dipende esclusivamente dalle preferenze personali, ma è consigliabile scegliere la prima.



Come si può notare, sono presenti due sezioni principali: la sezione “App” e la sezione “Gradle”. La prima contiene il codice sorgente e le risorse che saranno utilizzate per costruire l’applicazione, mentre la seconda contiene quegli script necessari a controllare come la nostra applicazione è costruita.

3.3. Sezione “App”

La sezione “App” contiene alcuni file statici o aggiuntivi usati dal codice:

- Android manifest
- Activities
- Risorse:
 - File di layout
 - Immagini
 - File audio
 - Stringhe dell’interfaccia utente

3.3.1. AndroidManifest.xml

Il file “AndroidManifest.xml” contiene le informazioni essenziali di un’app Android che vengono utilizzate dagli strumenti di compilazione (Android Build Tools), dal sistema Android e da Google Play. Al suo interno sono sempre indicati:

- Le componenti dell’app quali activities, servizi, fornitori di contenuti, ecc...
- I permessi richiesti dall’app per accedere a parti protette del sistema o di altre app.
- I requisiti hardware e software che determinano quali dispositivi possono installare l’app da Google Play.
- Il nome e l’icona dell’app.

```
<manifest ... >
  <application ... >
    <activity android:name="com.example.myapp.MainActivity" ... >
    </activity>
  </application>
</manifest>
```

```
<manifest ... >
  <uses-feature android:name="android.hardware.sensor.compass"
    android:required="true" />
  ...
</manifest>
```

```
<manifest ... >
  <uses-permission android:name="android.permission.SEND_SMS"/>
  ...
</manifest>
```

L’Android Manifest viene sempre generato automaticamente da Android Studio, ma può essere opportunamente modificato dallo sviluppatore in funzione delle sue esigenze. Di seguito è riportato un esempio di Android Manifest:



```
<?xml version="1.0" encoding="utf-8"?>
<manifest
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionCode="1"
    android:versionName="1.0">

    <!-- Beware that these values are overridden by the build.gradle file -->
    <uses-sdk android:minSdkVersion="15" android:targetSdkVersion="26" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:label="@string/app_name"
        android:supportRtl="true"
        android:theme="@style/AppTheme">

        <!-- This name is resolved to com.example.myapp.MainActivity
             based upon the namespace property in the 'build.gradle' file -->
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity
            android:name=".DisplayMessageActivity"
            android:parentActivityName=".MainActivity" />
    </application>
</manifest>
```

Il **livello di API** di un'app determina le funzioni a cui quest'ultima può accedere e le versioni del sistema Android con cui l'app è compatibile. Dal momento che nel corso degli anni vengono rilasciate nuove versioni di Android, gli sviluppatori devono cercare di adottare API quanto più recenti in modo tale da poter utilizzare le funzioni introdotte più recentemente e quindi fornire un'esperienza utente migliore. Bisogna sottolineare però che adottare API recenti può comportare una riduzione del numero di dispositivi compatibili con l'app.

Android Version	API Level/SDK	Version Name	Version Code
Android 12	Level 32	S_V2	Snow Cone
	Level 31	S	
Android 11	Level 30	R	Red Velvet Cake
Android 10	Level 29	Q	Quince Tart
Android 9	Level 28	P	Pie

Come si può evincere dalla tabella riportata, ad una versione di Android possono corrispondere più livelli di API, ma a ciascun livello di API è associata una singola versione di Android. Quando si crea un progetto Android bisogna definire tre livelli di API:

- **minSdkVersion**: è il livello minimo richiesto per installare l'app. È definito nel file Manifest e nel Gradle.

- **targetSdkVersion**: è il livello preso come target dell'app. È definito nel file Manifest e nel Gradle.
- **compileSdkVersion**: è il livello usato per compilare l'app. È definito nel Gradle.

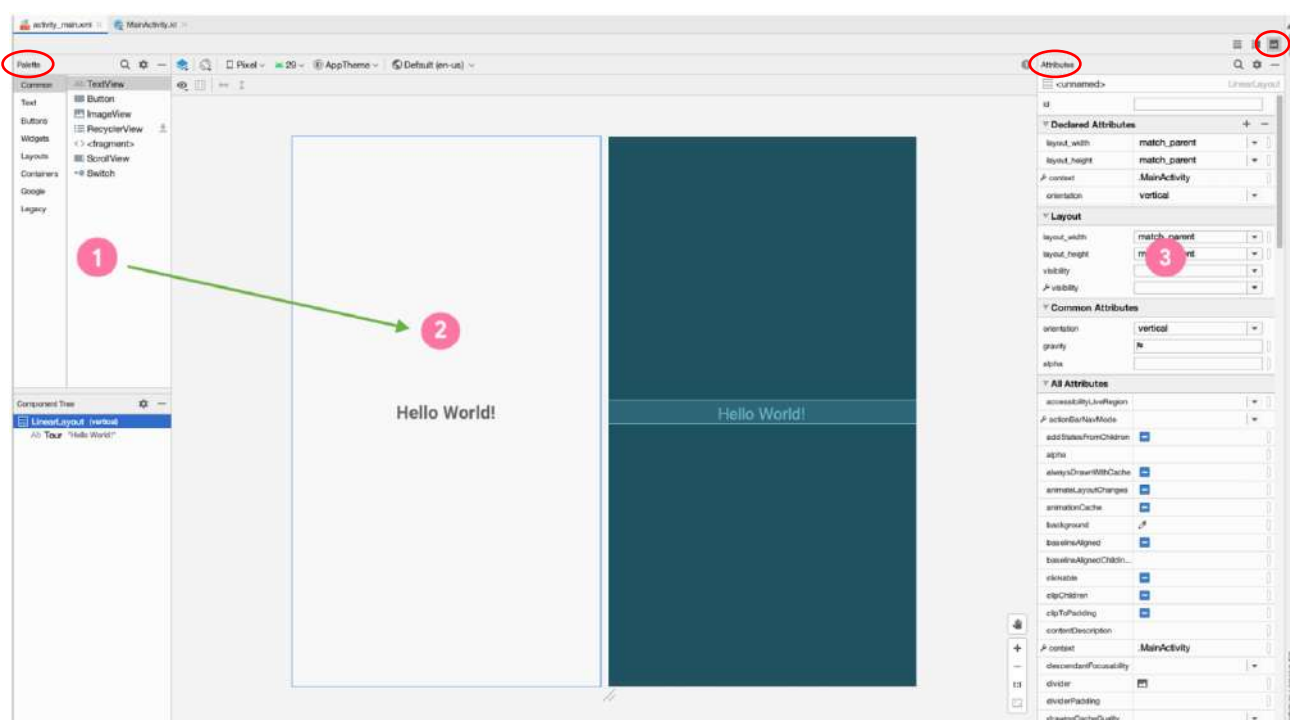
Fra i 3 livelli illustrati vale inoltre la seguente relazione:

$$\text{minSdkVersion} \leq \text{targetSdkVersion} \leq \text{compileSdkVersion}$$

Con l'arrivo di nuove versioni di Android, è una buona pratica testare la propria app con i nuovi livelli di API, ed eventualmente aumentare il **targetSdkVersion** e il **compileSdkVersion**, tenendo conto però che ciò potrebbe causare il malfunzionamento di alcune componenti del progetto, che andranno conseguentemente modificate. Il sistema di API di Android consente di determinare se un'applicazione è compatibile con un'immagine del sistema Android ancora prima di installare l'applicazione su un dispositivo.

3.3.2. Layout e risorse

Un **layout** altro non è che una composizione di Views, che a loro volta possono contenere altre Views. Si definisce View (o widget) tutte le componenti dell'interfaccia grafica in Android o, più semplicemente, tutti i blocchi che costituiscono l'interfaccia utente di un'app. Rientrano fra le Views, quindi, immagini (ImageView), testi (TextView) e bottoni (Button). In generale, una View è delimitata da un'area rettangolare sulla schermata in cui viene visualizzata ed è responsabile della rappresentazione a schermo di elementi grafici e della gestione di eventi quali quelli scaturiti dagli input dell'utente. Prima di approfondire le differenti tipologie di Views disponibili, va detto che ciascuna di esse è caratterizzata da specifici attributi, che non necessariamente sono presenti nelle altre. Il layout editor di Android studio permette anche il drag and drop di componenti dalla "Palette" nella design view, dove possiamo osservare una preview del layout che stiamo utilizzando. È possibile inoltre modificare gli attributi delle view nella parte destra dello schermo (finestra degli attributi).



Un **ViewGroup** è un contenitore che determina la disposizione delle Views contenute al suo interno, dette figlie. Esistono più tipi di ViewGroup, quali:

- **FrameLayout**: viene utilizzato qualora il ViewGroup debba contenere una sola View.
- **LinearLayout**: viene utilizzato per allineare in riga o in colonna più Views.
- **ConstraintLayout**: viene utilizzato per la realizzazione di layout complessi, e generalmente fornisce prestazioni superiori a quelle del LinearLayout.

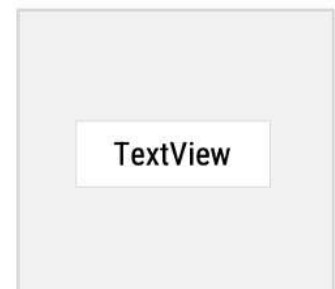


Ciascun tipo di ViewGroup sarà in seguito oggetto di analisi più approfondite, per adesso basta sapere che le caratteristiche di un ViewGroup sono definite nei file XML dove vengono definiti i layout da utilizzare nell'app. Per rendere il tutto più chiaro, di seguito è riportata la definizione di un FrameLayout e di un LinearLayout in XML:

<FrameLayout

```
android:layout_width="match_parent"
android:layout_height="match_parent">
<TextView
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:text="Hello World!"/>
```

</FrameLayout>



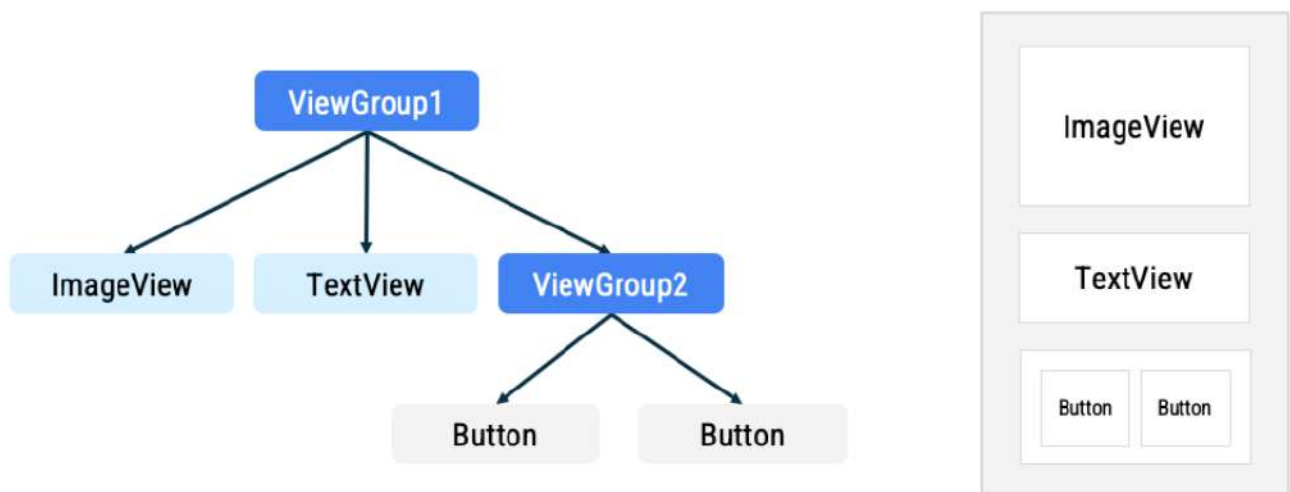
<LinearLayout

```
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical">
<TextView ... />
<TextView ... />
<Button ... />
```

</LinearLayout>



Normalmente un'interfaccia utente è costituita dalla combinazione di più layout, stabilendo una gerarchia tra questi:



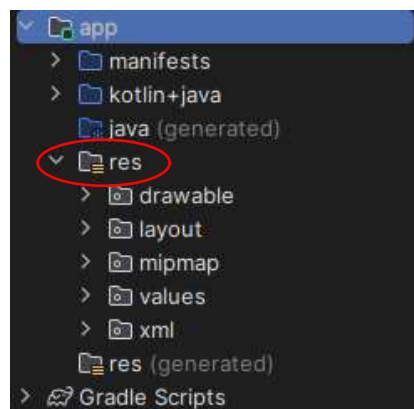
Come è possibile evincere dall'immagine, le Views figlie di un ViewGroup sono posizionate in corrispondenza del ViewGroup e sono disposte in modo da rientrare nei bordi di quest'ultimo.

Il file XML del layout appena creato sarà salvato in **res/layout/directory** dato che rappresenta una risorsa. Per esempio, se il layout è stato creato per l'interfaccia principale (main):

```
main
├── java
├── res
│   ├── drawable
│   ├── layout
│   │   └── activity_main.xml
│   ├── mipmap
│   └── values
```

Oltre a definire i nostri layout, possiamo anche aggiungere le nostre immagini, file audio, stringhe, l'icona dell'app e altre risorse.

I file XML dei layout sono, come già detto, parte delle risorse di un progetto, che sono organizzate all'interno di una cartella "res" (visibile in Android Studio).



A sua volta la cartella res è costituita da più cartelle, ovvero:

- **drawable:** contiene i file relativi alla rappresentazione di immagini e asset grafici.
- **layout:** contiene i file XML relativi ai layout dell'app.
- **mipmap:** contiene i file relativi all'icona dell'app per differenti risoluzioni dello schermo.
- **values:** contiene i file relativi a colori, stringhe, e stili utilizzati dall'app.

Le restanti cartelle non sono prese in considerazione perché generate automaticamente e di scarso interesse.

Per accedere ad una specifica risorsa si utilizza il suo ID, che per convenzione è composto da soli caratteri minuscoli ed eventuali underscore. Di default Android genera una classe R.java che contiene riferimenti a tutte le risorse dell'app; quindi, per accedere ad una risorsa via codice basta utilizzare la notazione R.tipo_risorsa.nome_risorsa. Per fare un esempio, con l'espressione R.layout.activity_main si accede al layout individuato dal percorso "res/layout/activity_main.xml".

All'interno di un file XML anche una singola View può disporre di un ID, ed è una buona pratica assegnargliene uno. Per farlo basta aggiungere l'attributo android:id alla View nel file XML in cui è definita, come illustrato nel seguente esempio:

```
<TextView  
    android:id="@+id/helloTextView"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="Hello World!"/>
```

Quindi, per fare riferimento al TextView appena definito all'interno della nostra app basta utilizzare l'espressione R.id.id.helloTextView.

3.3.3. Activity

Si definisce una **Activity** come il mezzo che consente all'utente di raggiungere un obiettivo principale. In termini pratici una Activity è una classe che definisce il funzionamento delle schermate presenti in un'app, e ricopre quindi un ruolo molto importante dato che ogni applicazione consiste in più schermate che interagiscono tra loro. La prima schermata mostrata all'avvio di una qualunque app Android prende il nome di MainActivity, ed è implementata nel file Kotlin MainActivity.kt. Per fornire un'idea di come sia strutturata la MainActivity viene riportato il seguente codice:

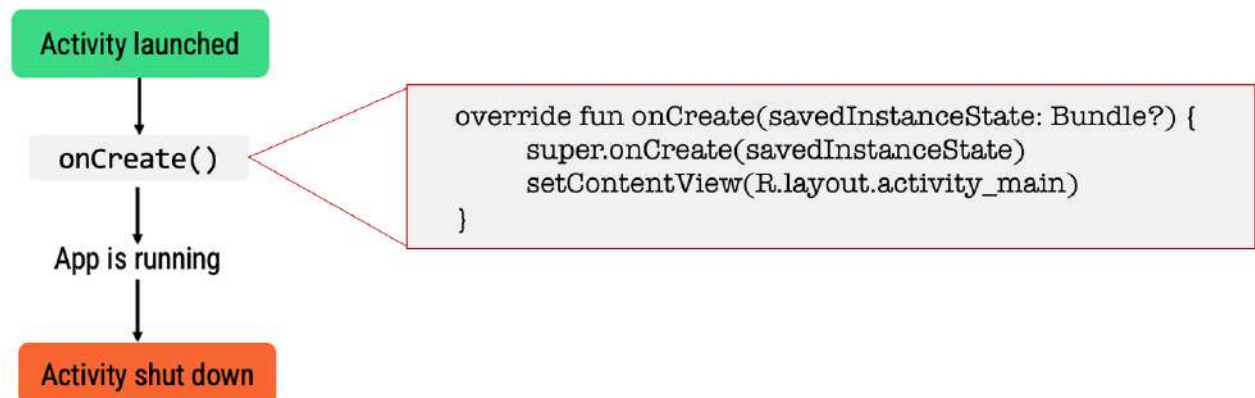
```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
    }  
}
```

It extends from the AppCompatActivity class, where we inherit behavior from the Android framework about how an Activity works.

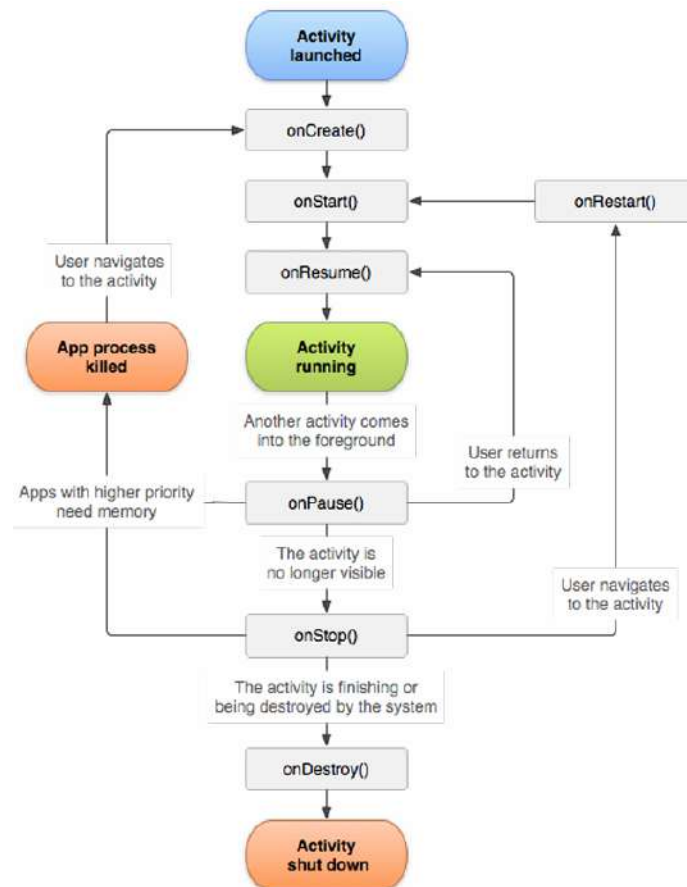
Within the class, we have one function that is overriding the onCreate function that was defined in the superclass.

La classe MainActivity utilizza AppCompatActivity per avere accesso a funzionalità che supportano la compatibilità su diverse versioni di Android e integrano funzionalità comuni delle activity.

Android lancia la nostra app tramite un'istanza di activity. Una volta che l'utente clicca sull'icona dell'applicazione nel proprio dispositivo, Android apre l'app lanciando la main activity. Il metodo onCreate è uno dei metodi del **ciclo di vita di un activity** Android e viene chiamato quando essa viene creata per la prima volta:

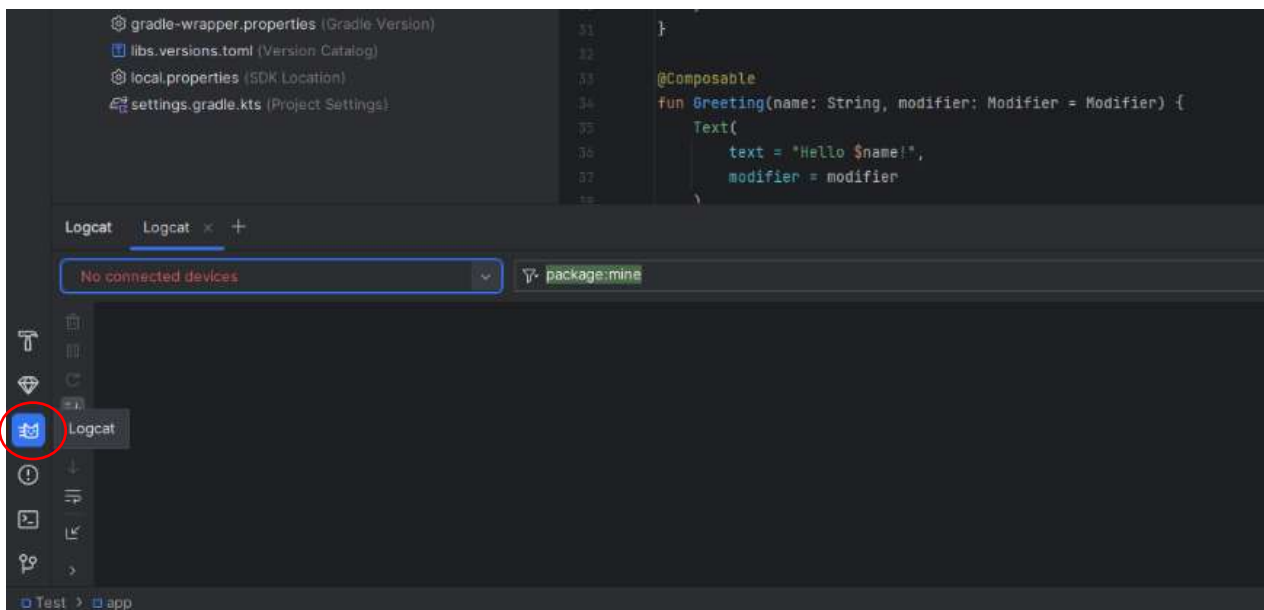


La classe Activity fornisce una serie di **callback** che consentono all'app di sapere quando uno stato cambia così da eseguire il corretto comportamento. Una funzione di callback è una funzione che viene passata come argomento ad un'altra funzione. Tali funzioni sono il componente essenziale per i meccanismi **asincroni**. La programmazione asincrona è un paradigma di programmazione che permette l'esecuzione di un programma suddivisa in più **thread** di esecuzione. La classe Activity fornisce un set di 7 funzioni di callback che, insieme, costituiscono il **ciclo di vita di una Activity**:

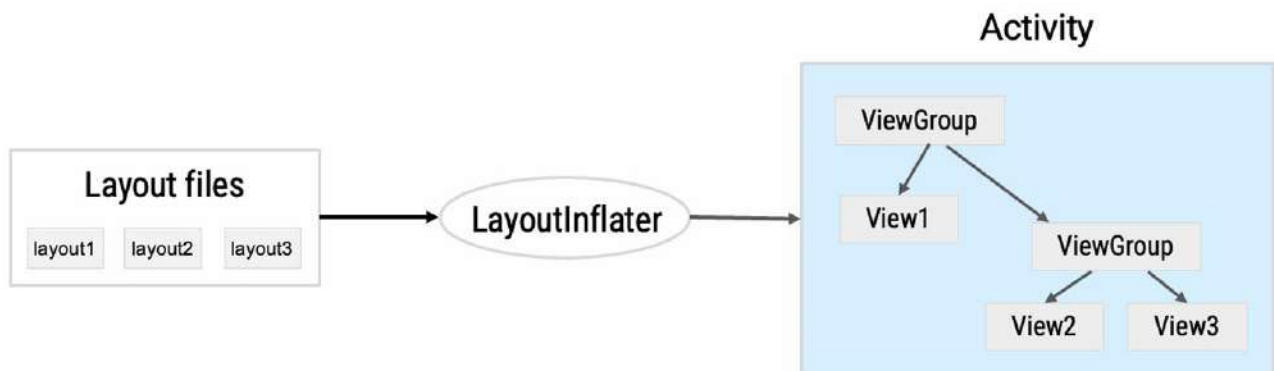


Android studio è dotato anche di una finestra **Logcat**, la quale consente di eseguire il debug dell'app mostrando i log del dispositivo in tempo reale. È possibile aggiungere i propri log attraverso la classe Log. A ogni log che viene mostrato sono associati data, ID del processo e del thread, tag, nome del pacchetto, priorità e messaggio. Ogni tag ha un colore univoco che consente di identificare il tipo di log, e i colori dipendono dalla priorità: FATAL, ERROR, WARNING, INFO, DEBUG o VERBOSE. La sintassi base per la creazione di un log è la seguente:

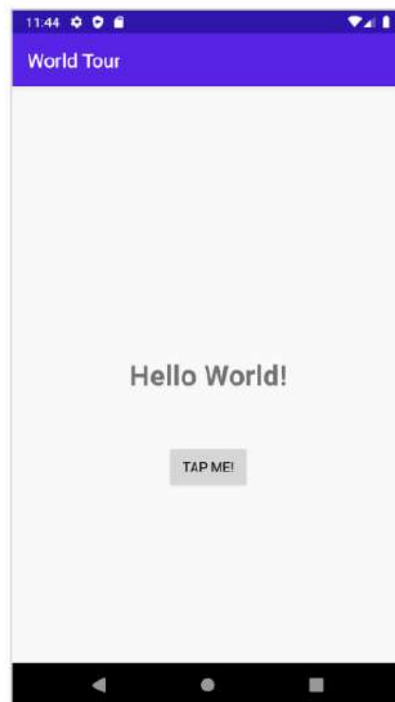
`Log.d(TAG, "messaggio")`



La **Layout inflation** è il processo per il quale un file XML che definisce un layout viene analizzato e trasformato in una gerarchia di oggetti View all'interno di un Activity Android per mezzo di un **LayoutInflater**:



Tramite le Activity, possiamo definire il comportamento che assumerà la nostra app in risposta a determinati input. Per esempio, potremmo modificare il file MainActivity in modo tale da mostrare la stringa “Hello world” quando il bottone viene pressato dall’utente:



Per fare ciò, dobbiamo ottenere un riferimento alla View interessata nella gerarchia delle view:

```
val resultTextView: TextView = findViewById(R.id.textView)
```

Il ottiene un riferimento a una TextView nell'activity usando il metodo findViewById. Questo metodo prende l'ID della view, che deve essere definito nel file XML del layout dell'activity, e restituisce un riferimento all'oggetto TextView.

Dopo aver ottenuto il riferimento, il testo della TextView può essere modificato impostando una nuova stringa per la proprietà text. In questo esempio, il testo "Hello World!" verrà cambiato in "Goodbye!" quando il pulsante viene premuto.

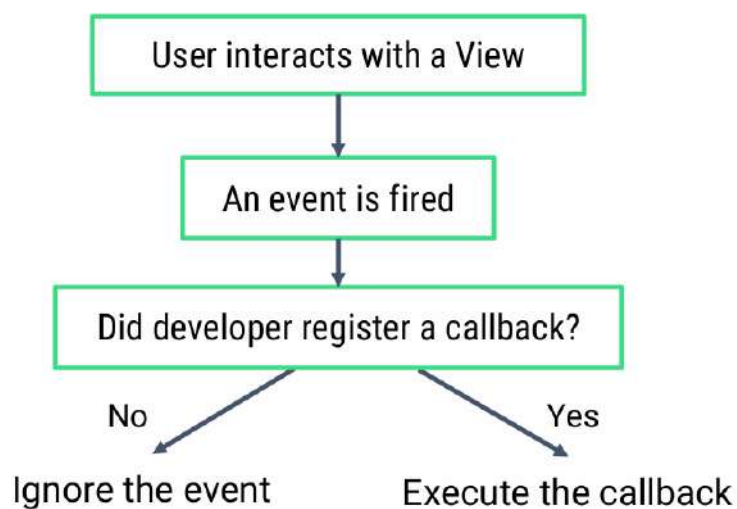
```
resultTextView.text = "Goodbye!"
```

Per fare in modo che il cambio di testo avvenga al click di un pulsante, bisogna impostare un `OnClickListener` sul pulsante stesso. Ciò non è mostrato poiché dobbiamo ancora definire cosa sia un **listener**, ma tipicamente si farebbe qualcosa del genere:

```
val button: Button = findViewById(R.id.button)

button.setOnClickListener {
    resultTextView.text = "Goodbye!"
}
```

Ci sono vari modi per gestire i listeners, ma tutti condividono la stessa logica di base:



Mostriamo un altro esempio di MainActivity:

```
class MainActivity : AppCompatActivity(), View.OnClickListener {

    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        val button: Button = findViewById(R.id.button)
        button.setOnClickListener(this)
    }

    override fun onClick(v: View?) {
        TODO("not implemented")
    }
}
```

In questo caso la MainActivity implementa l'interfaccia `OnClickListener` e sovrascrive (override) il metodo `onClick()`. Quando l'activity viene creata viene settato anche il click listener sull'oggetto Button. Un aspetto interessante da notare è che il metodo `onClick()` prende in input un parametro View, permettendoci di usare lo stesso metodo con diverse views.

Un altro modo di definire un `OnClickListener` è creare una classe che implementa l'interfaccia. In Kotlin, le interfacce che hanno un unico metodo astratto (Single Abstract Method, SAM) possono essere implementate tramite espressioni lambda, semplificando notevolmente il codice. Questo è particolarmente utile per interfacce come `View.OnClickListener`, che ha appunto un solo metodo astratto: `onClick()`.

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        ...  
        val button: Button = findViewById(R.id.button)  
        button.setOnClickListener({ view -> /* do something*/ })  
    }  
}
```

3.4. Gradle

Gradle è un toolkit di compilazione automatica che consente di configurare e gestire la creazione dei progetti.

Gradle permette di includere **plugin** specifici, ovvero programmi non autonomi che interagiscono con un altro programma per estendere la sua funzionalità originale:

```
apply plugin: 'kotlin-android'  
  
apply plugin: 'kotlin-android-extensions'
```

Il primo plugin consente di utilizzare Kotlin per lo sviluppo di applicazioni Android, integrando il supporto per Kotlin nel processo di build. Il secondo plugin aggiunge supporto per funzionalità Kotlin utili nello sviluppo di app Android, come la possibilità di accedere direttamente agli ID delle view nel codice Kotlin, semplificando l'interazione con l'interfaccia utente senza dover utilizzare `findViewById`.

Gradle permette di definire **proprietà Android**:

```
android {  
    compileSdkVersion 30  
    buildToolsVersion "30.0.2"  
  
    defaultConfig {  
        applicationId "com.example.sample"  
        minSdkVersion 19  
        targetSdkVersion 30  
    }  
}
```

Nel blocco di configurazione di Android possiamo settare tutte le specifiche della nostra app.

Gradle gestisce anche le **dependenze del progetto**, che sono librerie esterne richieste dall'app. Questo include:

- Aggiungere dipendenze ai moduli dell'app specificando le librerie nel blocco dependencies.
- Risoluzione automatica delle dipendenze da repository remoti come Maven Central o JCenter, assicurando che la versione corretta delle librerie sia sempre utilizzata e mantenuta aggiornata.

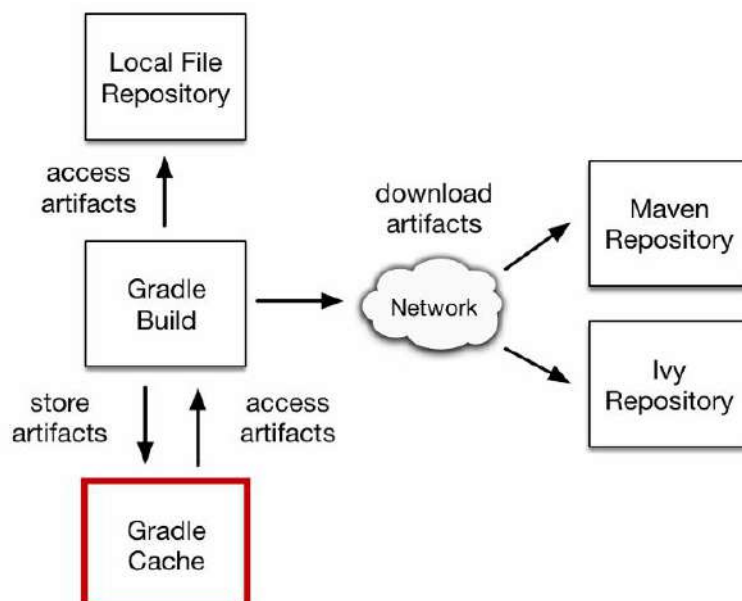
Le dipendenze sono rappresentate da moduli e dobbiamo comunicare al Gradle dove trovarle:

```
dependencies {  
    // Dependency on a local library module  
    implementation(project(":mylibrary"))  
  
    // Dependency on local binaries  
    implementation(fileTree(mapOf("dir" to "libs", "include" to listOf("*.jar"))))  
  
    // Dependency on a remote binary  
    implementation("com.example.android:app-magic:1.2.3")  
}
```

Il luogo dove i moduli sono salvati è chiamato **repository**, e può essere locale o remota.

```
repositories {  
    google()  
    jcenter()  
    maven {  
        url "https://maven.example.com"  
    }  
}
```

Il meccanismo di risoluzione salva i file sottostanti di una dipendenza in una cache locale (Gradle Cache) per costruzioni (build) future.



Un esempio di file Gradle potrebbe essere il seguente:

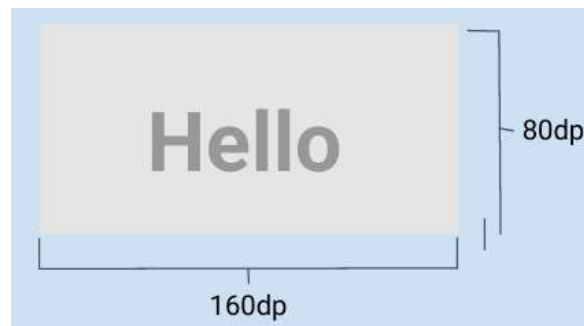
```
plugins {  
    id("com.android.application") version "7.3.0"  
}  
  
repositories {  
    google()  
    mavenCentral()  
}  
  
android {  
    compileSdkVersion(30)  
    defaultConfig {  
        applicationId = "org.gradle.samples"  
        minSdkVersion(16)  
        targetSdkVersion(30)  
        versionCode = 1  
        versionName = "1.0"  
    }  
}  
  
dependencies {  
    implementation("androidx.appcompat:appcompat:1.2.0")  
    implementation("com.google.android.material:material:1.2.0")  
    implementation("androidx.constraintlayout:constraintlayout:2.0.4")  
    testImplementation("junit:junit:4.13.1")  
    androidTestImplementation("androidx.test.ext:junit:1.1.2")  
    androidTestImplementation("androidx.test.espresso:espresso-core:3.3.0")  
}
```

4. Layout e listeners

4.1. Dispositivi android

Come ben sappiamo, esistono vari tipi di dispositivi android, tutti con le loro dimensioni e svariate forme. Proprio per questo motivo, gli sviluppatori necessitano di specificare dimensioni di layout che rimangano consistenti tra un dispositivo e l'altro. Per misurare le dimensioni di un layout bisogna evitare categoricamente il pixel (px), poiché diversi dispositivi potrebbero avere densità di pixel differenti, quindi lo stesso numero di pixel può corrispondere a differenti misure fisiche in dispositivi diversi. Per evitare questo problema si ricorre ai **density-independent pixels** (dp). Un dp è un'unità di pixel virtuale che equivale all'incirca a un pixel su uno schermo a media densità (ovvero 160 dpi, anche detta **densità di base**). Ciò che fa Android è tradurre questo valore virtuale nel corrispondente valore reale in pixel per ogni altra densità. Contrariamente ai pixels, i dp tengono anche conto della **densità dello schermo** (pixels per pollice), ecco perché usarli assicura che il design e i layout da noi progettati funzionino in dispositivi differenti.

Le view di android sono anch'esse misurate in dp:



Un dp è dato dalla seguente formula:

$$dp = \frac{(larghezza\ in\ pixels \cdot 160)}{densità\ dello\ schermo}$$

Gli schermi sono generalmente organizzati in diversi “secchi di densità” (density buckets):

Density qualifier	Description	DPI estimate
ldpi (mostly unused)	Low density	~120dpi
mdpi (baseline density)	Medium density	~160dpi
hdpi	High density	~240dpi
xhdpi	Extra-high density	~320dpi
xxhdpi	Extra-extra-high density	~480dpi
xxxhdpi	Extra-extra-extra-high density	~640dpi

Nonostante l'uso dei dp garantisca una certa sicurezza nel funzionamento dei layout da un dispositivo a un altro, bisognerebbe sempre testare personalmente l'app con dispositivi aventi differenti densità di schermo (e possiamo farlo grazie all'emulatore di Android studio).

Per quanto riguarda la dimensione dei testi, invece, è possibile ricorrere agli **scalable pixels** (sp). Un sp di default ha la stessa dimensione di un dp, ma si ridimensiona in base alla grandezza preferita dall'utente. È bene tenere a mente che gli sp non devono essere utilizzati come unità di misura per i layout.

Per esempio, possiamo utilizzare i dp per definire una distanza tra due view:

```
<Button android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/clickme"
        android:layout_marginTop="20dp" />
```

Possiamo invece utilizzare gli sp per specificare la grandezza del testo da noi desiderata:

```
<TextView android:layout_width="match_parent"
          android:layout_height="wrap_content"
          android:textSize="20sp" />
```

Quando Android disegna le informazioni sullo schermo, avvengono tre passaggi distinti in rapida successione:

1. **Measure (Misura):** in questa fase, Android calcola le dimensioni precise di ogni View. Qui si determinano larghezza e altezza di ogni componente dell'interfaccia utente, in base ai vincoli definiti nel layout e alle dimensioni dello schermo del dispositivo.
2. **Layout (Imposta Layout):** Dopo aver misurato le dimensioni, ogni View viene posizionata all'interno del layout. Questo passaggio decide la posizione esatta delle view sulla schermata, basandosi sulle regole stabilite dal layout manager (ad esempio, LinearLayout o RelativeLayout). Le posizioni sono calcolate rispetto al contenitore genitore e alle view adiacenti.
3. **Draw (Disegna):** Nell'ultima fase, Android effettivamente disegna le view sullo schermo, rendendo visibili i risultati delle prime due fasi. Questo comprende non solo le view stesse, ma anche eventuali effetti visivi come ombre o animazioni.

È importante notare come la forma di un widget non abbia alcuna importanza per Android quando si tratta disegnarla sullo schermo, poiché il sistema di disegno gestisce ogni elemento come se fosse confinato entro un rettangolo. Questo "rettangolo di bounding" rappresenta l'area massima entro cui la View può disegnare, e garantisce che il rendering sia efficiente e che le interazioni con l'utente (come il tocco) siano gestite correttamente:

What we see:



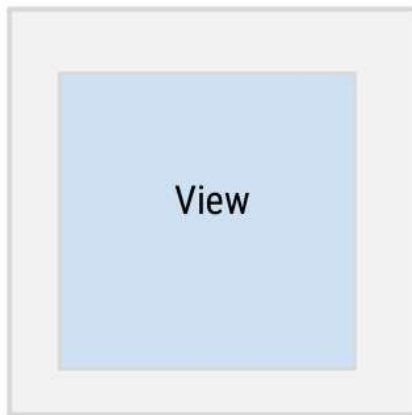
How it's drawn:



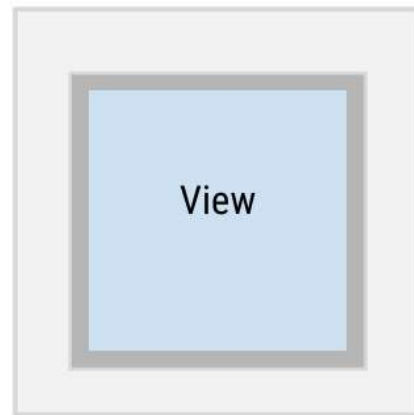
A questo punto è essenziale definire due concetti:

- **padding**: si riferisce allo spazio vuoto tra il contenuto di una View e i suoi bordi interni.
- **margin**: si riferisce allo spazio vuoto esterno tra la View e gli altri componenti nell'interfaccia utente.

View with margin

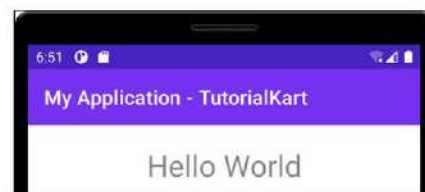
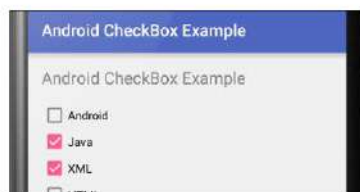
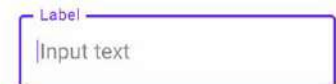
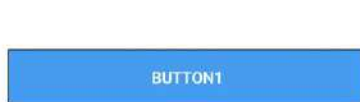


View with margin and padding



4.2. Views

Il layout descrive ciò che è contenuto in una schermata, ovvero i diversi componenti grafici. Classici esempi di componenti sono:



Ogni componente grafico è una sottoclasse della classe View di Android e, per questo motivo, ci si riferisce ai componenti con l'appellativo View.

L'interfaccia utente può essere definita in due modi:

- **Approccio dichiarativo**: definisce ciò che deve essere realizzato dal programma senza dire come deve essere implementato.
- **Approccio imperativo**: fornisce una descrizione dell'interfaccia utente tramite linee di codice.

```

class ImperativeActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // We define a LinearLayout
        val parentLayout = LinearLayout(this).apply {
            layoutParams = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            )
            orientation = LinearLayout.VERTICAL
        }

        // We define 3 Button
        3.forEach { index, -> Button(this@ImperativeActivity).apply {
            layoutParams = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            )
            text = "Button #$index"
            parentLayout.addView(this)
        }
        // We set the content view
        setContentView(parentLayout)
    }
}

```

Imperative

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <Button android:text="@string/button_0"
        android:id="@+id/button0"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
    <Button android:text="@string/button_1"
        android:id="@+id/button1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
    <Button android:text="@string/button_2"
        android:id="@+id/button2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
</LinearLayout>

```

Declarative



Tutte le Views in Android hanno la stessa struttura di base:

```

<ViewName
    Attribute1=Value1
    Attribute2=Value2
    Attribute3=Value3
    ...
    AttributeN=ValueN
/>

```

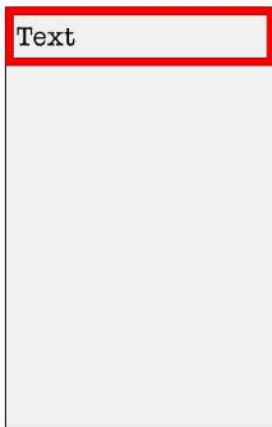
1. Iniziano con un carattere di apertura “<” seguito dal nome della View.
2. Contengono una serie di attributi che definiranno come la View apparirà sullo schermo.
3. Terminano con un carattere di chiusura “/>”.

Gli attributi più usati sono quelli che riguardano l’**allineamento di una View**, il che è un concetto importante ma allo stesso tempo noioso.

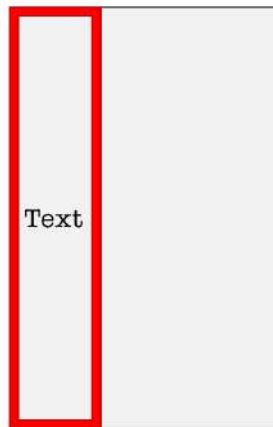
Ogni layout ha un insieme di attributi propri che definiscono le sue proprietà virtuali, tuttavia alcuni attributi sono comuni a tutti i layout, ad esempio:

- **wrap_content** → android:layout_width=”wrap_content”
- **match_parent** → android:layout_width=”match_parent”
- **fixed value** (usando i dp) → android:layout_width=”48dp”

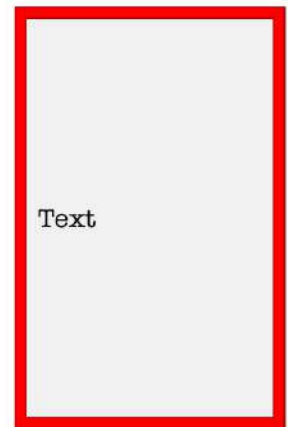
android:layout_width = "match_parent"
android:layout_height = "wrap_content"



android:layout_width = "wrap_content"
android:layout_height = "match_parent"



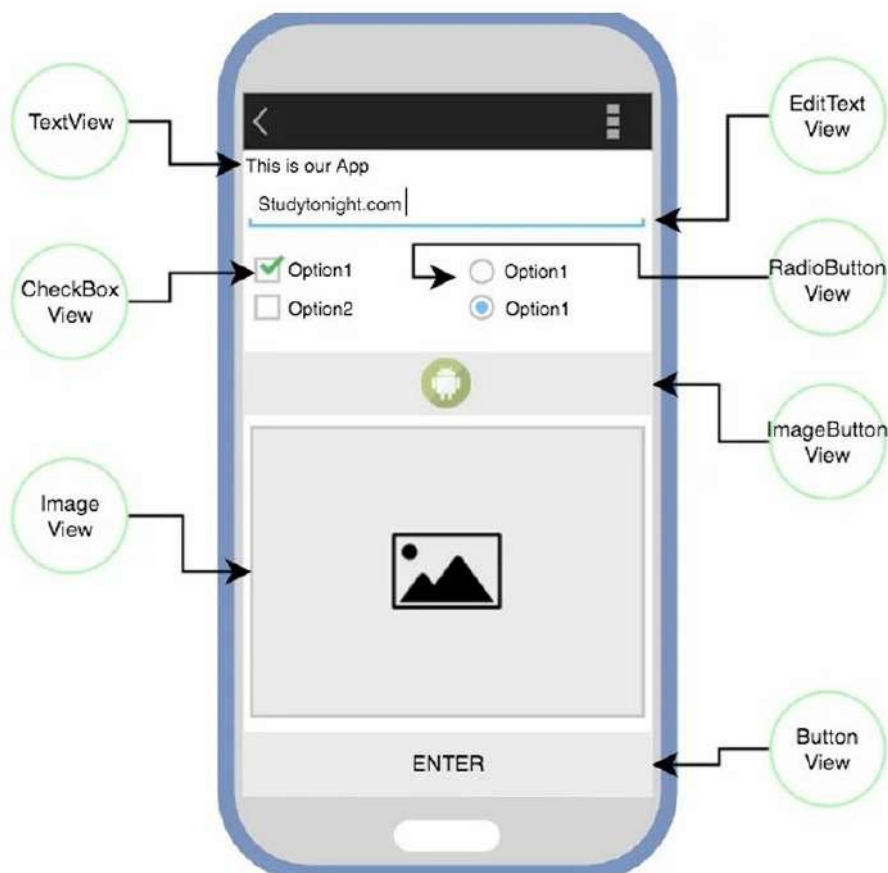
android:layout_width = "match_parent "
android:layout_height = "match_parent"



Altri attributi comuni riguardano i precedenti concetti di padding e margin:

padding	margin
android:paddingBottom	android:layout_marginBottom
android:paddingLeft	android:layout_marginLeft
android:paddingRight	android:layout_marginRight
android:paddingTop	android:layout_marginTop
android:padding	android:margin

Passiamo adesso alla rassegna delle View più comuni:



La **TextView** è la View più comunemente usata per visualizzare testo predefinito sullo schermo.

```
<TextView
    android:id="@+id/st»
    android:layout_width="wrap_content»
    android:layout_height="wrap_content»
    android:text="This is our App"/>
```

Alcuni dei suoi attributi più usati sono:

Attribute	Description
android:text	Used to specify the text to be displayed in the TextView
android:textSize	Using this attribute we can control the size of the text.
android:textColor	Using this attribute we can specify the color of our text.
android:textAllCaps	If set True, this will make the text appear in upper case.
android:letterSpacing	Using this attribute we can set the spacing between letters of the text.
android:hint	This attribute is used to show a default text, if no text is set in the TextView. Generally, when we populate a TextView using dynamic data coming from server(using the programmatic approach), then we set this attribute to show some default text in the TextView until data is fetched from server.

La **PlainTextView** è una particolare TextView che rende il testo modificabile.

```
<EditText
    android:id="@+id/et_address"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Write your email Address here"/>
```

Alcuni dei suoi attributi più usati sono:

Attribute	Description
android:inputType	Used to specify what the text entered should be like and for what purpose it will be used. If this is set to none, then the text cannot be edited: •text •textAutoComplete - Suggestions while user is typing in text. •textAutoCorrect - This will enable auto correct on user input text. •textPassword - Display the entered text in form of dots or stars. •phone - This will present only the numeric keyboard to users. •datetime •etc.
android:minLines	It provides the view, with a height equivalent to the specified number of lines on the screen
android:maxLines	It sets the maximum number of lines that the EditText view can accomodate, visually .
android:hint	It displays a hint message before anyone types in the EditText.
android:maxLength	It allows to specify the maximum number of characters that the user can enter into the field.

La **CheckBoxView** è un controllo grafico attraverso cui l'utente può effettuare selezioni multiple.

```
<CheckBox
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text=" I really enjoy this lesson."
    android:id="@+id/cb1"
    android:layout_centerHorizontal="true"
    android:checked="false"/>
```

android:checked="false/true" è l'attributo che indica se il quadratino è stato selezionato o meno.

La **RadioButtonView** è un controllo grafico attraverso cui l'utente può indicare una sola selezione.

```
<RadioButton
    android:id="@+id/radio_pirates»
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/pirates"
/>
```


I **RadioButton** vengono utilizzati all'interno di un **RadioGroup**. In un **RadioGroup**, la selezione di un'opzione tra le diverse proposte comporta la deselezione automatica di tutte le altre. Quindi, la differenza tra i **RadioButton** e le **CheckBox** è che, nel primo caso, è permessa un'unica scelta, mentre nel secondo caso l'utente può selezionare un numero qualsiasi di scelte (zero, una o più). Esistono casi in cui una **CheckBox** consente all'utente di abilitare o disabilitare una certa scelta da lui selezionata.

La **ImageView** permette di posizionare un'immagine nella View.

```
<ImageButton
    android:id="@+id/imgButton"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:scaleType="fitCenter"
    android:src="@drawable/img_nature"/>
```

Alcuni dei suoi attributi più usati sono:

Attribute	Description
android: maxHeight	Used to specify a maximum height for this view.
android: maxWidth	Used to specify a maximum width for this view.
android: src	Sets a drawable as the content for this ImageView.
android: scaleType	Controls how the image should be resized or moved to match the size of the ImageView.
android: tint	Tints the color of the image in the ImageView.
android: background	This property gives a background color to your ImageView. When your image is not able to entirely fill the ImageView, then background color is used to fill the area not covered by the image.

La **ImageButtonView** permette di usare un'immagine come pulsante.

```
<ImageButton
    android:id="@+id/imgButton"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:scaleType="fitCenter"
    android:src="@drawable/img_nature"/>
```

Alcuni dei suoi attributi più usati sono:

Attribute	Description
android: maxHeight	Used to specify a maximum height for this view.
android: maxWidth	Used to specify a maximum width for this view.
android: src	Sets a drawable as the content for this ImageView.
android: scaleType	Controls how the image should be resized or moved to match the size of the ImageView.
android: tint	Tints the color of the image in the ImageView.

La **ButtonView** è un componente che può essere cliccato dall'utente per eseguire una certa azione.

```
<Button
    android:id="@+id/btn_submit"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Submit"
    android:textColor="@android:color/holo_blue_dark"
/>
```

Alcuni dei suoi attributi più usati sono:

Attribute	Description
android: gravity	This can be used to set the position of any View on the app screen. The available value are right , left , center , center_vertical etc.
android: textSize	To set text size inside button
android: background	To set the background color of the button.

Quando si utilizza un Button, è necessario associare ad esso un certo comportamento o task da svolgere. Possiamo assegnare un metodo da noi definito in MainActivity.kt direttamente nel file XML durante la definizione del pulsante utilizzando l'attributo **android:onClick**:

```
<Button
    android:id="@+id/btn_submit"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Submit"
    android:textColor="@android:color/holo_blue_dark"
    android:onClick="study" />
```

↔

```
public void study(View view) {
    //Perform action on click
}
```

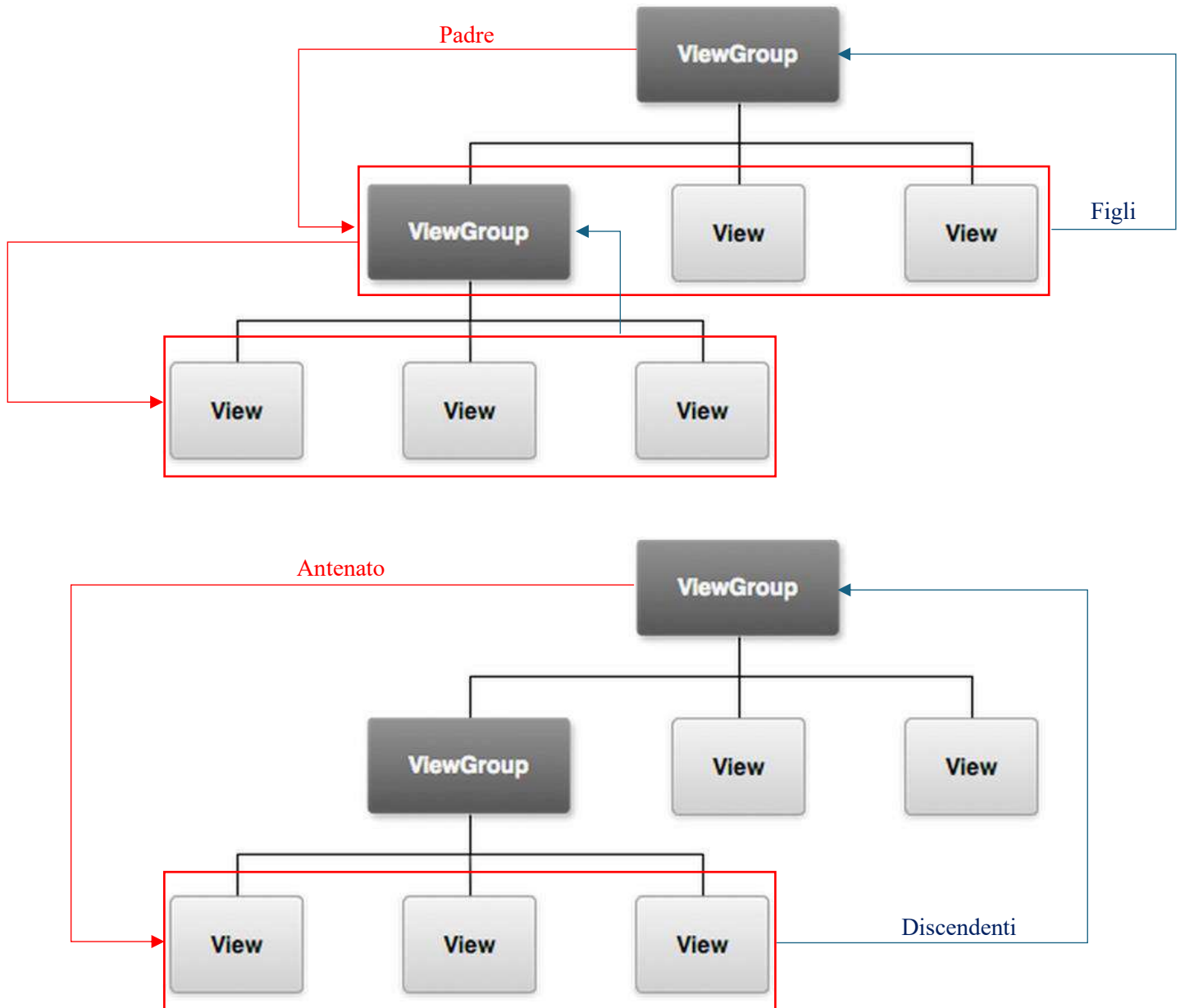
MainActivity.kt

Molte View di android (ad esempio la RadioButtonView) possiedono l'attributo onClick e tutte sono gestite allo stesso modo. Le regole per usarlo sono semplici:

- Il metodo deve essere pubblico.
- Il metodo non deve ritornare nulla.
- Il metodo deve accettare come parametro una View, come ad esempio quella che è stata cliccata.

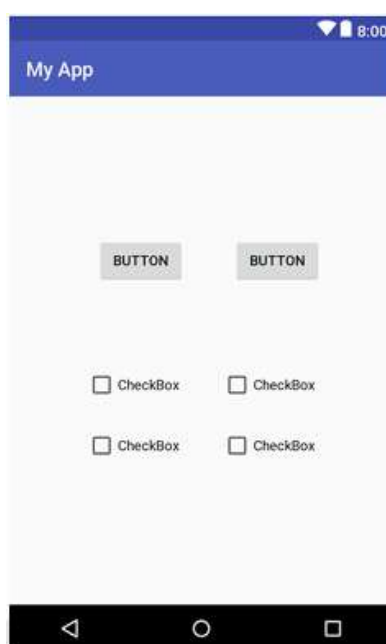
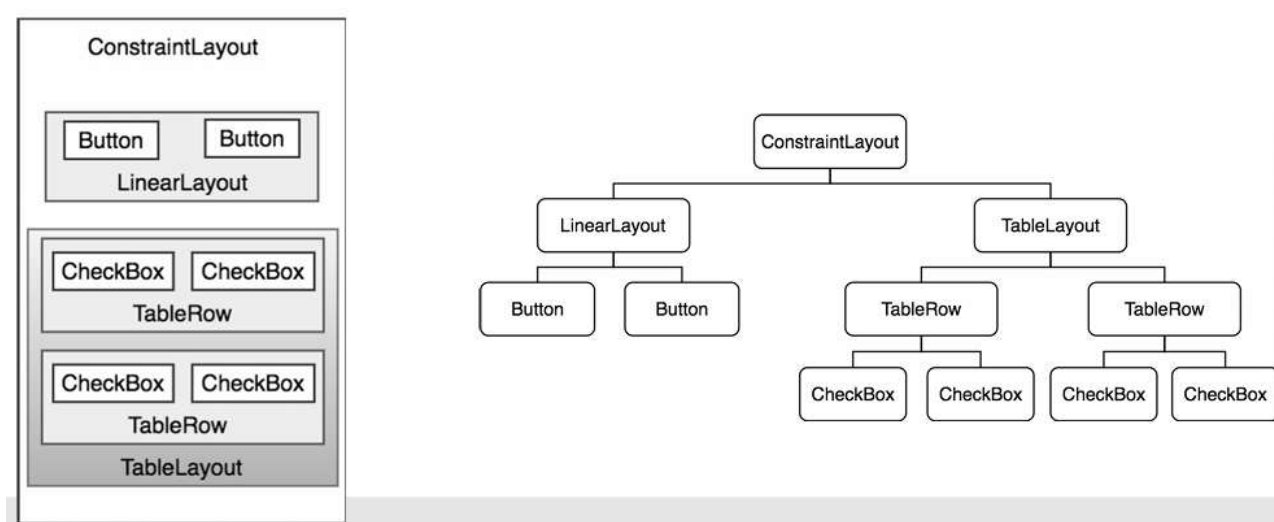
4.3. ViewGroups

Quando una View è composta da altre View viene chiamata **ViewGroup**, anch'essa una sottoclasse di View. Un ViewGroup altro non è che un contenitore che controlla come le Views sono organizzate e disposte sullo schermo.



Ogni View nell'interfaccia utente rappresenta un'area rettangolare dello schermo. Una View è responsabile per ciò che è disegnato in quel rettangolo e della risposta agli eventi che avvengono all'interno di quella parte di schermo (come un evento tocco).

Una schermata dell'interfaccia utente è costituita da una gerarchia di viste con una vista radice posizionata in cima all'albero e le viste figlie posizionate sui rami sottostanti. L'elemento figlio di una vista contenitore appare sopra la sua vista genitore ed è vincolato a comparire entro i limiti dell'area di visualizzazione della vista genitore.



Adesso approfondiremo alcuni dei layout più popolari di Android Studio.

Il **LinearLayout** permette a tutti gli elementi figli di essere allineati in una direzione, verticale o orizzontale. Tale direzione è definita dall'attributo **android:orientation**:

```
<LinearLayout
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical">
  <TextView ... />
  <TextView ... />
  <Button ... />
</LinearLayout>
```

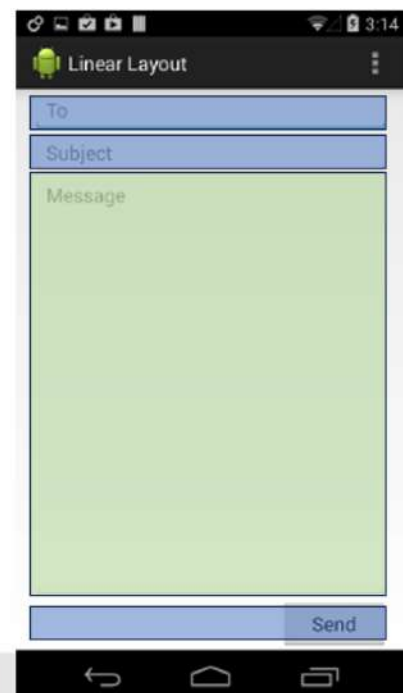


Il **LinearLayout** ci permette di assegnare un peso a ogni elemento figlio con l'attributo **android:layout_weight**, ovvero di assegnare un valore di “importanza” alla View in termini di quanto spazio dovrebbe occupare sullo schermo. Ci sono due tipi possibili di distribuzioni:

- **Uguale (equal)**: ogni elemento figlio ha la stessa importanza. Per ogni elemento figlio si settano:
 - **android:layout_height** = “0dp” per un layout verticale oppure **android:layout_width** = “0dp” per un layout orizzontale.
 - **android:layout_weight** = “1”
- **Ineguale (unequal)**: ogni elemento figlio ha la propria importanza.

Vediamo un esempio di pesi ineguali tra elementi figli:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:paddingLeft="16dp"
  android:paddingRight="16dp"
  android:orientation="vertical" >
  <EditText
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="@string/to" />
  <EditText
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="@string/subject" />
  <EditText
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:gravity="top"
    android:hint="@string/message" />
  <Button
    android:layout_width="100dp"
    android:layout_height="wrap_content"
    android:layout_gravity="right"
    android:text="@string/send" />
</LinearLayout>
```

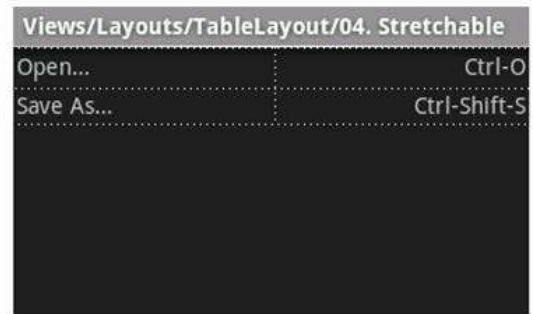


In questo esempio abbiamo tre **EditText** e un **Button** contenuti in un **LinearLayout**. L'attributo **weight** è settato esclusivamente nel terzo **EditText**, così si finisce per dare maggiore peso a questo specifico **EditText**.

Il **TableLayout** organizza i propri elementi figli in righe e colonne a formare celle. Ogni riga ha zero o più celle e ogni cella può contenere una View. Il numero di colonne è determinato dalle righe con il maggior numero di celle. Il TableLayout è composto da un certo numero di **TableRow**, ognuna delle quali definisce una riga.

```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:stretchColumns="1">
    <TableRow>
        <TextView
            android:text="@string/table_layout_4_open"
            android:padding="3dip" />
        <TextView
            android:text="@string/table_layout_4_open_shortcut"
            android:gravity="right"
            android:padding="3dip" />
    </TableRow>

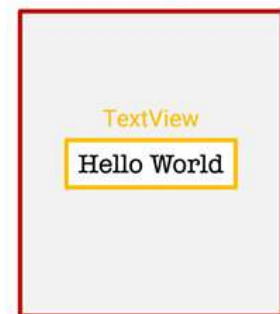
    <TableRow>
        <TextView
            android:text="@string/table_layout_4_save"
            android:padding="3dip" />
        <TextView
            android:text="@string/table_layout_4_save_shortcut"
            android:gravity="right"
            android:padding="3dip" />
    </TableRow>
</TableLayout>
```



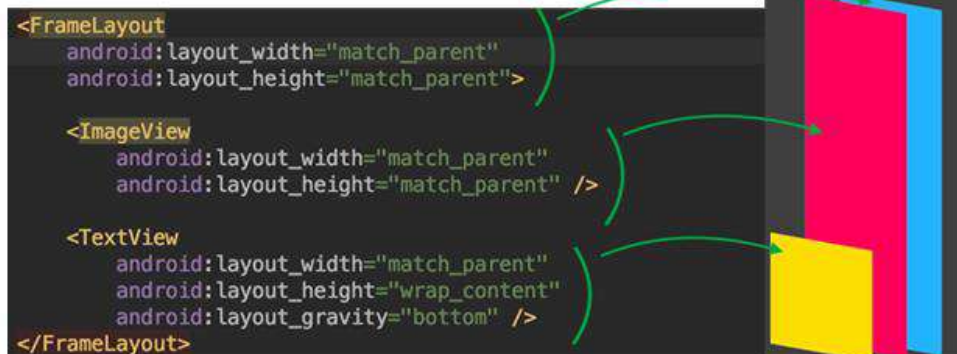
Il **FrameLayout** è progettato per bloccare un'area dello schermo e dovrebbe contenere una singola View perché è difficile organizzare più elementi figli senza che si sovrappongano. Nel caso si scelga di utilizzare comunque più Views, queste saranno organizzate secondo una strategia LIFO (Last-In-First-Out), ovvero l'elemento figlio aggiunto più recentemente sarà quello in cima:

```
<FrameLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:text="Hello World!"/>
</FrameLayout>
```

FrameLayout



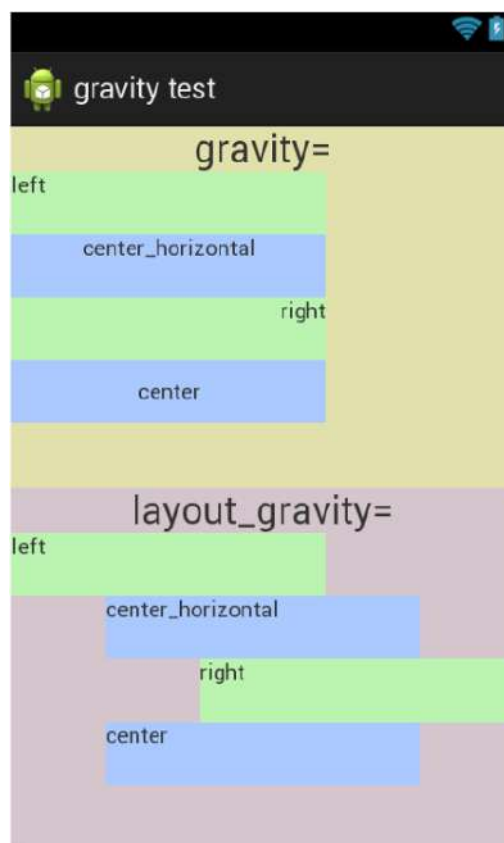
Single View



Multiple Views

L'attributo **layout_gravity** può essere usato per forzare la posizione di un elemento rispetto agli altri. Attenzione però, anche se spesso confusi gli attributi **gravity** e **layout_gravity** sono diversi:

- **gravity** imposta la posizione di un oggetto all'interno di un contenitore potenzialmente più grande. Quindi **wrap_content** e **gravity** insieme non hanno senso. Questo attributo viene spesso utilizzato nei **LinearLayout**.
- **layout_gravity** imposta la gravità della vista, o del layout, rispetto al suo genitore. La larghezza (o l'altezza) della vista deve essere inferiore a quella del genitore. Quindi, **match_parent** e **layout_gravity** insieme non hanno senso. Questo attributo viene spesso utilizzato nei **FrameLayout** e **LinearLayout**.



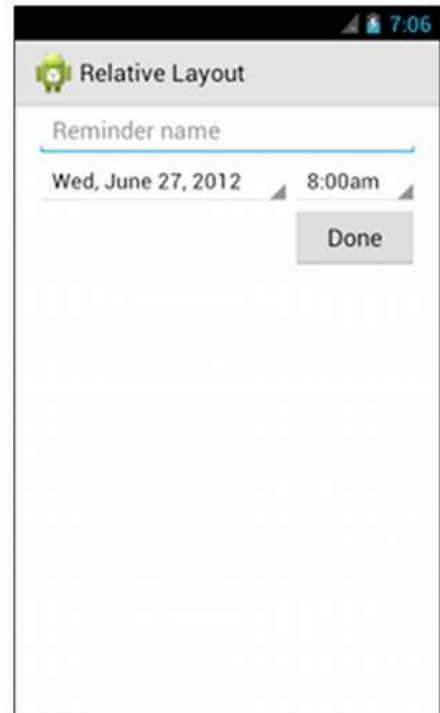
ViewGroup profondamente nidificate richiedono maggiori calcoli, in quanto le **Views** possono essere misurate più volte e ciò può causare un rallentamento dell'interfaccia utente e una mancanza di reattività. Una soluzione iniziale a questi problemi potrebbe essere l'uso di un **RelativeLayout** che permette di posizionare le **Views** in relazione ad elementi dello stesso livello o all'area principale (parent) del layout. Di default tutte le **Views** figlie sono disegnate nell'angolo in alto a sinistra del layout, quindi è necessario definire la posizione di ogni **View** usando le varie proprietà di layout:

- **android:layout_alignParentTop**: Se impostato su "true", allinea il bordo superiore di questa **View** al bordo superiore del parent.
- **android:layout_centerVertical**: Se impostato su "true", centra verticalmente questa **View** all'interno del parent.

- **android:layout_below**: Posiziona il bordo superiore di questa View sotto la View specificata con un ID di risorsa.
- **android:layout_toRightOf**: Posiziona il bordo sinistro di questa View a destra della View specificata con un ID di risorsa.

Queste sono solo alcune delle proprietà che possono essere usate.

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="16dp"
    android:paddingRight="16dp" >
    <EditText
        android:id="@+id/name"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="@string/reminder" />
    <Spinner
        android:id="@+id/dates"
        android:layout_width="Odp"
        android:layout_height="wrap_content"
        android:layout_below="@id/name"
        android:layout_alignParentLeft="true"
        android:layout_toLeftOf="@+id/times" />
    <Spinner
        android:id="@id/times"
        android:layout_width="96dp"
        android:layout_height="wrap_content"
        android:layout_below="@id/name"
        android:layout_alignParentRight="true" />
    <Button
        android:layout_width="96dp"
        android:layout_height="wrap_content"
        android:layout_below="@id/times"
        android:layout_alignParentRight="true"
        android:text="@string/done" />
</RelativeLayout>
```

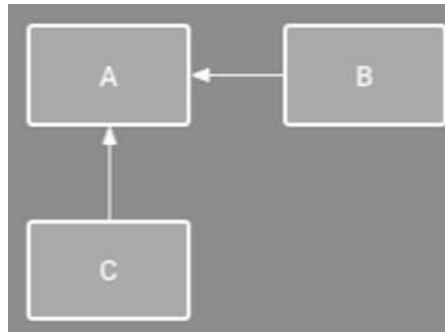


Per layout semplici il RelativeLayout è consigliabile e facile da utilizzare, infatti dovremo solo definire le relazioni tra gli elementi e il layout si occuperà del resto. Il RelativeLayout può risultare inutilmente complesso nel caso di design più intricati, e ciò può rendere i nostri file XML più difficili da leggere e mantenere.

Oggi il layout di default consigliato da Android è il **ConstraintLayout**, il quale duplica tutte le funzionalità del LinearLayout e del RelativeLayout permettendo allo sviluppatore di modellare comportamenti complessi del layout in modo più semplice, con meno gerarchia e annidamento.

4.4. ConstraintLayout

Il ConstraintLayout utilizza i cosiddetti **constraints** per ridurre la nidificazione, e ciò implica il poter misurare, impaginare e disegnare le Views in meno passaggi, aumentando così la velocità di visualizzazione dell'interfaccia utente. I constraints non sono altro che una restrizione o limitazione delle proprietà di una View che il layout tenta di rispettare. Il seguente esempio mostra come l'elemento B è vincolato ad essere sempre alla destra di A, mentre C è vincolato ad essere sempre sotto A:



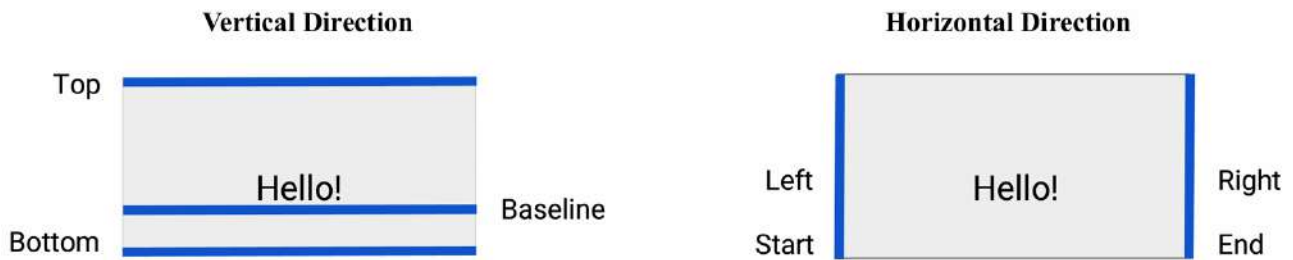
Alcuni constraints possono essere relativi al contenitore padre, dove per contenitore padre si intende il contenitore che racchiude gli elementi (ad esempio il ConstraintLayout stesso). Molti constraints si presentano in questa forma:

```
layout_constraint<SourceConstraint>_to<TargetConstraint>Of
```

Vediamo un esempio per capire quali sono le possibili posizioni da sostituire in SourceConstraint e TargetConstraint:

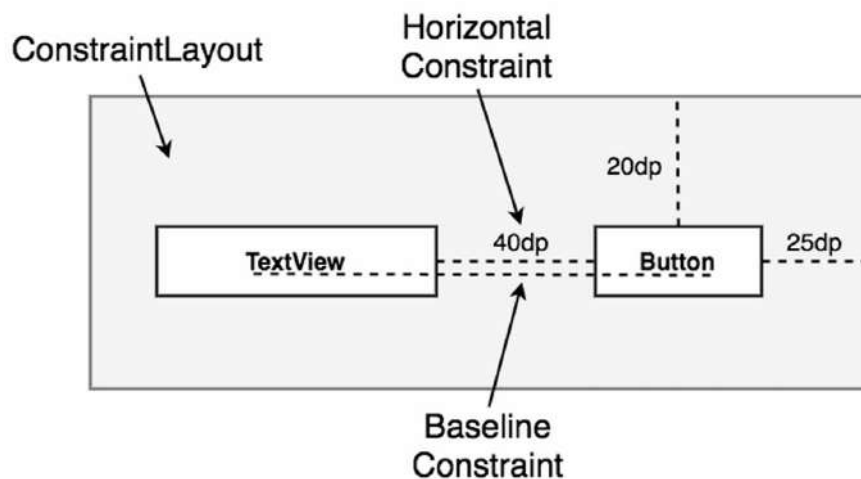


- **app:layout_constraintTop_toTopOf = "parent"** per dire “voglio allineare la parte superiore di questa TextView con la parte superiore del layout padre”.
- **app:layout_constraintLeft_toLeftOf = "parent"** per dire “voglio allineare la parte sinistra di questa TextView con la parte sinistra del layout padre”.
- In questo esempio ci sono anche due margini superiore e sinistro di 16 dp.



Per quanto riguarda la direzione orizzontale, sinistra e destra sono incluse per completezza, ma come buona abitudine bisognerebbe utilizzare di default **start** e **end**.

La **baseline** è una linea immaginaria sulla quale si appoggiano i caratteri di un testo. È la linea di riferimento che si usa per allineare il testo in una View, come quella mostrata nel seguente esempio:



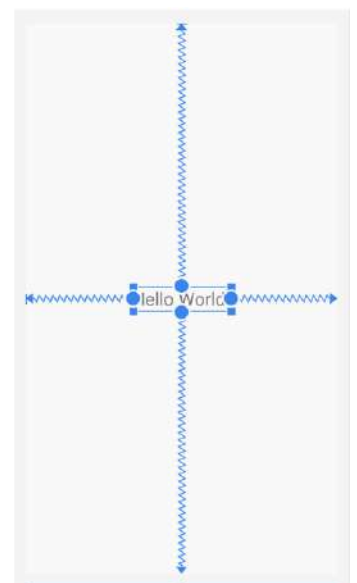
Osserviamo un semplicissimo esempio di ConstraintLayout:

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        ...

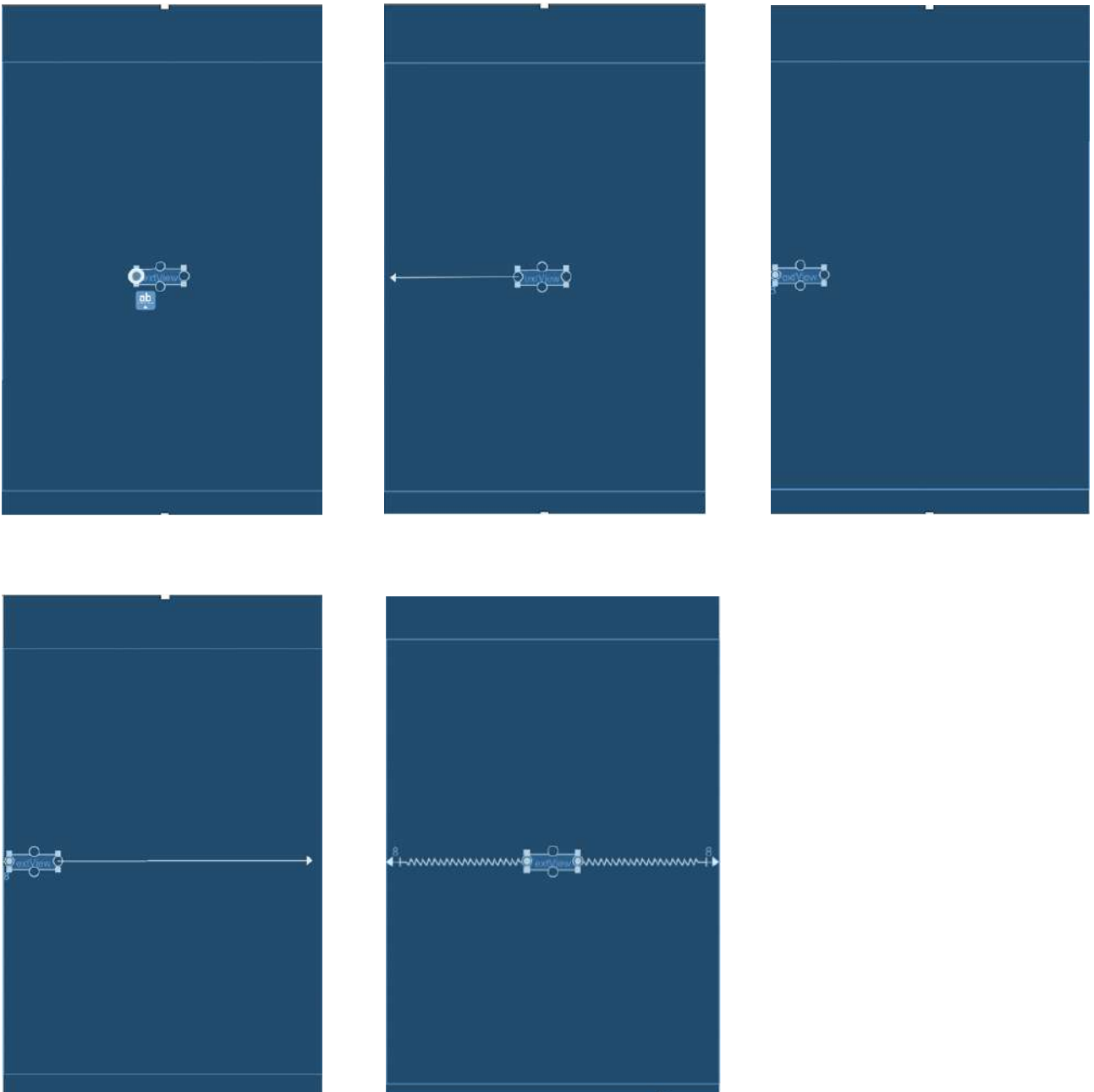
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

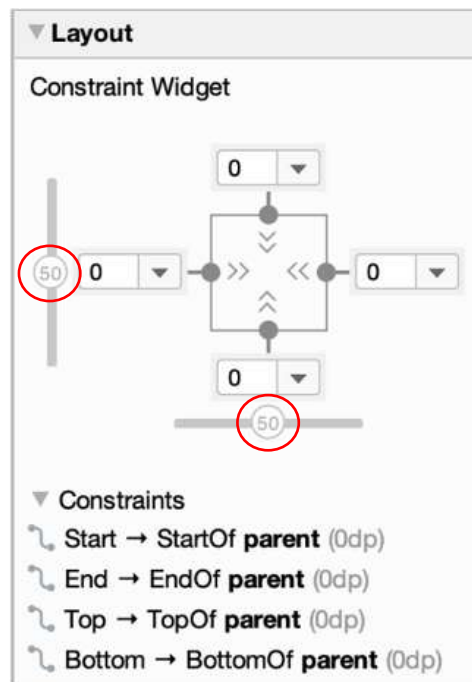


Un constraint può essere creato tramite codice XML, come mostrato nell'esempio precedente, oppure usando il layout editor di Android Studio. Nel layout editor ogni View in un ConstraintLayout è mostrata con quattro maniglie circolari, utili per vincolare rapidamente tale View all'inizio, alla fine, alla parte superiore o inferiore di altre View.

Possiamo trascinare la maniglia corrispondente per creare un vincolo all'inizio (start) del padre (parent) e, in seguito, trascinare la maniglia opposta per creare un vincolo alla fine (end) del padre.



Di default, questi due vincoli agiscono in modo uguale sull'oggetto (50% su ciascun lato), in modo tale che l'oggetto venga centrato. È possibile regolare la centratura trascinando il cursore apposito nel **Constraint Widget** della finestra degli attributi per spostare la View su uno dei due lati.



Nel Constraint Widget potremo osservare tre tipi di simboli, che indicano il tipo di constraint applicato alla parte iniziale, finale, superiore o inferiore della View corrente:

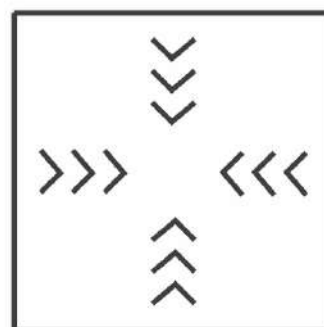
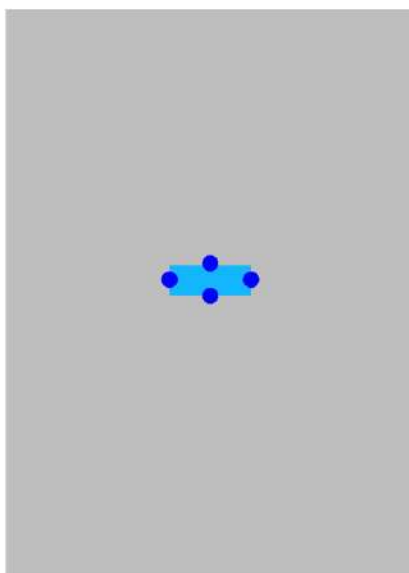
 Fixed

 Wrap content

 Match constraints

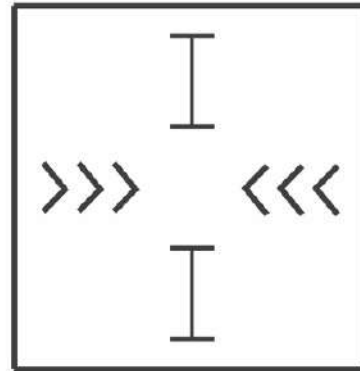
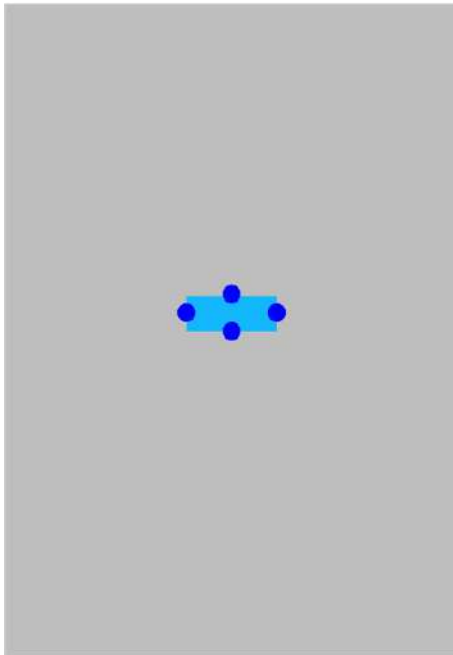
Vediamo una serie di esempi per comprendere al meglio quanto detto fino ad ora:

1. **wrap_content** per larghezza e altezza:



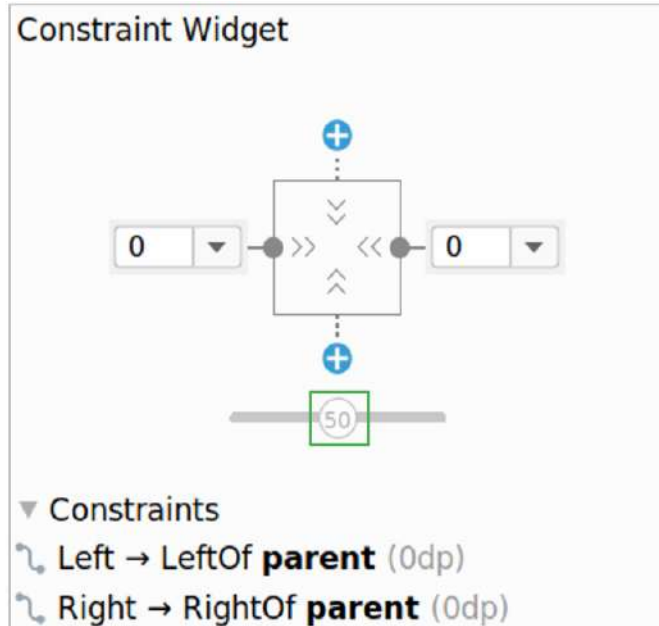
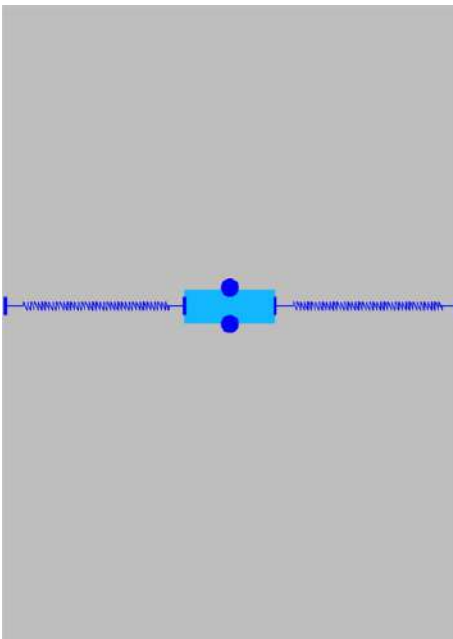
layout_width wrap_content
layout_height wrap_content

2. `wrap_content` per larghezza, altezza fixed:

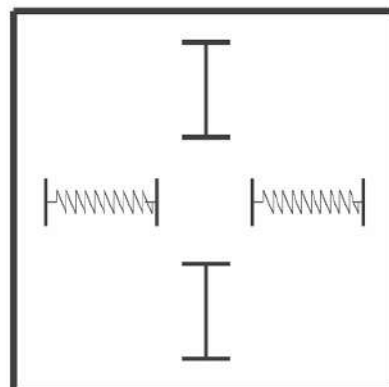
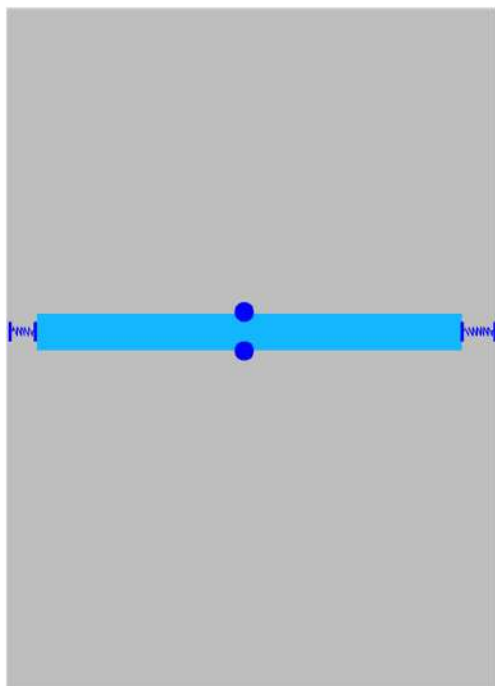


`layout_width` `wrap_content`
`layout_height` `48dp`

3. Centrare una View orizzontalmente:



4. Non si può usare `match_parent` su una View figlia, bisogna usare invece `match_constraint`:

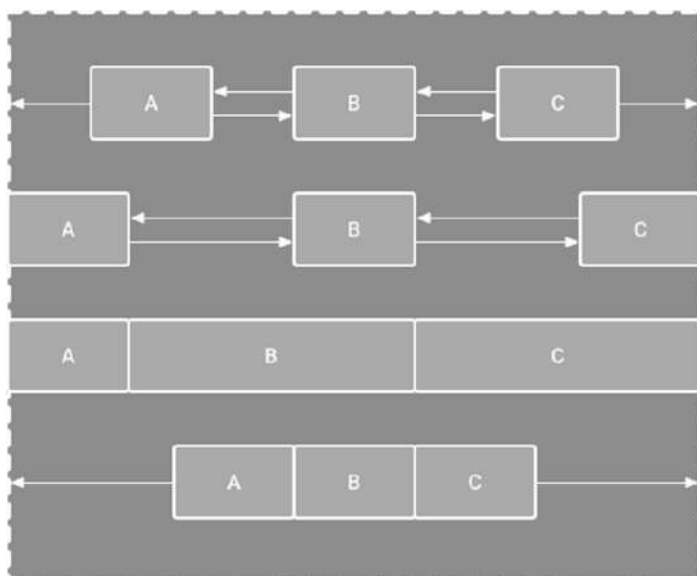


```
layout_width    0dp(match_constraint)
layout_height   48dp
```

Il **Chain Constraint** è un meccanismo utile a definire il comportamento di layout di due o più widget come gruppo. I widget sono concatenati quando ci sono constraint bidirezionali. I Chain Constraint sono dichiarati sull'asse verticale o orizzontale e configurati per definire la spaziatura e la dimensione dei widget nella catena. Questo tipo di constraint fornisce molte delle funzionalità del `LinearLayout`. Per creare una catena, bisogna seguire tre semplici passi nel layout editor:

1. Selezionare gli oggetti che vogliamo nella catena.
2. Cliccare con il mouse destro e selezionare la dicitura "Chains".
3. Creare una catena verticale o orizzontale.

Esistono vari stili di catene:



Spread Chain

Spread Inside Chain

Weighted Chain

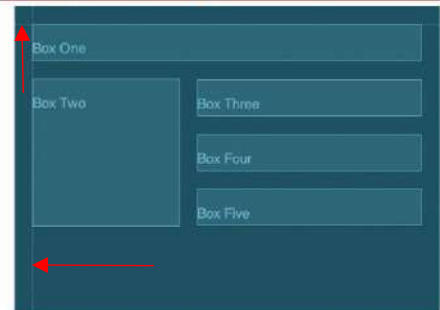
Packed Chain

Un'altra interessante funzionalità dei ConstraintLayout è la **Guideline**. Una Guideline serve a posizionare molteplici Views rispetto a una singola guida e può essere sia verticale sia orizzizontale. Le Guideline servono solo come riferimento allo sviluppatore e non saranno disegnate nel dispositivo una volta avviata l'app.

How guidelines appear in XML

```
<ConstraintLayout>
  <androidx.constraintlayout.widget.Guideline
    android:id="@+id/start_guideline"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    app:layout_constraintGuide_begin="16dp" />
  <TextView ...
    app:layout_constraintStart_toEndOf="@id/start_guideline" />
</ConstraintLayout>
```

How guidelines appear in Android Studio

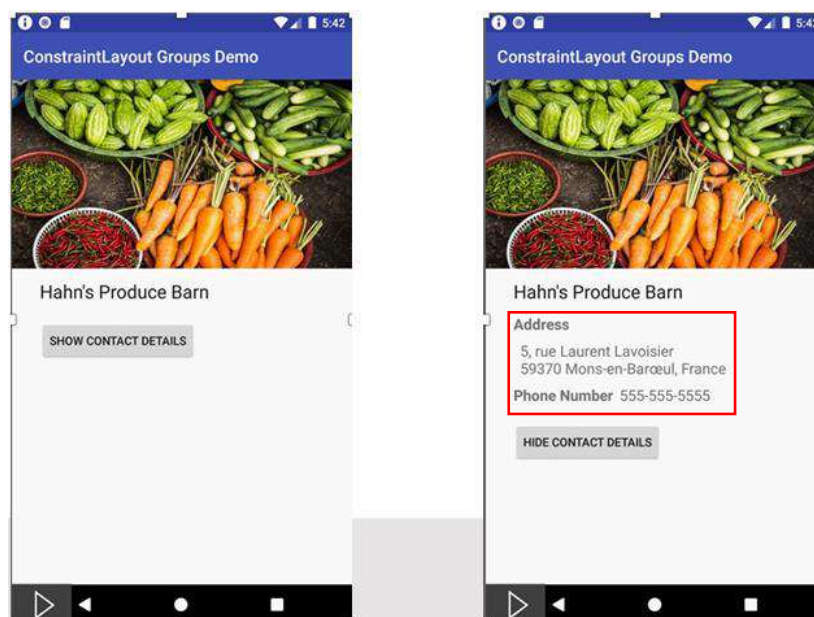


In XML possiamo settare le Guideline inserendo uno specifico numero di dp dall'inizio o dalla fine del layout, oppure inserendo una percentuale numerica tra 0 e 1:

```
<ConstraintLayout>
  <androidx.constraintlayout.widget.Guideline
    android:id="@+id/start_guideline"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    app:layout_constraintGuide_begin="16dp" />
  <TextView ...
    app:layout_constraintStart_toEndOf="@id/start_guideline" />
</ConstraintLayout>
```

layout_constraintGuide_begin
 layout_constraintGuide_end
 layout_constraintGuide_percent

Un'altra funzionalità da attenzionare è **Group**. Group permette di posizionare i widget in gruppi logici e controllare la visibilità di quei widget come una singola entità. Una volta definito un Group, cambiare la visibilità (visibile, non visibile o scomparso) dell'istanza del Group applicherà il cambiamento a tutti i membri di tale Group. Un singolo layout può contenere molteplici Group e un widget può appartenere a più di un Group.



```

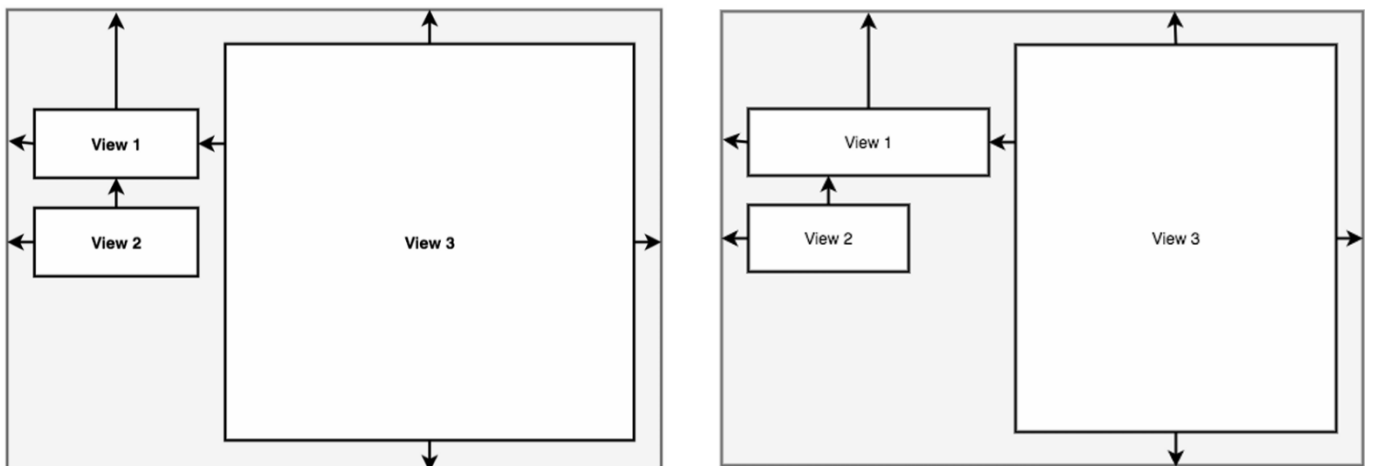
<androidx.constraintlayout.widget.Group
    android:id="@+id/group"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"

    app:constraint_referenced_ids="locationLabel,locationDetails"/>

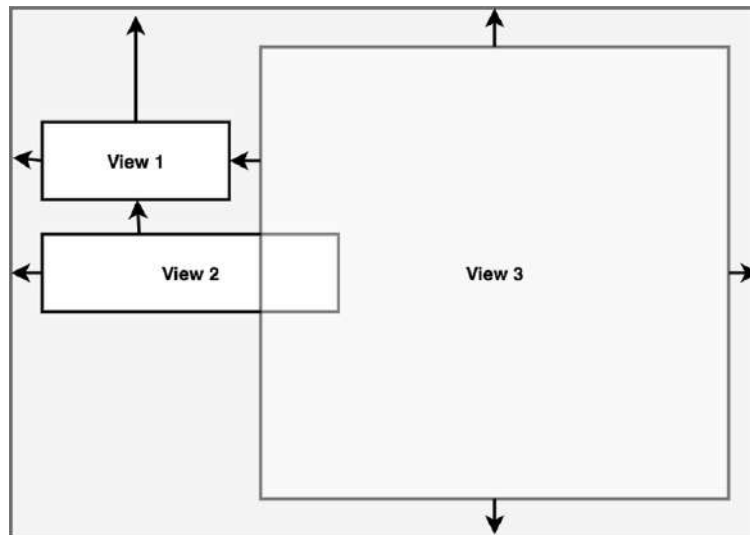
override fun onClick(v: View?) {
    if (group.visibility == View.GONE) {
        group.visibility = View.VISIBLE
        button.setText(R.string.hide_details)
    } else {
        group.visibility = View.GONE
        button.setText(R.string.show_details)
    }
}
}

```

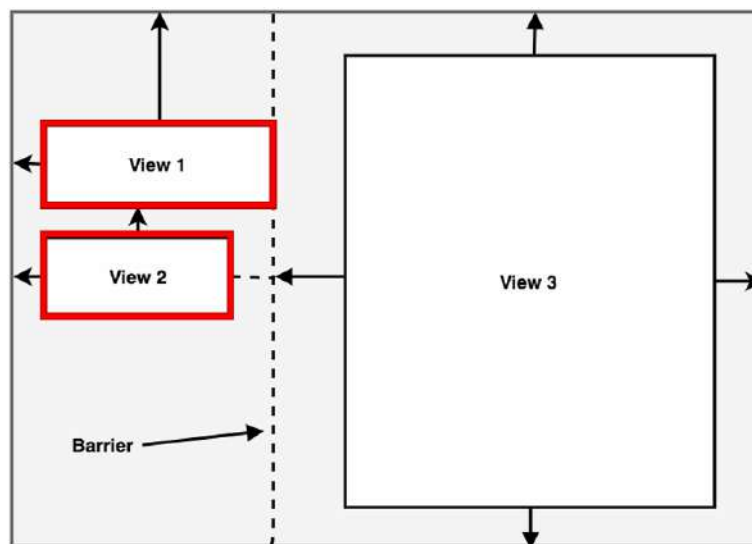
L'ultima funzionalità dei ConstraintLayout di cui parleremo è la **Barrier**. Una Barrier può essere verticale o orizzontale e la sua posizione è definita da un insieme di View di riferimento, quindi può essere variabile. Questa funzionalità è stata introdotta per risolvere un problema abbastanza ricorrente che si riscontra quando si usano View che si sovrappongono. Vediamo un esempio per capire di cosa stiamo parlando:



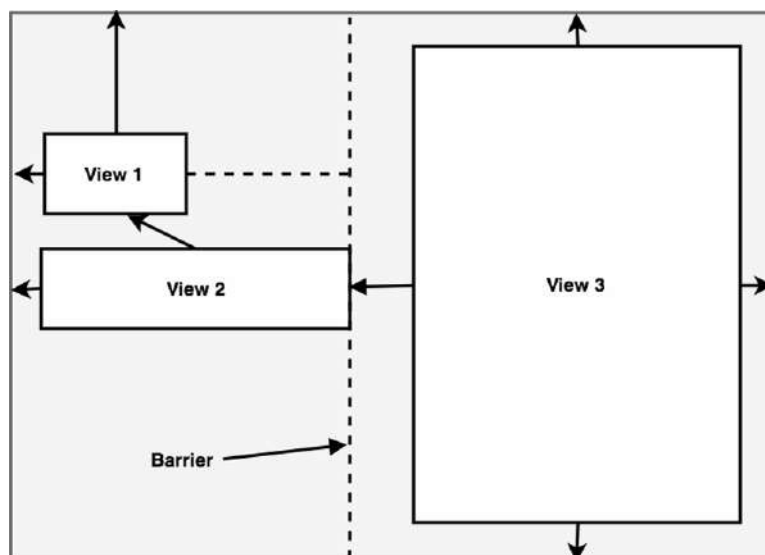
La View 3 è vincolata alla destra della View 1, quindi non si verificano problemi quando si aumenta la larghezza della View 1. Diversamente avviene se modifichiamo la larghezza della View 2, che non ha alcun constraint a destra:



Per risolvere il problema della sovrapposizione tra la View 2 e la View 3, usiamo una Barrier:



Le View 1 e 2 sono state settate come le View di riferimento per la Barrier e la View 3 è ora vincolata alla Barrier. La View 3 si adatterà alla View di dimensione massima:



4.5. Interazioni tra Kotlin e le View

Come abbiamo già visto, Android genera automaticamente una classe chiamata R.java con tutti i riferimenti alle risorse nell'applicazione. La sintassi per riferirsi a una qualsiasi risorsa è semplice:

`R.<resource_type>.<resource_name>`

Un'applicazione interagisce con le Views contenute entro un certo layout attraverso due meccanismi:

- **findViewById()**
- **ViewBinding**

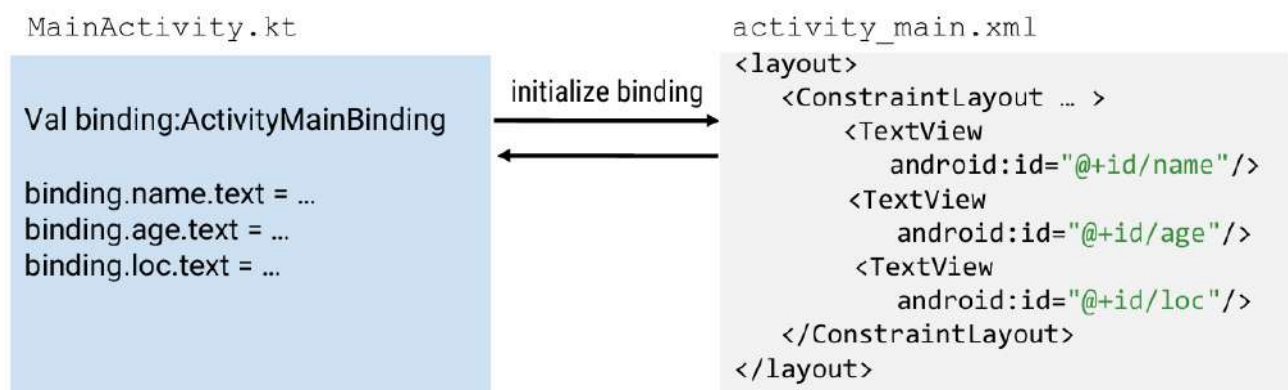
`findViewById()` è il metodo classico utilizzato per ottenere un riferimento a una View all'interno di un layout XML, dopo che il layout è stato "gonfiato" (inflate) tramite il metodo `setContentView()`.



Le due principali limitazioni sono:

- Questo metodo “attraversa” l’intera gerarchia delle View ogni volta (problema computazionale).
- Tutti gli ID sono globali per tutti i layout (alta possibilità di crash).

Per semplificare il codice ed evitare tali problemi è possibile ricorrere al **data binding**. Nel seguente esempio inizializzeremo un oggetto di binding specificatamente per il layout main activity, che potrà accedere alla View immediatamente invece di dover chiamare `findViewById` individualmente:



Per rendere possibile questo meccanismo bisogna modificare il file `build.gradle` del modulo e poi risincronizzare il gradle:

```

android {
    ...
    buildFeatures {
        viewBinding true
    }
}

```

Se il Binding è abilitato per un modulo, una classe di binding verrà generata per ogni file XML che il modulo contiene. Ogni classe di binding contiene riferimenti alla View radice e a tutte le View che possiedono un ID. Il nome della classe di binding è generato convertendo il nome del file XML in **Pascal** e aggiungendo la parola “binding” alla fine. I nomi in pascal case iniziano con una lettera maiuscola. Se il nome contiene più parole, tutte le parole iniziano con una lettera maiuscola.

Per esempio, consideriamo il layout `result_profile.xml` mostrato in di seguito:

```

<LinearLayout ... >
    <TextView
        android:id="@+id/name" />
    <ImageView
        android:cropToPadding="true" />
    <Button
        android:id="@+id/button"
        android:background="@drawable/rounded_button" />
</LinearLayout>

```

La classe di binding generata sarà `ResultProfileBinding` e avrà due campi, infatti la `TextView` e il `Button` hanno un ID mentre l'`ImageView` non ne ha.

Per impostare un'istanza di classe di binding da utilizzare in una Activity, bisogna eseguire i seguenti passi nel metodo `onCreate()`:

- Invocare il metodo statico `inflate()` incluso nella classe di binding generata.
- Prendere un riferimento alla View radice invocando il metodo `getRoot()`.
- Passare la View radice al metodo `setContentView()` per rendere la View attiva sullo schermo.

```

private lateinit var binding: ResultProfileBinding

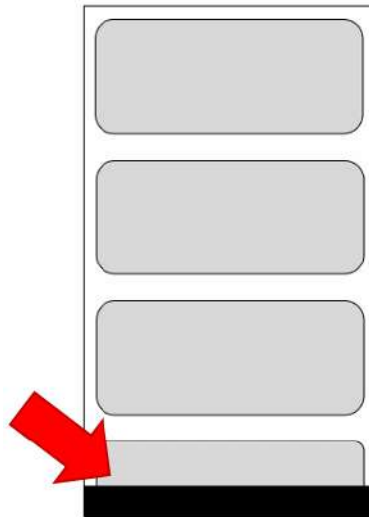
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = ResultProfileBinding.inflate(layoutInflater)
    val view = binding.root
    setContentView(view)
}

```

Una volta che la classe di binding è stata creata, è possibile fare riferimento alle varie View contenute al suo interno.

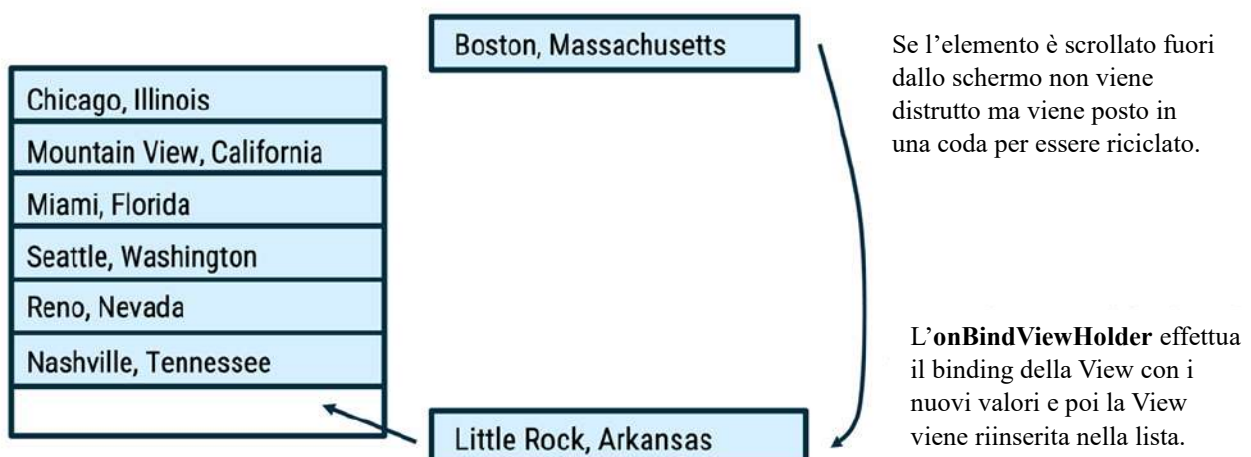
4.6. Liste dinamiche

È ormai consuetudine presentare all'utente una lista di elementi contenenti un sottoinsieme di informazioni che lo caratterizzano (come foto, titolo e sottotitolo) per aiutarlo a scegliere quale dettaglio consultare. Dal punto di vista dello sviluppo, l'implementazione di questi elementi potrebbe essere ridotta a cosa? Per esempio, si potrebbe definire un `LinearLayout` e inserire un insieme di `View` da popolare con le informazioni da visualizzare nella porzione di schermo visibile all'utente. Questo approccio ha tuttavia dei problemi, anche se solo un sottoinsieme di elementi viene visualizzato sullo schermo, l'applicazione deve creare il layout per tutti gli elementi che non sono visibili sullo schermo:

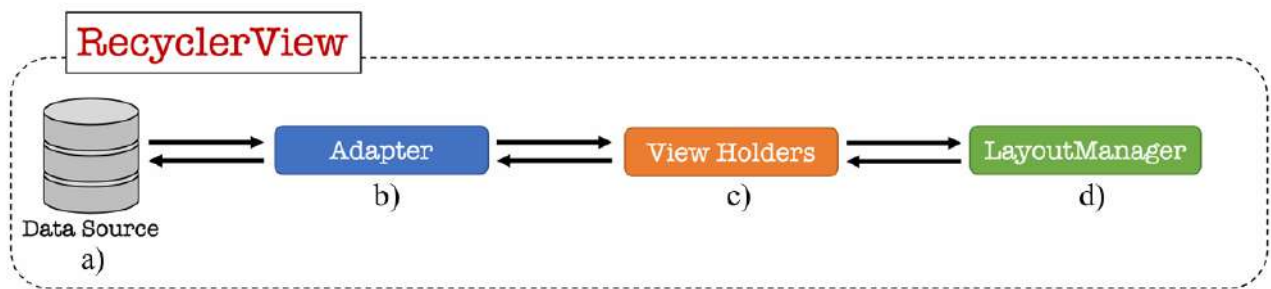


Ogni elemento mostrato deve essere salvato in memoria per poter essere in seguito mostrato all'utente senza doverlo ricreare. Questo approccio è quindi adatto solo per casi in cui è possibile lavorare con piccoli elenchi. Infatti, il caricamento di un grande insieme di elementi in una sola volta porta a una rapida saturazione della memoria, con ripercussioni negative sull'esperienza dell'utente e sulle prestazioni dell'applicazione.

Per risolvere questo problema si ricorre alla **RecyclerView**. Invece di creare `View` per elementi che non sono stati ancora scollati, questo componente tiene alcuni degli elementi precedentemente mostrati in una coda, chiamata **recycler bin**, per riutilizzarli a tempo debito. Quando si scorre la lista, la `RecyclerView` recupera un elemento dalla coda e lo popola con le nuove informazioni da mostrare all'utente. Questo componente supporta anche animazioni e transizioni di vario genere.



I componenti coinvolti nella creazione di una lista con RecyclerView sono:



a) Un **data set** usato per popolare la lista della RecyclerView (generalmente rappresentata da una List in Kotlin).

b) Un **Adapter** (RecyclerView.Adapter in Kotlin) che fornisce i dati e i layout che la RecyclerView mostra.

c) Un **View Holder** che definisce ogni elemento nella lista dinamica. È popolato tramite l'operazione di data binding sovrascritta dall'Adapter (RecyclerView.ViewHolder in Kotlin).

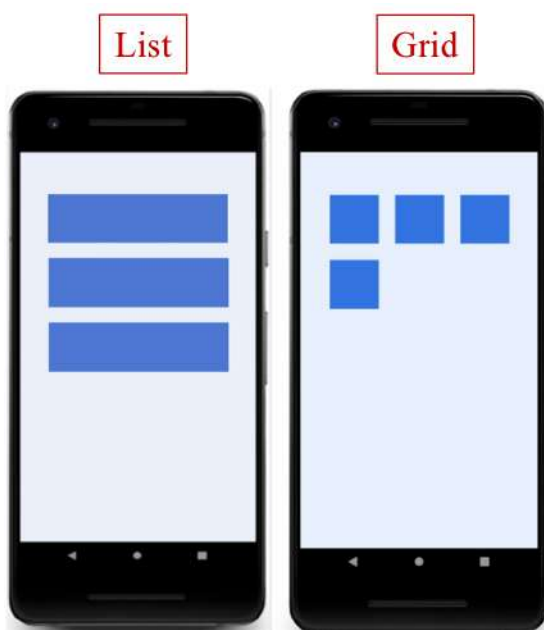
d) Un **layout manager** che organizza ogni elemento nella lista (come devono essere mostrati).

Per implementare la RecyclerView bisogna seguire alcuni passi:

1. **Aggiungere la RecyclerView al layout:**

```
<androidx.recyclerview.widget.RecyclerView  
    android:id="@+id/rv"  
    android:scrollbars="vertical"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"/>
```

2. **Decidere l'aspetto (lista o griglia):**



Nel metodo onCreate() in MainActivity.kt, una volta impostato il riferimento alla RecyclerView (tramite data binding o findViewById()), possiamo scegliere di mostrare, tramite il **LinearLayoutManager**, una lista:

```
recyclerView.layoutManager = LinearLayoutManager(this)
```

Oppure, tramite il **GridLayoutManager**, una griglia:

```
recyclerView.layoutManager = GridLayoutManager(this, 2)
```

In tutti e due i casi il parametro **this** indica il **Contesto** dell'Activity, nel secondo caso il parametro aggiuntivo indica quanti elementi ci sono in una singola riga.

È anche possibile usare altri layout manager o creare il nostro.

3. Disegnare l'aspetto e il comportamento di ogni elemento nella lista:

per fare ciò, dobbiamo creare il layout degli elementi della lista (basandoci sulle nostre preferenze) in **res/layout/item_view.xml**.

```
<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
    <TextView
        android:id="@+id/number"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</FrameLayout>
```

4. Definire l'Adapter:

per fare ciò dobbiamo seguire tre passaggi:

- a. Creare una nuova classe Adapter (MyAdapter) che estende da RecyclerView.Adapter.
- b. Definire una classe ViewHolder personalizzata che contenga le View nel layout per un singolo elemento della lista.
- c. Sovrascrivere (Override) i tre metodi dalla classe RecyclerView.Adapter:
 - i. onCreateViewHolder
 - ii. onBindViewHolder
 - iii. getItemCount

Il componente che si interfaccia con la sorgente dei dati è l'Adapter.

```
class MyAdapter(val data: List<Int>) : RecyclerView.Adapter<MyAdapter.MyViewHolder>() {
    class MyViewHolder(val row: View) : RecyclerView.ViewHolder(row) {
        val textView = row.findViewById<TextView>(R.id.number)
    }
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder {
        val layout = LayoutInflater.from(parent.context).inflate(R.layout.item_view,
            parent, false)
        return MyViewHolder(layout)
    }
    override fun onBindViewHolder(holder: MyViewHolder, position: Int) {
        holder.textView.text = data.get(position).toString()
    }
    override fun getItemCount(): Int = data.size
}
```

Questa funzione deve semplicemente ritornare quanti elementi sono presenti nella lista.

Prende un oggetto dalla lista ad una specifica posizione e inizializza i campi del contenitore.

```
class MyAdapter(val data: List<Int>) : RecyclerView.Adapter<MyAdapter.MyViewHolder>() {
    class MyViewHolder(val row: View) : RecyclerView.ViewHolder(row) {
        val textView = row.findViewById<TextView>(R.id.number)
    }
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder {
        val layout = LayoutInflater.from(parent.context).inflate(R.layout.item_view,
            parent, false)
        return MyViewHolder(layout)
    }
    override fun onBindViewHolder(holder: MyViewHolder, position: Int) {
        holder.textView.text = data.get(position).toString()
    }
    override fun getItemCount(): Int = data.size
}
```

```
<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
    <TextView
        android:id="@+id/number"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</FrameLayout>
```

item_view.xml

5. Effettuare i dovuti cambiamenti al file dell'Activity:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    val rv: RecyclerView = findViewById(R.id.rv)
    rv.layoutManager = LinearLayoutManager(this)

    rv.adapter = MyAdapter(IntRange(0, 100).toList())
}
```

Il risultato finale sarà:



Così come per ogni altra View, possiamo aggiungere dei click listeners a un'intera View di un elemento della lista o ad una vista figlia al suo interno. Poiché i nostri View holder vengono riciclati, abbiamo solo bisogno di impostare i click listeners una volta quando il ViewHolder è creato. Quando un elemento viene cliccato, si può ad esempio mostrare un messaggio **Toast** con il valore dalla TextView.

```
override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): IntViewHolder{
    val layout = LayoutInflater.from(parent.context).inflate(R.layout.item_view,
        parent, false)
    val holder = IntViewHolder(layout)
    holder.row.setOnClickListener {
        // Do something on click
    }
    return holder
}
```

4.6. Panoramica degli eventi di input

Un **evento** in Android può assumere svariate forme, ma è solitamente generato in risposta a un'azione esterna. Gli **eventi di input** sono i più frequenti in Android. Un esempio è l'interazione touch screen, per cui un evento viene generato e sarà notificato alla View in quel momento (se esiste). La View riceve un insieme di informazioni sulla natura dell'evento in questione (ad esempio le coordinate del punto di contatto) e, al fine di gestirlo, dovrà possedere un cosiddetto **Event Listener**.

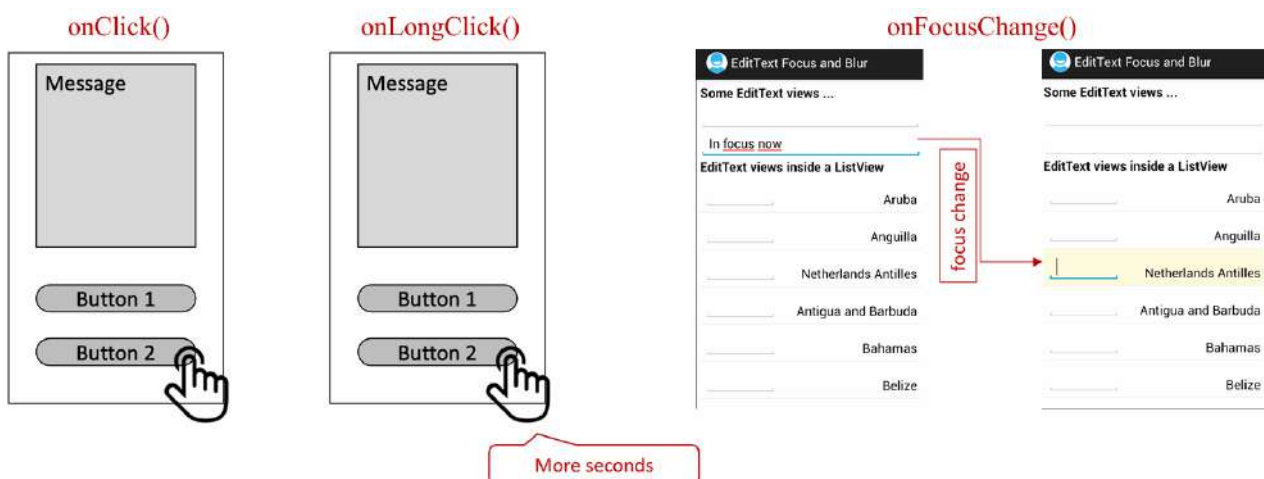
Un Event Listener è un'interfaccia nella classe View che contiene un singolo metodo callback. Questi metodi verranno chiamati dal framework Android quando la View presso la quale il listener è stato registrato sarà attivata in seguito all'interazione dell'utente con l'elemento nell'interfaccia utente (UI). La **registrazione di un Event Listener** è quel processo per mezzo del quale un **Event Handler** (gestore di eventi) viene registrato con un Event Listener in modo tale che l'handler venga chiamato quando l'Event Listener lancia l'evento. Per quanto riguarda l'Event Handler, quando avviene un evento e abbiamo registrato un Event Listener per quell'evento, l'Event Listener chiama l'Event Handler, che è concretamente il metodo che gestisce l'evento. Esistono diversi tipi di handler:

Handler	Listener	Description
onClick()	View.OnClickListener	This is called when the user either touches the item (when in touch mode), or focuses upon the item with the navigation-keys or trackball and presses the suitable "enter" key or presses down on the trackball.
onLongClick()	View.OnLongClickListener	This is called when the user either touches and holds the item (when in touch mode), or focuses upon the item with the navigation-keys or trackball and presses and holds the suitable "enter" key or presses and holds down on the trackball (for one second).
onFocusChange()	View.OnFocusChangeListener	This is called when the user navigates onto or away from the item, using the navigation-keys or trackball.
onKey()	View.OnKeyListener	This is called when the user is focused on the item and presses or releases a hardware key on the device.
onTouch()	View.OnTouchListener	This is called when the user performs an action qualified as a touch event, including a press, a release, or any movement gesture on the screen (within the bounds of the item).
onCreateContextMenu()	View.onCreateContextMenuListener	This is called when a Context Menu is being built (as the result of a sustained "long click"). We will discuss Context Menu after.

Touch mode indicates whether the last user interaction was performed with the touch screen.

- When the user is not in **touch mode**, we talk about the **trackball mode**, **navigation mode** or **keyboard navigation**, so do not be surprised if you encounter these terms.

Esempi pratici di questi eventi sono:



Il seguente esempio mostra come registrare un `onClickListener` per un `Button`:

```
protected void onCreate(savedInstanceState: Bundle) {  
    ...  
    val button: Button = findViewById(R.id.corky)  
    // Register the onClick listener with the implementation above  
    button.setOnClickListener { view ->  
        // do something when the button is clicked  
    }  
    ...  
}
```

Oppure:

```
class ExampleActivity : Activity(), OnClickListener {  
  
    protected fun onCreate(savedInstanceState: Bundle) {  
        val button: Button = findViewById(R.id.corky)  
        button.setOnClickListener(this)  
    }  
  
    // Implement the OnClickListener callback  
    fun onClick(v: View) {  
        // do something when the button is clicked  
    }  
}
```

Nel precedente esempio il metodo `onClick()` non ha valore di ritorno, ma altri metodi degli `Event Listener` devono ritornare un booleano:

- **True** per indicare che l'evento è stato gestito e dovrebbe interrompersi.
- **False** per indicare che l'evento non è stato gestito e/o dovrebbe essere passato ad altri listeners.

In particolare:

- **onLongClick()** ritorna `False` se l'evento deve continuare in qualsiasi altro `OnClickListener`.
- **onKey()** ritorna `False` se l'evento deve continuare in qualsiasi altro `onKeyListener`.
- **onTouch()** ritorna `False` se l'evento deve continuare in qualsiasi altro `onTouchListener`. La cosa importante è che questo evento può avere più azioni che si susseguono.

Vediamo un esempio:

```
package com.pwm.onlongclicklistener

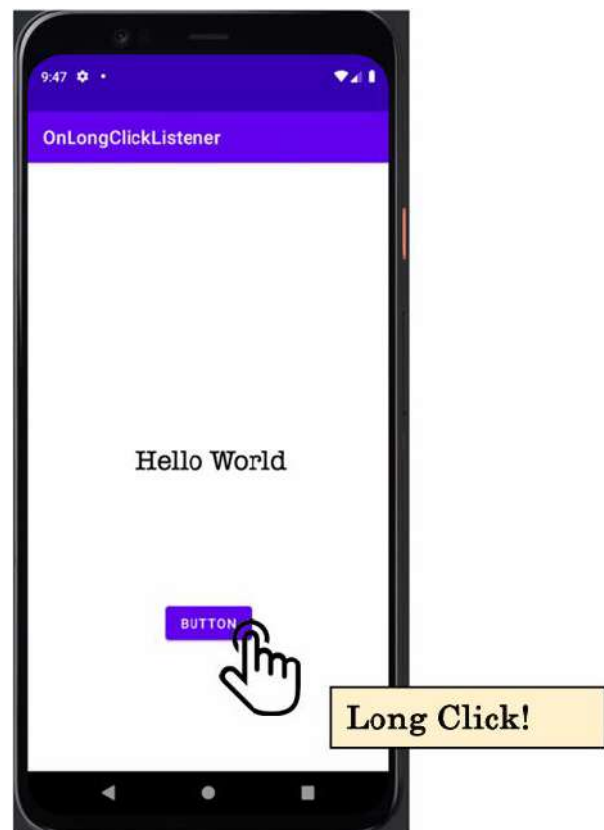
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import
com.pwm.onlongclicklistener.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        binding.button.setOnClickListener {
            binding.textView.text = "onClickListener"
        }

        binding.button.setOnLongClickListener {
            binding.textView.text = "onLongClickListener"
            true
        }
    }
}
```



```

package com.pwm.onlongclicklistener

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import com.pwm.onlongclicklistener.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        binding.button.setOnClickListener {
            binding.textView.text = "onClickListener"
        }

        binding.button.setOnLongClickListener {
            binding.textView.text = "onLongClickListener"
            true
        }
    }
}

```



Risultato finale: ritornando True, stiamo dicendo ad Android che l'evento è stato gestito e dovrebbe terminare adesso.

Ritornando False, invece:

```
package com.pwm.onlongclicklistener

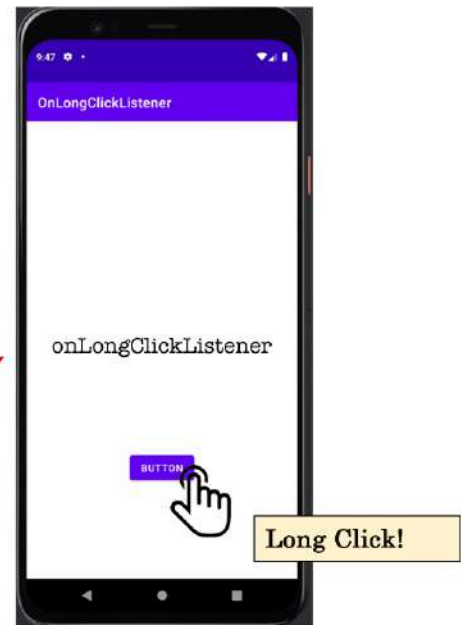
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import com.pwm.onlongclicklistener.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        binding.button.setOnClickListener {
            binding.textView.text = "onClickListener"
        }

        binding.button.setOnLongClickListener {
            binding.textView.text = "onLongClickListener"
            false
        }
    }
}
```



```
package com.pwm.onlongclicklistener

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import com.pwm.onlongclicklistener.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        binding.button.setOnClickListener {
            binding.textView.text = "onClickListener"
        }

        binding.button.setOnLongClickListener {
            binding.textView.text = "onLongClickListener"
            false
        }
    }
}
```



Risultato finale: ritornando False, stiamo dicendo ad Android che l'evento non è stato gestito e/o dovrebbe essere passato ad un altro `onClickListener`, che in questo caso è `setOnClickListener`.

La gestione degli eventi relativi all'interazione dell'utente (tocco) con lo schermo va ben oltre la risposta a un singolo tocco del dito su una View:

- **Oggetto MotionEvent** per descrivere azioni effettuate dall'utente finale:
 - ACTION_DOWN: quando l'utente tocca per la prima volta lo schermo.
 - ACTION_UP: quando l'utente solleva le dita dallo schermo.
 - ACTION_MOVE: movimenti vari tra ACTION_DOWN e ACTION_UP.
 - ...
- Rilevare più di un tocco alla volta (**multi-touch**):
Quando l'utente tocca lo schermo in diversi punti, i punti di contatto sono detti puntatori:
 - ACTION_POINTER_DOWN
 - ACTION_POINTER_UP
 - ...
- Rilevare la distanza percorsa dall'utente per far scorrere uno o più punti di contatto sulla superficie dello schermo.



AGGIUNGERE ROBE SULLA RECYCLER DA ESERCITAZIONE 5

5. Navigazione in-app

5.1. Activity e Intent multipli

Una **Activity** è la classe che permette di rappresentare il concetto di schermo in un'applicazione. La prima schermata che appare dopo l'avvio dell'applicazione è conosciuta come **MainActivity**:

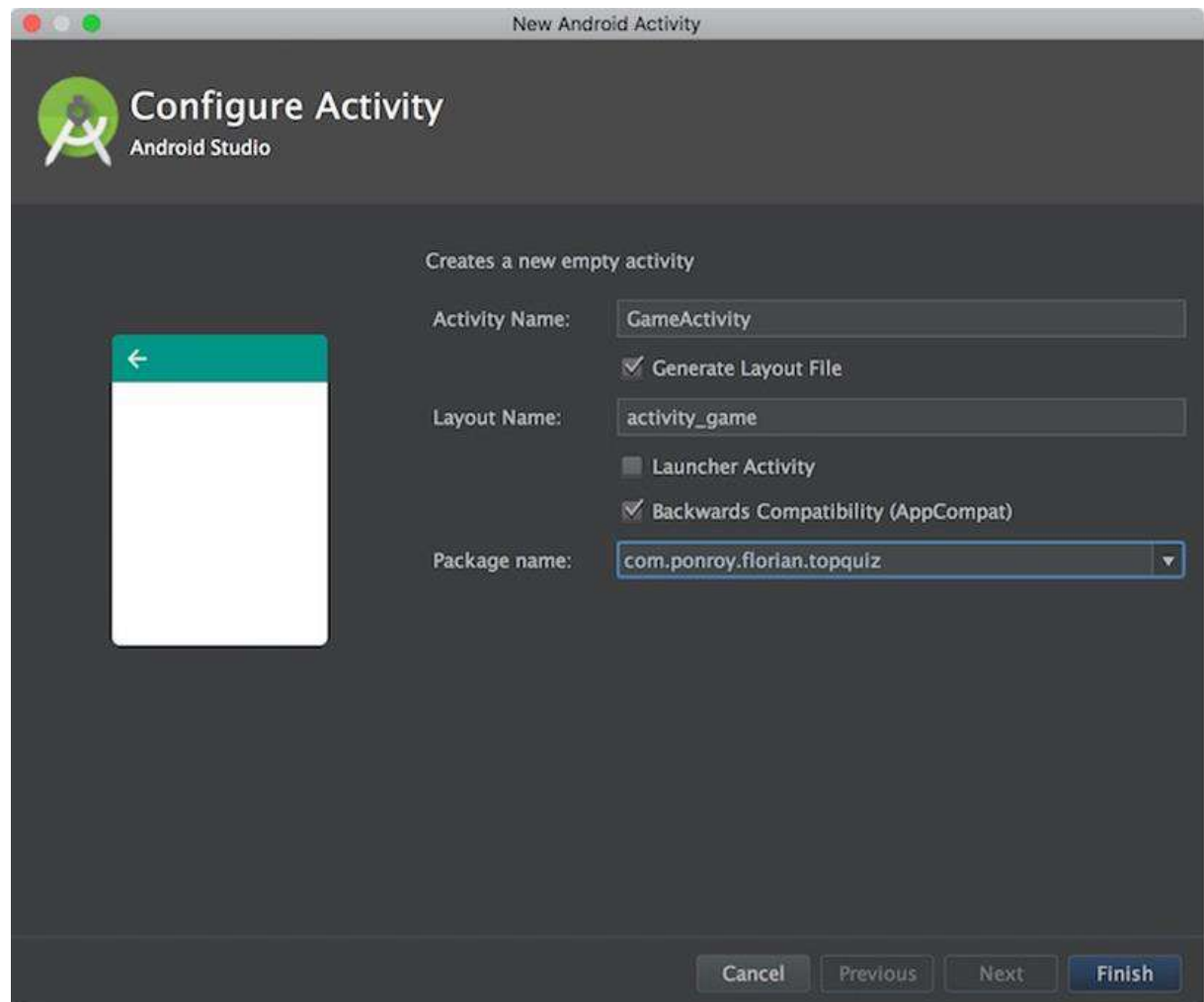
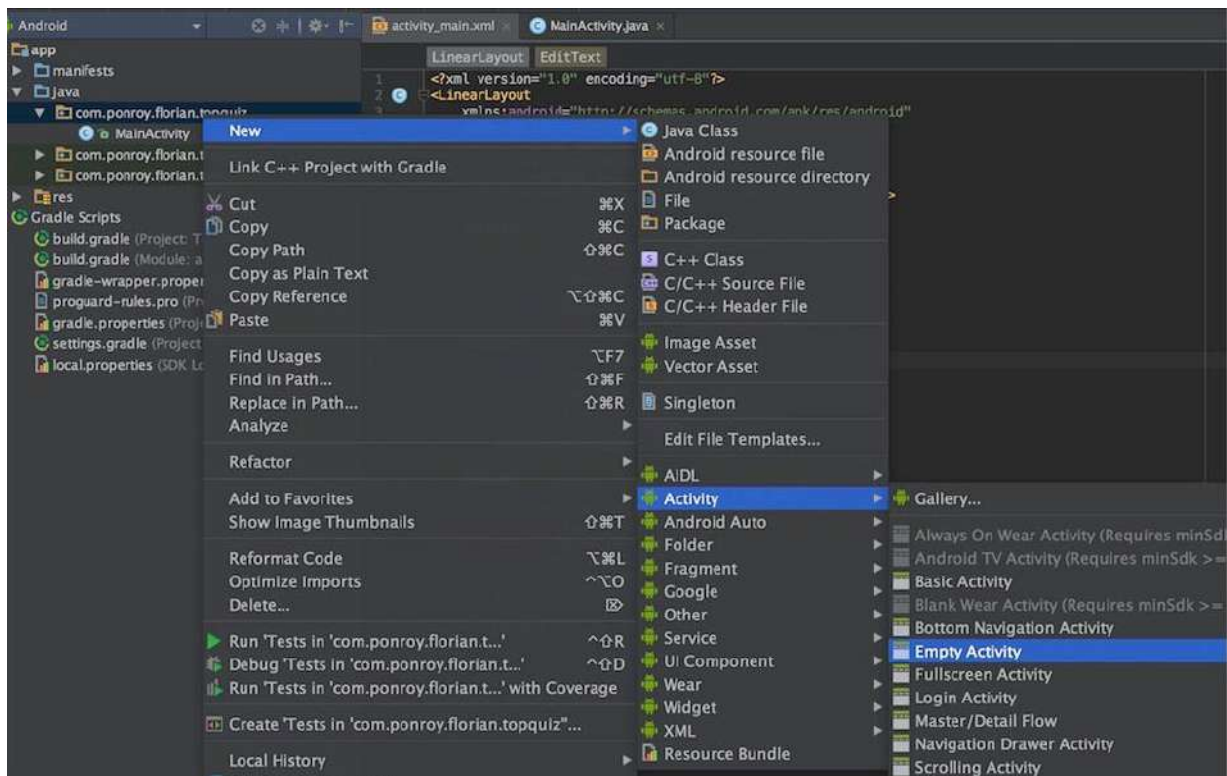
```
class MainActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
    }  
  
}
```

Ogni Activity è una sottoclasse della classe Activity, la quale è responsabile per la creazione della schermata o di una finestra che contiene l'interfaccia utente. Non si può estendere o ereditare dalla classe Activity ma solo dalle sue sottoclassi. Nell'esempio soprastante, la classe Activity è **AppCompatActivity**: ci permette di risolvere in modo trasparente problemi relativi alle differenze nelle varie versioni di Android.

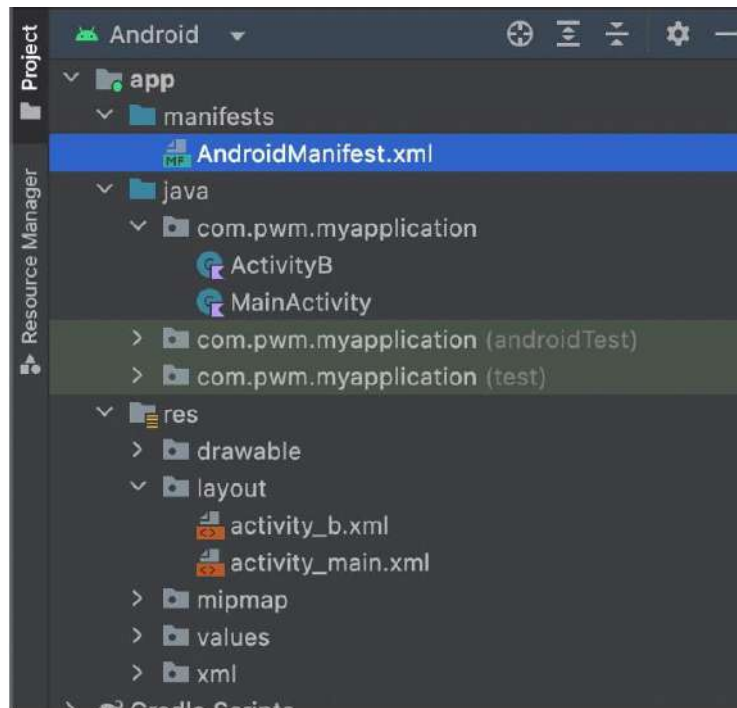
Man mano che aggiungiamo funzionalità alla nostra app potrebbe avere senso separare le varie features in differenti schermate all'interno dell'app. Un modo di ottenere diverse schermate nella nostra app è quello di implementarle come singole Activity, ognuna con il suo specifico obbiettivo. Questo metodo può rendere il codice più semplice da mantenere e riusare. Per implementare la logica di molteplici schermate, dovremo usare:

1. Una nuova Activity.
2. Gli **Intent**.

Creare una nuova Activity è semplicissimo:



Una volta eseguiti questi passaggi, possiamo ispezionare Android Studio per scoprire quali cambiamenti sono avvenuti:

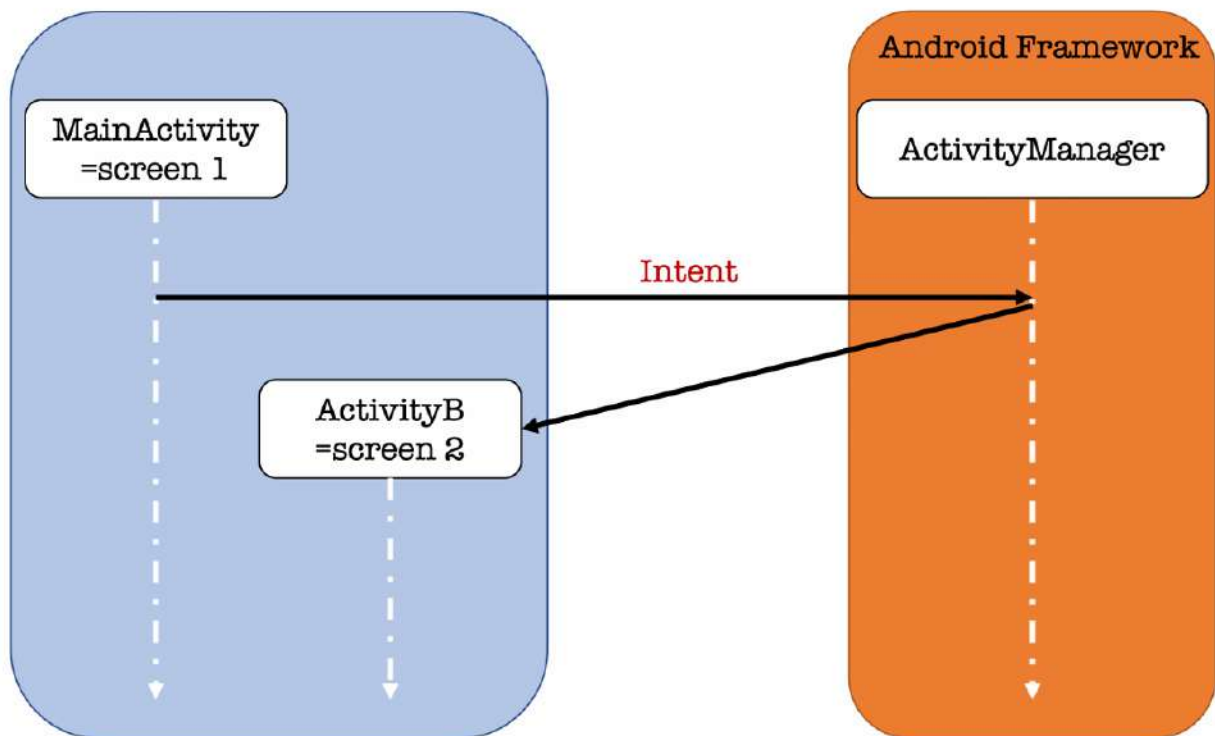


Nella directory **java** troveremo un nuovo file con estensione .kt (ad esempio ActivityB.kt), mentre nel percorso **res/layout** troveremo un nuovo file .xml (activity_b.xml). Nel file **AndroidManifest.xml** troveremo un'altra interessante informazione: ci sono due Activity, di cui una è quella di **avvio** (startup).

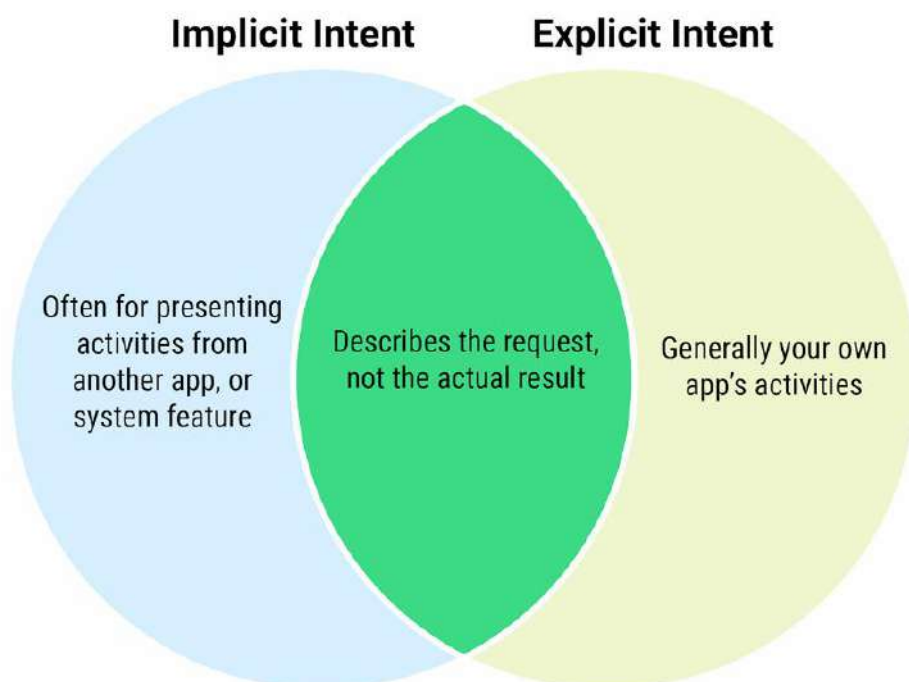
```
    android:theme="@style/Theme.MyApplication"
    tools:targetApi="31">
    <activity
        android:name=".ActivityB"
        android:exported="false" />
    <activity
        android:name=".MainActivity"
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />

            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>
</manifest>
```

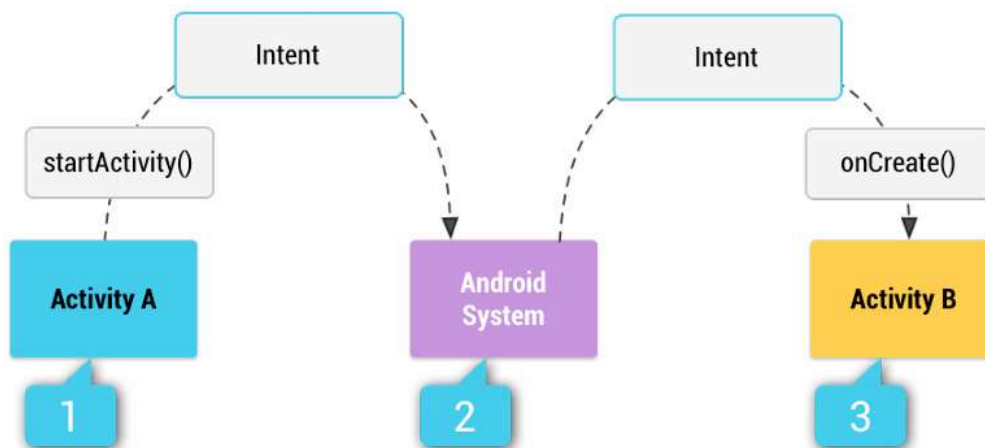
Ma cosa è questo **Intent** che appare anche nell'esempio soprastante? Per dare una prima idea grafica:



Un **Intent** è un oggetto di tipo **messaggio** che può essere usato per richiedere un'azione da un altro componente dell'applicazione. Esistono Intent impliciti e Intent espliciti, ma li approfondiremo meglio a breve.



Vediamo un esempio di meccanismo di risoluzione degli Intent:



1. ActivityA crea un Intent con una descrizione dell'azione e lo passa a startActivity().
2. Il sistema Android cerca fra tutte le applicazioni nel manifest per trovare un **Intent Filter** che corrisponda all'Intent ricevuto. Un Intent Filter è un'espressione nel file manifest di un'applicazione che specifica il tipo di Intent che il componente vuole ricevere.
3. Quando viene trovata una corrispondenza, il sistema avvia una matching Activity (ActivityB) invocando il suo metodo onCreate() e passandogli l'Intent.

Per creare un Intent si possono includere alcune informazioni:

1. **Azione:** campo che specifica l'azione generica da eseguire. Due delle molte azioni che esistono su Android sono:
 - a. **ACTION_VIEW**: quando si dispone di informazioni che una Activity può mostrare all'utente, ad esempio una foto da visualizzare in un'applicazione galleria o un indirizzo da visualizzare in un'applicazione di mappe.
 - b. **ACTION_SEND**: quando si dispone di dati che l'utente può condividere tramite un'altra applicazione, come un'applicazione di e-mail o social.
2. **Dati:** campo che specifica i dati sui quali operare, come un record di contatto nel database dei contatti, espresso come oggetto URI (Uniform Resource Identifier).

Esempi di azioni e dati sono:

- **ACTION_VIEW** `content://contacts/people/1` -- Display information about the person whose identifier is "1".
- **ACTION_DIAL** `content://contacts/people/1` -- Display the phone dialer with the person filled in.
- **ACTION_VIEW** `tel:123` -- Display the phone dialer with the given number filled in. Note how the VIEW action does what is considered the most reasonable thing for a particular URI.
- **ACTION_DIAL** `tel:123` -- Display the phone dialer with the given number filled in.
- **ACTION_EDIT** `content://contacts/people/1` -- Edit information about the person whose identifier is "1".
- **ACTION_VIEW** `content://contacts/people/` -- Display a list of people, which the user can browse through. This example is a typical top-level entry into the Contacts application, showing you the list of people. Selecting a particular person to view would result in a new intent { **ACTION_VIEW** `content://contacts/people/N` } being used to start an activity to display that person.

3. **ComponentName:** campo che differenzia un Intent esplicito da uno implicito. Se viene fornito un nome, l'Intent dovrebbe essere consegnato solamente a quello specifico componente (Intent esplicito). Viceversa, il sistema decide quale componente dovrebbe ricevere l'Intent in base alle altre informazioni (Intent implicito).
4. **Categoria:** stringa contenente informazioni aggiuntive sul tipo di componente che dovrebbe gestire l'Intent. Più categorie possono essere incluse in un Intent, ma la maggior parte degli Intent non ha bisogno di una categoria. Alcune comuni categorie sono:
 - a. **CATEGORY_BROWSABLE:** questa categoria indica che l'Activity di destinazione (target) può essere lanciata da un browser Web per visualizzare i dati referenziati da un link. In altre parole, se un'applicazione contiene un'Activity contrassegnata da questa categoria, essa può essere avviata dal browser quando l'utente clicca su un collegamento (ad esempio un URL). Questo è utile per permettere a determinate pagine web di aprire direttamente parti specifiche di un'applicazione.
 - b. **CATEGORY_LAUNCHER:** questa categoria è associata all'Activity principale di un'applicazione, quella che viene avviata per prima e che è elencata nel programma di avvio delle applicazioni del sistema (ovvero il launcher di Android). In pratica, è l'Activity che viene mostrata come icona dell'applicazione sulla schermata principale del dispositivo e che viene avviata quando l'utente clicca su di essa.
5. **Extra:** coppie chiave-valore che contengono informazioni aggiuntive necessarie per eseguire l'azione richiesta. I dati aggiuntivi possono essere aggiunti con vari metodi `putExtra()`, ognuno dei quali accetta due parametri: il nome della chiave e il valore.
6. **Flags:** insieme di metadati associati ad un Intent in Android. I flags vengono utilizzati per specificare come l'Intent dovrebbe essere gestito dal sistema operativo Android quando viene avviato. Questi metadati possono influenzare il comportamento del sistema in vari modi, come la gestione delle Activities in termini di lancio, riutilizzo, o posizionamento nella pila delle attività (task stack).

5.2. Intent espliciti e impliciti

Un **Intent esplicito** richiede l'avvio di una specifica Activity all'interno della nostra applicazione. In questo caso dobbiamo quindi fare riferimento direttamente al nome del componente (Activity/Service) che desideriamo avviare. L'uso più comune è lanciare un'Activity che risiede nella stessa applicazione che ha emesso l'Intent. Per implementare un Intent esplicito dobbiamo:

1. **Creare un'istanza della classe sorgente:** questo significa che dobbiamo avere un'istanza dell'Activity/Service da cui stiamo avviando l'Intent. Ad esempio, se stiamo avviando l'Intent da un'Activity chiamata MainActivity, quella sarà la nostra classe sorgente.
2. **Creare l'Intent e passare i seguenti parametri:**
 - a. **Context dell'Activity da cui parte l'Intent:** Questo è il contesto dell'Activity attuale (la sorgente) che stiamo usando per lanciare l'Intent.
 - b. **Riferimento dell'Activity da lanciare:** Questo è il nome della classe dell'Activity che vogliamo avviare (la destinazione).
3. **Aggiungere dati (se necessario):** possiamo includere dati aggiuntivi nell'Intent usando il metodo `putExtra()`. Ciò è utile se vogliamo passare informazioni dalla sorgente alla destinazione.
4. **Invocare il metodo `startActivity()` per avviare l'Activity specificata.**

```
fun viewNoteDetail() {  
    val intent = Intent(this, NoteDetailActivity::class.java)  
    intent.putExtra(NOTE_ID, note.id)  
    startActivity(intent)  
}
```

Spesso ci sarà capitato di cliccare su un file nel nostro smartphone e vedere una schermata di questo tipo:



La domanda del sistema operativo è: con quale applicazione vuoi aprire quel file?

Un **Intent implicito** è utile quando un'applicazione non può eseguire una certa azione, ma altre applicazioni probabilmente possono. Il caso più comune è quando si chiede all'utente di scegliere l'applicazione con cui vuole aprire/leggere qualcosa. In questi casi, il tipo di azione e di dati su cui lavorare identifica un certo Intent. Ad esempio:

- **Tipo di azione:** ACTION_VIEW
- **Tipo di dati:** l'URL di una pagina web
- **Effetto risultante:** suggerisce al sistema Android di cercare e lanciare un'Activity in grado di gestire un browser web per visualizzare il contenuto dell'URL.

```
fun sendEmail() {  
    val intent = Intent(Intent.ACTION_SEND)  
    intent.type = "text/plain"  
    intent.putExtra(Intent.EXTRA_EMAIL, emailAddresses)  
    intent.putExtra(Intent.EXTRA_TEXT, "How are you?")  
  
    if (intent.resolveActivity(packageManager) != null) {  
        startActivity(intent)  
    }  
}
```

Se si desidera che l'applicazione implementi una funzione che possa risolvere un certo compito, come si potrebbe fare? Possiamo dichiarare uno o più elementi **<intent-filter>** per ognuna delle componenti dell'applicazione. Per ogni intent-filter da aggiungere all'applicazione bisogna:

1. Definire un elemento **<intent-filter>** nel file manifest.
2. Annidare l'elemento nella componente dell'applicazione (ad esempio l'elemento **<activity>**) e settare esplicitamente un valore per **android:exported** che può assumere un valore booleano:
 - a. **True:** la componente dell'app è accessibile alle altre applicazioni.
 - b. **False:** la componente dell'app non è accessibile alle altre applicazioni.

All'interno di **<intent-filter>** possiamo specificare:

- **<action>**: dichiara l'azione che può essere accettata attraverso l'attributo name.
- **<category>**: dichiara la categoria che può essere accettata tramite l'attributo name.
- **<data>**: dichiara il tipo di dati accettati, usando uno o più attributi che specificano vari aspetti dell'URI dei dati (schema, host, porta, percorso) e del tipo MIME.

Vediamo un esempio:

```

1      <activity android:name="ShareActivity" android:exported="false">
2
3      <intent-filter>
4          <action android:name="android.intent.action.SEND"/>
          <category android:name="android.intent.category.DEFAULT"/>
          <data android:mimeType="text/plain"/>
        </intent-filter>
      </activity>

```

1. Si tratta del nome (name) dell'Activity.
2. Indica che l'Activity non è accessibile dalle altre app.
3. Definizione di un <intent-filter> per quell'Activity.
4. L'Activity può ricevere un Intent di tipo ACTION_SEND (qualcuno manda dati) per dati di tipo testo.

Attenzione: Usare un intent-filter non è un modo sicuro per impedire ad altre app di avviare i nostri componenti. Sebbene gli intent-filter limitino un componente a rispondere solo a determinati tipi di intent impliciti, un'altra app può potenzialmente avviare un componente della nostra app utilizzando un intent esplicito se lo sviluppatore riesce a determinare i nomi dei nostri componenti. Se è importante che **solo la nostra app** sia in grado di avviare uno dei nostri componenti, allora è meglio non dichiarare intent-filter nel manifesto. Invece, possiamo impostare l'attributo **exported** a "false" per quel componente. Allo stesso modo, per evitare di eseguire inavvertitamente il servizio di un'altra app, ricordiamoci di utilizzare sempre un intent esplicito per avviare il nostro servizio.

Vediamo un altro esempio:

```

<activity android:name="MainActivity" android:exported="true">
  <intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
  </intent-filter>
</activity>

```

```

<activity android:name="ShareActivity" android:exported="false">
  <intent-filter>
    <action android:name="android.intent.action.SEND"/>
    <category android:name="android.intent.category.DEFAULT"/>
    <data android:mimeType="text/plain"/>
  </intent-filter>
  <intent-filter>
    <action android:name="android.intent.action.SEND"/>
    <action android:name="android.intent.action.SEND_MULTIPLE"/>
    <category android:name="android.intent.category.DEFAULT"/>
    <data android:mimeType="application/vnd.google.panorama360+jpg"/>
    <data android:mimeType="image/*"/>
    <data android:mimeType="video/*"/>
  </intent-filter>
</activity>

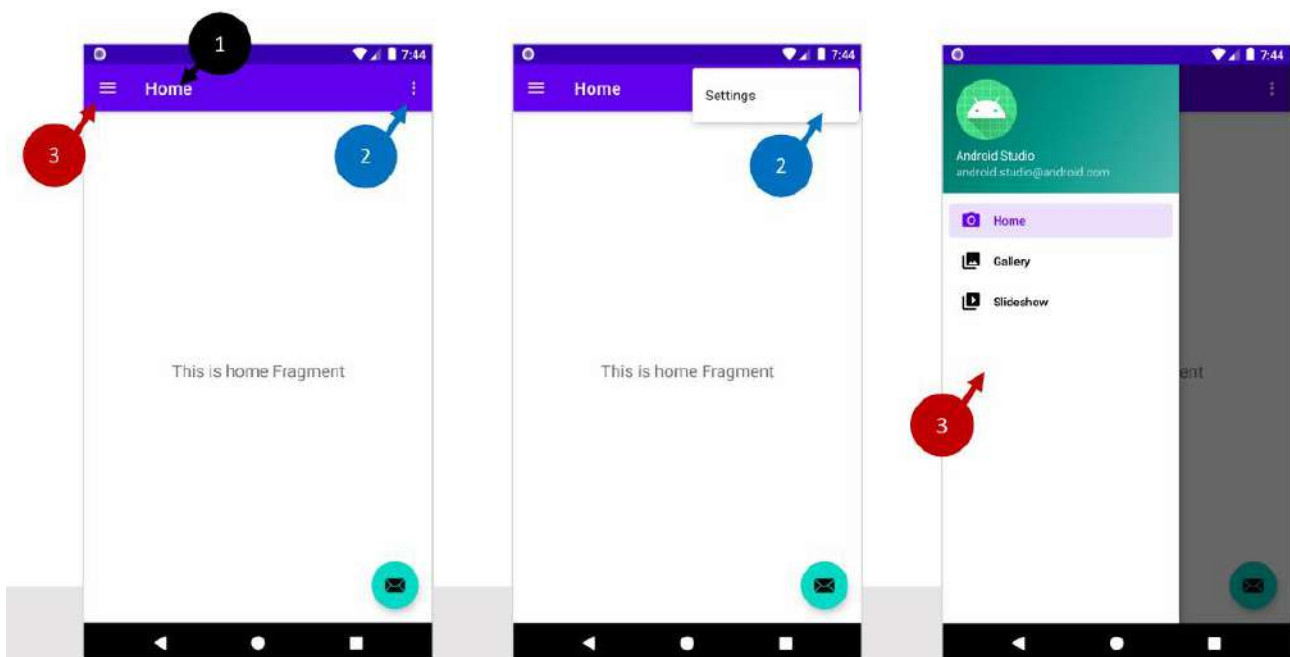
```


La prima Activity, MainActivity, è il punto principale di accesso all'applicazione:

- ACTION_MAIN indica che è il punto principale di accesso.
- CATEGORY_LAUNCHER indica che l'icona di questa Activity deve essere collocata nel programma di avvio delle applicazioni di sistema.

La seconda Activity, ShareActivity, è stata pensata per facilitare la condivisione di contenuto testuale e multimediale. Se android:exported fosse stato settato a true, altri componenti esterni avrebbero potuto accedere a ShareActivity emettendo un Intent implicito corrispondente a uno dei due intent-filter.

5.3. Barra delle applicazioni, navigation drawer e menu



Nella parte superiore dello schermo troviamo la **barra dell'applicazione**, la quale mostra il nome dell'Activity (o dell'app) e fornisce all'utente l'accesso a importanti funzionalità in modo predicibile, come ad esempio il **menu di overflow** che mostra delle opzioni di menu aggiuntive. Supporta anche la **navigazione** (navigation drawer) o il **cambio delle View** (con schede o liste a discesa).

Esistono diversi tipi di menu offerti dal framework: menu delle opzioni, menu contestuale e menu popup.



Le risorse del menu sono localizzate nella directory **res/menu** e sono definite attraverso un tag **menu**.

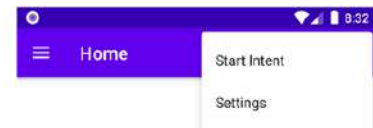
```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">
    <item
        android:id="@+id/action_settings"
        android:orderInCategory="100"
        android:title="@string/action_settings"
        app:showAsAction="never" />
</menu>
```

Per avere più opzioni non dobbiamo fare altro che aggiungere più elementi:

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">

    <item android:id="@+id/action_intent"
        android:title="@string/action_intent" />

    <item
        android:id="@+id/action_settings"
        android:orderInCategory="100"
        android:title="@string/action_settings"
        app:showAsAction="never" />
</menu>
```



Ora abbiamo un nuovo elemento del menu che non è “inflateato” (non ho idea di come si possa tradurre) nel layout e non fa nulla quando lo clicchiamo.

Per inflare il menu, dobbiamo sovrascrivere (override) il metodo `onOptionsItemSelected()` all'interno dell'Activity:

```
override fun onOptionsItemSelected(menu: MenuItem): Boolean {
    inflater.inflate(R.menu.main, menu)
    return true
}
```

Per gestire le opzioni, dobbiamo sovrascrivere (override) il metodo `onOptionsItemSelected()` all'interno dell'Activity:


```

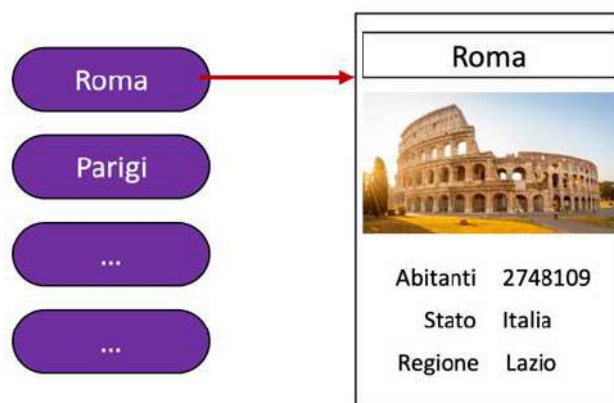
override fun onOptionsItemSelected(item: MenuItem): Boolean {
    when (item.itemId) {
        R.id.action_intent -> {
            val intent = Intent(Intent.ACTION_WEB_SEARCH)
            intent.putExtra(SearchManager.QUERY, "pizza")
            if (intent.resolveActivity(packageManager) != null) {
                startActivity(intent)
            }
        }
        else -> Toast.makeText(this, item.title, Toast.LENGTH_LONG).show()
    }
    ...
}

```

5.4. Fragment

I fragment nel corso del tempo hanno acquisito così tanta importanza che sono stati incorporati come parte integrante del framework Android. Il loro obbiettivo è quello di rendere modulare l'interfaccia utente. Un Fragment occupa una porzione di un'Activity, ma non gioca il ruolo di un semplice raggruppamento di View, bensì appare come una sub-Activity con il suo ciclo di vita. Queste caratteristiche rendono il fragment separabile da altri fragment nella stessa interfaccia rendendolo riutilizzabile autonomamente e anche combinabile con nuovi componenti. Vediamo un esempio per capire al meglio la potenza di un Fragment.

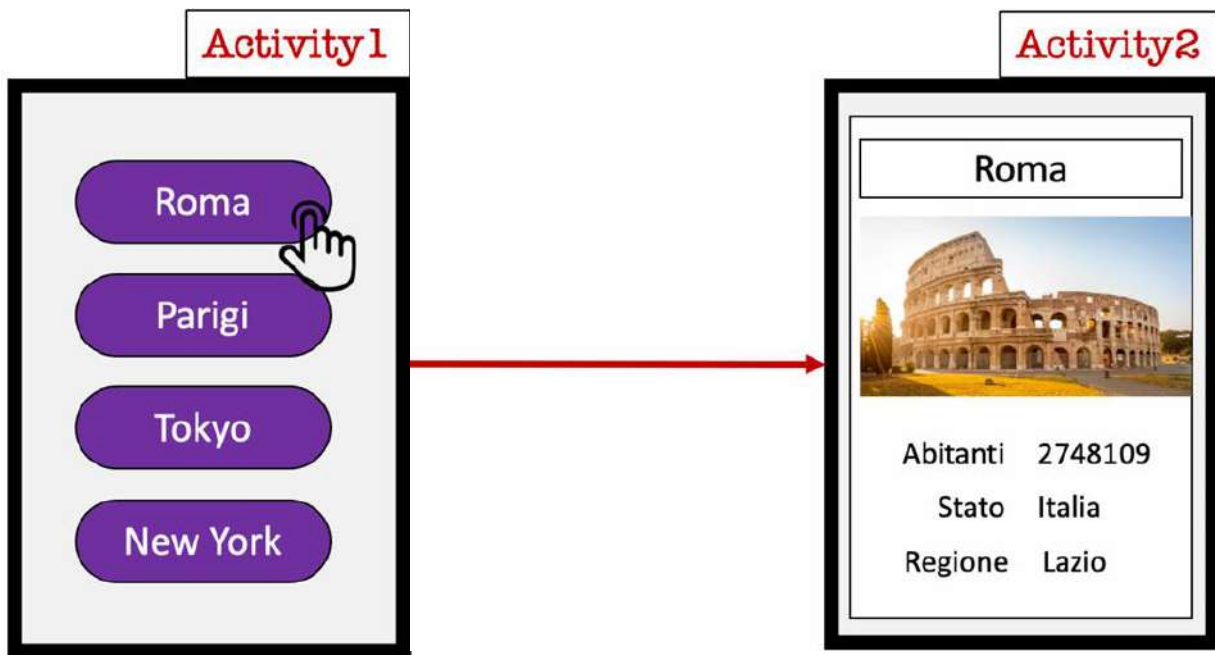
Consideriamo un'applicazione che permette agli utenti di selezionare una di cento città e mostra loro le informazioni riguardanti l'opzione selezionata:



Ma quali e quanti componenti saranno necessari?

- **Due Activity:**
 - Schermata 1: RecyclerView per mostrare i nomi delle città.
 - Schermata 2: mostra le informazioni riguardanti la città selezionata.
- Un **Intent esplicito** per la comunicazione.

Graficamente:



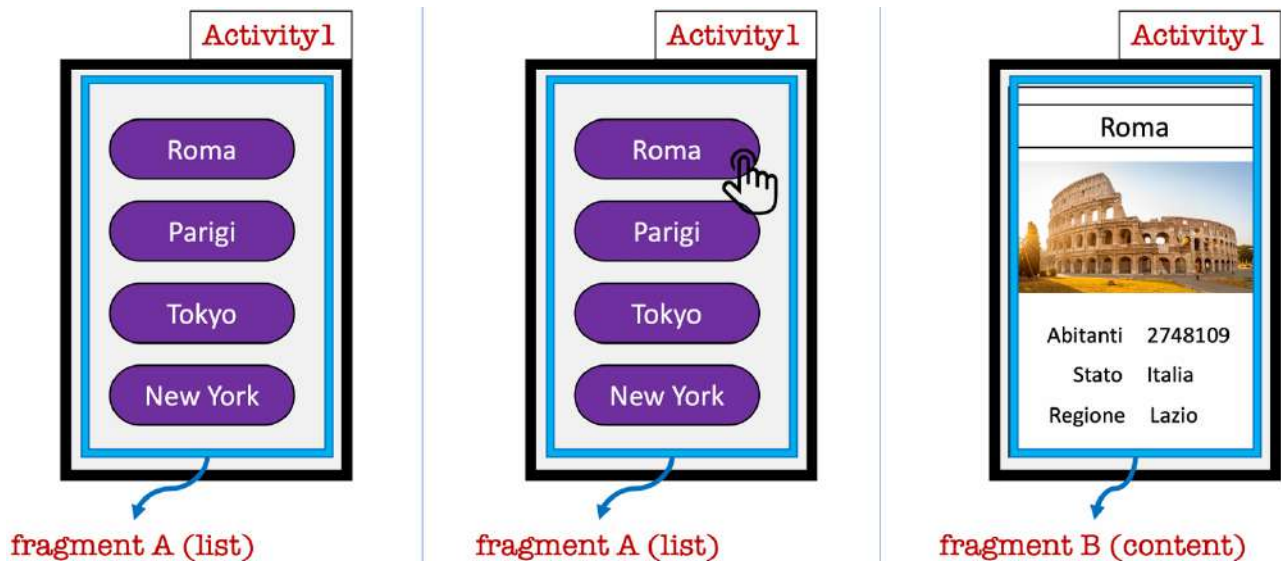
Se usassimo la stessa logica per un tablet, ci sarebbe un sacco di spazio sprecato:



Una soluzione più efficace è mettere la lista e il contenuto in due fragment della stessa Activity:



Ovviamente selezionare un oggetto diverso del Fragment A aggiorna il Fragment B. In questo senso i Fragment sono solo una porzione dell'Activity. Questa stessa esatta logica funziona anche per uno smartphone:



I Fragment richiedono una dipendenza dalla libreria **AndroidX Fragment**. Per includere questa libreria al nostro progetto bisogna aggiungere le seguenti dipendenze nel file build.gradle della nostra app:

```
dependencies {  
    val fragment_version = "1.6.2"  
    // Kotlin  
    implementation("androidx.fragment:fragment-ktx:$fragment_version")  
}
```

Per creare un Fragment bisogna estendere la classe **AndroidX Fragment** e sovrascrivere (override) i suoi metodi per inserire la logica della nostra app, in modo simile a quanto faremmo per creare una classe Activity. Per creare un Fragment minimale che definisce il proprio layout, bisogna fornire la risorsa layout del Fragment al costruttore di base, come mostrato nell'esempio seguente:

```
class ExampleFragment : Fragment(R.layout.example_fragment)
```

Oppure è possibile sovrascrivere (override) il metodo callback **onCreateView()** nel quale è possibile inflare il layout:

```
class DetailFragment : Fragment() {  
  
    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?,  
        savedInstanceState: Bundle?): View? {  
        return inflater.inflate(R.layout.detail_fragment, container, false)  
    }  
}
```

La libreria Fragment fornisce anche delle classi di base più specializzate, come ad esempio la **DialogFragment**, che mostra dei dialog fluttuanti. Usare questa classe per creare un dialog è una buona alternativa rispetto all'uso dei metodi del dialog helper nella classe Activity, in quanto i Fragment gestiscono automaticamente la creazione e la pulizia dei dialog.

Generalmente, il Fragment deve essere incorporato in una AndroidX FragmentActivity per contribuire con una porzione di interfaccia utente al layout dell'Activity. FragmentActivity è la classe base per AppCompatActivity, quindi se stiamo già sottoclassando AppCompatActivity per fornire retrocompatibilità alla nostra app, allora non abbiamo bisogno di cambiare la classe base della nostra Activity.

In seguito, dobbiamo aggiungere il Fragment alla gerarchia delle View dell'Activity, e lo si può fare in due modi:

- Definendo il Fragment nel file di layout della nostra Activity (approccio basato su XML).
- Definendo un contenitore di Fragment nel file di layout della nostra Activity e poi aggiungere da codice il Fragment dall'interno della nostra Activity (approccio programmatico).

In ogni caso dovremo aggiungere una **FragmentContainerView** che definisce la posizione in cui il Fragment dovrebbe essere posizionato all'interno della gerarchia delle View dell'Activity.

Vediamo un primo esempio del primo approccio:

```
<!-- res/layout/example_activity.xml -->
<androidx.fragment.app.FragmentContainerView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragment_container_view"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:name="com.example.ExampleFragment" />
```

Questo è un esempio di layout di Activity che contiene una singola FragmentContainerView. L'attributo **android:name** specifica il nome della classe del Fragment da istanziare. Quando il layout dell'Activity è inflato, il Fragment specificato è istanziato, viene chiamato il metodo **onInflate()** sul Fragment appena istanziato e una **FragmentTransaction** viene creata per aggiungere il Fragment al **FragmentManager**.

Adesso vediamo un esempio un po' più pratico dello stesso approccio. Definiamo `fragment_first.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".FirstFragment">

    <!-- TODO: Update blank fragment layout -->
    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:text="@string/hello_blank_fragment" />

</FrameLayout>
```

Hello blank fragment

In seguito, definiamo la classe `FirstFragment`:

```
import androidx.fragment.app.Fragment

class FirstFragment : Fragment(R.layout.fragment_first){

}
```

FirstFragment.kt

Ciò che rimane da fare è aggiungere il `Fragment` all'Activity:

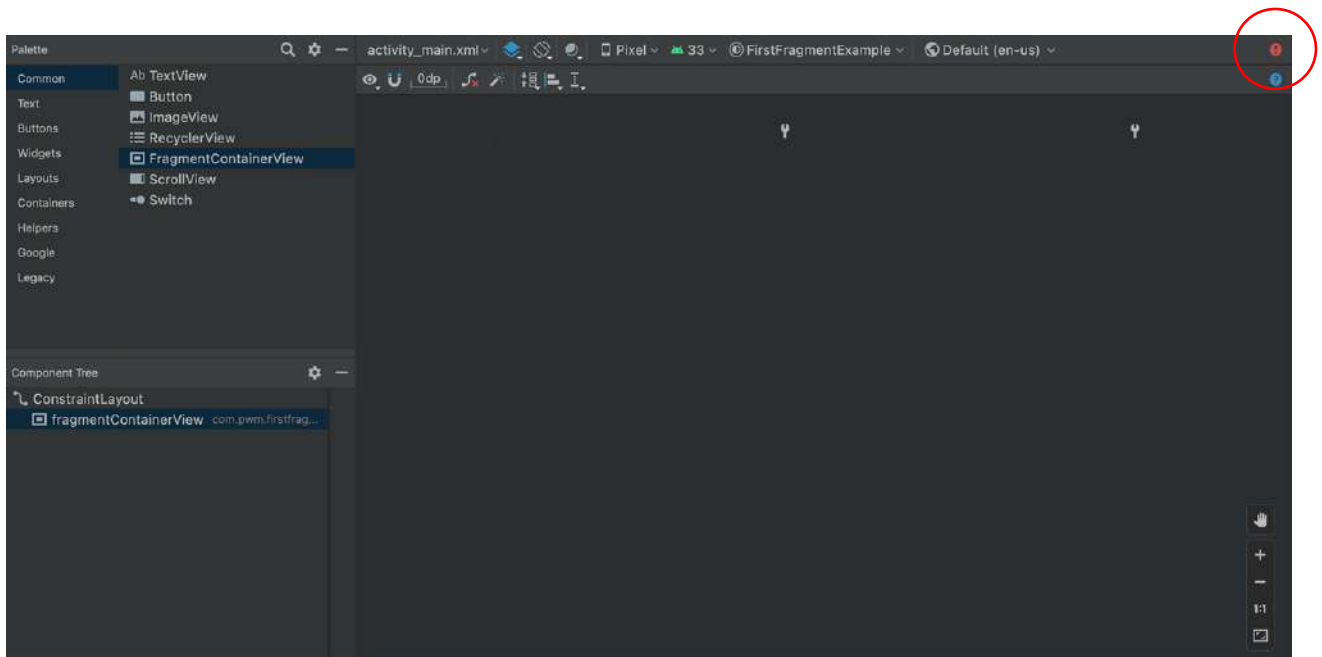
```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/fragmentContainerView"
        android:name="com.pwm.firstfragmentexample.FirstFragment"
        android:layout_width="Odp"
        android:layout_height="wrap_content"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

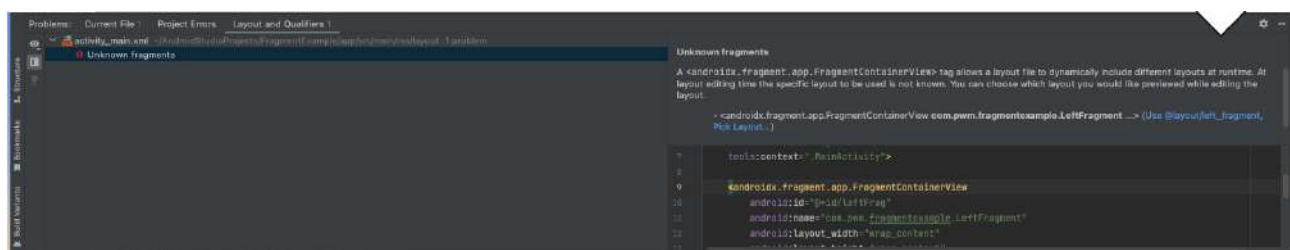
</androidx.constraintlayout.widget.ConstraintLayout>
```

activity_main.xml

Dopo aver aggiunto la `FragmentContainerView` noteremo che non viene più mostrato il test di layout, bensì in alto a destra figurerà un errore:



Cliccando sull'avviso ci verranno mostrati dettagli aggiuntivi sull'errore:



Il problema è che il layout è inserito al run time e non al preview time, quindi non si può considerare un vero e proprio errore. Se proprio vogliamo vedere la preview del layout, dobbiamo cliccare manualmente sull'opzione contrassegnata in blu "Pick Layout".

Il risultato finale sarà:



Passando all'approccio programmatico, per aggiungere un Fragment al layout della nostra Activity, il layout dovrebbe includere una `FragmentManager` che serva appunto da contenitore per il Fragment, come mostrato di seguito:

```
<!-- res/layout/example_activity.xml -->
<androidx.fragment.app.FragmentContainerView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragment_container_view"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Diversamente dall'approccio XML, l'attributo `android:name` non è usato sulla `FragmentContainerView`, quindi non viene istanziato alcuno specifico Fragment. Mentre la nostra Activity è in esecuzione, possiamo eseguire delle **Fragment transactions** come l'aggiunta, la rimozione o la sostituzione di un Fragment.

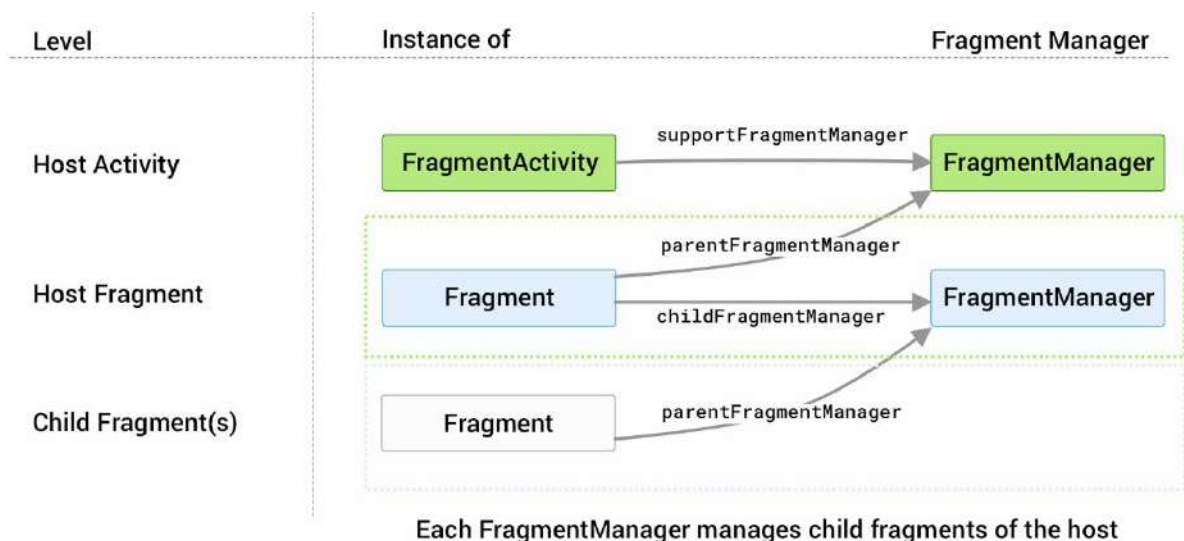
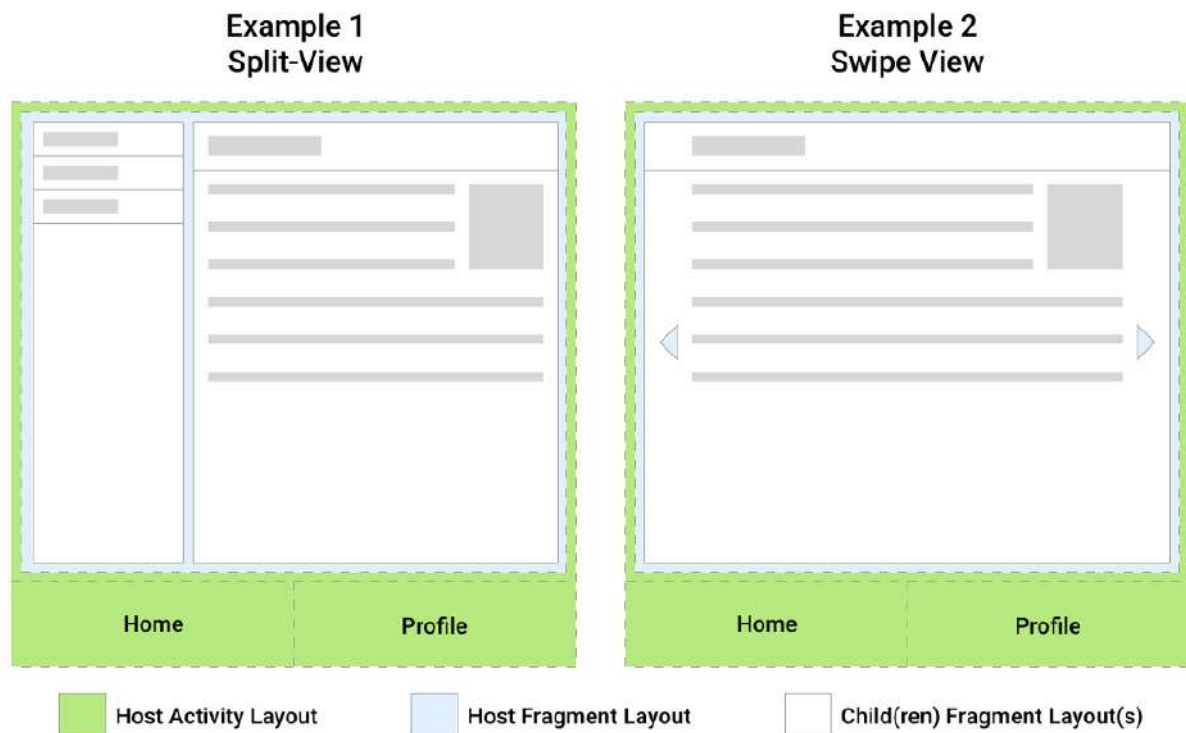
Vediamo un esempio:

in `FragmentActivity`, prendiamo un'istanza del `FragmentManager`, che può essere usata per creare una `FragmentManager`. In seguito, istanziamo il Fragment all'interno del metodo `onCreate()` della nostra Activity usando `FragmentManager.add()`, passando l'ID del ViewGroup del contenitore nel layout e la classe Fragment che vogliamo aggiungere, e in seguito facciamo un commit della transaction:

```
class ExampleActivity : AppCompatActivity(R.layout.example_activity) {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        if (savedInstanceState == null) {
            supportFragmentManager.commit {
                setReorderingAllowed(true)
                add<ExampleFragment>(R.id.fragment_container_view)
            }
        }
    }
}
```


5.5. Overview su FragmentManager e Transactions

FragmentManager è la classe responsabile dell'esecuzione di azioni sui Fragment della nostra app, come l'aggiunta, la rimozione o la sostituzione di un Fragment e la loro aggiunta al back stack. È possibile accedere al FragmentManager da un'Activity o da un Fragment. La classe **FragmentManager** e le sue sottoclassi (come **AppCompatActivity**) hanno accesso al **FragmentManager** attraverso il metodo **getSupportFragmentManager()**. I Fragment possono ospitare uno o più Fragment figli. All'interno di un Fragment, possiamo ottenere un riferimento al **FragmentManager** che gestisce i figli del Fragment tramite il metodo **getChildFragmentManager()**. Se invece dobbiamo accedere al suo host **FragmentManager**, possiamo farlo attraverso il metodo **getParentFragmentManager()**. Vediamo un esempio grafico:



Al runtime, il `FragmentManager` può aggiungere, rimuovere, sostituire ed eseguire altre azioni con i `Fragment` in risposta all'interazione dell'utente. Ogni insieme di modifiche che vengono applicate al `Fragment` è chiamato **Transaction**. Nel contesto di una singola `Transaction`, possono essere eseguite molteplici azioni. Per esempio, una `Transaction` può aggiungere o sostituire molteplici `Fragment`. Possiamo ottenere un'istanza di `FragmentTransaction` dal `FragmentManager` invocando il metodo `beginTransaction()`, come mostrato di seguito:

```
val fragmentManager = ...
val fragmentTransaction = fragmentManager.beginTransaction()
```

L'ultima operazione di una qualsiasi `Transaction` deve essere un **commit**, la quale segnala al `FragmentManager` che tutte le operazioni sono state aggiunte con successo alla `Transaction`:

```
val fragmentManager = ...
// The fragment-ktx module provides a commit block that automatically
// calls beginTransaction and commit for you.
fragmentManager.commit {
    // Add operations here
}
```

ATTENZIONE: `commit()` lavora in modo **asincrono**. Ciò significa che la chiamata a `commit` non esegue immediatamente la `Transaction`. La `Transaction` è programmata per essere eseguita sul thread principale dell'interfaccia utente non appena è in grado di farlo. Se necessario, ad ogni modo, possiamo ricorrere al metodo `commitNow()` per eseguire immediatamente la `Transaction` del `Fragment` sul thread dell'interfaccia utente (per la maggior parte dei casi però, `commit()` è sufficiente).

Le operazioni principali svolte in una `Transaction` sono:

- **add()**: inserisce il `Fragment` da aggiungere al contenitore. Il `Fragment` aggiunto passa allo stato `RESUMED`.
- **remove()**: rimuove il `Fragment` recuperato dal `FragmentManager` tramite `findFragmentById()` o `findFragmentByTag()`. Il `Fragment` rimosso passa allo stato `DESTROYED`.
- **replace()**: rimpiazza uno dei `Fragment` con un altro. La chiamata a `replace()` è equivalente a chiamare `remove()` sul contenitore e aggiungere un nuovo `Fragment` allo stesso contenitore.

Vediamo un esempio:

```
// Create new fragment
val fragmentManager = // ...

// Create and commit a new transaction
fragmentManager.commit {
    setReorderingAllowed(true)
    // Replace whatever is in the fragment_container view with this fragment
    replace<ExampleFragment>(R.id.fragment_container)
}
```

In questo esempio, una nuova istanza di `ExampleFragment` rimpiazza il `Fragment`, se esiste, che si trova in quel momento nel contenitore del layout identificato da `R.id.fragment_container`.

ATTENZIONE: l'ordine con cui si eseguono operazioni in una `FragmentManager` è significativo, in particolare quando si utilizza il metodo `setCustomAnimations()`. Questo metodo applica le animazioni fornite a tutte le operazioni dei `Fragment` che la seguono:

```
supportFragmentManager.commit {  
    setCustomAnimations(enter1, exit1, popEnter1, popExit1)  
    add<ExampleFragment>(R.id.container) // gets the first animations  
    setCustomAnimations(enter2, exit2, popEnter2, popExit2)  
    add<ExampleFragment>(R.id.container) // gets the second animations  
}
```

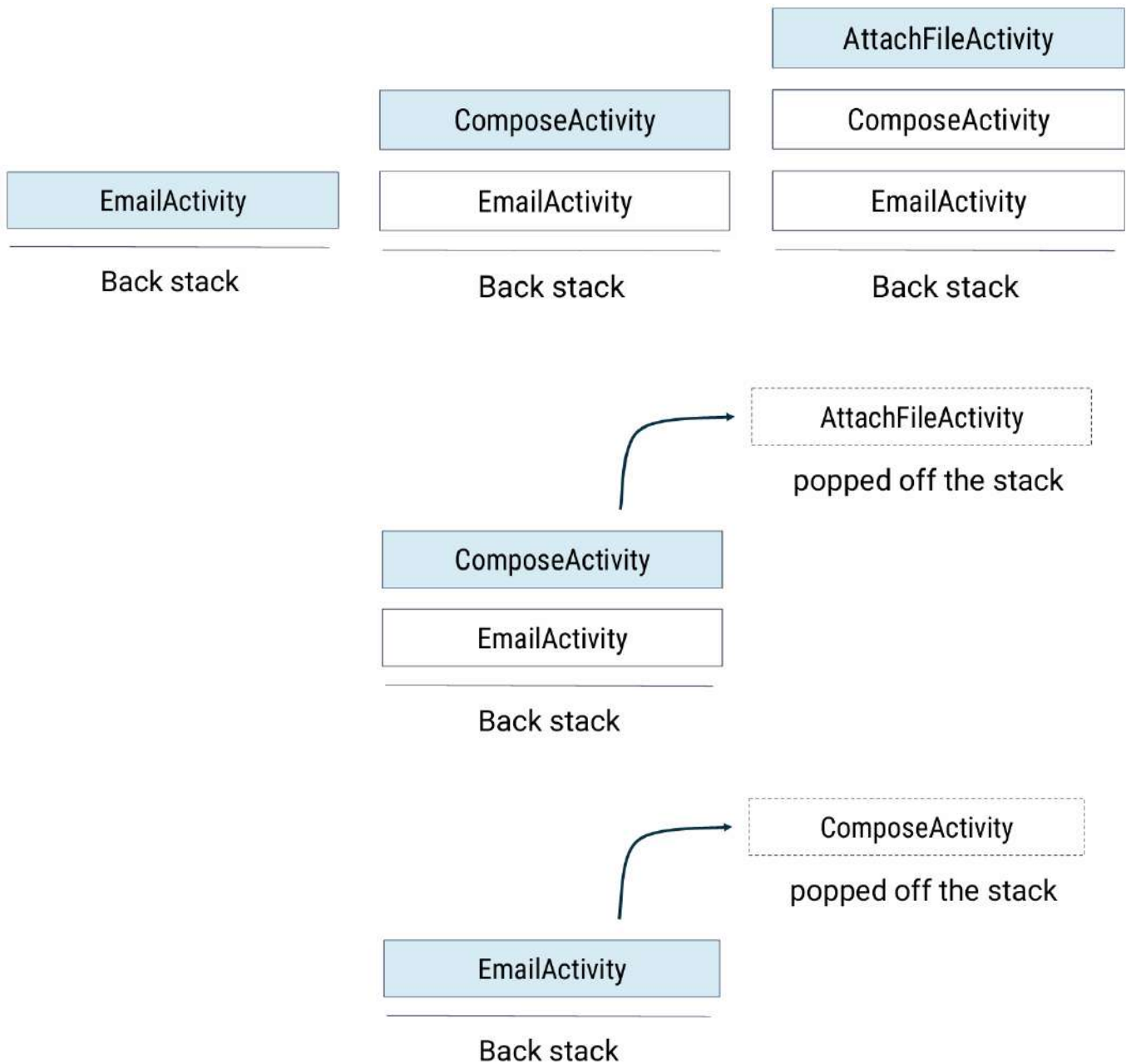
Il `FragmentManager` ci permette anche di gestire manualmente il `Backstack`:

```
supportFragmentManager.commit {  
    replace<ExampleFragment>(R.id.fragment_container)  
    setReorderingAllowed(true)  
    addToBackStack(null)  
}  
...  
val fragment: ExampleFragment =  
    supportFragmentManager.findFragmentById(R.id.fragment_container) as ExampleFragment
```

In questo esempio, un metodo come `popBackStack()` farebbe il contrario.

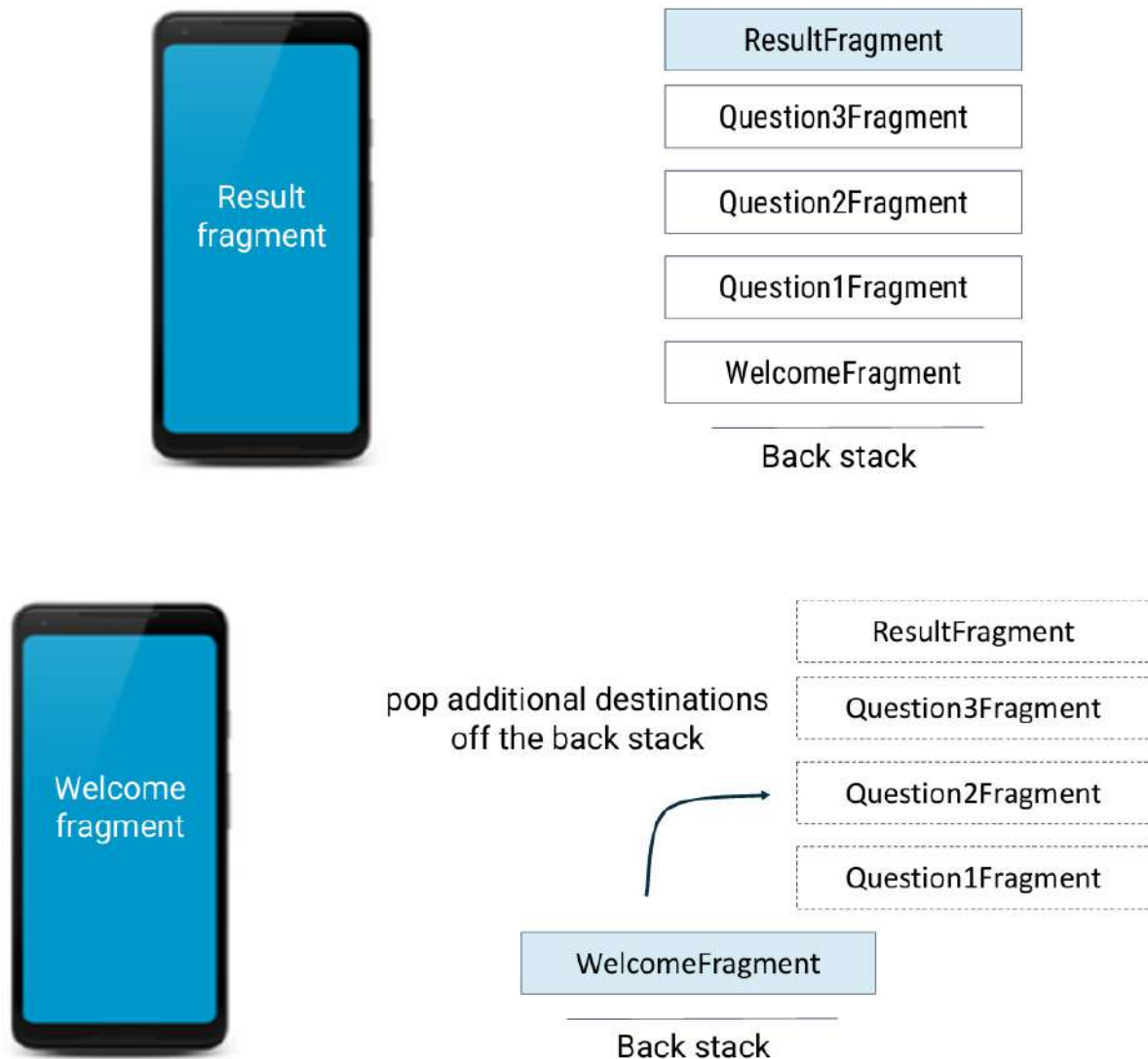
5.6. Backstack

Vediamo come funziona il Backstack di un'Activity:

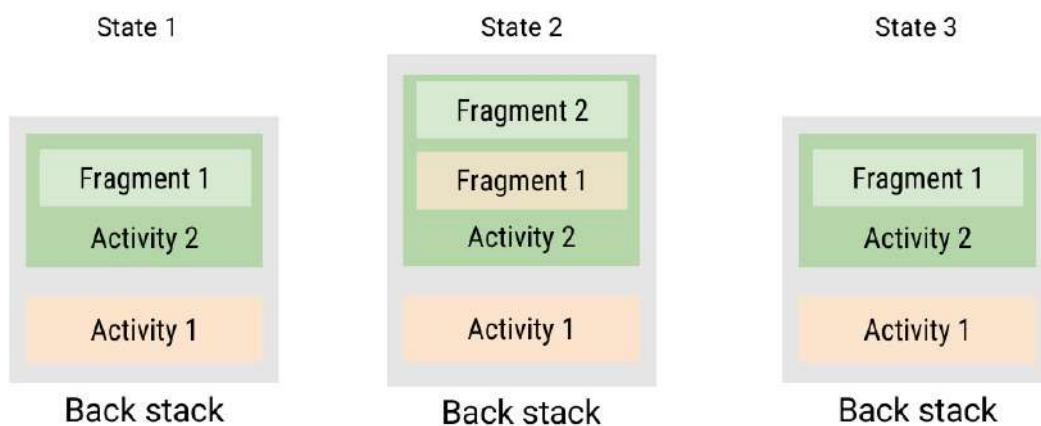


Si tratta di una pila LIFO (Last-In-First-Out).

Il backstack dei Fragment in Android funziona in modo simile al backstack delle Activity, ma è gestito all'interno di una singola Activity. Ogni volta che aggiungiamo, rimuoviamo o sostituiamo un Fragment, abbiamo l'opzione di aggiungere l'operazione al backstack. Questo permette all'utente di navigare indietro attraverso i Fragment precedentemente visualizzati premendo il tasto indietro.



Vediamo un esempio grafico di backstack con Activity e Fragment:

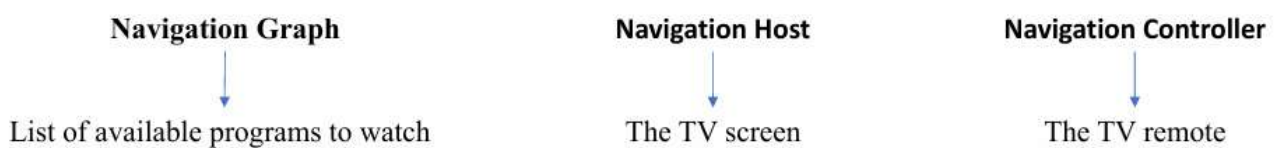


5.7. Navigazione all'interno di un'app

La componente **Navigation** è una collezione di librerie, strumenti e integrazioni dell'IDE utili alla creazione di percorsi di navigazione all'interno di un'app. Tale componente funziona meglio se si opta per il paradigma “un'Activity, molti Fragment” e presuppone una Activity per un grafico con molti Fragment di destinazione. È composto da tre parti che lavorano insieme:

- **Navigation Graph**: un insieme di Fragment (o Activity) di destinazione che l'app può offrire e mostrare.
- **Navigation Host**: il contenitore che mostra le destinazioni elencate nel navigation graph.
- **Navigation Controller**: il mezzo per navigare tra le diverse destinazioni.

Un pratico esempio per capire meglio i concetti appena spiegati:



Per usare la componente Navigation, dobbiamo aggiungere delle dipendenze al file build.gradle:

```
implementation "androidx.navigation:navigation-fragment-ktx:$nav_version"
implementation "androidx.navigation:navigation-ui-ktx:$nav_version"
```

Per scegliere la release recente più stabile della nav_version possiamo consultare la pagina delle release notes della libreria Navigation.

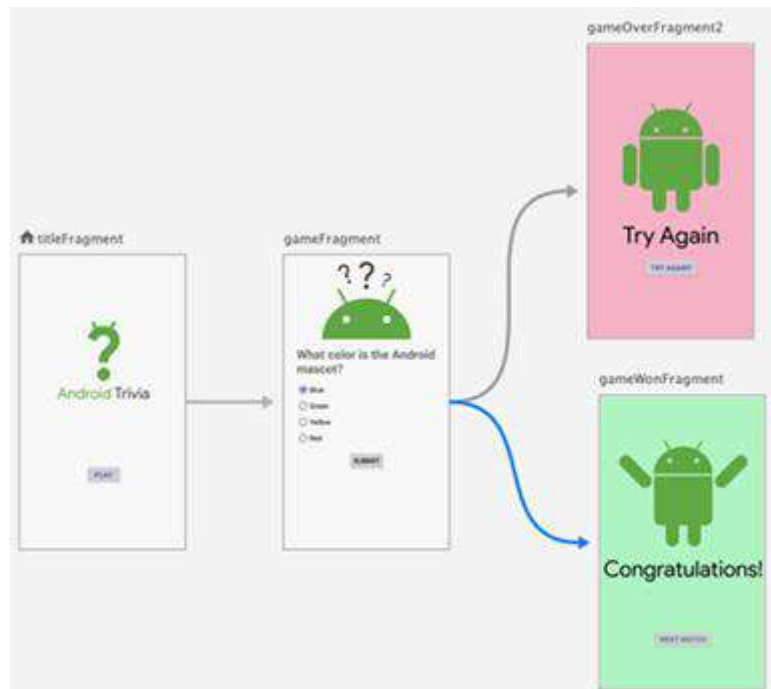
Iniziamo con il nostro **host di navigazione**, che è un contenitore vuoto in cui le destinazioni vengono scambiate in entrata e in uscita durante la navigazione dell'utente nell'applicazione. Un host di navigazione deve implementare l'interfaccia **NavHost**, la cui implementazione di default è **NavHostFragment**. Per dichiarare il NavHostFragment usiamo l'approccio XML, facendo attenzione a qualche attributo che approfondiremo più avanti:



- L'attributo **android:name** ha un valore che è il nome della classe completamente qualificata del nostro Fragment.
- L'attributo **app:defaultNavHost** è settato a true per assicurarci che questo host di navigazione intercetterà le pressioni del pulsante “Indietro” del sistema.
- L'attributo **app:navGraph** punta al navigation graph che elenca tutte le destinazioni del nostro Fragment.

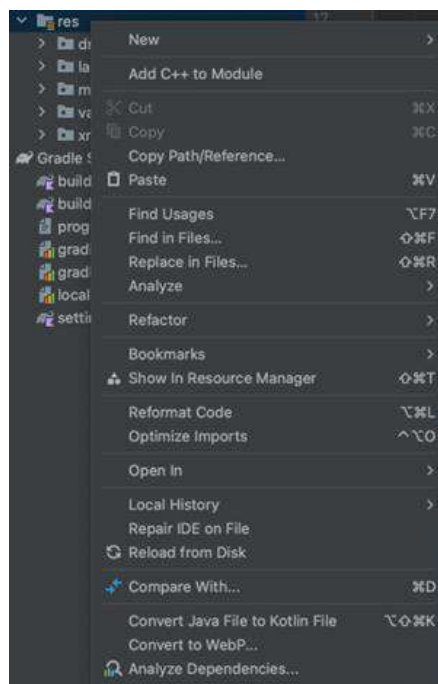
L'obiettivo è quello di creare un navigation graph, che è un file di risorse che contiene tutte le destinazioni e le azioni per navigare tra loro. Questo nuovo file di risorse si trova nella directory

res/navigation e si presenta sottoforma di file XML contenente tutte le destinazioni (dei Fragment o delle Activity) che possono essere raggiunte e le azioni a loro associate utili a navigare tra loro. Opzionalmente contiene anche animazioni per l'entrata o l'uscita da una destinazione.

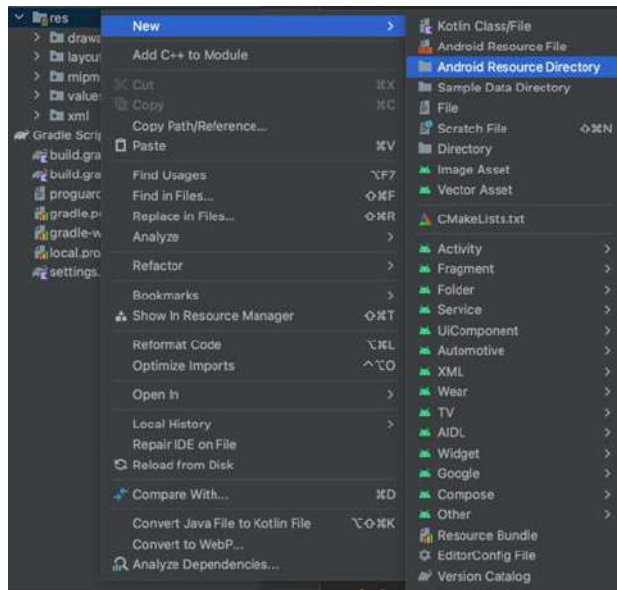


Come creiamo concretamente questo file? Ecco il tutorial:

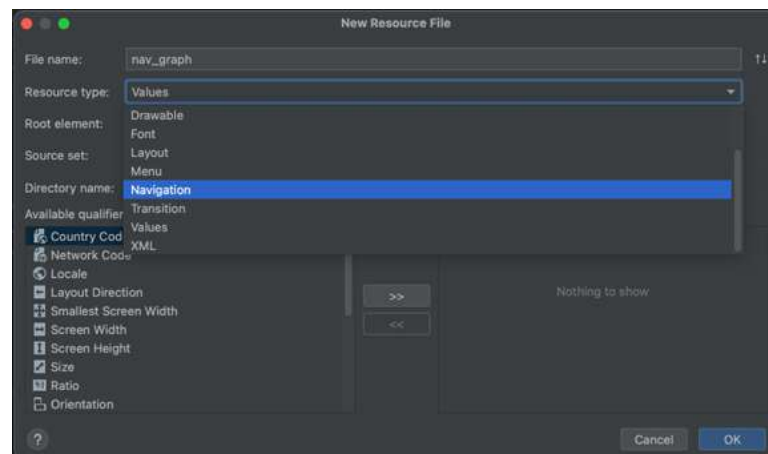
1. Nella finestra del progetto, navighiamo alla directory res ed effettuiamo un click destro sulla cartella res:



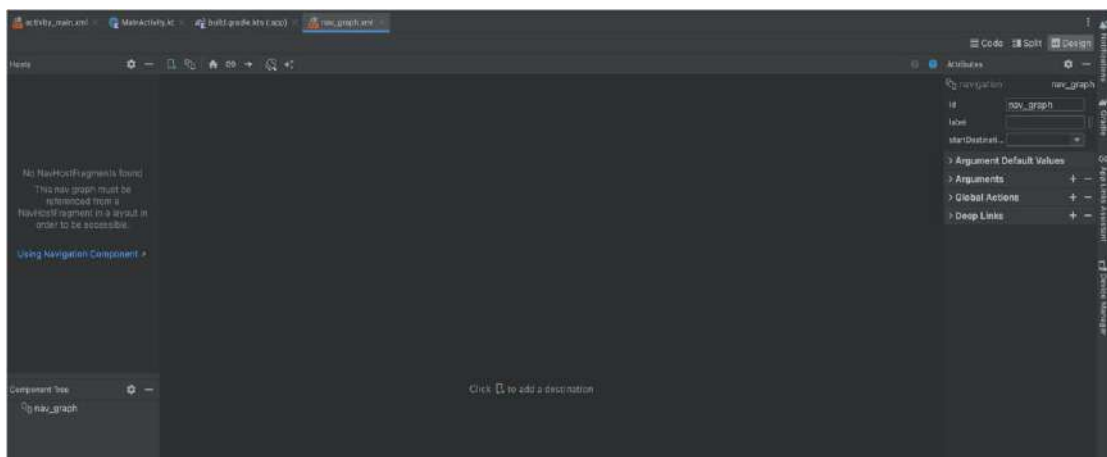
2. Selezionare la voce "New" → "Android resource file" dal menu contestuale:



3. Selezionare la voce “Resource type” e settarla a “Navigation”, impostando anche un nome per il file (ad esempio nav_graph):



4. Cliccare “OK” per creare il file.
5. Una volta che il navigation graph è stato creato, si aprirà nel Navigation Editor. Qui potremo aggiungere le destinazioni, definire le connessioni tra loro e configurare proprietà aggiuntive.



Una volta che usiamo i Fragment, è nostro interesse specificare tutte le destinazioni raggiungibili. Le destinazioni dei Fragment sono denotate da un **tag di azione** nel navigation graph. Le azioni possono essere definite direttamente in XML dichiarando l'azione oppure nel Navigation Editor trascinando dalla sorgente alla destinazione. Quest'ultima operazione da automaticamente all'azione un ID nella forma **action_<sourceFragment>_to_<destinationFragment>**.

```
<fragment
    android:id="@+id/welcomeFragment"
    android:name="com.example.android.navigation.WelcomeFragment"
    android:label="fragment_welcome"
    tools:layout="@layout/fragment_welcome" >

    <action
        android:id="@+id/action_welcomeFragment_to_detailFragment"
        app:destination="@id/detailFragment" />

</fragment>
```

Specificare un percorso di destinazione assegna un nome all'azione, ma non la esegue. Per seguire un percorso, dobbiamo usare un **NavController**:

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        val navController = findNavController(R.id.myNavHostFragment)
    }

    fun navigateToDetail() {
        navController.navigate(R.id.action_welcomeFragment_to_detailFragment)
    }
}
```

5.8. Informazioni aggiuntive sulla navigazione in-app

In molti casi, il nostro Fragment di destinazione avrà bisogno di alcuni dati per costruirsi. La componente Navigation ha un plugin del Gradle chiamato **Safe args** che genera semplici classi di oggetti e costruttori per una navigazione sicura e l'accesso a qualsiasi argomento associato. Safe args è fortemente consigliato quando si naviga passando dati tra le destinazioni per assicurare la **sicurezza di tipo** (misura con cui un linguaggio di programmazione previene o avvisa rispetto agli errori di tipo).

Nel file build.gradle del progetto:

```
buildscript {
    repositories {
        google()
    }
    dependencies {
        classpath "androidx.navigation:navigation-safe-args-gradle-plugin:$nav_version"
    }
}
```

Nel file build.gradle dell'app o modulo:

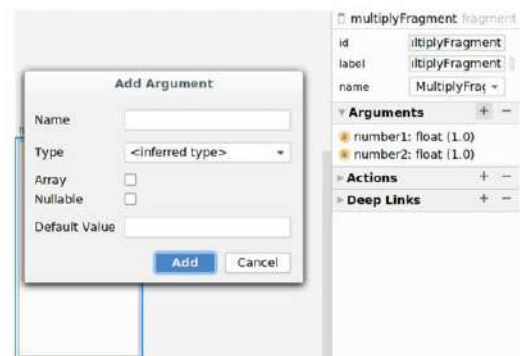
```
apply plugin: "androidx.navigation.safeargs.kotlin"
```

Dopo aver settato il necessario per Safe args, dobbiamo seguire una serie di passi per poter mandare dati tra i Fragment:

1. Creare gli argomenti che il Fragment di destinazione si aspetta.
2. Creare un'azione per collegare da sorgente a destinazione.
3. Impostare gli argomenti nel metodo dell'azione a <Source>FragmentDirections.
4. Navigare rispettando quell'azione e utilizzando il Navigation Controller.
5. Recuperare gli argomenti nel Fragment di destinazione.

Per esempio, supponiamo di avere un'app che prende in un Fragment due numeri e li manda al Fragment successivo per moltiplicarli. Definiamo gli argomenti nel Fragment di destinazione che li riceverà. Android Studio mette a disposizione un modo agevole per creare argomenti per un Fragment all'interno del Navigation Editor, il quale crea il codice XML nel nostro navigation graph per noi.

```
<fragment
    android:id="@+id/multiplyFragment"
    android:name="com.example.arithmetic.MultiplyFragment"
    android:label="MultiplyFragment" >
    <argument
        android:name="number1"
        app:argType="float"
        android:defaultValue="1.0" />
    <argument
        android:name="number2"
        app:argType="float"
        android:defaultValue="1.0" />
</fragment>
```

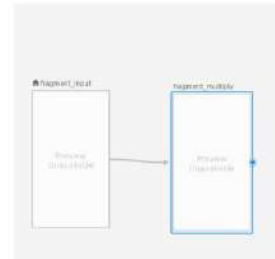


In nav_graph.xml:

```
<fragment
    android:id="@+id/fragment_input"
    android:name="com.example.arithmetic.InputFragment">

    <action
        android:id="@+id/action_to_multiplyFragment"
        app:destination="@id/multiplyFragment" />

</fragment>
```



In InputFragment.kt:

```
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    binding.button.setOnClickListener {
        val n1 = binding.number1.text.toString().toFloatOrNull() ?: 0.0
        val n2 = binding.number2.text.toString().toFloatOrNull() ?: 0.0

        val action = InputFragmentDirections.actionToMultiplyFragment(n1, n2)
        view.findNavController().navigate(action)
    }
}

class MultiplyFragment : Fragment() {
    val args: MultiplyFragmentArgs by navArgs()
    lateinit var binding: FragmentMultiplyBinding
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        val number1 = args.number1
        val number2 = args.number2
        val result = number1 * number2
        binding.output.text = "${number1} * ${number2} = ${result}"
    }
}
```

In this case, we update a TextView with the product of the two numbers.

È possibile fare uso di molti altri tipi oltre a Float:

Type	Type Syntax app:argType=<type>	Supports Default Values	Supports Null Values
Integer	"integer"	Yes	No
Float	"float"	Yes	No
Long	"long"	Yes	No
Boolean	"boolean"	Yes ("true" or "false")	No
String	"string"	Yes	Yes
Array	above type + "[]" (for example, "string[]" "long[]")	Yes (only "@null")	Yes
Enum	Fully qualified name of the enum	Yes	No
Resource reference	"reference"	Yes	No

Oltre ai tipi, possiamo usare anche le classi:

Type	Type Syntax app:argType=<type>	Supports Default Values	Supports Null Values
Serializable	Fully qualified class name	Yes (only "@null")	Yes
Parcelable	Fully qualified class name	Yes (only "@null")	Yes

La **serializzazione** è il processo tramite cui si prende lo stato di un oggetto e lo si converte in un flusso di dati per la trasmissione:

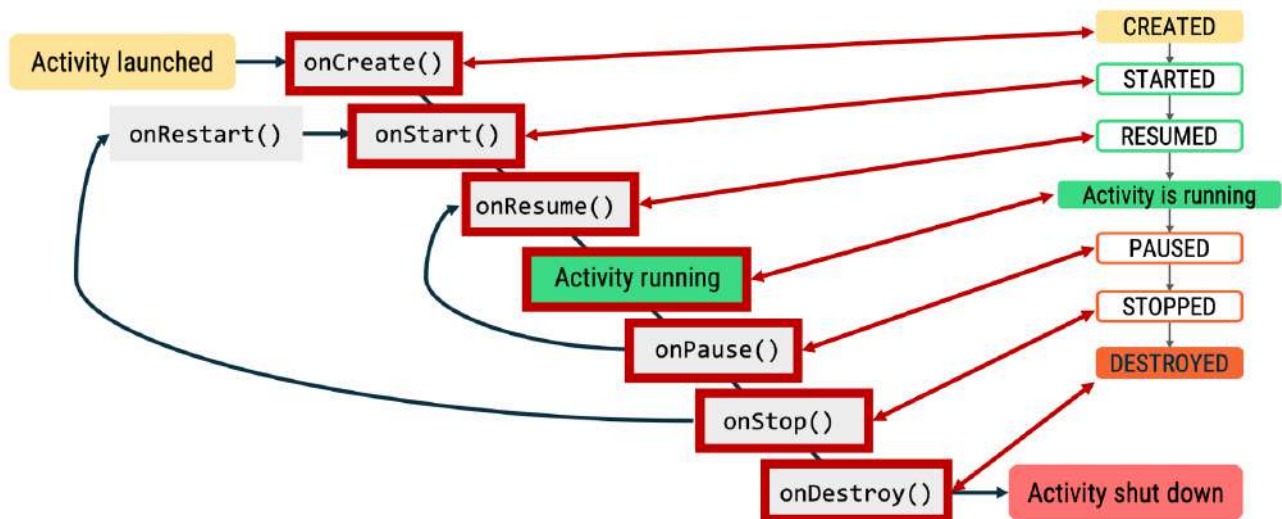
- **Serializable** è l'interfaccia che si implementa per una pura classe JVM.
- **Parcelable** applica anch'essa la serializzazione, ma in un modo di gran lunga più ottimizzato per Android.

5.9. Ciclo di vita delle Activity/Fragment

È importante conoscere gli stati del ciclo di vita in modo tale che lo sviluppatore possa implementare dei comportamenti appropriati nell'app in base a ciò che si aspetta l'utente. È responsabilità dello sviluppatore dell'app gestire i cambi degli stati in modo appropriato e senza causare crash o sprecare risorse di sistema. Un esempio di ciò che un buon sviluppatore dovrebbe fare è preservare i dati e lo stato dell'utente quando:

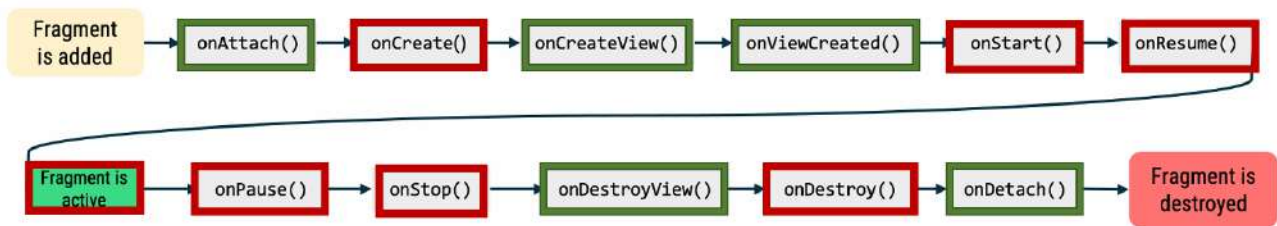
- Esce temporaneamente dall'app e ritorna.
- È interrotto da un altro evento (ad esempio una chiamata telefonica).
- Ruota il dispositivo.

Il framework Android mette a disposizione metodi callback per sapere quando un componente (Activity/Fragment) sta entrando in un certo stato del suo ciclo di vita. Vediamo in primis il ciclo di vita e gli stati di una Activity:

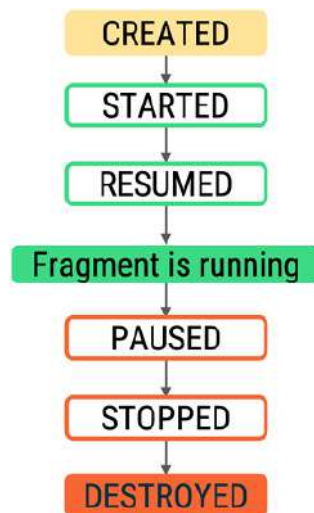


State	Callbacks	Meaning	Description
Created	onCreate()	Activity is being initialized.	Activity is created, the inflation of UI is done, and other app startup logic is performed
Started	onStart()	Activity is visible to the user.	Activity becomes visible to the user
Resumed	onResume()	Activity has input focus.	Activity gains input focus , so the user can interact with the activity. The activity stays in resumed state until system triggers activity to be paused
Paused	onPause()	Activity does not have input focus.	Activity has lost focus (not in foreground). Or, the Activity is still visible, but user is not actively interacting with it
Stopped	onStop()	Activity is no longer visible.	Release resources that aren't needed anymore. Moreover, save any persistent state that the user is in the process of editing so they don't lose their work
Destroyed	onDestroy()	Activity is destroyed.	Perform any final cleanup of resources. Don't rely on this method to save user data (do that earlier)

Passiamo adesso al ciclo di vita e agli stati di un Fragment:



Alcuni metodi di callback sono uguali a quelli delle Activity (rettangoli rossi), mentre altri sono nuovi (rettangoli verdi). Per quanto riguarda gli stati invece rimangono uguali:



State	Callbacks	Meaning	Description
Initialized	<code>onAttach()</code>	Fragment is attached to host.	Called when a fragment is attached to a context .
Created	<code>onCreate()</code>	Fragment is being initialized.	-
	<code>onCreateView()</code>	Fragment's layout is being initialized.	Called to create the view hierarchy associated with the fragment. Recommend for only inflating the layout in this method, and moving logic that modifies the returned View to <code>onViewCreated()</code> .
	<code>onViewCreated()</code>	Fragment's layout is initialized.	Called when view hierarchy has already been created . Perform any remaining initialization here.
Started	<code>onStart()</code>	Fragment is started and visible.	-
Resumed	<code>onResume()</code>	Fragment has input focus.	-
Paused	<code>onPause()</code>	Fragment no longer has input focus.	-
Stopped	<code>onStop()</code>	Fragment is not visible.	-
Destroyed	<code>onDestroyView()</code>	Fragment's layout is destroyed.	Called when view hierarchy of fragment is removed .
	<code>onDestroy()</code>	Fragment's resources are destroyed.	-
	<code>onDetach()</code>	Fragment is removed from host.	Called when fragment is no longer attached to the host .

6. Architettura di un'app Android

Una tipica app Android contiene molteplici componenti, incluse Activity e Fragment. Nello sviluppo di un'app bisogna tenere bene a mente che i dispositivi mobile hanno risorse limitate, quindi, in un qualunque momento, il sistema potrebbe decidere di killare alcuni processi dell'app per fare spazio ad altri. Date le condizioni di questo ambiente, è possibile che i componenti dell'applicazione vengano lanciati singolarmente e in ordine sparso e che il sistema operativo o l'utente possano distruggerli in qualsiasi momento. Visto che questi eventi non sono sotto il nostro controllo, non dovremmo salvare o memorizzare qualsiasi dato o stato dell'applicazione nelle componenti della stessa, e i componenti dell'app non dovrebbero dipendere l'uno dall'altro. Ma se non dobbiamo usare i componenti dell'app, come ne impostiamo il design? È sicuramente importante definire un'**architettura** che permetta all'applicazione di godere di scalabilità, che ne aumenti la robustezza e che la renda più facile da testare. L'architettura di un'app definisce i confini tra le parti dell'app e le responsabilità che ogni parte dovrebbe avere. L'architettura di un'app deve seguire pochi specifici principi:

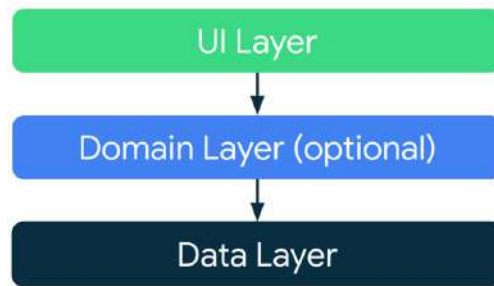
- **Separazione dei problemi:** è un comune errore scrivere tutto il codice in una Activity o Fragment. Mantenendo queste classi più snelle possibile, possiamo evitare molti problemi legati al ciclo di vita del componente e migliorare la testabilità di queste classi.
- **UI controllata dai modelli di dati:** i modelli di dati rappresentano i dati di un'app. Sono indipendenti dagli elementi dell'UI e da altri componenti nell'app.
- **Singola sorgente di verità:** quando un nuovo tipo di dato è definito nell'app, dovremmo assegnargli un Single Source Of Truth (pattern SSOT). L'SSOT è il proprietario di quei dati e solo lui potrà modificarli o mutarli. Per farlo, l'SSOT espone i dati utilizzando un tipo immutabile e, per modificare i dati, espone funzioni o riceve eventi che altri tipi possono chiamare.

Ma perché abbiamo bisogno di una buona architettura?

- Una buona architettura migliora la manutenibilità, la qualità e la robustezza dell'intera app.
- Permette all'app di essere scalabile e molte persone (o molti team) possono contribuire allo stesso lavoro con conflitti di codice minimi. Poiché l'architettura conferisce coerenza al progetto, i nuovi membri del team possono rapidamente prendere confidenza ed essere più efficienti in meno tempo.
- È più facile da testare, poiché incoraggia l'uso di tipi che sono generalmente più semplici da controllare.
- I bug possono essere investigati in modo metodico con processi ben definiti.

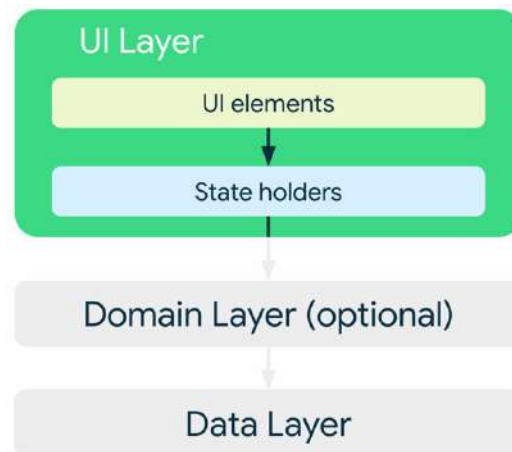
Considerando i comuni principi architetturali, ogni applicazione dovrebbe almeno avere due livelli:

- **UI layer:** mostra i dati dell'applicazione sullo schermo.
- **Data layer:** contiene la logica di business dell'app e si occupa di esporre i dati dell'applicazione.
- **Domain layer** (opzionale): semplifica e permette il riuso delle interazioni tra l'UI layer e il Data layer.

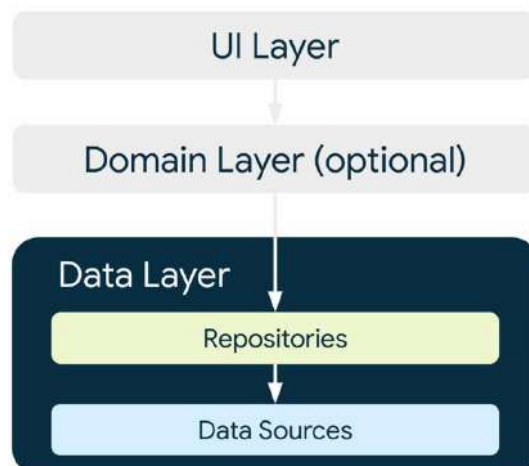


Il ruolo dell'UI layer (o **layer di presentazione**) è di mostrare i dati dell'applicazione sullo schermo. Ogni volta che i dati cambiano, o per l'interazione utente (come il premere un bottone) oppure per input esterni (risposta della rete), l'UI dovrebbe aggiornarsi per riflettere i cambiamenti. L'UI layer è composto da due cose:

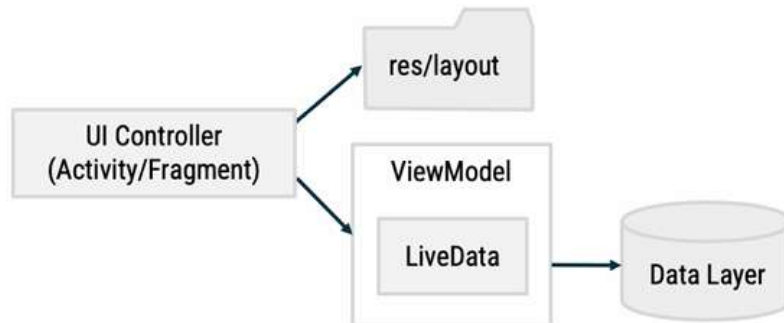
- **Elementi UI** che mostrano i dati sullo schermo (elementi costruiti con le View).
- **State holders** che trattengono i dati, li espongono all'UI e ne gestiscono la logica (ad esempio un ViewModel).



Il data layer dell'app contiene la **logica di business**, ovvero ciò che dà valore alla nostra app. La logica di business è fatta di regole che determinano come la nostra app crei, immagazzini e modifichi i dati. Il data layer è fatto di **repositories**, ognuno dei quali può contenere da 0 a molte sorgenti di dati. Dovremmo creare una classe repository per ogni differente tipo di dato che vogliamo gestire nella nostra app. Ogni classe di sorgente dati dovrebbe avere la responsabilità di lavorare con una sola sorgente di dati, la quale può essere un file, una sorgente di rete o un database locale.



Android Jetpack è un insieme di librerie che semplificano lo sviluppo di app Android, assicurando che siano facili da mantenere e retrocompatibili con le versioni precedenti di Android. Jetpack comprende inoltre le librerie del pacchetto androidx.*. La guida di Jetpack all'architettura di un'app parla dei comuni principi architetturali da seguire. Uno di questi è la separazione dei problemi (di cui abbiamo già discusso brevemente).



Questa grafica non mostra uno specifico pattern di design da seguire, bensì un modo di base per separare e isolare i “posti dove vengono fatte le cose”. L'Activity o Fragment è responsabile di mostrare i dati e catturare l'input utente oltre agli eventi di sistema Android, comportandosi di fatto come un controller per l'UI. Il ViewModel contiene tutti i dati necessari a disegnare l'UI e le funzioni che la mantengono aggiornata. A breve parleremo di come ciò avvenga grazie ai LiveData.

6.1. UI layer: ViewModel

Il framework Android gestisce i cicli di vita di controller UI come Activity e Fragment. Il framework potrebbe decidere di distruggere o ricreare un controller UI in risposta alle interazioni utente o a eventi di sistema che vanno oltre il nostro controllo. Se ciò avviene, ogni dato temporaneo salvato nel controller viene perso. Poiché i controller UI sono in primo luogo responsabili di mostrare le informazioni all'utente e di gestire gli eventi di input utente, non vogliamo addossargli la responsabilità aggiuntiva di gestione dei dati. Per questo motivo abbiamo bisogno di un'entità separata per gestire i dati, ovvero un **ViewModel**.

Per abilitare i ViewModel nella nostra app, dobbiamo aggiungere alcune dipendenze al file Gradle del nostro modulo:

```
dependencies {
    implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:$lifecycle_version"
    implementation "androidx.activity:activity-ktx:$activity_version"
}
```

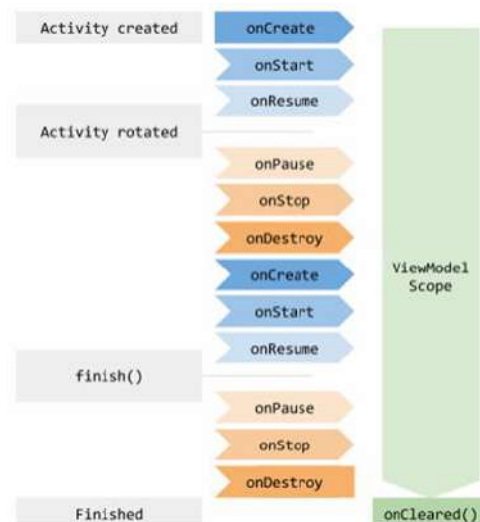
Un ViewModel:

- Prepara e gestisce i dati per l'UI, mentre l'Activity o Fragment mostra i dati dal ViewModel.
- Non dovrebbe mantenere riferimenti diretti alle View nella gerarchia delle View o all'Activity/Fragment stesso.
- Separa la proprietà dei dati dalla logica del controller UI.

Gli oggetti ViewModel:

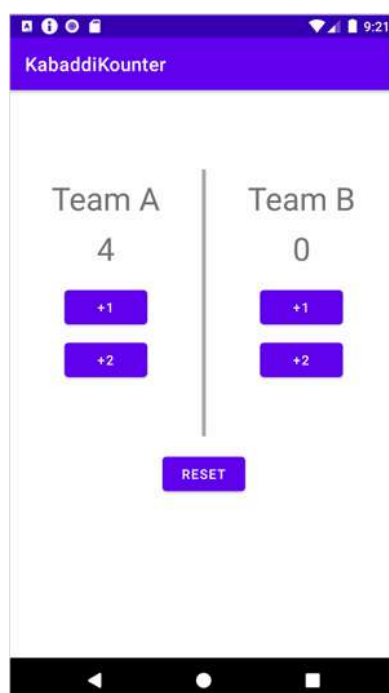
- Sono assegnati a un ciclo di vita (Activity e Fragment hanno cicli di vita).
- Sono automaticamente conservati durante le modifiche alla configurazione, in modo che i dati che contengono siano immediatamente disponibili per l'Activity o l'istanza del Fragment successiva.
- Rimangono in memoria finché il ciclo di vita a cui sono assegnati non cessa definitivamente.

Il seguente diagramma mostra il tempo di vita di un ViewModel in relazione all'Activity a cui è associato, man mano che questa va incontro ai suoi vari cambiamenti del ciclo di vita:



Da notare che non dobbiamo più manualmente salvare e ripristinare lo stato da un Bundle perché possiamo recuperare la stessa istanza del ViewModel anche dopo cambi di configurazione (come la rotazione dello schermo) e ricreare di nuovo l'UI. Il ViewModel rimane in memoria finché l'Activity non termina e, in seguito, il metodo onCleared() del ViewModel viene chiamato per ripulire le risorse.

Per vedere come un ViewModel viene usato in un'app, prendiamo in esempio la seguente applicazione "KabaddiKounter":



Ogni team può incrementare i suoi punti cliccando i tasti +1 e +2. Prima di mostrare il codice per creare un ViewModel, bisogna dire che ViewModel è una classe astratta in Kotlin, quindi non può essere istanziata ma deve essere sottoclassata. Prima di tutto creiamo una classe ScoreViewModel che estende la classe astratta ViewModel:

```
class ScoreViewModel : ViewModel() {
    var scoreA : Int = 0
    var scoreB : Int = 0
    fun incrementScore(isTeamA: Boolean) {
        if (isTeamA) {
            scoreA++
        }
        else {
            scoreB++
        }
    }
}
```

ScoreViewModel immagazzina due risultati interi, uno per il team A e l'altro per il team B. Nella MainActivity carichiamo e usiamo il ViewModel:

```
class MainActivity : AppCompatActivity() {
    // Delegate provided by androidx.activity.viewModels
    val viewModel: ScoreViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        val scoreViewA: TextView = findViewById(R.id.scoreA)
        scoreViewA.text = viewModel.scoreA.toString()
    }
}
```

Qui la dicitura “delegate” si riferisce al concetto di delegare la responsabilità di eseguire una specifica funzione a un'altra entità o metodo. Viene utilizzato il metodo di delega **by viewModels()** fornito dalla libreria `androidx.activity.viewModels` per ottenere un'istanza del ViewModel. Questo semplifica l'uso del ViewModel, eliminando la necessità di gestirlo manualmente. La delega automatizza il processo di creazione e gestione del ciclo di vita del ViewModel. La parola chiave **by** in Kotlin può essere interpretata come "fornito da" (provided by). In Kotlin, una proprietà dichiarata come `val` ha un getter predefinito (per ottenere il valore), mentre una proprietà `var` ha sia un getter che un setter (per ottenere e impostare il valore). Questi metodi getter e setter sono generati automaticamente per ogni proprietà, e non c'è bisogno di definirli esplicitamente. Invece, quando si utilizza `by`, si sta dichiarando che l'implementazione del getter o del getter/setter per quella proprietà è **fornita da un'altra funzione o entità** (cioè è stata delegata). Quindi, anziché utilizzare i metodi getter e setter predefiniti, si affida il compito di gestire la proprietà a una funzione esplicita fornita dopo `by`.

Ritornando all'esempio, aggiungiamo un click listener sul bottone +1 per il team A, il quale incrementa il punteggio chiamando il ViewModel. In seguito, aggiorniamo la TextView con il nuovo punteggio. Nel metodo `onCreate()` del MainActivity:

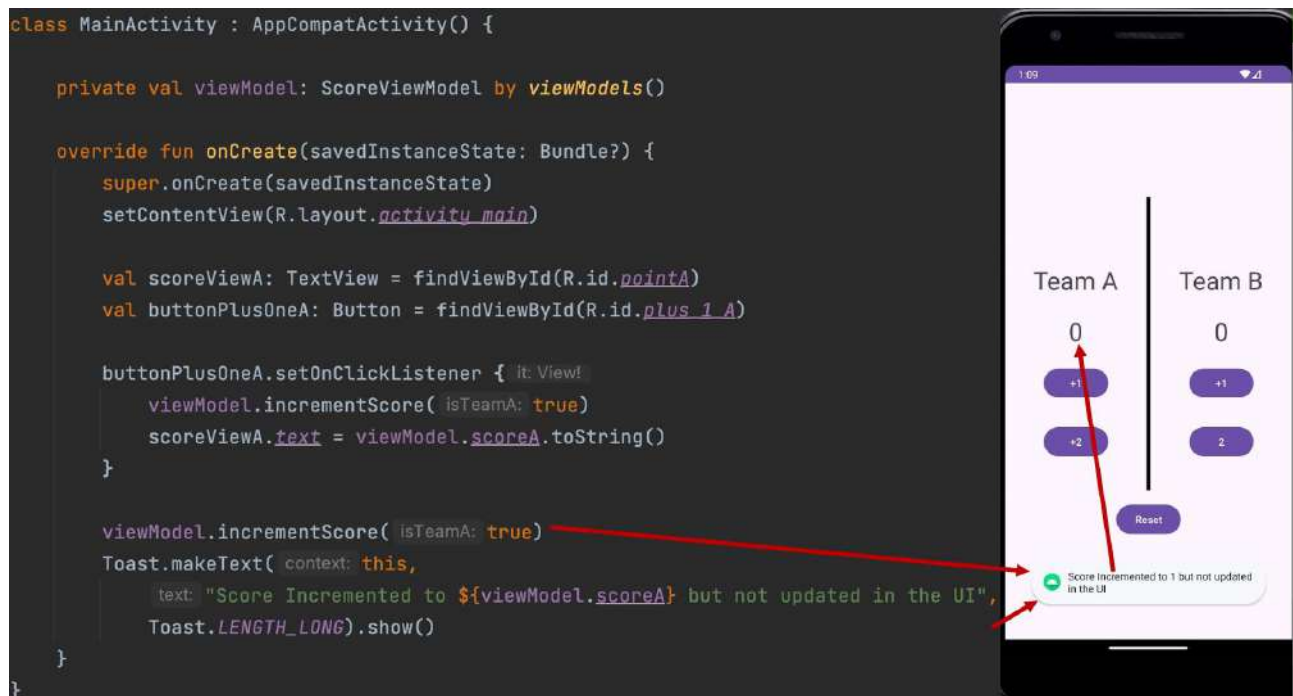
```

val scoreViewA: TextView = findViewById(R.id.scoreA)
val plusOneButtonA: Button = findViewById(R.id.plusOne_teamA)

plusOneButtonA.setOnClickListener {
    viewModel.incrementScore(true)
    scoreViewA.text = viewModel.scoreA.toString()
}

```

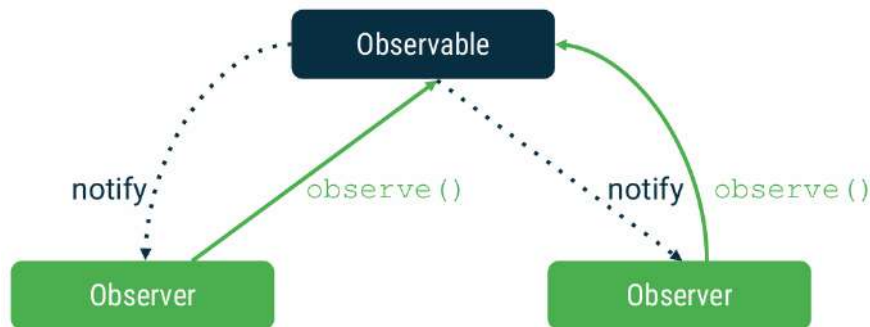
Potrebbe esserci un problema in questo approccio. Supponiamo che il programmatore abbia scritto codice in modo tale che `viewModel.incrementScore(true)` possa essere invocato fuori dal `setOnClickListener` oppure che, dopo averlo invocato, non vi sia uno statement del tipo `binding.scoreView.text = ...`:



Come si può notare, il codice aggiornerebbe solo l'istanza dell'oggetto (incrementando il punteggio del corrispondente team) ma il valore nell'interfaccia non sarebbe aggiornato: solo il `ViewModel` cambia ma non il valore nell'UI.

6.2. UI layer: ViewModel + LiveData

L'**Observer Design Pattern** (modello progettuale osservatore) è un modello in cui un oggetto (soggetto o **osservabile**) mantiene una lista di oggetti dipendenti (**osservatori**) da notificare quando ci sono cambiamenti di stato nel soggetto. Gli osservatori ricevono le modifiche di stato dal soggetto e eseguono il codice appropriato, inoltre possono essere aggiunti o rimossi in qualsiasi momento.



L'osservabile è di fatto il **LiveData** e gli osservatori possono essere aggiunti e impiegati per stare in ascolto di cambiamenti nei dati.

LiveData è un contenitore di dati consapevole del ciclo di vita e che può essere osservato. È essenzialmente un "wrapper" (involucro) attorno a qualsiasi tipo di dato. Ad esempio, un oggetto `LiveData<Int>` contiene un valore intero (`Int`). Poiché **LiveData** funge da involucro attorno ai dati, possiamo osservare le modifiche su tali dati. Gli **osservatori** in questo contesto sarebbero i controller dell'interfaccia utente (UI), come le **Activity** o i **Fragment**. Questi vogliono sapere quando i dati sottostanti cambiano, in modo da aggiornare l'interfaccia utente.

```
observe(owner: LifecycleOwner, observer: Observer)
removeObserver(observer: Observer)
```

È importante notare che, nella firma del metodo, un osservatore viene aggiunto insieme a un **LifecycleOwner**. L'osservatore riceverà aggiornamenti solo quando il **LifecycleOwner** (proprietario del ciclo di vita) si trova in uno stato attivo.

LiveData non ha metodi pubblici per aggiornare i dati memorizzati (possiamo pensarlo come read-only), mentre **MutableLiveData** mette a disposizione il metodo pubblico **setValue(T)**:

<code>LiveData<T></code>	<code>MutableLiveData<T></code>
• <code>getValue()</code>	• <code>getValue()</code> • <code>setValue(value: T)</code>

Di solito, **MutableLiveData** viene utilizzata come variabile privata nel **ViewModel**, il quale poi espone un'istanza di oggetto **LiveData** immutabile agli osservatori, come l'interfaccia utente (UI). In questo modo, si applica il principio di separazione dei problemi, in quanto il **ViewModel** è responsabile di modificare i dati e di eseguire la logica, mentre l'interfaccia utente (UI) reagisce semplicemente ai cambiamenti dei dati.

Tornando all'esempio di app KabaddiKounter:

```
class ScoreViewModelLiveData : ViewModel() {  
    private val _scoreA = MutableLiveData<Int>(value: 0)  
    val scoreA: LiveData<Int>  
        get() = _scoreA  
  
    private val _scoreB = MutableLiveData<Int>(value: 0)  
    val scoreB: LiveData<Int>  
        get() = _scoreB  
  
    fun incrementScore(isTeamA: Boolean){  
        if (isTeamA){  
            _scoreA.value = _scoreA.value!! + 1  
        } else {  
            _scoreB.value = _scoreB.value!! + 1  
        }  
    }  
}
```

Noi non vogliamo che un oggetto esterno o un osservabile possa cambiare i risultati. Notiamo che `_scoreA` è inizializzato come `MutableLiveData` che contiene un `Int` pari a 0. Quando sarà il momento di cambiare il risultato, dovremo modificare solo `_scoreA`. Se un qualsiasi chiamante esterno prova ad accedere alla proprietà pubblica `_scoreA`, la classe ritornerà `_scoreA` come `LiveData` immutabile. Detto questo, modifichiamo in modo appropriato `MainActivity.kt`:

```
class MainActivity : AppCompatActivity() {  
    val viewModel: ScoreViewModelLiveData by viewModels()  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        val scoreViewA: TextView = findViewById(R.id.scoreA)  
        val buttonPlusOneA: Button = findViewById(R.id.plus_1_A)  
  
        val scoreA_Observer = Observer<Int> {  
            newValue -> {  
                scoreViewA.text = newValue.toString()  
            }  
        }  
  
        viewModel.scoreA.observe(owner: this, scoreA_Observer)  
  
        buttonPlusOneA.setOnClickListener { it: View? -> {  
            viewModel.incrementScore(isTeamA: true)  
        }  
  
        viewModel.incrementScore(isTeamA: true)  
        Toast.makeText(context: this,  
            text: "Score Incremented to ${viewModel.scoreA} but not updated in the UI",  
            Toast.LENGTH_LONG).show()  
    }  
}
```

Create an object of the Class that contains the LiveData

Create observer to update team A score on screen

Add the observer onto scoreA LiveData in ViewModel

The update of UI was removed

```

class MainActivity : AppCompatActivity() {

    val viewModel: ScoreViewModelLiveData by viewModel()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val scoreViewA: TextView = findViewById(R.id.pointA)
        val buttonPlusOneA: Button = findViewById(R.id.plus_1_A)

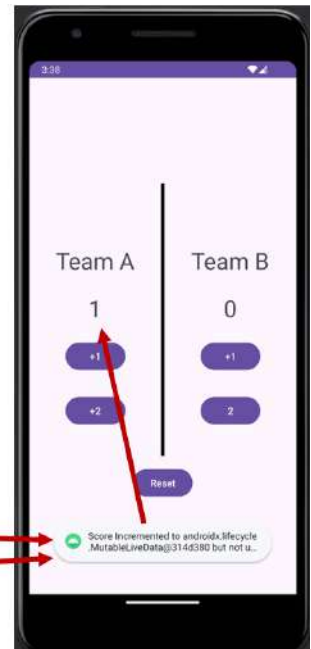
        val scoreA_Observer = Observer<Int> {
            newValue ->
            scoreViewA.text = newValue.toString()
        }

        viewModel.scoreA.observe(owner: this, scoreA_Observer)

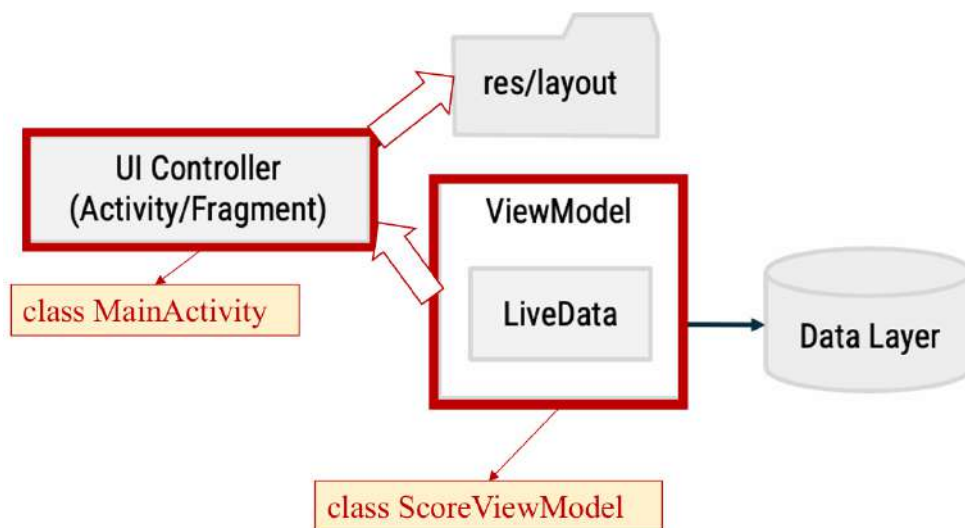
        buttonPlusOneA.setOnClickListener { it: View?
            viewModel.incrementScore(isTeamA: true)
        }

        viewModel.incrementScore(isTeamA: true)
        Toast.makeText(context: this,
            text: "Score Incremented to ${viewModel.scoreA} but not updated in the UI",
            Toast.LENGTH_LONG).show()
    }
}

```



Ciò che fa il ViewModel è dire al controller di aggiornare il layout:



Il flusso logico seguito è quindi il seguente:



Sarebbe più semplice se l'istanza del ViewModel comunicasse direttamente con le View, senza appoggiarsi ai controller UI come intermediari.



Ciò è possibile passando gli oggetti ViewModel al **Data Binding**, in modo tale da automatizzare alcune delle comunicazioni tra le View e gli oggetti ViewModel. Il programmatore è libero di scegliere l'approccio da lui preferito, ma dovrebbe tenere bene a mente che la seconda modalità è più efficiente perché non fa uso di entità intermedie. Vediamo quindi adesso come usare il Data Binding con ViewModel.

6.3. UI layer: ViewModel + LiveData + Data Binding

Prima di tutto, è necessario definire un **layout di data binding**. I file di layout di data binding sono leggermente diversi e iniziano con un **tag radice**, seguito da un **elemento data** e un **elemento View radice**.

```
<?xml version="1.0" encoding="utf-8"?>
<layout xmlns:android="http://schemas.android.com/apk/res/android">
    <data>
        <variable name="user" type="com.example.User"/>
    </data>
    <LinearLayout
        android:orientation="vertical"
        android:layout_width="match_parent"
        android:layout_height="match_parent">
        <TextView android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@{user.firstName}"/>
        <TextView android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@{user.lastName}"/>
    </LinearLayout>
</layout>
```

```
<?xml version="1.0" encoding="utf-8"?>
<layout xmlns:android="http://schemas.android.com/apk/res/android">
    <data>
        <variable name="user" type="com.example.User"/>
    </data>
    <LinearLayout
        android:orientation="vertical"
        android:layout_width="match_parent"
        android:layout_height="match_parent">
        <TextView android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@{user.firstName}"/>
        <TextView android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@{user.lastName}"/>
    </LinearLayout>
</layout>
```

La variabile User (un'istanza della data class User) all'interno dei dati descrive una proprietà che può essere usata all'interno di questo layout.

```
data class User(val firstName: String, val lastName: String)
```

Le espressioni all'interno del layout sono scritte nelle proprietà degli attributi usando la sintassi `@{}`. Nell'esempio, il testo della TextView è settato alla proprietà `firstName` della variabile `user`.

In seguito, è necessario generare una **classe di binding**. Per ogni file di layout viene generata una classe di binding. Di default, il nome della classe è basato sul nome del file di layout convertito in Pascal case, con l'aggiunta del suffisso Binding:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    val binding: ActivityMainBinding = DataBindingUtil.setContentView(
        this, R.layout.activity_main)

    binding.user = User("Test", "User")
}
```

Le **espressioni di Binding** offrono la possibilità di includere operatori:

- Mathematical: + - / * %
- String concatenation: +
- Logical: && ||
- Binary: & | ^
- Unary: + - ! ~
- Shift: >> >>> <<
- Comparison: == > < >= <= (the < needs to be escaped as <)
- instanceof
- Grouping: ()
- Literals, such as character, String, numeric, null
- Cast
- Method calls
- Field access
- Array access: []
- Ternary operator: ?:

```
android:text="@{String.valueOf(index + 1)}"
android:visibility="@{age > 13 ? View.GONE : View.VISIBLE}"
android:transitionName="@{"image_" + id}"
```

E di lavorare con le collections:

```
<data>
    <import type="android.util.SparseArray"/>
    <import type="java.util.Map"/>
    <import type="java.util.List"/>
    <variable name="list" type="List<String>"/>
    <variable name="sparse" type="SparseArray<String>"/>
    <variable name="map" type="Map<String, String>"/>
    <variable name="index" type="int"/>
    <variable name="key" type="String"/>
</data>

...
android:text="@{list[index]}"
...
android:text="@{sparse[index]}"
...
android:text="@{map[key]}"
```


Concludiamo quindi l'esempio dell'app KabaddiKounter:

1. Innanzitutto, specifichiamo il ViewModel direttamente nel file XML usando il data tag di un binding:

```
<layout>
    <data>
        <variable>
            name="viewModel"
            type="com.example.kabaddikounter.ScoreViewModel" />
        </data>
    <ConstraintLayout ...>
        <TextView ...
            android:id="@+id/scoreViewA"
            android:text="@{viewModel.scoreA.toString()}" />
        ...
    </ConstraintLayout>
</layout>
```

2. Colleghiamo il ViewModel al data binding:

```
class MainActivity : AppCompatActivity() {
    val viewModel: ScoreViewModel by viewModels()
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val binding: ActivityMainBinding = DataBindingUtil
            .setContentView(this, R.layout.activity_main)

        binding.viewModel = viewModel
        binding.lifecycleOwner = this
        binding.plusOneButtonA.setOnClickListener {
            viewModel.incrementScore(true)
        }
        ...
    }
}
```

Il LifecycleOwner è l'Activity

3. Lasciamo la classe ScoreViewModel esattamente come prima:

```
class ScoreViewModel : ViewModel() {
    private val _scoreA = MutableLiveData<Int>(0)
    val scoreA : LiveData<Int>
        get() = _scoreA
    private val _scoreB = MutableLiveData<Int>(0)
    val scoreB : LiveData<Int>
        get() = _scoreB
    fun incrementScore(isTeamA: Boolean) {
        if (isTeamA) {
            _scoreA.value = _scoreA.value!! + 1
        } else {
            _scoreB.value = _scoreB.value!! + 1
        }
    }
}
```

L'enorme differenza sta nel fatto che, con il secondo approccio, non abbiamo più bisogno di un oggetto osservatore e non dobbiamo più aggiornare manualmente la TextView del risultato quando il bottone +1 viene cliccato.



6.4. Persistenza

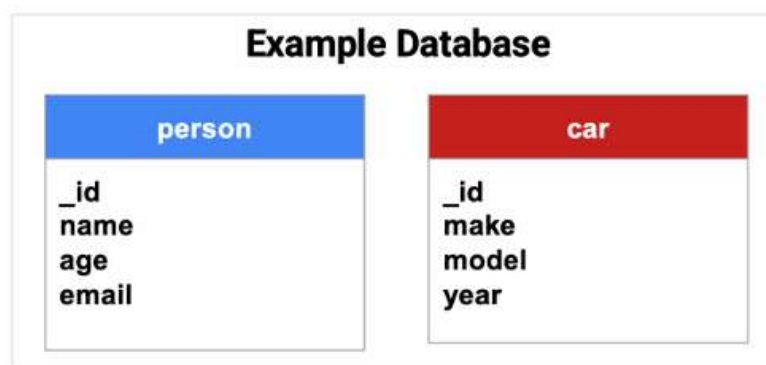
Android utilizza un file system simile ai file system basati su disco di altre piattaforme. Esistono varie opzioni disponibili per salvare i dati dell'app:

- **App-specific files** (File specifici per l'app): Sono file destinati esclusivamente all'uso da parte dell'app. Esempi di questi file includono file di dati strutturati come file JSON, file di testo semplice e file multimediali. Questi file non sono accessibili da altre app.
- **Shared files** (File condivisi): Sono file che l'app intende condividere con altre app, come file multimediali o documenti. Questi file possono essere letti e modificati da altre app.
- **Preferences** (Preferenze): Permettono di salvare dati primitivi privati sotto forma di coppie chiave-valore. Android fornisce l'API **SharedPreferences** per facilitare il salvataggio di questi dati.
- **Databases** (Basi di dati): Consente di memorizzare dati strutturati in un database privato dell'app. Questi dati non sono accessibili da altre app e vengono gestiti internamente dall'app.

Noi concentreremo il nostro studio sui database, ma cosa è un database?

Un database è una raccolta di dati strutturati che possono essere facilmente consultati, ricercati e organizzati, composta da:

- Tabelle
- Righe
- Colonne



Per accedere e modificare un database relazionale ci si serve tipicamente di SQL, il quale permette di:

- **Creare nuove tabelle**
- **Interrogare i dati (query):** `SELECT * from colors;`
- **Inserire nuovi dati:** `INSERT INTO colors VALUES ("red", "#FF0000");`
- **Aggiornare i dati:** `UPDATE colors SET hex="#DD0000" WHERE name="red";`
- **Cancellare i dati:** `DELETE FROM colors WHERE name = "red";`

SQLite è un database relazionale leggero e senza server, è ampiamente utilizzato in applicazioni mobili e incorporate grazie alla sua semplicità e al basso consumo di risorse. Android utilizza di default SQLite per memorizzare i dati dell'applicazione. Su Android SQLite ha le seguenti principali caratteristiche:

- **Dimensioni ridotte:** SQLite è molto leggero e richiede poche risorse di sistema, rendendolo ideale per le applicazioni mobili, dove le risorse hardware possono essere limitate.
- **Transazioni atomiche:** SQLite supporta le transazioni atomiche, il che significa che tutte le operazioni di scrittura avvengono in modo sicuro e affidabile. Ciò evita corruzione dei dati, garantendo che ogni transazione venga completata o annullata interamente.
- **Sintassi SQL completa:** SQLite supporta la sintassi completa di SQL, rendendolo facile da usare per una vasta gamma di applicazioni. Questo permette agli sviluppatori di utilizzare la piena potenza del linguaggio SQL per gestire e interrogare i dati.
- **Supporto per le chiavi esterne:** SQLite supporta le chiavi esterne, che consentono la creazione di relazioni tra tabelle, permettendo una gestione più complessa e strutturata dei dati.

Uno dei principi fondanti dei database SQL è lo **schema**, ovvero una dichiarazione formale di come il database è organizzato. Su Android è spesso utile creare una classe complementare nota come **classe contratto**, la quale descrive esplicitamente la struttura dello schema in modo sistematico e auto documentato. Un buon modo di organizzare una classe contratto è quello di includere definizioni globali per l'intero database al livello radice della classe e creare una classe interna per ogni tabella. Ognuna di queste classi interne enumera le colonne della tabella corrispondente:

```
object FeedReaderContract {  
    // Table contents are grouped together in an anonymous object.  
    object FeedEntry : BaseColumns {  
        const val TABLE_NAME = "entry"  
        const val COLUMN_NAME_TITLE = "title"  
        const val COLUMN_NAME_SUBTITLE = "subtitle"  
    }  
}
```

Una volta definita la struttura del database, dobbiamo implementare metodi che creino e gestiscano il database e le tabelle:

```
private const val SQL_CREATE_ENTRIES =  
    "CREATE TABLE ${FeedEntry.TABLE_NAME} (" +  
        "${BaseColumns._ID} INTEGER PRIMARY KEY," +  
        "${FeedEntry.COLUMN_NAME_TITLE} TEXT," +  
        "${FeedEntry.COLUMN_NAME_SUBTITLE} TEXT)"  
  
private const val SQL_DELETE_ENTRIES = "DROP TABLE IF EXISTS ${FeedEntry.TABLE_NAME}"
```


Proprio come i file che salviamo nella memoria interna del dispositivo, Android ripone il nostro database nella cartella privata della nostra app. I nostri dati saranno sicuri perché, di default, questa area non è accessibile ad altre app o all'utente stesso.

La classe **SQLiteOpenHelper** contiene un utile insieme di API per gestire il nostro database. Quando usiamo questa classe, il sistema esegue le operazioni potenzialmente più lunghe (creazione e aggiornamento del database) solo quando è necessario e non durante l'avvio dell'app. Tutto ciò che dobbiamo fare è chiamare i metodi **getWritableDatabase()** o **getReadableDatabase()**. Per usare SQLiteOpenHelper dobbiamo creare una sottoclasse che sovrascriva (override) i metodi **onCreate()** e **onUpgrade()**, mentre sovrascrivere il metodo **onDowngrade()** non è obbligatorio.

```
class FeedReaderDbHelper(context: Context) : SQLiteOpenHelper(context, DATABASE_NAME, null, DATABASE_VERSION) {
    override fun onCreate(db: SQLiteDatabase) {
        db.execSQL(SQL_CREATE_ENTRIES)
    }
    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        // This database is only a cache for online data, so its upgrade policy is
        // to simply to discard the data and start over
        db.execSQL(SQL_DELETE_ENTRIES)
        onCreate(db)
    }
    override fun onDowngrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        onUpgrade(db, oldVersion, newVersion)
    }
    companion object {
        // If you change the database schema, you must increment the database version.
        const val DATABASE_VERSION = 1
        const val DATABASE_NAME = "FeedReader.db"
    }
}
```

Per accedere al database, dobbiamo istanziare la nostra sottoclasse SQLiteOpenHelper.

```
val dbHelper = FeedReaderDbHelper(context)
// Gets the data repository in write mode
val db = dbHelper.getWritableDatabase
```

// Create a new map of values, where column names are the keys

```
val values = ContentValues().apply {
    put(FeedEntry.COLUMN_NAME_TITLE, title)
    put(FeedEntry.COLUMN_NAME_SUBTITLE, subtitle)
}
```

// Insert the new row, returning the primary key value of the new row

```
val newRowId = db?.insert(FeedEntry.TABLE_NAME, null, values)
```

Creiamo un'istanza della sottoclasse SQLiteOpenHelper e un database scrivibile.

Insieme delle coppie chiave/valore (dove la chiave è il nome della colonna) da inserire nel DB

Inserimento dei valori nella tabella:

- **Arg1**: nome della tabella.
- **Arg2**: indica cosa fare nel caso ContentValues sia vuoto. Se specifichiamo il nome di una colonna, il framework inserisce una riga e setta il valore di quella colonna a null. Facendo come in questo esempio, ossia specificando null come arg2, il framework non inserisce una riga quando non ci sono valori.
- **Arg3**: i valori da inserire.

Il metodo `insert()` ritorna l'ID della nuova riga creata oppure -1 se un qualche tipo di errore si è verificato durante l'inserimento dei dati (ciò potrebbe avvenire se c'è un conflitto con dati preesistenti nel database).

Per leggere dati da un database, dobbiamo usare il metodo **query()**, passandogli il nostro criterio di selezione e le colonne desiderate. I risultati di una query ci vengono ritornati in un **oggetto cursore**:

```
val db = dbHelper.readableDatabase

// Define a projection that specifies which columns from the database
// you will actually use after this query.
val projection = arrayOf(BaseColumns._ID, FeedEntry.COLUMN_NAME_TITLE, FeedEntry.COLUMN_NAME_SUBTITLE)

// Filter results WHERE "title" = 'My Title'
val selection = "${FeedEntry.COLUMN_NAME_TITLE} = ?"
val selectionArgs = arrayOf("My Title")

// How you want the results sorted in the resulting Cursor
val sortOrder = "${FeedEntry.COLUMN_NAME_SUBTITLE} DESC"

val cursor = db.query(
    FeedEntry.TABLE_NAME, // The table to query
    projection,            // The array of columns to return (pass null to get all)
    selection,             // The columns for the WHERE clause
    selectionArgs,         // The values for the WHERE clause
    null,                 // don't group the rows
    null,                 // don't filter by row groups
    sortOrder              // The sort order
)
```

Istanza di un database leggibile.

Colonne che devono essere ritornate dalla query.

Filtrare il risultato per la colonna che ha titolo uguale a "My Title".

Se necessario, si possono filtrare i risultati in base a un qualche criterio.

Esecuzione della query.

Il terzo e quarto argomento (selection e selectionArgs) sono combinati per creare una clausola WHERE. Dato che gli argomenti sono forniti separatamente dalla query di selezione, vengono evasi prima di essere combinati.

Prima di leggere i dati, è necessario chiamare uno dei metodi di movimento del cursore. Dato che inizialmente il cursore parte da -1, dobbiamo spostare il cursore sulla prima riga dei risultati chiamando il metodo **moveToNext()**. Questo metodo posiziona il cursore sulla prima riga e ritorna true finché ci sono righe da leggere, altrimenti ritorna false. Per leggere il valore di una colonna in una riga, possiamo utilizzare uno dei metodi get, come **getString()** o **getLong()**, a seconda del tipo di dato. Per ottenere il valore di una colonna è necessario conoscere il suo indice, che può essere recuperato con i metodi **getColumnIndex()** o **getColumnIndexOrThrow()**. Quest'ultimo lancia un'eccezione se la colonna non esiste, mentre il primo restituisce -1 in caso di errore. Una volta terminata l'iterazione sui risultati, è essenziale chiamare il metodo **close()** sul cursore per rilasciare le risorse associate. Non chiudere il cursore può portare a perdite di memoria.

```
val itemIds = mutableListOf<Long>()
with(cursor) {
    while (moveToNext()) {
        val itemId = getLong(getColumnIndexOrThrow(BaseColumns._ID))
        itemIds.add(itemId)
    }
}
cursor.close()
```

Per cancellare righe da una tabella, dobbiamo fornire un criterio di selezione che identifichi tali righe al metodo **delete()**. Il meccanismo funziona in maniera analoga agli argomenti di selezione del metodo **query()**. Il metodo **delete()** ritorna il numero di righe cancellate:

```
// Define 'where' part of query.
val selection = "${FeedEntry.COLUMN_NAME_TITLE} LIKE ?"
// Specify arguments in placeholder order.
val selectionArgs = arrayOf("MyTitle")
// Issue SQL statement.
val deletedRows = db.delete(FeedEntry.TABLE_NAME, selection, selectionArgs)
```

Quando dobbiamo modificare un sottoinsieme dei valori del nostro database, possiamo usare il metodo **update()**. L'aggiornamento della tabella combina la sintassi di **ContentValues** del metodo **insert()** con quella di **WHERE** del metodo **delete()**:

```
val db = dbHelper.writableDatabase

// New value for one column
val title = "MyNewTitle"
val values = ContentValues().apply {
    put(FeedEntry.COLUMN_NAME_TITLE, title)
}

// Which row to update, based on the title
val selection = "${FeedEntry.COLUMN_NAME_TITLE} LIKE ?"
val selectionArgs = arrayOf("MyOldTitle")
val count = db.update(
    FeedEntry.TABLE_NAME,
    values,
    selection,
    selectionArgs)
```

Dato che **getWritableDatabase()** e **getReadableDatabase()** sono metodi costosi da chiamare quando il database è chiuso, fintantoché ne abbiamo bisogno dovremmo lasciare la connessione del nostro database aperta il più a lungo possibile. Tipicamente, è ottimale chiudere il database nel metodo **onDestroy()** dell'Activity chiamante:

```
override fun onDestroy() {
    dbHelper.close()
    super.onDestroy()
}
```

Fino ad ora abbiamo visto che, per interagire con un DB, dobbiamo effettuare una serie di operazioni che possono essere più o meno complesse a seconda dell'obiettivo. Ma perché è necessario svolgere queste operazioni? Non potremmo scrivere una query in cui i campi da cercare sono già inseriti nella stringa di codice?

Per dare una risposta a questi interrogativi, facciamo un semplice esempio:

Users

ID	Username	Password
1	admin	secret
2	moderator	123
3	guest	password
99	ghost	secret

Username:

Password:

Query

```
tableUsername = getRequestString("Username")
tablePassword = getRequestString("Password")
sqlString = "SELECT *
            FROM Users
            WHERE Username=" + tableUsername +
              "AND Password="+ tablePassword;
```

Nella tabella **Users** abbiamo alcune voci con ID, nome utente e password. Viene mostrato un esempio di un modulo di login con due campi: uno per il nome utente e uno per la password. Quando l'utente inserisce le informazioni, il modulo invia questi dati al server tramite un pulsante "Submit". La query SQL è costruita concatenando il nome utente e la password direttamente nella stringa SQL: la variabile `tableUsername` contiene il valore del nome utente (es. "admin") e la variabile `tablePassword` contiene il valore della password (es. "secret"). La query risultante diventa:

```
SELECT * FROM Users WHERE Username='admin' AND Password='secret'
```

Users

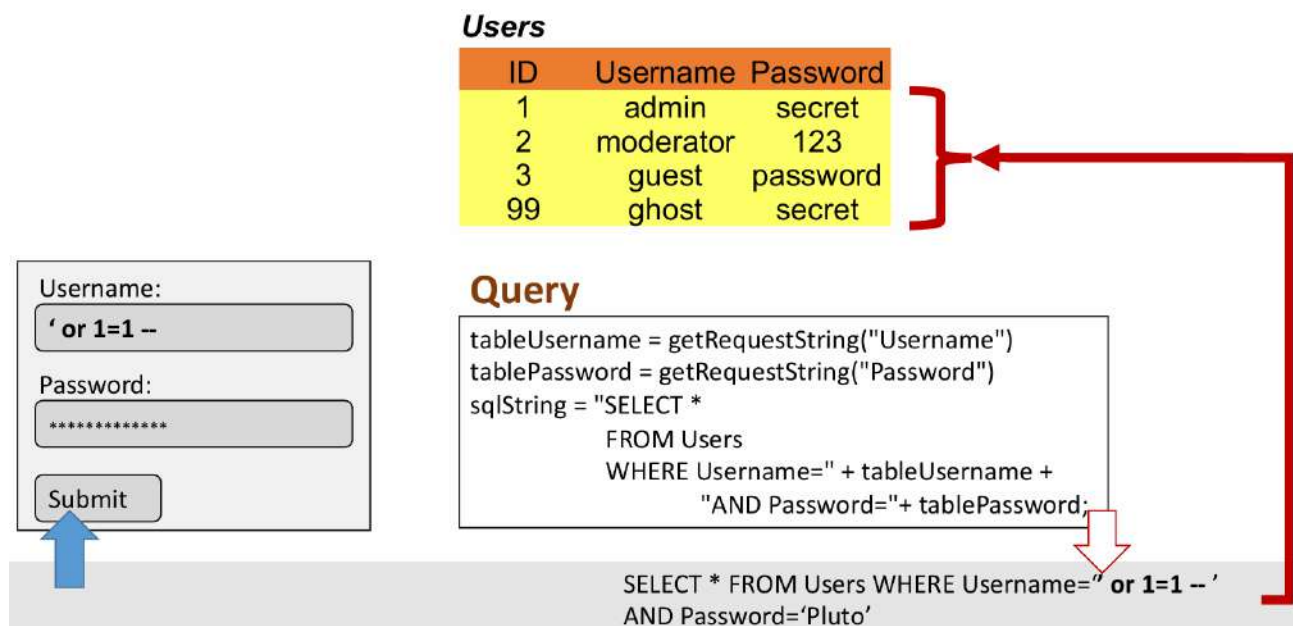
ID	Username	Password
1	admin	secret

Query

```
tableUsername = getRequestString("Username")
tablePassword = getRequestString("Password")
sqlString = "SELECT *
            FROM Users
            WHERE Username=" + tableUsername +
              "AND Password="+ tablePassword;
```

```
SELECT * FROM Users WHERE Username='admin' AND
Password='secret'
```

Questa query è corretta, ma è vulnerabile ad attacchi di **SQL Injection**, un attacco informatico che sfrutta una vulnerabilità nel codice SQL non protetto. Se un utente malintenzionato inserisse input malevolo nei campi username o password, potrebbe manipolare la query per accedere al sistema senza avere credenziali valide. Cambiamo quindi in modo malevolo l'input utente:



Questa query sarebbe sempre vera, perché la condizione '1'=1' è sempre soddisfatta, permettendo all'attaccante di accedere al sistema senza conoscere la password reale. La query ritornerebbe infatti tutte le righe contenute nel DB, poiché non è stata eseguita alcuna **sanificazione dell'input**.

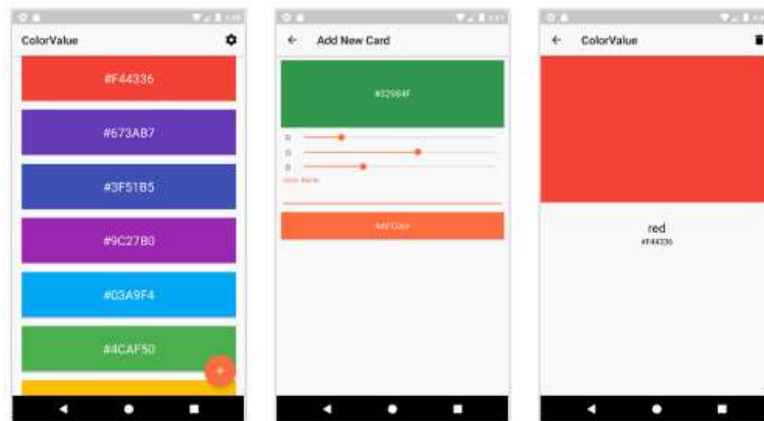
6.5. Room

Android fornisce delle API per aiutarci a creare e gestire un database SQLite direttamente nella nostra app. Interagire direttamente con il database è potentissimo, ma richiede una grande quantità di tempo e attenzione per usarlo in modo appropriato e per evitare errori. Per esempio, non abbiamo a disposizione la verifica al tempo di compilazione delle query SQL non elaborate. Inoltre, dovremo scrivere molto codice **boilerplate** (un testo in linguaggio informatico che è possibile riutilizzare con modifiche minime o nulle in diversi contesti) per convertire le query SQL in oggetti di dati.

Fortunatamente, Android Jetpack ha introdotto la **libreria di persistenza Room**. Room offre un livello di astrazione che ci permette di interagire più facilmente con un database nella nostra app. Nello specifico, Room è una **libreria di mappatura relazionale ad oggetti** che converte dati dalla loro rappresentazione in un database in oggetti che noi possiamo manipolare direttamente nel codice dell'app. Può anche invertire il processo, effettuando un push dei dati dagli oggetti al database. Ci sono tre componenti principali in Room:

- **Entità:** rappresenta una tabella all'interno del database.
- **DAO:** contiene i metodi utilizzati per accedere al database.
- **Database:** contiene il gestore del database (database holder) e rappresenta il punto di accesso principale per la sottostante connessione ai dati relazionali e persistenti dell'app.

Non c'è alcun limite al numero di classi entità o DAO, ma devono essere uniche all'interno del database. Vediamo l'esempio dell'app "ColorValue", che permette di visualizzare una lista di colori, aggiungere nuovi colori e visualizzare informazioni dettagliate sui singoli colori.



Dichiareremo tre classi:

- **Color**: la tabella dei colori.
- **ColorDao**: la classe contenente tutti i metodi.
- **ColorDatabase**: la classe usata come punto di accesso.

Quando lavoriamo con la libreria Room, è necessario usare le **annotazioni**. In generale, le annotazioni sono delle note che spiegano o evidenziano parole o passaggi in un testo. In modo simile, nel contesto dei linguaggi di programmazione, le annotazioni collegano i metadati al nostro codice e forniscono informazioni extra al compilatore. Così come in Java usavamo l'annotazione `@Override` per sovrascrivere i metodi di una superclasse in una sottoclasse, possiamo usare le annotazioni `@Entity`, `@Dao`, `@Database` rispettivamente per contrassegnare una classe Entity, DAO o database. Ogni annotazione può prendere dei parametri e può autogenerare codice per noi:

`@Entity(tableName="colors")`

Un'entità Room include campi per ogni colonna nella corrispondente tabella del database, incluse una o più colonne che fanno la chiave primaria. Il seguente codice fornisce un esempio di una semplice entità che definisce una tabella User con colonne per l'ID, nome e cognome:

```
@Entity
data class User(
    @PrimaryKey val id: Int,

    val firstName: String?,
    val lastName: String?
)
```

Di default, Room usa il nome della classe come nome della tabella del database. Se vogliamo che la tabella abbia un nome diverso, dovremo settare la proprietà **tableName** dell'annotazione `@Entity`:

```
@Entity(tableName = "users")
data class User (
    @PrimaryKey val id: Int,
    @ColumnInfo(name = "first_name") val firstName: String?,
    @ColumnInfo(name = "last_name") val lastName: String?
)
```


Come si nota dall'esempio, se vogliamo specificare dei nomi per le colonne, possiamo aggiungere l'annotazione **@ColumnInfo** al campo e passargli il nome desiderato per la colonna.

Ogni entità Room deve definire una chiave primaria che identifichi univocamente ogni riga nella corrispondente tabella del database. Il modo più diretto per farlo è annotare una singola colonna con **@PrimaryKey**:

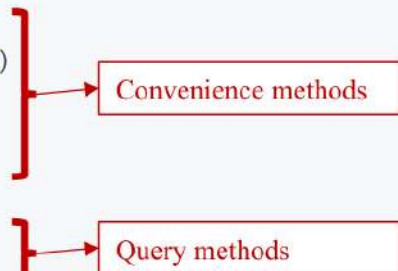
```
@PrimaryKey val id: Int
```

Se abbiamo bisogno di istanze di un'entità che devono essere identificate univocamente da una combinazione di molteplici colonne, possiamo definire una **chiave primaria composta** facendo una lista di tali colonne nella proprietà **primaryKey** di **@Entity**:

```
@Entity(primaryKeys = ["firstName", "lastName"])
data class User(
    val firstName: String?,
    val lastName: String?
)
```

Invece di accedere al DB in modo diretto, possiamo servirci delle classi DAO. Nel DAO, che dichiareremo come interfaccia o classe astratta, andremo a definire le interazioni con il database. Room crea un'implementazione DAO e verifica tutte le nostre query DAO al tempo di compilazione:

```
@Dao
interface UserDao {
    @Insert
    fun insertAll(vararg users: User)
    @Delete
    fun delete(user: User)
    @Query("SELECT * FROM user")
    fun getAll(): List<User>
}
```



The diagram shows two red brackets on the right side of the code. The first bracket groups the `@Insert` and `@Delete` annotations and points to a box labeled "Convenience methods". The second bracket groups the `@Query` annotation and points to a box labeled "Query methods".

Notiamo che l'interfaccia contiene metodi annotati con **@Query**, **@Insert**, **@Update** e **@Delete**, che sono le operazioni che vogliamo eseguire sul nostro database. Room mette a disposizione le **annotazioni di convenienza** per definire metodi che eseguano semplici inserimenti, aggiornamenti e cancellazioni senza che ci sia richiesto di scrivere alcuno statement SQL.

L'annotazione **@Insert** ci permette di definire metodi che inseriscono i loro parametri nella tabella appropriata del database. Il seguente codice mostra esempi di metodi **@Insert** validi che inseriscono uno o più oggetti User nel database:

```
@Dao
interface UserDao {
    @Insert
    fun insertUsers(vararg users: User)

    @Insert
    fun insertBothUsers(user1: User, user2: User)

    @Insert
    fun insertUsersAndFriends(user: User, friends: List<User>)
}
```

Ogni parametro di un metodo `@Insert` deve essere o un'istanza di una classe di entità dati di Room annotata con `@Entity` oppure una collection di istanze di tali classi, ognuna delle quali punta ad un database. Quando un metodo `@Insert` viene chiamato, Room inserisce ogni istanza di entità passata nella corrispondente tabella del database.

L'annotazione `@Update` ci permette di definire metodi che aggiornino specifiche colonne in una tabella del database. Come i metodi `@Insert`, anche i metodi `@Update` accettano istanze di entità dati come parametri:

```
@Dao
interface UserDao {
    @Update
    fun updateUsers(vararg users: User)
}
```

L'annotazione `@Delete` ci permette di definire metodi che cancellino specifiche colonne da una tabella del database. Come i metodi `@Insert`, anche i metodi `@Delete` accettano istanze di entità dati come parametri:

```
@Dao
interface UserDao {
    @Delete
    fun deleteUsers(vararg users: User)
}
```

L'annotazione `@Query` ci permette di scrivere degli statement SQL e di esporli come metodi DAO. Questi metodi sono utili per interrogare i dati dal database della nostra app o quando necessitiamo di eseguire operazioni di inserimento, aggiornamento e cancellazione più complesse.

Vediamo un esempio di una semplice query che ritorna tutti gli utenti dalla tabella user:

```
@Query("SELECT * FROM user")
fun loadAllUsers(): Array<User>
```

Vediamo un esempio di query che ritorna un sottoinsieme di colonne:

```
data class NameTuple(
    @ColumnInfo(name = "first_name") val firstName: String?,
    @ColumnInfo(name = "last_name") val lastName: String?
)
@Query("SELECT first_name, last_name FROM user")
fun loadFullName(): List<NameTuple>
```

Vediamo un esempio di query che accetta dei semplici parametri in input:

```
@Query("SELECT * FROM user WHERE age > :minAge")
fun loadAllUsersOlderThan(minAge: Int): Array<User>

@Query("SELECT * FROM user WHERE age BETWEEN :minAge AND :maxAge")
fun loadAllUsersBetweenAges(minAge: Int, maxAge: Int): Array<User>

@Query("SELECT * FROM user WHERE first_name LIKE :search " +
    "OR last_name LIKE :search")
fun findUserWithName(search: String): List<User>
```

Come evidenziato nell'esempio, ogni parametro che la funzione necessita di passare alla query deve fare riferimento al nome preciso dell'argomento della funzione.

Vediamo un esempio di query che accetta in input una collection di parametri:

```
@Query("SELECT * FROM user WHERE region IN (:regions)")
fun loadUsersFromRegions(regions: List<String>): List<User>
```

Vediamo un esempio di query verso tabelle multiple:

```
@Query(
    "SELECT * FROM book " +
    "INNER JOIN loan ON loan.book_id = book.id " +
    "INNER JOIN user ON user.id = loan.user_id " +
    "WHERE user.name LIKE :userName"
)
fun findBooksBorrowedByUserNameSync(userName: String): List<Book>
```

Vediamo un esempio di query che ritorna una multimap:

```
@Query(
    "SELECT * FROM user" +
    "JOIN book ON user.id = book.user_id"
)
fun loadUserAndBookNames(): Map<User, List<Book>>
```

Infine, il seguente codice definisce una classe AppDatabase che contiene il database:

```
@Database(entities = [User::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}
```

AppDatabase definisce la configurazione del database e serve da punto di accesso principale dell'app ai dati persistenti. La classe del database deve soddisfare le seguenti condizioni:

- La classe deve essere annotata con @Database che include un array di entità che elenca tutte le entità dati associate al database.
- La classe deve essere astratta ed estendere RoomDatabase.
- Per ogni classe DAO associata al database, la classe database deve definire un metodo astratto che abbia zero argomenti e ritorni un'istanza della classe DAO.

Dopo aver correttamente definito l'entità dati, il DAO e l'oggetto database, possiamo usare il seguente codice per creare un'istanza del database:

```
val db = Room.databaseBuilder(
    applicationContext,
    AppDatabase::class.java, "database-name"
).build()
```

In seguito, possiamo usare i metodi astratti di AppDatabase per ottenere un'istanza del DAO:

```
val userDao = db.userDao()
val users: List<User> = userDao.getAll()
```

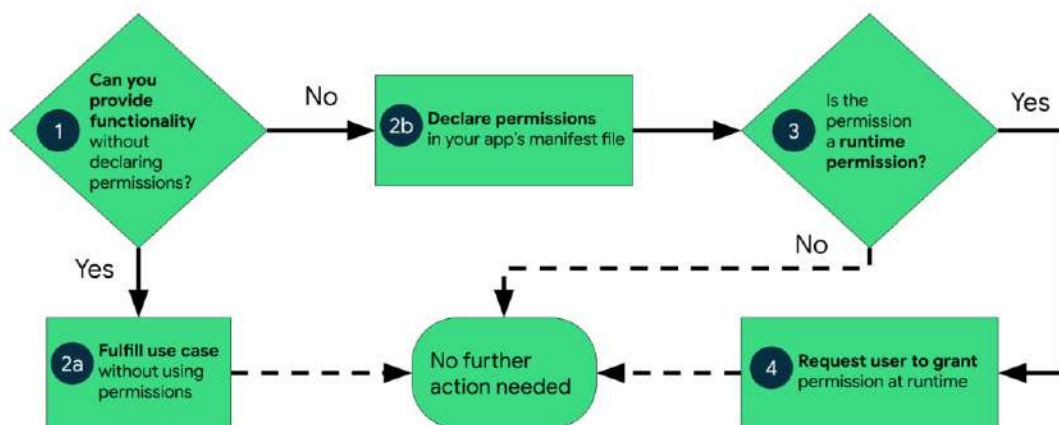
7. Permessi Android e connessione ad Internet

7.1. Permessi

I permessi delle app aiutano a supportare la privacy dell'utente proteggendo l'accesso a:

- **Dati riservati:** per esempio, le informazioni di contatto dell'utente.
- **Azioni limitate:** per esempio, connettersi a un dispositivo accoppiato e registrare l'audio.

Definire i permessi per una specifica applicazione è un compito da non sottovalutare. L'importanza di questo processo è così alta che Google ha definito un appropriato flusso di lavoro per gestire i permessi:



Android suddivide i permessi in varie categorie che includono:

- **Permessi al tempo di installazione**
- **Permessi al tempo di esecuzione**
- **Permessi speciali**

Ogni categoria indica la portata dei dati riservati a cui la nostra app può accedere e delle azioni limitate che può compiere quando il sistema concede alla nostra app quel permesso.

Ogni categoria di permessi ha un **livello di protezione**:

- **Permessi normali:** Questi permessi permettono all'app di accedere a dati o risorse che si trovano al di fuori del "sandbox" dell'app stessa, ma non comportano un grande rischio per la privacy dell'utente o per il funzionamento di altre app. Il sistema operativo Android concede automaticamente questi permessi all'installazione dell'app, senza richiedere il consenso dell'utente. Gli utenti inoltre non possono revocare questi permessi.
- **Permessi di firma:** Questi permessi possono essere utilizzati solo dalle app che sono state firmate con lo stesso certificato dell'app che ha definito il permesso. Ad esempio, un gruppo di app appartenenti a uno stesso account sviluppatore potrebbe collaborare se tutte sono state firmate con lo stesso certificato. Il sistema concede questi permessi al momento dell'installazione se l'app soddisfa i requisiti.
- **Permessi pericolosi:** Questi permessi consentono all'app di accedere a dati o risorse che coinvolgono informazioni private dell'utente o potrebbero influenzare i dati memorizzati o il funzionamento di altre app. Richiedono particolare attenzione poiché riguardano la sicurezza e la privacy degli utenti.

Il livello di protezione per ogni permesso è basato sulla sua categoria. Per maggiori informazioni si consiglia di consultare la pagina di riferimento dell'API per i permessi. Un esempio di ciò che troveremo è il seguente:

BLUETOOTH

```
public static final String BLUETOOTH
```

Allows applications to connect to paired bluetooth devices.

Protection level: normal

Constant Value: "android.permission.BLUETOOTH"

I **permessi al tempo di installazione** danno alla nostra app un limitato accesso ai dati riservati o lasciano eseguire all'app azioni limitate che coinvolgono in minima parte il sistema o le altre app. Android include diversi sottotipi di permessi al tempo di installazione, tra cui i permessi normali e di firma. Quando dichiariamo permessi al tempo di installazione nella nostra app, un app store presenta un avviso di permessi al tempo di installazione all'utente quando questo osserva la pagina dei dettagli dell'app:

Version 1.234.5 may request access to

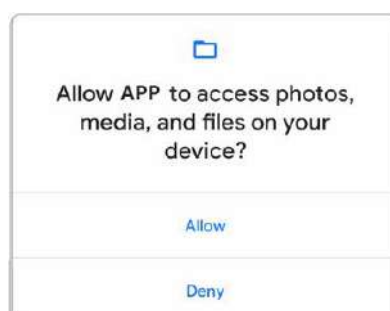


Other

- have full network access
- view network connections
- prevent phone from sleeping
- Play Install Referrer API
- view Wi-Fi connections
- run at startup
- receive data from Internet

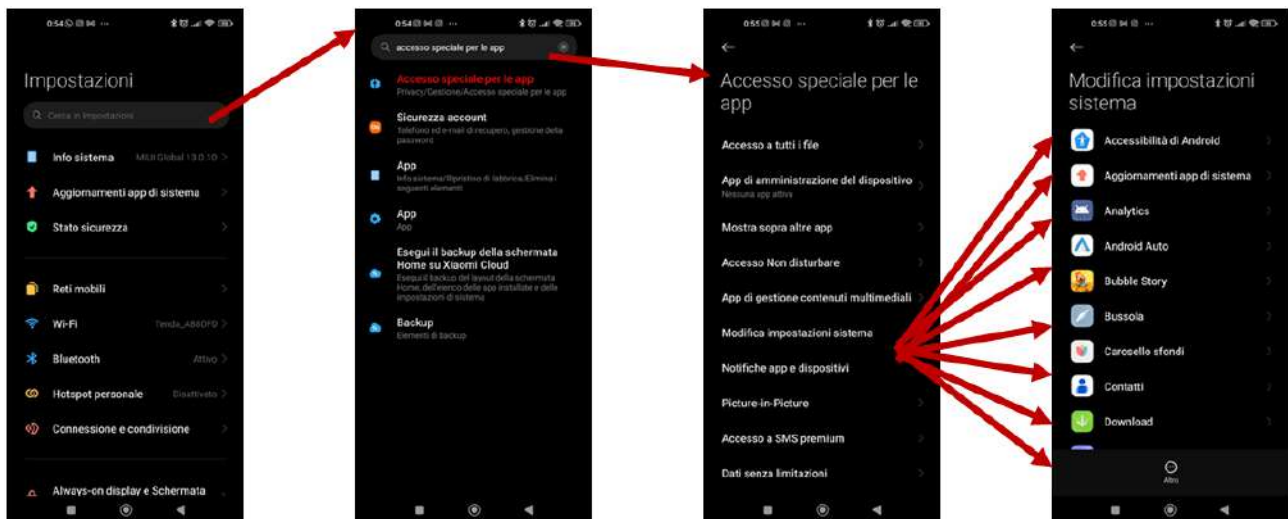
Date queste informazioni, l'utente può decidere se fidarsi dello sviluppatore dell'app e se si sente a suo agio nel concedere questi permessi all'app. Scegliere di usare un'app solo con gli appropriati permessi è un modo eccellente di tenere sotto controllo i permessi delle app nei dispositivi Android fin dall'inizio.

I **permessi al tempo di esecuzione**, conosciuti anche come permessi pericolosi, danno alla nostra app accesso addizionale a dati riservati o lasciano che l'app possa eseguire delle azioni limitate che potrebbero influenzare il sistema e le altre app in modo più sostanziale. Per accedere a determinati dati riservati o azioni limitate abbiamo bisogno di richiedere questi permessi nella nostra app, quindi piuttosto che assumere che siano stati concessi in precedenza, è meglio verificarli e, se necessario, richiederli prima di ogni accesso. Quando richiediamo i permessi al tempo di esecuzione, il sistema mostra un prompt di questo tipo:

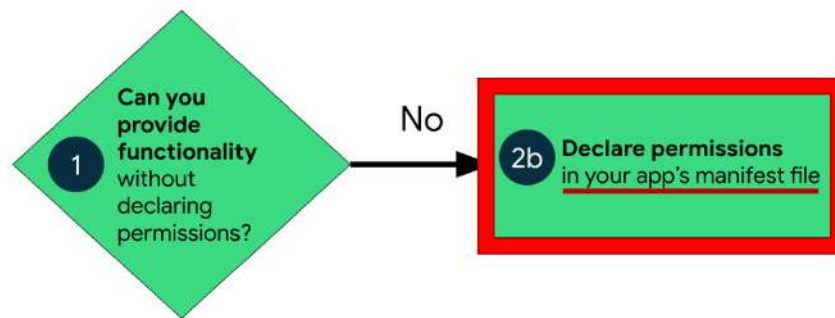


Molti permessi al tempo di esecuzione accedono ai dati privati dell'utente, un tipo speciale di dati riservati che includono informazioni potenzialmente sensibili. Esempi di dati privati dell'utente includono posizione e informazioni di contatto.

I **permessi speciali** proteggono l'accesso a risorse di sistema che sono sensibili o non direttamente legate alla privacy dell'utente. Queste ultime corrispondono a specifiche operazioni che un'applicazione può eseguire, come ad esempio monitorare il consumo dei dati. La maggior parte delle applicazioni non dovrebbe usare questo tipo di autorizzazione e ogni potenziale loro utilizzo deve essere sempre approvato dall'utente. Di seguito si mostra come accedere alla lista delle app che fanno uso di tali permessi:

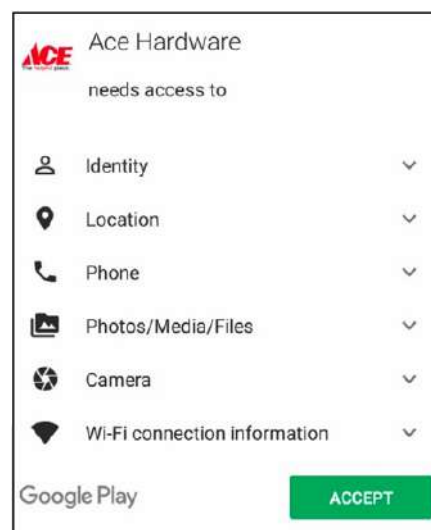


7.2. Dichiarare i permessi della nostra app



Se la nostra app richiede determinati permessi, dobbiamo dichiararli nel suo file manifest. Il processo di richiedere un permesso dipende dal tipo di quel permesso:

- **Se il permesso è al tempo di installazione**, come un normale permesso o un permesso di firma, allora tale permesso è concesso automaticamente all'installazione.
- **Se il permesso è al tempo di esecuzione o speciale**, e se la nostra app è installata in un dispositivo che usa Android 6.0 (livello di API 23) o superiore, allora dovremo richiedere da noi tale permesso. Prima di Android 6.0, infatti, i permessi erano richiesti sempre al momento dell'installazione dell'app sul dispositivo (ve lo ricordavate?):



Per dichiarare un permesso che la nostra app potrebbe richiedere, dobbiamo usare l'appropriato elemento **<uses-permission>** nel file manifest della nostra app. Per esempio, un'app che avrà bisogno di accedere alla fotocamera avrà questa linea di codice in AndroidManifest.xml:

```
<manifest ...>
  <uses-permission android:name="android.permission.CAMERA"/>
  <application ...>
    ...
  </application>
</manifest>
```

Bisogna stare attenti quando si dichiarano permessi nel file manifest, in quanto potrebbero rendere la nostra app inutilizzabile. Un pratico esempio è dato dai permessi legati all'hardware. Anche se un'applicazione richiede un permesso associato a un hardware specifico (ad esempio, la

fotocamera), in molti casi l'app può comunque funzionare senza di esso. Per gestire questa situazione, è consigliato dichiarare l'uso dell'hardware in modo opzionale. Questo si fa impostando l'attributo **android:required** su false all'interno del tag `<uses-feature>` nel file manifest.

```
<manifest ...>
  <application>
    ...
  </application>
  <uses-feature android:name="android.hardware.camera"
               android:required="false" />
</manifest>
```

In questo modo, l'app non dipenderà necessariamente dalla presenza di tale hardware per funzionare e potrà essere installata e utilizzata anche su dispositivi che non dispongono della fotocamera o di altro hardware dichiarato opzionale. Per controllare se un dispositivo possiede uno specifico pezzo di hardware, possiamo utilizzare il metodo **hasSystemFeature()**, come mostrato nel seguente esempio:

```
// Check whether your app is running on a device that has a front-facing camera.
if (applicationContext.packageManager.hasSystemFeature(
    PackageManager.FEATURE_CAMERA_FRONT)) {
    // Continue with the part of your app's workflow that requires a
    // front-facing camera.
} else {
    // Gracefully degrade your app experience.
}
```

Se un certo pezzo di hardware non è disponibile, abbiamo il dovere di disabilitare quella feature dell'app.

Una tipica funzionalità è quella di permettere al dispositivo di eseguire **operazioni di rete**, come ad esempio il download di contenuti. Anche in questo caso, dobbiamo dichiarare tali permessi nel file `AndroidManifest.xml`. I permessi Internet fanno parte della categoria di permessi normali, quindi non dobbiamo fare altro che aggiungere le due seguenti righe di codice:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

INTERNET

```
public static final String INTERNET
```

Allows applications to open network sockets.

Protection level: normal

Constant Value: "android.permission.INTERNET"

ACCESS_NETWORK_STATE

```
public static final String ACCESS_NETWORK_STATE
```

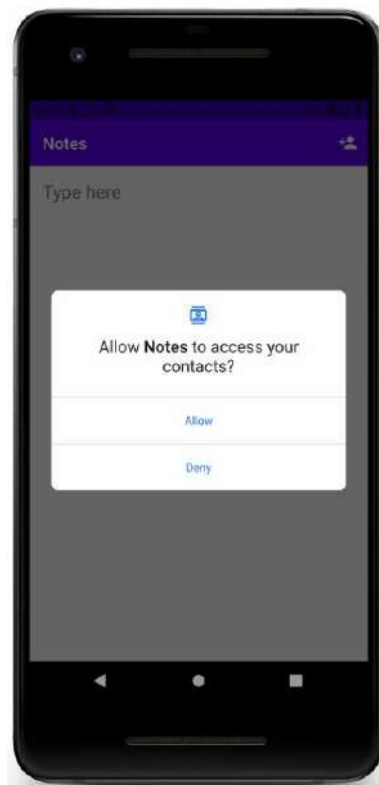
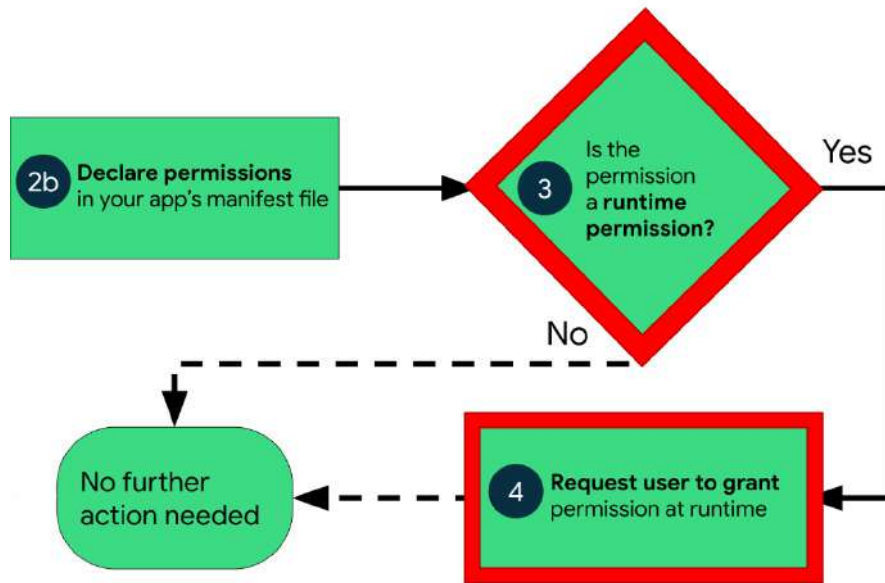
Allows applications to access information about networks.

Protection level: normal

Constant Value: "android.permission.ACCESS_NETWORK_STATE"

Adesso vedremo come i permessi pericolosi richiedano maggiore lavoro.

7.3. Richiedere i permessi al tempo di esecuzione

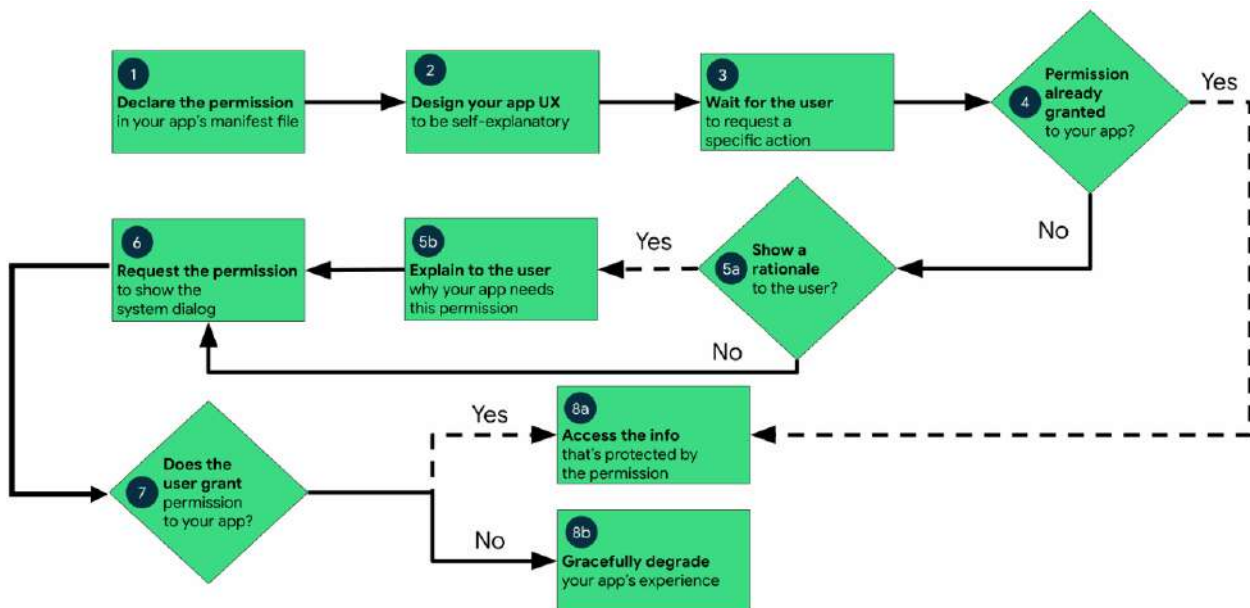


Ogni app Android viene eseguita in un sandbox ad accesso limitato. Se l'app ha necessità di accedere a risorse o informazioni che si trovano al di fuori del proprio sandbox, dovremo dichiarare un permesso al tempo di esecuzione e allestire una richiesta di permesso che fornisca questo accesso. I principi di base da seguire per richiedere permessi sono:

- **Chiedere un permesso nel suo determinato contesto**, quando l'utente inizia ad interagire con la feature che lo richiede.

- **Non bloccare l'utente.** Dobbiamo sempre fornire l'opzione di cancellare un flusso UI educativo, come quello che espone la spiegazione della richiesta di quel permesso.
- **Se l'utente nega o revoca un permesso di cui una feature ha bisogno, dobbiamo degradare con grazia la nostra applicazione** in modo tale che l'utente possa continuare ad utilizzare l'app, possibilmente disabilitando la feature che richiede quel permesso.
- **Non assumere alcun comportamento di sistema.** Per esempio, non possiamo assumere che i permessi appaiano nello stesso gruppo di permessi.

Per dichiarare permessi al tempo di esecuzione dobbiamo seguire un percorso specifico:



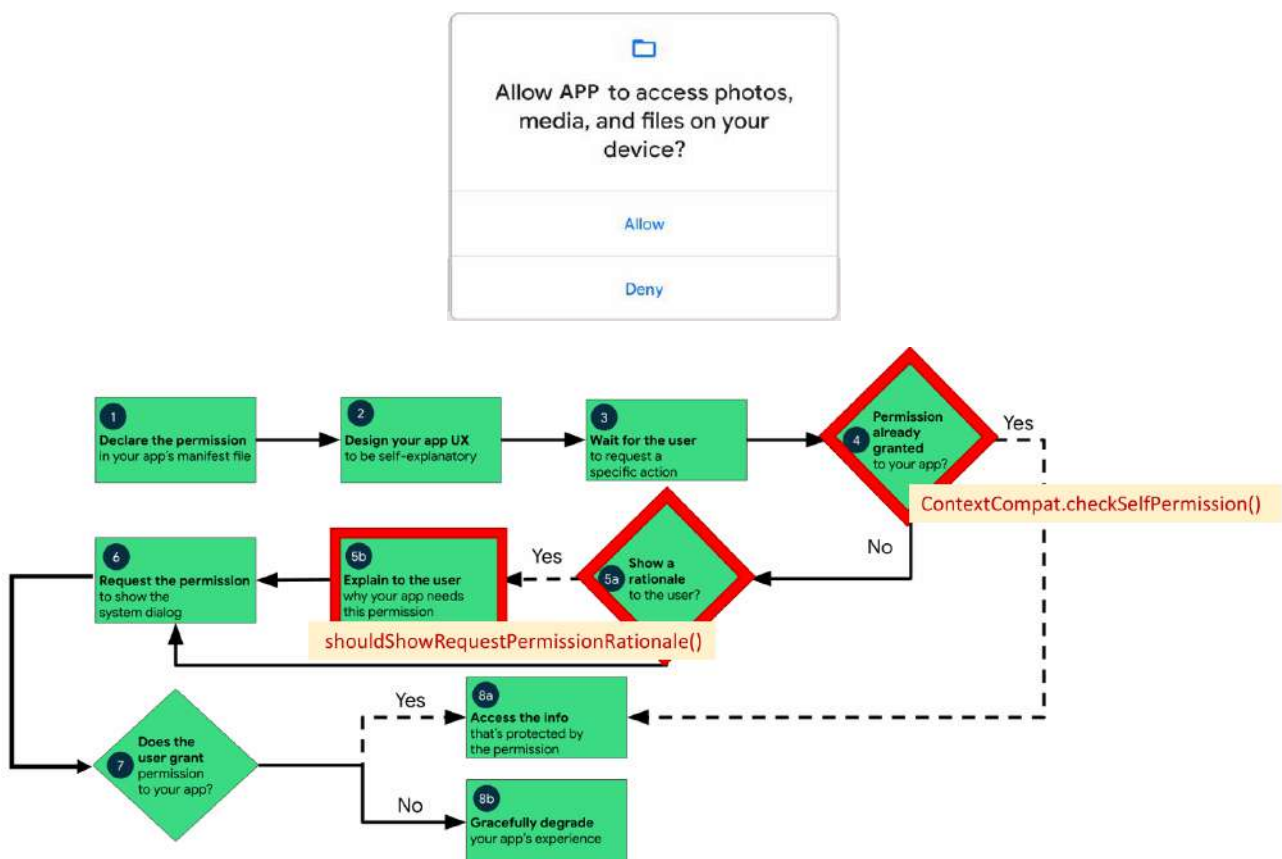
1. Dichiarare i permessi nel file AndroidManifest.xml.
2. Progettare una user-experience (UX) per l'app in modo tale che gli utenti sappiano quali azioni potrebbero richiedere la concessione di permessi alla nostra app per accedere ai dati privati dell'utente.
3. Aspettare che l'utente invochi quell'azione che richiede accesso ai dati privati dell'utente. A quel punto la nostra app può richiedere i necessari permessi al tempo di esecuzione per accedere a quei dati.
4. Controllare se l'utente ha già consentito i permessi al tempo di esecuzione che l'app richiede. Se è così, la nostra app può già accedere ai dati privati dell'utente, altrimenti si passa al prossimo step.
5. Controllare se la nostra app debba mostrare la spiegazione all'utente, illustrando perché l'applicazione ha bisogno che l'utente conceda un particolare permesso al tempo di esecuzione. Se il sistema determina che la nostra app non deve mostrare la spiegazione, si passa al prossimo step (7) direttamente, senza mostrare un elemento UI che spieghi quanto detto.
6. Se il sistema determina che la nostra app deve mostrare la spiegazione, allora presentiamola all'utente tramite un elemento UI. In questa spiegazione, dobbiamo illustrare chiaramente a quali dati la nostra app sta tentando di accedere e quali vantaggi l'app può fornire all'utente se egli concede quel permesso al tempo di esecuzione.
7. Richiedere il permesso al tempo di esecuzione che la nostra app richiede per accedere ai dati privati dell'utente. Il sistema mostra un prompt di permesso al tempo di esecuzione (dialog).

8. Controllare la risposta dell'utente.
9. Se l'utente ha concesso il permesso alla nostra app, potremo accedere ai suoi dati privati. Se invece l'utente ha negato il permesso, dovremo degradare con grazia la nostra applicazione in modo tale che possa continuare a fornire funzionalità all'utente senza le informazioni protette da quel permesso.

Tornando al punto 4, come controlliamo se i permessi sono stati già concessi? Per farlo, dobbiamo passare il permesso in questione al metodo **ContextCompat.checkSelfPermission()**, il quale può ritornare:

- **PERMISSION_GRANTED**: il permesso è stato concesso.
- **PERMISSION_DENIED**: il permesso è stato negato.

Nel caso il metodo ritorni **PERMISSION_DENIED**, allora bisogna chiamare il metodo **shouldShowRequestPermissionRationale()**. Se questo metodo ritorna true, allora bisogna mostrare un elemento UI educativo all'utente che descriva perché la feature che vuole abilitare necessita di quel particolare permesso:



Tornando adesso al punto 7, come richiediamo il permesso? Per permettere al sistema di gestire il codice di richiesta associato con una richiesta di permessi, bisogna aggiungere alcune dipendenze alle seguenti librerie nel file build.gradle del nostro modulo:

- **androidx.activity**, versione 1.2.0 o successive.
- **androidx.fragment**, versione 1.3.0 o successive.

In seguito, è possibile usare una delle due seguenti classi:

- Per richiedere un singolo permesso, usiamo **RequestPermission**.
- Per richiedere molteplici permessi allo stesso tempo, usiamo **RequestMultiplePermissions**.

Ora vediamo quali sono i passi da seguire per utilizzare il contratto RequestPermission (tale processo è praticamente analogo per il contratto RequestMultiplePermissions):

1. Nella logica di inizializzazione della nostra Activity o Fragment, dobbiamo passare un'implementazione di **ActivityResultCallback** in una chiamata a **registerForActivityResult()**. La ActivityResultCallback definisce come l'app gestisca la risposta dell'utente alla richiesta di permessi.
2. Mantenere un riferimento al valore di ritorno di registerForActivityResult(), il quale è di tipo **ActivityResultLauncher**.
3. Per mostrare il dialog dei permessi di sistema quando necessario, dobbiamo chiamare il metodo **launch()** sull'istanza di ActivityResultLauncher che abbiamo salvato nel precedente step.
4. Dopo la chiamata a launch(), il dialog per i permessi di sistema appare. Quando l'utente effettua la sua scelta, il sistema invoca asincronamente la nostra implementazione di ActivityResultCallback, che abbiamo definito in uno step precedente.

```
// Register the permissions callback, which handles the user's response to the
// system permissions dialog. Save the return value, an instance of
// ActivityResultLauncher. You can use either a val, as shown in this snippet,
// or a lateinit var in your onAttach() or onCreate() method.
val requestPermissionLauncher =
    registerForActivityResult(RequestPermission())
    ) { isGranted: Boolean ->
        if (isGranted) {
            // Permission is granted. Continue the action or workflow in your
            // app.
        } else {
            // Explain to the user that the feature is unavailable because the
            // feature requires a permission that the user has denied. At the
            // same time, respect the user's decision. Don't link to system
            // settings in an effort to convince the user to change their
            // decision.
        }
    }
```

Questa porzione di codice rappresenta ciò che abbiamo spiegato nei primi due punti ed è eseguita dopo la chiamata al metodo launch().


```

when {
    ContextCompat.checkSelfPermission(
        CONTEXT ✎,
        Manifest.permission.REQUESTED_PERMISSION ✎
    ) == PackageManager.PERMISSION_GRANTED -> {
        // You can use the API that requires the permission.
    }
    ActivityCompat.shouldShowRequestPermissionRationale(
        this, Manifest.permission.REQUESTED_PERMISSION ✎) -> {
        // In an educational UI, explain to the user why your app requires this
        // permission for a specific feature to behave as expected, and what
        // features are disabled if it's declined. In this UI, include a
        // "cancel" or "no thanks" button that lets the user continue
        // using your app without granting the permission.
        showInContextUI(...)
    }
    else -> {
        // You can directly ask for the permission.
        // The registered ActivityResultCallback gets the result of this request.
        requestPermissionLauncher.launch(
            Manifest.permission.REQUESTED_PERMISSION ✎)
    }
}

```

Questa porzione di codice rappresenta invece ciò che abbiamo spiegato negli ultimi due punti.

Per capire al meglio ciò che abbiamo spiegato, vediamo un esempio completo comprensivo di output grafico.

AndroidManifest.xml
<pre> <?xml version="1.0" encoding="utf-8"?> <manifest xmlns:android="http://schemas.android.com/apk/res/android" xmlns:tools="http://schemas.android.com/tools"> <uses-permission android:name="android.permission.RECORD_AUDIO"/> <application ... <activity android:name=".MainActivity" android:exported="true"> <intent-filter> <action android:name="android.intent.action.MAIN" /> <category android:name="android.intent.category.LAUNCHER" /> </intent-filter> </activity> </application> </manifest> </pre>

Il permesso di cui si vuole fare richiesta riguarda la registrazione audio.

```

class MainActivity : AppCompatActivity() {
    private val TAG = "PermissionDemo"
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        setupPermission()
    }

    private fun setupPermission(){
        val permission = ContextCompat.checkSelfPermission(this, android.Manifest.permission.RECORD_AUDIO)
        if(permission == PackageManager.PERMISSION_GRANTED){
            Log.i(TAG, "Permission enabled")
        } else if (shouldShowRequestPermissionRationale(android.Manifest.permission.RECORD_AUDIO)) {
            Log.i(TAG, "Request for rational permission")
        } else {
            Log.i(TAG, "Request for the permission")
            requestPermission.launch(android.Manifest.permission.RECORD_AUDIO)
        }
    }
}

```

MainActivity.kt

Non abbiamo ancora definito il metodo di callback che gestisce il risultato del contratto.

L'esempio mostrato è stato costruito con le condizioni if, ma lo stesso codice può essere cambiato per usare la condizione when. Definiamo sempre nello stesso file la callback che gestisce il risultato del contratto:

```

class MainActivity : AppCompatActivity() {
    private val TAG = "PermissionDemo"
    private lateinit var binding: ActivityMainBinding
    private val requestPermission = registerForActivityResult(ActivityResultContracts.RequestPermission()){
        isGranted: Boolean ->
        if(isGranted){
            Log.i(TAG, "Permission enabled")
        } else {
            Log.i(TAG, "Explain the reason")
        }
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        setupPermission()
    }

    private fun setupPermission(){
        ...
    }
}

```

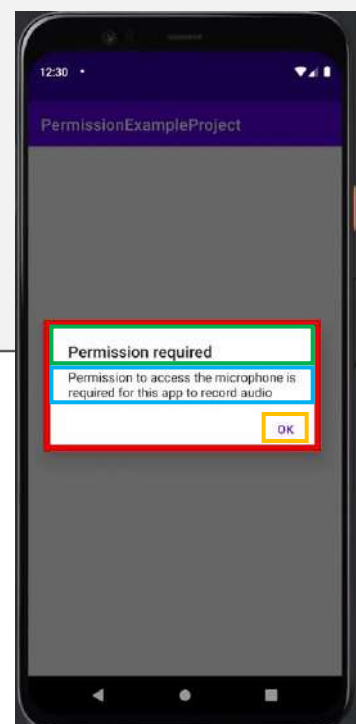
MainActivity.kt

In questo modo otterremo il seguente output:

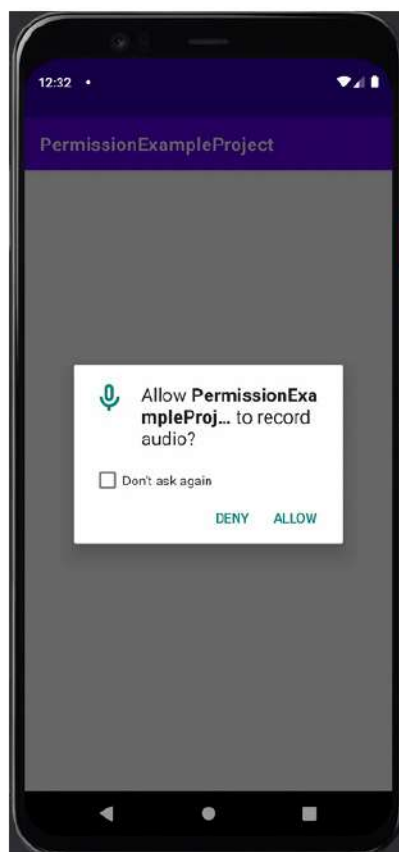
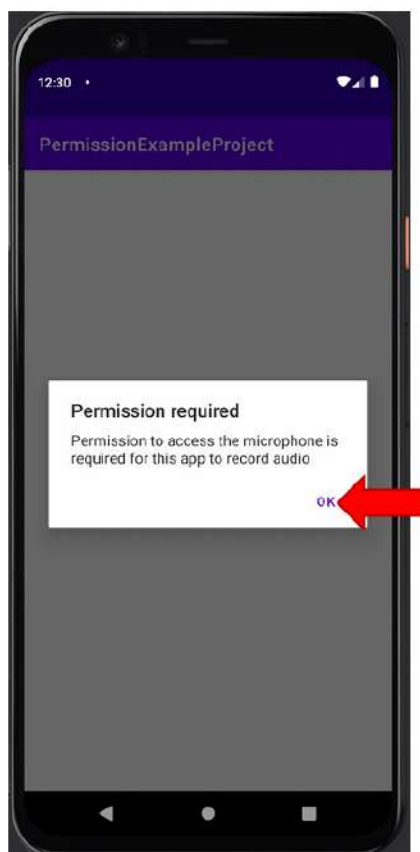


Come gestiamo l'opzione "deny"? Dobbiamo mostrare una spiegazione! Modifichiamo quindi il metodo `setupPermission()` nel file `MainActivity.kt`:

```
private fun setupPermission(){
    val permission = ContextCompat.checkSelfPermission(this, android.Manifest.permission.RECORD_AUDIO)
    if(permission == PackageManager.PERMISSION_GRANTED){
        Log.i(TAG, "Permission enabled")
    } else if (shouldShowRequestPermissionRationale(android.Manifest.permission.RECORD_AUDIO)) {
        Log.i(TAG, "Request for rational permission")
        val builder = AlertDialog.Builder(this)
        builder.setMessage("Permission to access the microphone is required for this app to record audio")
        builder.setTitle("Permission required")
        builder.setPositiveButton("OK"){
            dialog, id ->
            Log.i(TAG, "Clicked")
            requestPermission.launch(android.Manifest.permission.RECORD_AUDIO)
        }
        val dialog = builder.create()
        dialog.show()
    } else {
        Log.i(TAG, "Request for the permission")
        requestPermission.launch(android.Manifest.permission.RECORD_AUDIO)
    }
}
```



Cliccando il tasto “OK”, il risultato sarà:



7.4. Permessi di rete e HTTP

Molte app richiedono una connessione alla rete per mandare e ricevere dati. I permessi Internet giocano un ruolo cruciale nel funzionamento delle app Android: permettono all'app di eseguire numerosi compiti come mandare e ricevere dati, accedere al web e altro ancora. In ogni caso, avere a disposizione i permessi Internet rappresenta una potenziale minaccia alla privacy e alla sicurezza dei dati dell'utente. Per eseguire operazioni di rete nella nostra applicazione, il manifest deve includere i seguenti permessi:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

Il primo permette alle applicazioni di aprire socket di rete, mentre il secondo permette di accedere alle informazioni che riguardano la rete.

La maggior parte delle app connesse alla rete usano il protocollo HTTP (Hyper-Text Transfer Protocol) per mandare e ricevere dati. Tale protocollo è localizzato al livello applicativo (sopra il protocollo TCP) e si presta a un modello di comunicazione di tipo client-server. Assumiamo di dover aprire la pagina <https://example.com/index.html>, il browser aprirà una connessione con il server e invierà un messaggio richiedendo i contenuti del documento HTML index.html che si trova alla radice del server web <https://example.com>. Il messaggio mandato dal browser è chiamato **richiesta HTTP**. Al ricevimento della richiesta, il server si occuperà di recuperare i contenuti del file richiesto e mandarli in risposta. Il messaggio mandato dal server web è chiamato **risposta HTTP**. Una volta ricevuto il documento HTML, il browser inizierà ad analizzare la pagina per renderizzare e caricare il contenuto, e scoprirà che il documento HTML indica che per la sua visualizzazione sono necessari altri dispositivi remoti.

Il seguente è un esempio di **richiesta HTTP**:

```
PUT /somewhere/fun HTTP/1.1
Host: origin.example.com
Content-Type: video/h264
Content-Length: 1234567890987
Expect: 100-continue
```

Il campo evidenziato indica il tipo di richiesta. I metodi più conosciuti sono **GET** e **POST**.

Il **metodo GET** è utilizzato per richiedere dati da un server. I dati che devono essere inviati al server sono scritti direttamente nell'URL:

```
www.example.com/register.php?firstname=peter&name=miller&age=55&gender=male
```

Vantaggi principali:

I parametri possono essere salvati nell'URL insieme all'indirizzo del sito. Ciò permette di salvare la query (richiesta) come segnalibro, rendendo facile richiamarla in un secondo momento.

Svantaggi principali:

- **Assenza di protezione dei dati:** I dati inviati tramite GET vengono inclusi nell'URL, il che significa che possono essere facilmente visualizzati o tracciati, ad esempio, attraverso la cronologia del browser.

- **Limite di lunghezza massima:** Gli URL hanno una lunghezza massima che può essere trasmessa. Se i dati da inviare sono troppo grandi, questo limite può essere problematico, rendendo il metodo GET inadatto per inviare dati molto lunghi o complessi.

Con il **metodo POST**, i parametri vengono inviati al server nel corpo della richiesta HTTP, rendendoli invisibili all'utente nell'URL.

Vantaggi principali:

- **Sicurezza:** Se devi trasmettere dati sensibili al server (come password o informazioni private), il metodo POST offre una maggiore discrezione. I dati non vengono memorizzati nella cache, né compaiono nella cronologia del browser o nell'URL.
- **Flessibilità:** Consente di inviare dati di qualsiasi tipo e dimensione. A differenza del metodo GET, non c'è un limite di lunghezza per i dati trasmessi.

Svantaggi principali:

- **Ri-invio dei dati:** Quando aggiorni una pagina web contenente un modulo (ad esempio, facendo clic su "Indietro" o aggiornando la pagina), i dati inseriti devono essere nuovamente inviati. Questo può risultare scomodo per l'utente, poiché potrebbe dover ripetere l'invio di un modulo.
- **Impossibilità di salvare la richiesta come segnalibro:** Dato che i parametri non sono nell'URL, i dati inviati con POST non possono essere salvati in un segnalibro insieme all'indirizzo del sito.

Ecco una lista dei metodi utilizzabili in una richiesta HTTP:

HTTP method ↕	RFC ↕	Request has Body ↕	Response has Body ↕	Safe ↕	Idempotent ↕	Cacheable ↕
GET	RFC 7231	Optional	Yes	Yes	Yes	Yes
HEAD	RFC 7231	Optional	No	Yes	Yes	Yes
POST	RFC 7231	Yes	Yes	No	No	Yes
PUT	RFC 7231	Yes	Yes	No	Yes	No
DELETE	RFC 7231	Optional	Yes	No	Yes	No
CONNECT	RFC 7231	Optional	Yes	No	No	No
OPTIONS	RFC 7231	Optional	Yes	Yes	Yes	No
TRACE	RFC 7231	No	Yes	Yes	Yes	No
PATCH	RFC 5789	Yes	Yes	No	No	No

Mostriamo invece adesso un esempio di **risposta HTTP**:

```
Header { HTTP/1.1 200 OK
          Date: Mon, 27 Jul 2009 12:28:53 GMT
          Server: Apache
          Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
          ETag: "34aa387-d-1568eb00"
          Accept-Ranges: bytes
          Content-Length: 51
          Vary: Accept-Encoding
          Content-Type: text/plain
Body { Hello World! My content includes a trailing CRLF.
```

Il primo campo evidenziato è il **codice di stato** che informa il client dell'adempimento (o meno) della sua richiesta. Alcuni comuni codici di stato sono:

- 2xx = Successo
 - 200 (OK): risposta standard per risposte HTTP che hanno successo.
- 4xx = Errore Client
 - 400 (Bad Request): errore di sintassi nella richiesta.
 - 401 (Unauthorized): il client non è autorizzato ad accedere a quella risorsa.
 - 404 (Not Found): la risorsa richiesta non è stata trovata.
- 5xx = Errore Server
 - 500 (Internal Server Error): messaggio di errore generico privo di dettagli.
 - 503 (Service Unavailable) il server è momentaneamente indisponibile (generalmente questa condizione è temporanea).

Il secondo campo evidenziato rappresenta il tipo di contenuto della risposta:

Type/subtype labels

- text/plain - the text format (extension .txt)
- text/html - the HTML text
- application/octet-stream - the byte stream of an executable program
- image/jpeg, audio/mp3, video/mp4 - audio and video
- application/msword - word document
-

Un ultimo aspetto da trattare è la differenza tra HTTP e HTTPS. Tramite il primo, gli utenti si collegano a un sito tramite **connessione non sicura** e questo lascia qualsiasi dato in transito in balia di attacchi informatici. Tramite il secondo, gli utenti si collegano a un sito tramite **connessione sicura**, la quale cripta il canale di trasmissione dei dati per metterlo al sicuro dall'accesso di terze parti potenzialmente malintenzionate.

Le app moderne richiedono l'utilizzo del protocollo HTTPS, quindi se proviamo a effettuare delle semplici richieste HTTP, il sistema operativo bloccherà qualsiasi comunicazione con il mondo esterno. Per questo motivo, è obbligatorio nel manifest indicare che alla nostra app è permesso di comunicare tramite protocollo HTTP:

AndroidManifest.xml -

```
<?xml version="1.0" encoding="utf-8"?>
<manifest ...>
    <uses-permission android:name="android.permission.INTERNET" />
    <application
        ...
        android:usesCleartextTraffic="true"
        ...>
    </application>
</manifest>
```

7.5. Servizi web e API Rest

Riepilogando, possiamo dire che HTTP è il fondamento della comunicazione dei dati nel World Wide Web. La sua principale funzione è quella di specificare come i messaggi sono formattati e trasmessi, e come i server web e i browser dovrebbero rispondere ai vari comandi. La comunicazione di base avviene tra un client (come un browser web) e un server (il luogo dove è ospitato il contenuto web). L'evoluzione di Internet e l'uso del web ha portato alla luce il bisogno di interazioni tra le applicazioni in differenti contesti. Sfortunatamente, il modello classico del web non è adatto a un'interazione del genere dato che si basa su una semplice idea:

- L'utente, usando un browser, trasmette un URL, o digitandolo direttamente oppure cliccando su un link.
- Il server web ritorna una pagina HTML che è renderizzata graficamente dal browser.

Questo pattern presenta due punti critici:

- Gli URL sono troppo rudimentali per esprimere richieste complesse e articolate.
- HTML è una specie di ibrido tra un linguaggio di descrizione del contenuto dell'informazione e un linguaggio per descrivere la presentazione. Non c'è una chiara separazione tra forma e contenuto.

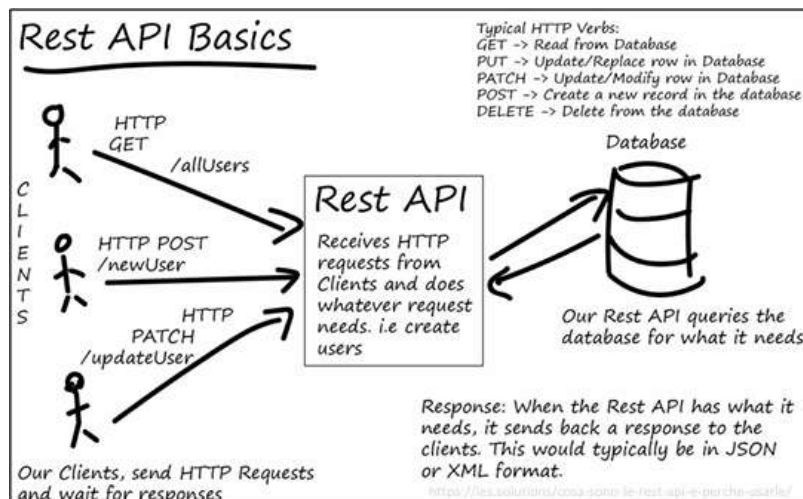
Queste considerazioni hanno portato a definire un nuovo modello noto come **servizio web**. Gli obiettivi primari erano:

- Continuare ad utilizzare HTTP come protocollo di trasporto dell'informazione, in modo tale da riutilizzare l'esistente infrastruttura e mantenere la semplicità del modello di interazione.
- Trovare uno strumento più appropriato per descrivere gli elementi della conversazione: la richiesta e la risposta.

I servizi web sono funzionalità/servizi esposte da un server che possono essere invocate da programmi esterni tramite la rete e che, dopo l'elaborazione, restituiscono un risultato. Le funzionalità che un servizio web offre prendono il nome di **API**, ovvero **Application Programming Interface**. Molti servizi web esposti tra gli anni '90 e i primi 2000 vennero fatti con una tecnologia molto utile ma non sempre facilmente approcciabile, ovvero la cosiddetta **SOAP** (Simple Object Access Protocol), basato sul protocollo HTTP e sul formato XML. Ad ogni modo, il suo ruolo è stato degradato nel tempo dalla semplicità ed efficienza del modello **REST** (Representational State Transfer) basato sul protocollo HTTP e sul formato **JSON**, il quale non aggiungeva alcuna superstruttura addizionale bensì manteneva tutti gli strumenti già in possesso per il recupero delle informazioni.

Nel modello REST, sono quattro le operazioni che vengono solitamente richieste al server:

- Con il metodo GET chiederemo solamente di leggere dei dati.
- Con il metodo POST creeremo nuovi dati.
- Con il metodo PUT modificheremo i dati.
- Con il metodo DELETE cancelleremo i dati.

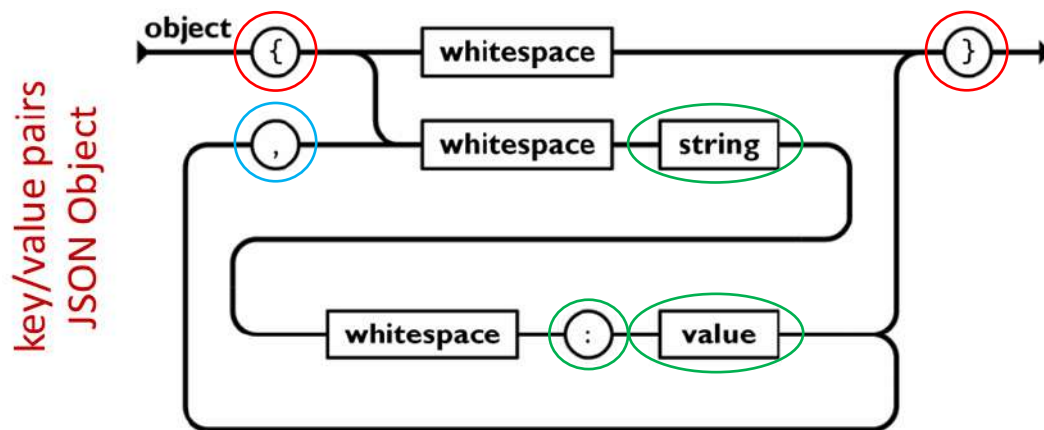


Ogni richiesta verrà inviata a un particolare **indirizzo web**, e quell'URL sarà unico per la risorsa sulla quale l'azione è richiesta. Supponendo che il servizio abbia indirizzo web www.mioservizioweb.org/api/ e che ogni prodotto sia identificato da un intero (ad esempio, una chiave nel database). Il progettista del servizio potrebbe decidere quanto segue:

- Per richiedere la lettura dei dati del prodotto con ID uguale a 1, viene invocata una richiesta **GET** all'indirizzo www.mioservizioweb.org/api/prodotti/1.
- Per richiedere la lettura dei dati di tutti i prodotti in offerta, viene inviata una richiesta **GET** all'URL www.mioservizioweb.org/api/offerte.
- Per inserire i dati di un nuovo prodotto, viene inviata una richiesta **POST** all'indirizzo www.mioservizioweb.org/api/prodotti/nuovo.
- Per modificare i dati del prodotto esistente con ID uguale a 1, viene inviata una richiesta **PUT** all'indirizzo www.mioservizioweb.org/api/prodotti/1.
- Per eliminare il prodotto con ID 1, viene inviata una richiesta **DELETE** a www.mioservizioweb.org/api/prodotti/1.

Tipicamente nelle API REST i dati sono convertiti in **formato JSON** (JavaScript Object Notation) che è un formato di testo che è completamente indipendente dal linguaggio di programmazione e ideale per scambi di dati. JSON è basato su due strutture dati universali che lo rendono supportabile da tutti i moderni linguaggi di programmazione:

- **Un insieme di coppie chiave/valore:** In molti linguaggi ciò può essere fatto mediante un oggetto, un record, una struct, un dizionario, una hash table, una lista di chiavi o un array associativo.
- **Una lista ordinata di valori:** in molti linguaggi ciò può essere realizzato mediante un array, vettore, lista o sequenza.

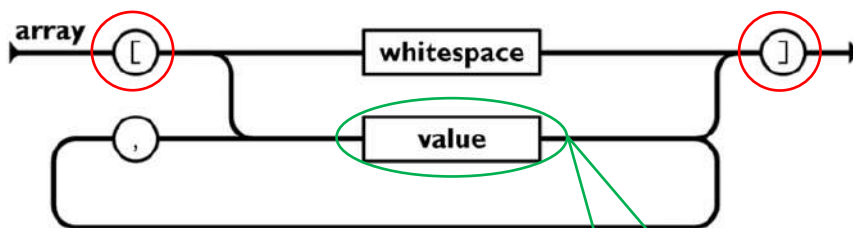


Un **oggetto JSON** inizia con una “ { ” e finisce con una “ } ”.

Ogni chiave è seguita da un carattere “ : ”, che è a sua volta seguito da un valore.

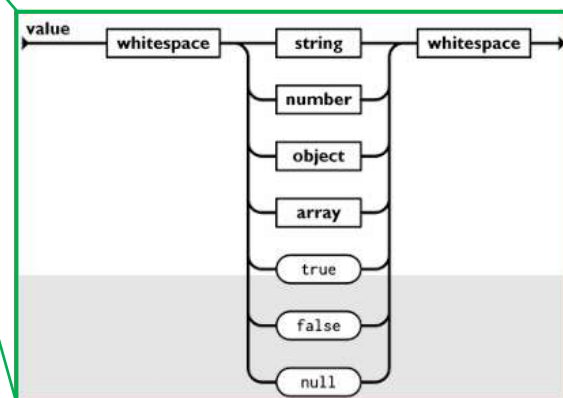
Una coppia chiave valore è separata da un carattere “ , ”.

an ordered list of values
JSON array

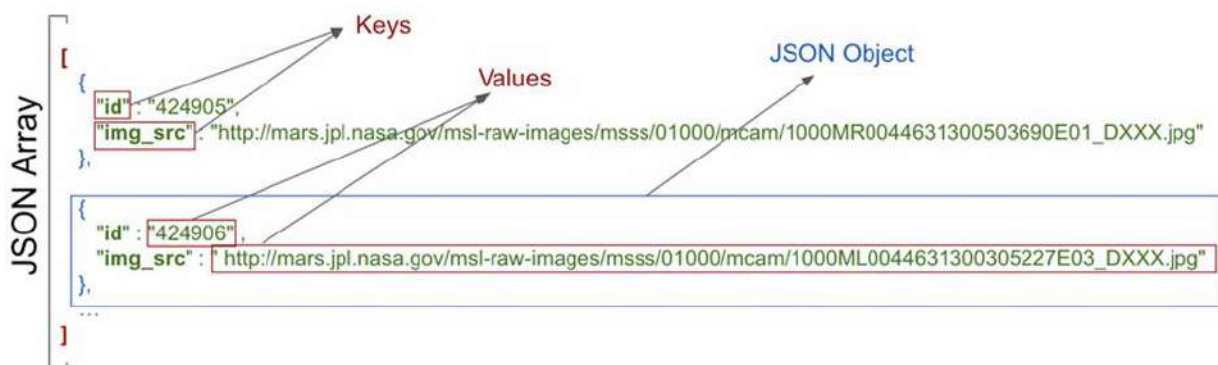
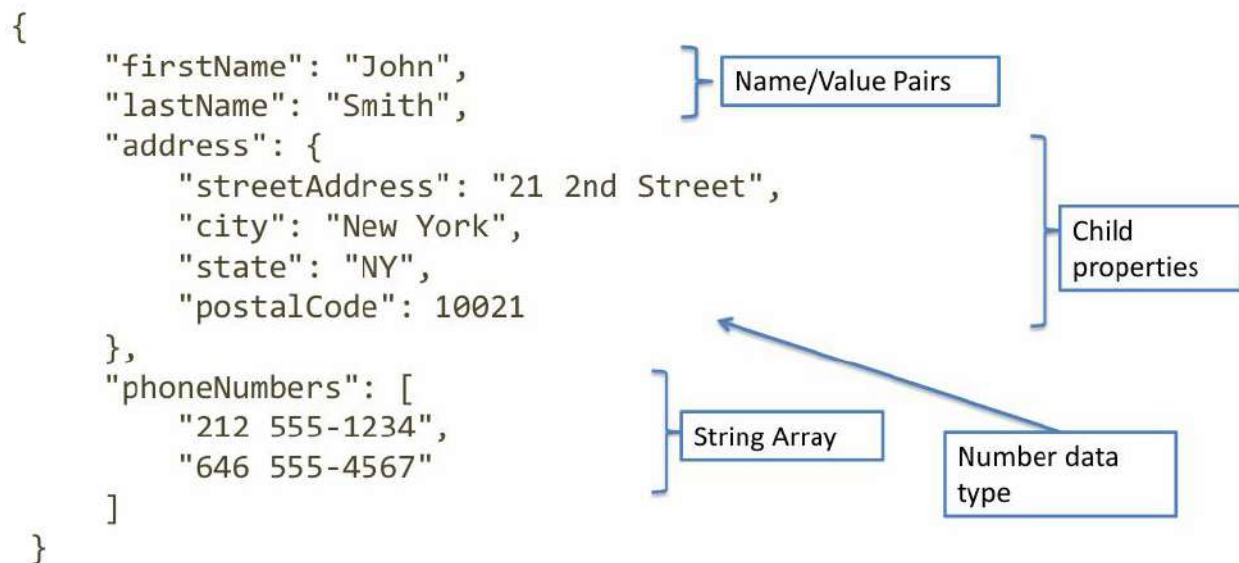


Un **array JSON** inizia con una “ [” e finisce con una “] ”.

I valori sono separati da un carattere “ , ”. Un valore può essere una stringa tra virgolette, un numero, True, False, Null, un oggetto o un array.



Vediamo adesso un esempio di oggetto JSON e uno di array JSON:



Avendo delle similarità, sia XML che JSON possono essere adottati:

```
<employees>
  <employee>
    <firstName>John</firstName> <lastName>Doe</lastName>
  </employee>
  <employee>
    <firstName>Anna</firstName> <lastName>Smith</lastName>
  </employee>
  <employee>
    <firstName>Peter</firstName> <lastName>Jones</lastName>
  </employee>
</employees>
```

```
{"employees": [
  { "firstName": "John", "lastName": "Doe" },
  { "firstName": "Anna", "lastName": "Smith" },
  { "firstName": "Peter", "lastName": "Jones" }
]}
```

JSON è simile a XML perché:

- Entrambi sono **autodescrittivi** (human readable).
- Entrambi sono **gerarchici** (valori all'interno di valori).
- Entrambi possono essere analizzati e **usati da molti linguaggi di programmazione**.
- Entrambi possono essere recuperati utilizzando un **XMLHttpRequest**.

JSON è diverso (e migliore) rispetto a XML perché:

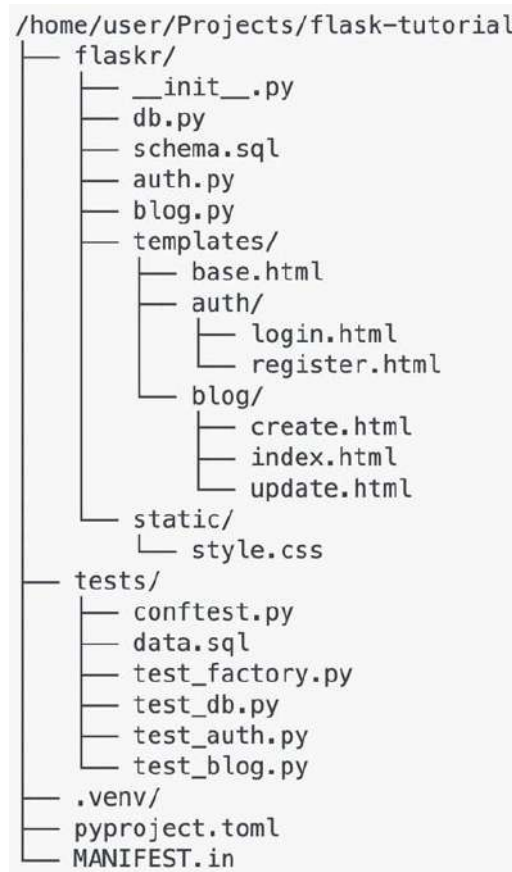
- JSON non utilizza un **tag di fine**.
- JSON è più **breve**.

- JSON è più **veloce** a leggere e scrivere.
- JSON può usare **array**.

La più grande differenza è che XML deve essere analizzato da un parser XML mentre JSON può essere analizzato da una funzione JavaScript standard.

7.5. Flask (ha tutto ciò di cui abbiamo bisogno per il progetto finale)

Ecco un esempio di struttura generale di progetto di applicazione Flask:



La directory del progetto conterrà:

- **flask/**: un package Python contenente il codice della nostra applicazione e i file.
- **tests/**: directory contenente moduli di test.
- **.venv/**: ambiente virtuale Python dove Flask e altre dipendenze vengono installate.
- **File di installazione** che dicono a Python come installare il nostro progetto.
- **Version control config**, come git. Dovremmo abituarci a usare un qualche tipo di controllo di versione per i nostri progetti, non importa quanto siano grandi.
- Ogni altro file di progetto che potremmo aggiungere in futuro.

Le directory più usate per il progetto saranno la prima e la terza.

Prima di usare Flask dobbiamo eseguirne il setup correttamente:

1. Prima di tutto dobbiamo installare Flask in un ambiente virtuale:
<https://flask.palletsprojects.com/en/3.0.x/installation/>

2. L'output sarà simile al seguente:

```
server_prova -- zsh -- 166x49
Last login: Tue May 14 19:36:39 on console
(federicoconcone@MacBook-Pro-di-Federico-2 ~ % cd Desktop
(federicoconcone@MacBook-Pro-di-Federico-2 Desktop % mkdir server_prova
(federicoconcone@MacBook-Pro-di-Federico-2 Desktop % cd server_prova
(federicoconcone@MacBook-Pro-di-Federico-2 server_prova % python3 -m venv .venv
(federicoconcone@MacBook-Pro-di-Federico-2 server_prova % source .venv/bin/activate
(.venv) federicoconcone@MacBook-Pro-di-Federico-2 server_prova % pip install flask
Collecting flask
  Using cached flask-3.0.3-py3-none-any.whl (101 kB)
Collecting itsdangerous>=2.1.2
  Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting click>=8.1.3
  Using cached click-8.1.7-py3-none-any.whl (97 kB)
Collecting importlib-metadata>=3.6.0
  Using cached importlib_metadata-7.1.0-py3-none-any.whl (24 kB)
Collecting Jinja2>=3.1.2
  Downloading Jinja2-3.1.4-py3-none-any.whl (133 kB)
    |#####| 133 kB 2.3 MB/s
Collecting Werkzeug>=3.0.0
  Downloading Werkzeug-3.0.3-py3-none-any.whl (227 kB)
    |#####| 227 kB 11.9 MB/s
Collecting blinker>=1.6.2
  Downloading blinker-1.8.2-py3-none-any.whl (9.5 kB)
Collecting zipp>=0.5
  Using cached zipp-3.18.1-py3-none-any.whl (8.2 kB)
Collecting MarkupSafe>=2.0
  Using cached MarkupSafe-2.1.5-cp39-cp39-macosx_10_9_universal2.whl (18 kB)
Installing collected packages: zipp, MarkupSafe, Werkzeug, Jinja2, itsdangerous, importlib-metadata, click, blinker, flask
Successfully installed Jinja2-3.1.4 MarkupSafe-2.1.5 Werkzeug-3.0.3 blinker-1.8.2 click-8.1.7 flask-3.0.3 importlib-metadata-7.1.0 itsdangerous-2.2.0 zipp-3.18.1
```

3. Creiamo la directory **flaskr**:

```
server_
[ (.venv) federicoconcone@MacBook-Pro-di-Federico-2 server_prova % mkdir flaskr
[ (.venv) federicoconcone@MacBook-Pro-di-Federico-2 server_prova % ls -al
total 0
drwxr-xr-x  4 federicoconcone  staff   128 15 Mag 12:47 .
drwx-----@ 33 federicoconcone  staff  1056 15 Mag 12:44 ..
drwxr-xr-x  6 federicoconcone  staff   192 15 Mag 12:44 .venv
drwxr-xr-x  2 federicoconcone  staff    64 15 Mag 12:47 flaskr
```

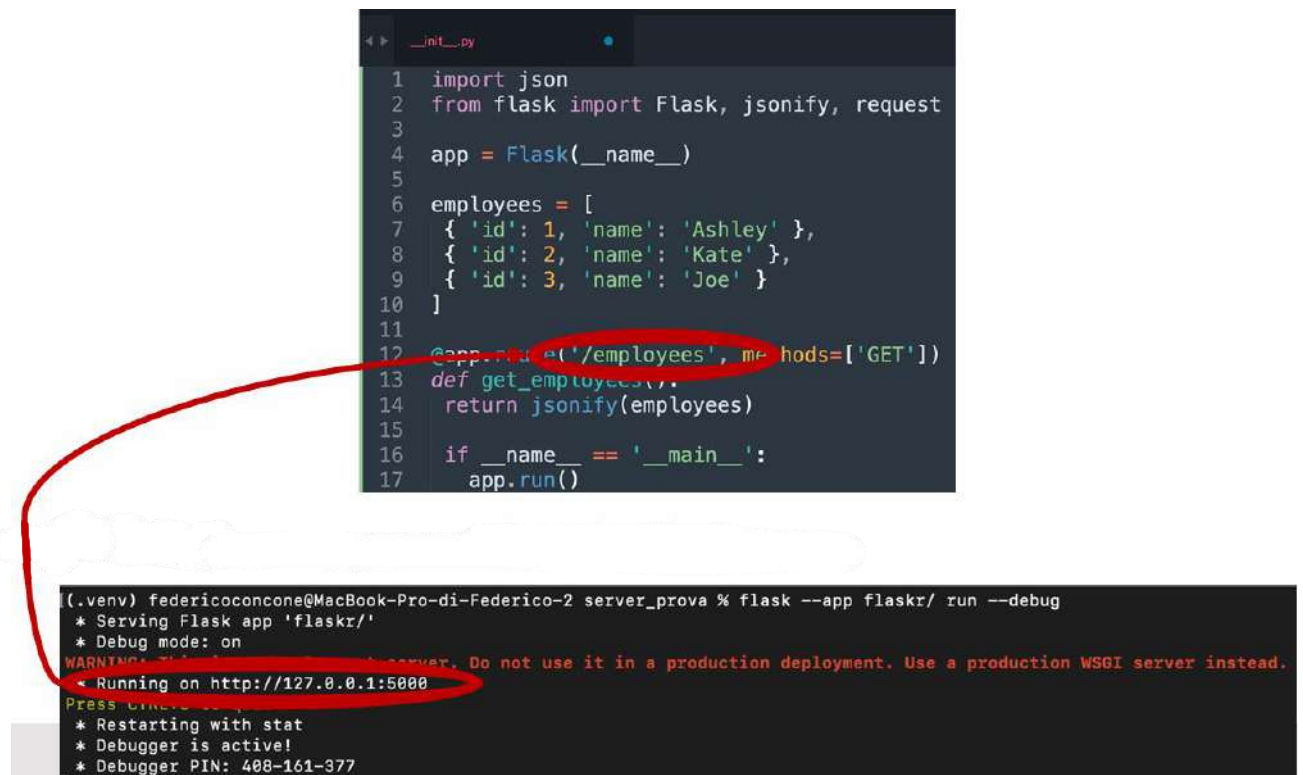
4. Creiamo il file **__init__.py** dentro la directory flaskr:

```
[ (.venv) federicoconcone@MacBook-Pro-di-Federico-2 server_prova % ls -al flaskr
total 0
drwxr-xr-x  3 federicoconcone  staff    96 15 Mag 12:49 .
drwxr-xr-x  5 federicoconcone  staff   160 15 Mag 12:49 ..
-rw-r--r--@ 1 federicoconcone  staff     0 15 Mag 12:49 __init__.py
```

5. Scriviamo in **__init__.py** il seguente codice:

```
__init__.py
1 import json
2 from flask import Flask, jsonify, request
3
4 app = Flask(__name__)
5
6 employees = [
7     { 'id': 1, 'name': 'Ashley' },
8     { 'id': 2, 'name': 'Kate' },
9     { 'id': 3, 'name': 'Joe' }
10 ]
11
12 @app.route('/employees', methods=['GET'])
13 def get_employees():
14     return jsonify(employees)
15
16 if __name__ == '__main__':
17     app.run()
```

6. Posizioniamoci nella directory `server_prova` (o come l'abbiamo chiamata) ed eseguiamo il server Flask:

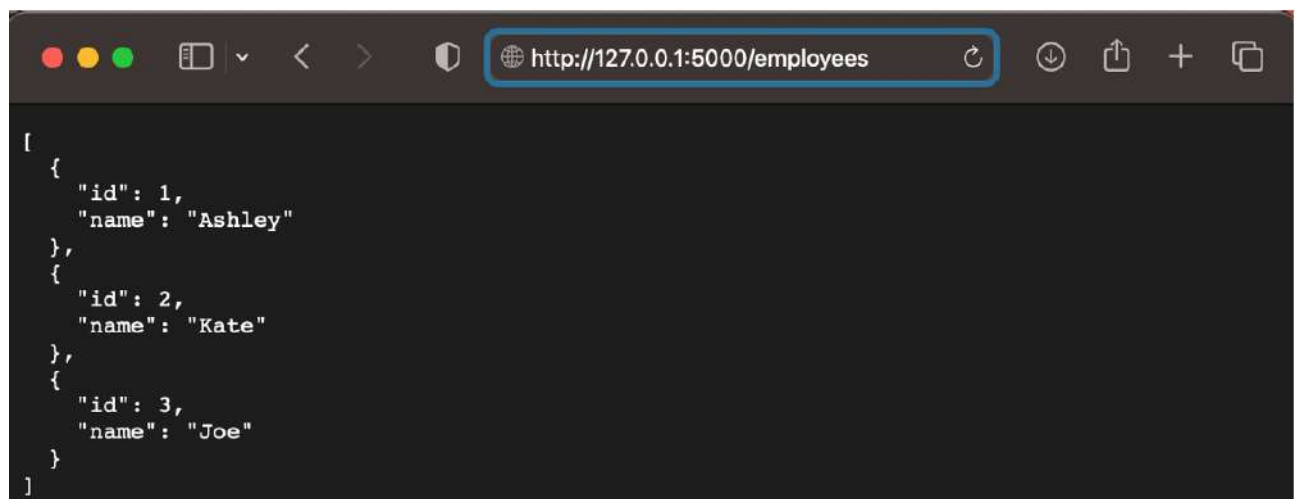


The image shows a code editor window with a Python file named `__init__.py`. The code defines a Flask application with a list of employees and a GET endpoint for `/employees`. A red circle highlights the `@app.route('/employees', methods=['GET'])` line. A red arrow points from this line to a terminal window below. The terminal shows the command `flask --app flaskr/ run --debug` being executed in the `server_prova` directory. The output indicates the server is running on `http://127.0.0.1:5000`, which is also circled in red. A warning message is displayed: "WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead."

```
1 import json
2 from flask import Flask, jsonify, request
3
4 app = Flask(__name__)
5
6 employees = [
7     {'id': 1, 'name': 'Ashley'},
8     {'id': 2, 'name': 'Kate'},
9     {'id': 3, 'name': 'Joe'}
10 ]
11
12 @app.route('/employees', methods=['GET'])
13 def get_employees():
14     return jsonify(employees)
15
16 if __name__ == '__main__':
17     app.run()
```

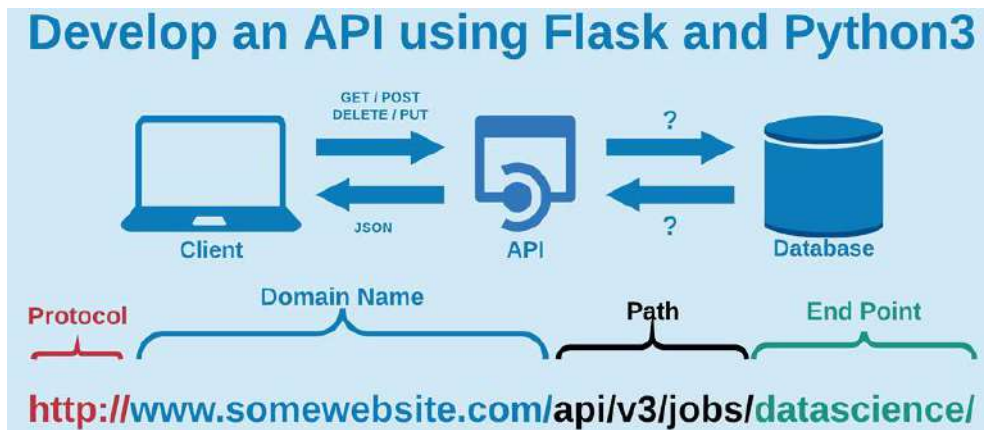
```
(.venv) federicoconcone@MacBook-Pro-di-Federico-2 server_prova % flask --app flaskr/ run --debug
* Serving Flask app 'flaskr/'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 408-161-377
```

7. Tramite browser, ci stiamo connettendo all'URL <http://127.0.0.1:5000/employees> :



IMPORTANTE: le configurazioni appena mostrate e altre funzionalità che servono a runnare il server Flask sono diverse da quelle appena mostrate. Nel portale studenti è possibile recuperare il codice corretto per runnare il server Flask e comunicare con l'app Android. Dopodichè, l'unica cosa importante da fare è definire gli endpoint in base alle specifiche dell'app.

Una **funzione view** è costituita da codice che scriviamo per rispondere alle richieste della nostra applicazione. Flask usa dei pattern per associare l'URL della richiesta in arrivo alla view che dovrebbe gestirla:



Un **Blueprint** è un modo per organizzare un gruppo di View in relazione tra loro e altro codice. Esistono molti Blueprint a seconda delle necessità degli sviluppatori. Ad esempio, se volessimo creare un Blueprint di nome “pwm”, dovremmo farlo nel seguente modo:

```
flaskr/pwm.py
from flask import (
    Blueprint, flash, g, redirect, render_template,
    request, session, url_for, jsonify, current_app
)
from . import db

bp = Blueprint('pwm', __name__, url_prefix='/pwm')
```

The name of the Blueprint

Indicate where the Blueprint is defined

The url_prefix will be prepended to all the URLs associated with the Blueprint

Dopo aver definito il Blueprint, possiamo aggiungere tutti i metodi che possono essere invocati:

Example of endpoint WITHOUT parameter

```
@bp.route('/user', methods=['GET'])
def user():
    if request.method == 'GET':
        connection = db.getdb()
        resp = []
        try:
            cursor = connection.cursor(dictionary=True)
            cursor.execute("SELECT * FROM User")

            users = cursor.fetchall()
            for user in users:
                resp.append({'id': user['idUser'],
                            'username': user['username'],
                            'password': user['password'],
                            'nome': user['nome'],
                            'cognome': user['cognome']
                            })

        except db.IntegrityError:
            resp.append({'error': 'Error retrieving users'})
        finally:
            cursor.close()
    return jsonify(resp)
```

Example of endpoint WITH parameter

```
@bp.route('/user/<int:user_id>', methods=['GET'])
def get_user(user_id):
    connection = db.getdb()
    resp = {}
    try:
        cursor = connection.cursor(dictionary=True)
        cursor.execute("SELECT * FROM User WHERE idUser = %s", (user_id,))
        user = cursor.fetchone()
        if user:
            resp = {'id': user['idUser'],
                    'username': user['username'],
                    'password': user['password'],
                    'nome': user['nome'],
                    'cognome': user['cognome']}
        else:
            resp = {'error': 'User not found'}
    except db.IntegrityError:
        resp = {'error': 'Error retrieving user'}
    finally:
        cursor.close()
    return jsonify(resp)
```

Infine, dobbiamo registrare il Blueprint nel file `__init__.py`:

flaskr/ __init__.py

```
import os
from flask import Flask
from . import db, pwm

def create_app(test_config=None):
    # create and configure the app
    app = Flask(__name__)
    app.config.from_mapping(
        SECRET_KEY='dev',
        DB_HOST='localhost',
        DB_USER='root',
        DB_PASSWORD='mysqlpwm',
        DB_DATABASE='pwm'
    )

    db.init_app(app)
    app.register_blueprint(pwm.bp)

    if test_config is None:
        # load the instance config, if it exists, when not testing
        app.config.from_pyfile('config.py', silent=True)
    else:
        # load the test config if passed in
        app.config.from_mapping(test_config)

    # ensure the instance folder exists
    try:
        os.makedirs(app.instance_path)
    except OSError:
        pass

    return app

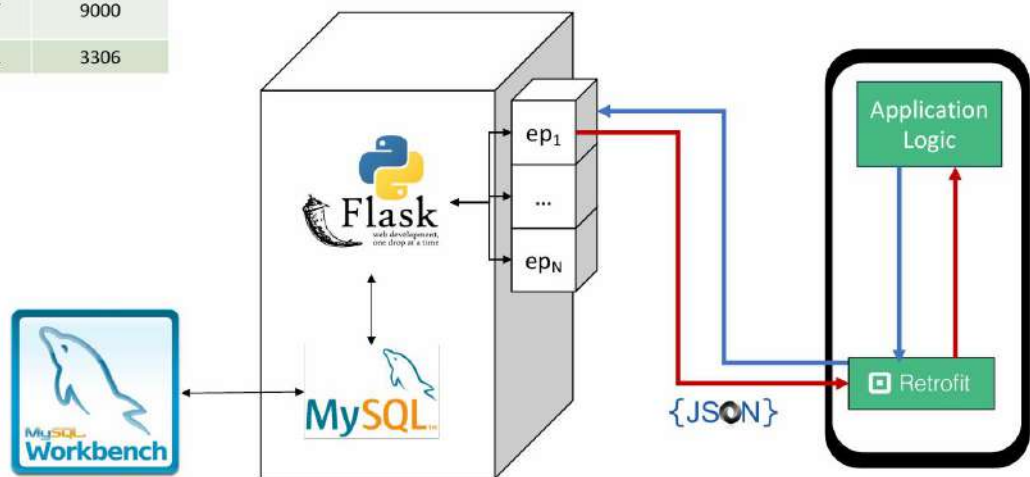
app = create_app(None)
```


Di default, Flask utilizza SQLite. Ad ogni modo, il database da utilizzare per il progetto finale sarà **MySQL**. Quando creeremo un'istanza di un server MySQL per il progetto finale, dovremo porre particolare attenzione ai campi:

- DB_HOST
- DB_USER
- DB_PASSWORD
- DB_DATABASE

L'architettura del progetto finale sarà così costituita:

Entity	IP	Port
Server	127.0.0.1	-
Flask	127.0.0.1 0.0.0.0	9000
MySQL Server	127.0.0.1	3306



7.6. Retrofit

Anche se potremmo usare le classi di rete di base che sono fornite dal framework Android, è raccomandato sfruttare una libreria di rete poiché queste gestiscono per noi molti dettagli in secondo piano in modo efficiente e robusto. Una delle librerie più usate è appunto **Retrofit**, che è una libreria di rete che trasforma la nostra API HTTP in un'interfaccia Kotlin e Java, permette l'elaborazione delle richieste e risposte in oggetti utilizzabili dalla nostra app e fornisce supporto di base per analizzare comuni tipi di risposta (come XML o JSON) ma può essere esteso per supportare anche altri tipi di risposta. Per utilizzare Retrofit nel nostro progetto, inseriamo la seguente riga di codice nel file gradle:

```
implementation 'com.squareup.retrofit2:retrofit:(insert latest version)'
```

Retrofit da solo non può gestire la deserializzazione del corpo di risposta HTTP, quindi devono essere integrate alcune librerie di conversione.

Libreria di persistenza	Dipendenza da includere
Krypton	implementation group: 'com.abubusoft', name: 'krypton-retrofit-converter', version: '3.1.0'
Gson	implementation group: 'com.squareup.retrofit2', name: 'converter-gson', version: '2.3.0'
Jackson	implementation group: 'com.squareup.retrofit2', name: 'converter-jackson', version: '2.3.0'
Moshi	implementation group: 'com.squareup.retrofit2', name: 'converter-moshi', version: '2.3.0'
Protobuf	implementation 'com.squareup.retrofit2', name: 'converter-protobuf', version: '2.3.0'
Wire	implementation group: 'com.squareup.retrofit2', name: 'converter-wire', version: '2.3.0'
Simple XML	implementation group: 'com.squareup.retrofit2', name: 'converter-simplexml', version: '2.3.0'

```
implementation("com.squareup.retrofit2:retrofit:2.1.0")
```

```
implementation("com.squareup.retrofit2:converter-gson:2.1.0")
```

Facciamo un semplice esempio di uso di Retrofit. Creiamo un'istanza di Retrofit in una classe separata:

```
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

object RetrofitInstance{
    private const val BASE_URL = "https://example.com/"

    fun getInstance(): Retrofit {
        return Retrofit.Builder().baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
    }
}
```


Poi creiamo un'interfaccia Retrofit che descriva gli endpoint API. La dicitura “end/point” deve essere sostituita con quella che vogliamo usare, mentre ExampleResponse è un modello di dati utile a contenere i dati recuperati:

```
import retrofit2.Call
import retrofit2.http.Body
import retrofit2.http.GET

interface ApiInterface {
    @GET("end/point")
    fun getExampleData(): Call<ExampleResponse>
}
```

È possibile implementare anche gli altri metodi HTTP:

```
interface UserAPI {
    @GET (URI)
    fun nomeFunzione (Possibile_input): Call<TipoDiRitorno>

    @DELETE (URI)

    @PUT (URI)

    @POST (URI)
    //-----
}
```

Infine, utilizziamo Retrofit nella nostra Activity o Fragment:

```
class MainActivity : AppCompatActivity() {
    private lateinit var apiInterface: ApiInterface
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        getApiInterface()
        getExampleData()
    }

    private fun getApiInterface() {
        apiInterface = RetrofitInstance.getInstance().create(ApiInterface::class.java)
    }

    private fun getExampleData(){
        val call = apiInterface.getExampleData()
        call.enqueue(object : Callback<ExampleResponse> {
            override fun onResponse(call: Call<ExampleResponse>, response: Response<ExampleResponse>) {
                if (response.isSuccessful && response.body() != null){
                    // TODO: Process data
                }
            }
        })

        override fun onFailure(call: Call<ExampleResponse>, t: Throwable) {
            t.printStackTrace()
        }
    }
}
```

Implements the method declared into the interface!

7.7. Visualizzazione delle immagini

Glide è una libreria di caricamento immagini di terze parti in Android. Permette di ottenere prestazioni ottimizzate per uno scorrimento più fluido, supporta immagini, fotogrammi video e GIF animate. Per utilizzare la libreria Glide bisogna aggiungere questa riga di codice al file gradle:

```
implementation "com.github.bumptech.glide:glide:$glide_version"
```

Tale codice lega Glide al ciclo di vita dell'Activity o Fragment e carica l'immagine all'URL specificato nella ImageView data:

```
Glide.with(fragment)
      .load(url)
      .into(imageView);
```

8. Stack tecnologico

Nello sviluppo delle applicazioni mobile, i programmatori software usano il termine **tech stack** per riferirsi a un insieme di tecnologie che costituiscono la piattaforma, così come l'aspetto dell'applicazione mobile. Ci poniamo alcuni interrogativi:

- Quali sono i possibili tech stacks per lo sviluppo di un'applicazione mobile?
- Quale di questi è considerato il migliore e perché?

La scelta dipende:

- Dalla piattaforma
- Dal target di utenti finale
- Dai processi dei dati
- Dalle funzioni chiave dell'applicazione
- Dal budget del progetto

In generale, esistono quattro tipi di app:

- App native
- App cross-platform
- PWA (Progressive Web App)
- App ibride

Le **app native** sono progettate per funzionare su uno specifico sistema operativo (Android, iOS ecc.). Sono scritte usando dei linguaggi di programmazione specifici, che possono variare da un sistema operativo all'altro e il cui codice non può essere semplicemente tradotto da un linguaggio ad un altro. I **punti di forza** di un'app nativa sono:

- **Performance migliorata:** le app native sono più veloci, più efficienti, più sicure e lavorano agevolmente nei sistemi operativi dei dispositivi mobile.
- **Sicurezza:** poiché si interfacciano direttamente con il sistema operativo, le app native hanno degli standard di sicurezza più alti, rendendole più sicure di qualsiasi altro tipo di app. Rafforzano la protezione dei dati utente e sono ampiamente usate dalle compagnie e dagli attori che mettono la sicurezza al primo posto, come nel caso delle app di banking.
- **Connettività avanzata:** le app native possono usare più facilmente i vari strumenti e le risorse costruite nei dispositivi mobile, come la fotocamera, il Bluetooth, i contatti telefonici, eccetera, senza il bisogno di integrare e/o sviluppare componenti esterne.
- **User experience al top:** l'interfaccia utente propria delle app native fornisce una migliore esperienza per gli utenti. Ciò avviene perché le app nativamente sviluppate hanno a disposizione dei componenti UI tipici della piattaforma ed elementi usati sia dalle app sviluppate dai produttori dei dispositivi sia da sviluppatori di terze parti.

Ma perché utenti di differenti piattaforme hanno difficoltà a cambiare dispositivo? Nonostante gli utenti passino tra app diverse, diventano **familiari con elementi, stili e comportamenti comuni**. Alcuni esempi da citare sono:

- L'effetto di "ripple" sui pulsanti in Android.
- La barra di navigazione su iOS.

- Le animazioni dell'apparizione delle schermate che variano tra i sistemi operativi.
- Lo stile di navigazione, i caratteri utilizzati, e altri dettagli.

Questo significa che, quando gli utenti cambiano dispositivo o sistema operativo, possono trovare difficoltà nell'adattarsi a nuovi modelli di interazione, poiché i loro automatismi e abitudini sono stati costruiti su una piattaforma specifica.

I **punti deboli** di un'app nativa sono:

- **Differenti linguaggi di programmazione:** ogni app nativa è adatta a una specifica piattaforma o sistema operativo. Quindi, gli sviluppatori devono creare due differenti app per due differenti sistemi operativi senza poter riutilizzare alcun elemento. Questa caratteristica ha anche alcuni svantaggi minori in termini di manutenzione, dato che il team di sviluppatori dovrà mantenere due differenti codebases.
- **Costose:** sviluppare, mantenere e aggiornare il codebase di ogni app nativa è più costoso, in quanto è necessario sviluppare l'app per ogni sistema operativo in cui deve essere eseguita. Per questo motivo gli sviluppatori avranno il doppio del lavoro da fare, in quanto saranno costretti a modificare, risolvere bug e aggiornare due codebases interamente differenti.
- **Processo lavorativo che consuma tempo eccessivo:** dato che le app native richiedono agli sviluppatori che facciano il doppio del lavoro, è naturale che ci mettano molto di più a gestire e mantenere un'app nativa.

Esempi famosi di app native sono Pokemon GO e Google Maps.

I framework **multiplatforma** sono stati creati per risolvere uno dei principali svantaggi delle **applicazioni native**: la necessità di scrivere codice diverso per ogni sistema operativo. Le app create con un framework multiplatforma possono funzionare su ogni OS supportato dal framework, rendendo il processo di sviluppo più semplice, veloce e accessibile. Queste app sono la soluzione ideale quando si deve creare un'app mobile per diverse piattaforme con **tempo o risorse limitate**. I **punti di forza** di un'app cross-platform sono:

- **Più economico:** lo sviluppo di app multiplatforma costa meno rispetto allo sviluppo di app native, perché non serve un team di sviluppo separato per ogni sistema operativo (iOS, Android, ecc.).
- **Processo più veloce:** le modifiche o i miglioramenti possono essere effettuati su un'unica base di codice, accelerando il processo.
- **Scrivi una volta, esegui ovunque:** il codice scritto può essere eseguito su diverse piattaforme (ad esempio, usando HTML5).
- **Coerenza del comportamento:** sviluppare un'app multiplatforma garantisce coerenza nel comportamento dell'app su tutte le piattaforme supportate, ossia l'app si comporterà sempre allo stesso modo, indipendentemente dal sistema operativo utilizzato.

I **punti deboli** di un'app cross-platform sono:

- **Integrazioni difficili:** il codice di un'app multiplatforma sviluppata con JavaScript o HTML5 rende più complessa l'integrazione di funzionalità che comunicano a basso livello con il sistema operativo, come Bluetooth, Fotocamera, Rubrica, GPS, ecc.

- **Interfaccia utente:** creare un'interfaccia utente che rispetti le linee guida di progettazione richieste dai vari sistemi operativi è una sfida. Ad esempio, iOS e Android utilizzano icone e schemi di navigazione diversi.
- **Prestazioni inferiori:** Questi framework richiedono un componente intermedio (uno strato) che ha il compito di comunicare con le parti native, traducendo le istruzioni del framework in istruzioni comprensibili per i sistemi operativi nativi. Questo strato aggiuntivo può rallentare le prestazioni dell'applicazione.

Un esempio popolare di framework cross-platform è **flutter**. Creato da Google, ha un SDK (Software Development Kit) che contiene tutto ciò di cui abbiamo bisogno per lo sviluppo di app cross-platform. Flutter ci permette di proporre un'ottima user-experience, inoltre tutte le app sviluppate in questo modo funzionano su dispositivi Android, iOS, Linux, Mac, Windows ecc.

I punti di forza di Flutter sono:

- Alta performance
- Creazione di elementi UI che risaltano all'occhio mediante personalizzazione dei widget
- Testing approfondito
- Efficienza comprovata

I punti deboli di Flutter sono:

- App più pesanti (40% in più rispetto alle app native)
- Supporto limitato per le librerie
- Supporto limitato per alcune funzionalità, come ad esempio l'accessibilità
- L'aspetto e il feeling non sono nativi
- Il framework, nonostante sia di Google, è ancora giovane e i suoi cambiamenti frequenti richiedono un maggiore sforzo per restare al passo.

Altri framework utilizzabili oltre a Flutter sono **Xamarin** e **Cordova** (Apache). Un esempio (particolare) di app cross-platform è l'app "Modena volley".

Progressive Web App (PWA) è un termine originariamente coniato da Google che si riferisce alle "applicazioni web" che sono sviluppate e pubblicate come normali pagine web, ma si comportano in modo simile alle app native quando usate in dispositivi mobile. Le PWA non sono app stand-alone perché la loro operatività dipende dal browser che l'utente sta utilizzando. Ciò significa che, sebbene queste app possano avere un'icona sulla schermata principale del dispositivo, cliccandoci sopra si apre effettivamente il browser web, anche se l'utente potrebbe non rendersene conto. Le PWA sono in grado di interfacciarsi con alcune funzionalità del dispositivo, come l'invio di notifiche push, in modo simile alle app native. I **punti di forza** delle PWA sono:

- **App web:** essendo delle web app, le PWA non devono essere personalizzate per ogni sistema operativo e possono essere aggiornate in tempo reale, senza bisogno di download.
- **Non richiedono un vero download:** Le PWA occupano poco spazio nella memoria del dispositivo poiché non vengono effettivamente installate come un'app nativa.
- **Mantenere l'app è più semplice:** La manutenzione di una PWA è più facile, veloce e economica rispetto a una app nativa, poiché non deve essere adattata a ciascun sistema operativo specifico.

I **punti deboli** delle PWA sono:

- **Limitazioni delle funzionalità e problemi di compatibilità:** Apple ostacola le PWA sui suoi smartphone limitando l'accesso a determinate funzionalità dei dispositivi, il che crea veri problemi di compatibilità. Le PWA funzionano solo su dispositivi Apple con iOS 11.3 o versioni più recenti.
- **Alto consumo della batteria:** le PWA consumano più batteria rispetto alle app native perché sono sviluppate utilizzando tecnologie web, piuttosto che linguaggi di codifica nativi.
- **Richiedono una connessione Internet:** alcune funzioni delle PWA non sono disponibili offline.
- **Dipendenza dal browser:** le PWA sono app basate sul web, quindi funzionano nel browser. Ciò significa che l'esperienza utente può variare a seconda del browser, poiché non sempre vengono aggiornati correttamente dalle aziende che li sviluppano, e alcune funzionalità potrebbero non essere presenti.
- **Interfaccia utente non nativa:** le PWA sono sviluppate con tecnologie web come HTML, CSS e JavaScript, anziché con linguaggi nativi come Kotlin o Swift (utilizzati per le app native). Per alcuni utenti, questo può tradursi in un'esperienza utente di qualità inferiore.

Un esempio di app PWA è **Pinterest**.

Le **app ibride** possono essere viste come applicazioni software che combinano elementi di app native e app web. Una volta scaricate da uno store digitale e installate localmente sul dispositivo, gli elementi di sviluppo web (ad esempio HTML5 e JavaScript) sono connessi a tutte le funzionalità offerte dalla piattaforma mobile tramite un browser integrato nell'app. Simili alle app sviluppate tramite framework, le app ibride aggiungono uno strato aggiuntivo tra il codice sorgente e la piattaforma di destinazione, risultando in una velocità leggermente inferiore rispetto alle versioni native o web della stessa app. I **punti di forza** delle app ibride sono:

- **Sviluppo veloce:** un'unica base di codice per più piattaforme riduce notevolmente il tempo di sviluppo delle app.
- **Più economiche rispetto alle app native:** richiedono lo sviluppo di un solo codice sorgente, il che le rende meno costose da sviluppare.
- **Manutenzione più semplice:** poiché le app ibride utilizzano un'unica base di codice, la loro manutenzione risulta più semplice.
- **Integrazione dei componenti nativi più semplice** rispetto ad altre soluzioni.

I **punti deboli** delle app ibride sono:

- **Minore reattività:** a causa dello strato aggiuntivo tra il codice sorgente e la piattaforma di destinazione, le app ibride tendono ad essere meno reattive rispetto alle app native.
- **Esperienza utente povera:** implementare animazioni complesse, grafica e interfacce hardware è più difficile nelle app ibride. Di conseguenza, il codice viene eseguito più lentamente rispetto a quello nativo, compromettendo l'esperienza utente.
- **Problemi con i plug-in:** le app ibride necessitano di plug-in per interfacciarsi con le specifiche del dispositivo e dei diversi sistemi operativi. Sebbene questi plug-in aiutino le app a funzionare correttamente sui dispositivi, gli sviluppatori possono incontrare problemi durante la creazione e la manutenzione. Poiché i plug-in sono creati da terze parti, è probabile che gli sviluppatori incontrino difficoltà nel mantenerli aggiornati.

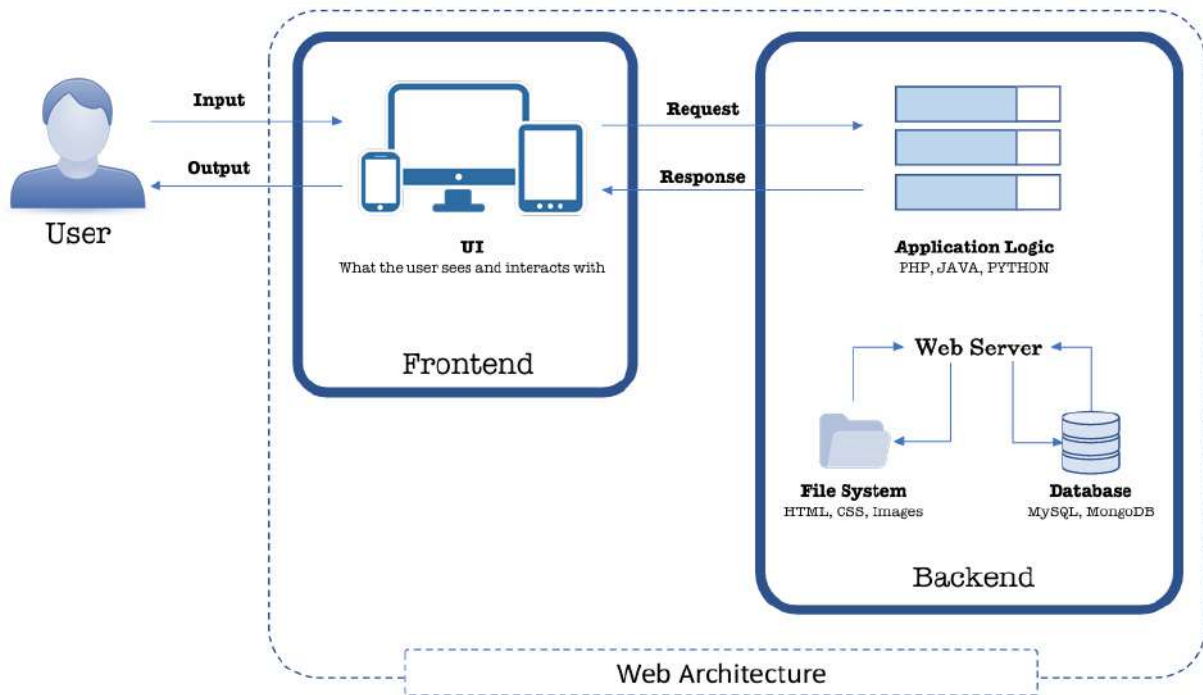
Un esempio di app ibrida è **Gmail**.

In conclusione, possiamo dire che non esiste una soluzione unica quando sviluppiamo un app. L'ecosistema si evolve continuamente, quindi la nostra app deve tenersi in linea con i nostri obiettivi.

- **Se abbiamo bisogno di un'app il più presto possibile**, dovremmo optare per una cross-platform o PWA. Sicuramente una singola codebase velocizzerà di molto il tempo di sviluppo.
- **Se non abbiamo molte risorse a disposizione** come tempo, soldi o risorse di altro tipo, dovremmo optare per un'app ibrida o cross-platform. In aggiunta, un'app ibrida ci dà l'opportunità di testare il mercato con investimenti minimi. Se i nostri risultati sono soddisfacenti, potremmo decidere di svilupparne una versione nativa in seguito.
- **Se vogliamo che la nostra app sia veloce e stabile**, dovremmo optare per un'app nativa. Le app native forniscono velocità, affidabilità, alte performance e funzionalità personalizzate essenziali per raggiungere i nostri obiettivi.

9. HTML

9.1. Introduzione al web



La seguente immagine illustra come il browser funga da interfaccia tra gli utenti e il web, facilitando sia la visualizzazione e l'interazione con il contenuto, sia la navigazione tra diverse pagine web:



I compiti principali di un browser sono:

- **Connessione al server:** il browser stabilisce una connessione con il server web per comunicare.
- **Inviare richieste:** quando l'utente inserisce un indirizzo URL o interagisce con un contenuto, il browser invia una richiesta HTTP al server per ottenere le informazioni richieste.
- **Ricevere richieste:** il browser riceve la risposta dal server, che può essere il contenuto richiesto o eventuali dati o istruzioni aggiuntive.

- **Presentare il contenuto:** il browser visualizza il contenuto al pubblico, utilizzando tecnologie chiave come:
 - **HTML (HyperText Markup Language):** definisce la struttura della pagina web.
 - **CSS (Cascading Style Sheets):** gestisce lo stile e la presentazione grafica del contenuto.
 - **Javascript:** fornisce interattività dinamica e controlla il comportamento del contenuto nelle pagine web.

HTML è un linguaggio di markup contenente tag per la presentazione di:

- Testo (carattere, dimensione, colore)
- Immagini
- Link ad altre pagine web
- ...

Tali tag sono racchiusi dai simboli “<>”.

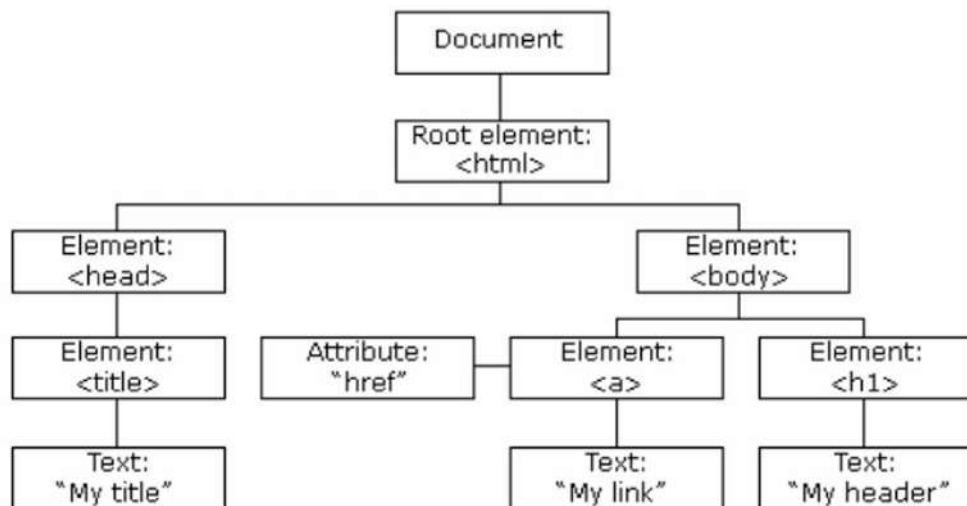
```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>This is a Heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

Il **DOM** (Document Object Model) di W3C è una piattaforma e interfaccia neutrale dal punto di vista linguistico che permette ai programmi e agli script di accedere e aggiornare dinamicamente il contenuto, la struttura e lo stile di un documento. Alcune delle sue caratteristiche principali sono:

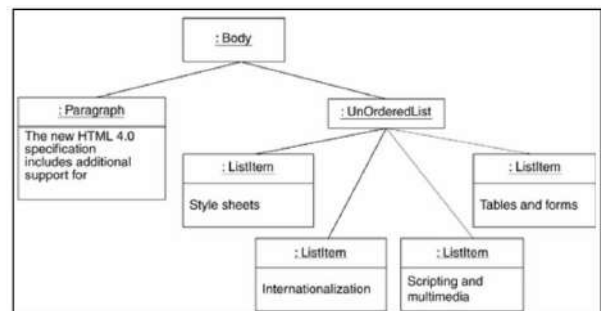
- **Struttura dati:** il DOM rappresenta la struttura del contenuto sotto forma di una struttura ad albero, in cui ogni nodo rappresenta una parte del documento (ad esempio un elemento HTML).
- **Modifiche dinamiche:** il DOM consente di prelevare, cambiare, aggiungere o eliminare elementi HTML. Questo permette di aggiornare il contenuto di una pagina web senza doverla ricaricare completamente.
- **Compatibilità universale:** il DOM è supportato da tutti i principali browser, il che significa che le modifiche dinamiche al contenuto della pagina sono possibili su qualsiasi browser.
- **Modifica tramite JavaScript:** JavaScript può interagire direttamente con il DOM per manipolare dinamicamente il contenuto e lo stile di una pagina web, permettendo funzionalità interattive avanzate.



I documenti sono trattati come oggetti caratterizzati da attributi e comportamenti. Va da sé che un documento è visto come una collezione di oggetti. DOM fornisce un'interfaccia orientata agli oggetti per gli elementi delle pagine, accessibili da script e programmi.

```

<p>The <em>new</em> <strong>HTML 4.0</strong>
specification includes additional support</p>
<ul>
  <li>Style sheets</li>
  <li>Internazionalization</li>
  <li>Accessibility</li>
  <li>Tables and forms</li>
  <li>Scripting and multimedia</li>
</ul>
  
```



CSS è un linguaggio che definisce lo stile di un documento HTML, ovvero come gli elementi HTML dovrebbero essere mostrati. Il CSS permette di adattare le pagine web in base a diverse caratteristiche, come:

- **Risoluzioni differenti:** il CSS può gestire la visualizzazione su schermi con diverse risoluzioni, come schermi ad alta definizione o schermi più piccoli.
- **Dispositivi differenti:** il CSS consente di adattare il contenuto per dispositivi diversi, come smartphone, tablet o computer.
- **Preferenze differenti:** il CSS può essere utilizzato per personalizzare l'aspetto delle pagine in base a preferenze degli utenti, come la scelta dei colori.
- **Media differenti:** il CSS permette di gestire la visualizzazione di contenuti per media diversi, come testo e video.

Il CSS applica queste personalizzazioni in un modo standard, garantendo che i browser interpretino e visualizzino correttamente il contenuto in modo coerente su diverse piattaforme e dispositivi.

```

<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-color: lightblue;
}

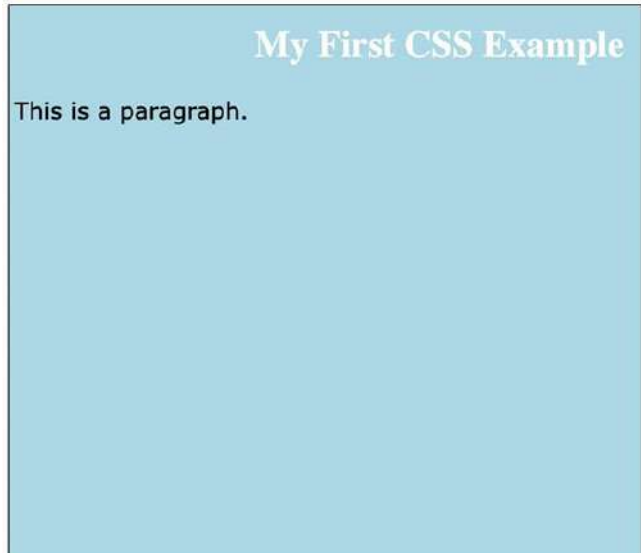
h1 {
  color: white;
  text-align: center;
}

p {
  font-family: verdana;
  font-size: 20px;
}
</style>
</head>
<body>

<h1>My First CSS Example</h1>
<p>This is a paragraph.</p>

</body>
</html>

```



JavaScript è un linguaggio simile a Java (ma non è Java) basato sugli oggetti (ma non orientato agli oggetti). L'utente non definisce né classi né ereditarietà. JavaScript è incluso nelle pagine HTML utilizzando il tag `<script>` ed è spesso utilizzato per migliorare l'interfaccia utente con animazioni su bottoni o menu. È usato anche per effetti speciali che migliorano l'esperienza visiva e interattiva della pagina. JavaScript può essere impiegato per gestire la logica di business all'interno di una pagina web, come ad esempio:

- **Validazione di un form:** Verifica che i dati inseriti dall'utente siano corretti prima di inviarli al server.
- **Analisi semplici:** JavaScript può eseguire operazioni semplici o complesse di calcolo e logica direttamente nel browser senza dover comunicare con un server.

JavaScript può interagire con tutti gli elementi della pagina HTML utilizzando il DOM. Può inoltre interagire con il Browser caricando nuove immagini, manipolando la history, ecc...

Il codice JavaScript in una pagina HTML viene interpretato ed eseguito sequenzialmente mentre la pagina viene letta (parsing). È preferibile scrivere funzioni JavaScript che implementano logiche specifiche che verranno eseguite solo quando necessario, ovvero quando si verifica un determinato **evento** (come il clic su un pulsante, il caricamento della pagina, o l'inserimento di dati in un form). Un evento attiva o "triggera" lo script, il che significa che l'esecuzione del codice dipende dall'interazione dell'utente o da altri eventi. Non tutti gli elementi HTML supportano tutti i tipi di eventi. Alcuni elementi, come le immagini o i paragrafi, possono non rispondere a certi tipi di eventi, a seconda del loro utilizzo e delle capacità previste dal browser. La seguente tabella elenca diversi tipi di eventi JavaScript associati a elementi HTML e i relativi **gestori di eventi** (event handlers):

Evento	Elemento	Descrizione	Event handler
Blur	Windows, frames, and all form elements	User removes input focus from window, frame, or form element.	onBlur
Click	Buttons, radio buttons, check boxes, Submit buttons, Reset buttons, links	User clicks form element or link.	onClick
Change	Text fields, text areas, select lists	User changes value of element.	onChange
Error	Images, windows	The loading of a document or image causes an error.	onError
Focus	Windows, frames, and all form elements	User gives input focus to window, frame, or form element.	onFocus
Load	Document body	User loads the page in the Navigator.	onLoad
Mouse out	Areas, links	User moves mouse pointer out of an area—client-side image map—or link.	onMouseout
Mouse over	Links	User moves mouse pointer over a link.	onMouseOver
Reset	Forms	User resets a form: clicks a Reset	onReset

Vediamo un primo esempio di JavaScript:

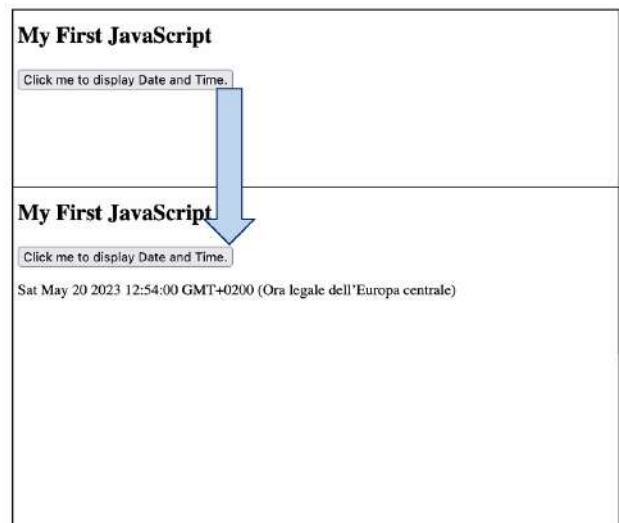
```
<!DOCTYPE html>
<html>
<script>
fun click(){
  document.getElementById("demo").innerHTML = "My First JavaScript";
}
</script>
<body>

<h2>My First JavaScript</h2>

<button type="button"
onclick="click()">
Click me to display Date and Time.</button>

<p id="demo"></p>

</body>
</html>
```



Vediamo un secondo esempio:

```
<!DOCTYPE html>
<html>
<script>
function main() {
  document.write( 'I am in control now, ha ha ha!' );
}
</script>
<head>
<title>test page</title>
</head>
<body onload="main()">
<p>This is the normal content. </p>
</body>
</html>
```

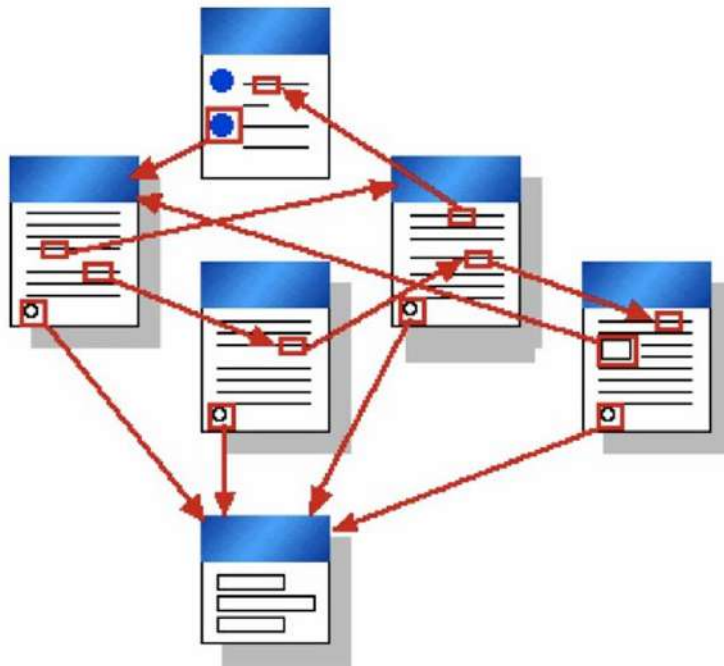
I am in control now, ha ha ha!

9.2. Introduzione ad HTML

L'**ipertesto (Hypertext)** è un insieme di documenti connessi tramite link direzionali chiamati **ipercollegamenti (Hyperlink)**. È una rete (o **grafo**) in cui i documenti rappresentano i **nodi**. Tramite i link possiamo navigare l'ipertesto andando da un documento a un altro. Le principali caratteristiche dell'ipertesto sono:

- **La lettura non è più lineare:** qualsiasi documento nell'ipertesto può essere il prossimo.
- **Ci sono opzioni di lettura illimitate:** la lettura non è più solo sequenziale.

Se i documenti non sono costituiti solamente da testo ma da generici elementi multimediali (immagini, audio, video), allora non si parla più di ipertesto bensì di **ipermedia (Hypermedia)**.



HTML sta per Hyper Text Markup Language ed è il linguaggio di markup standard per la creazione di pagine web. HTML descrive la struttura di una pagina web tramite una serie di elementi, i quali etichettano pezzi di contenuto tipo “Questo è il titolo”, “Questo è un paragrafo”, ecc. Un esempio di documento HTML è il seguente:

<code><!DOCTYPE html></code>	Definisce che questo documento è di tipo HTML5
<code><html></code>	Elemento radice di una pagina HTML
<code><head></code>	Contiene meta-informazioni riguardanti la pagina HTML
<code><title>Page Title</title></code>	Specifica un titolo per la pagina HTML
<code></head></code>	
<code><body></code>	Definisce il corpo del documento e rappresenta un contenitore per tutti i contenuti visibili
<code><h1>My First Heading</h1></code>	Definisce una voce grande
<code><p>My first paragraph.</p></code>	Definisce un paragrafo
<code></body></code>	
<code></html></code>	

Un browser web deve leggere i documenti HTML e mostrarli correttamente:



Sono state create svariate versioni di HTML, l'ultima è HTML5:

Berners-Lee idea:
put together
hypermedia and
Internet



HTML
• CERN
• 1991

HTML 2.0
• IETF
• 1995



HTML 3.2
• W3C
• 1997

W3C: World Wide
Web Consortium

HTML 4.01
• 1999

XHTML 1.0
• W3C
• 2000
• XML-based;
discontinued

WHATWG: Web Hypertext
Application Technology
Working Group



HTML5
• WHATWG
• and W3C
• 2014

Tutte le versioni fino a HTML 4 vengono comunemente chiamate HTML, ciò significa che **HTML**, **XHTML** e **HTML5** sono diversi tra loro.

XHTML è praticamente una versione più aggiornata e rigorosa di HTML 4, quindi non ci sono chissà quali differenze tra le due versioni:

- HTML accetta che alcuni elementi possano non contenere il tag di chiusura. XHTML si aspetta invece che tutti gli elementi (senza eccezioni) includano un tag di chiusura.
- XHTML richiede che tutti i valori degli attributi (ad esempio la grandezza del font) siano tra virgolette, anche quelli numerici. Questo requisito non esiste in HTML.

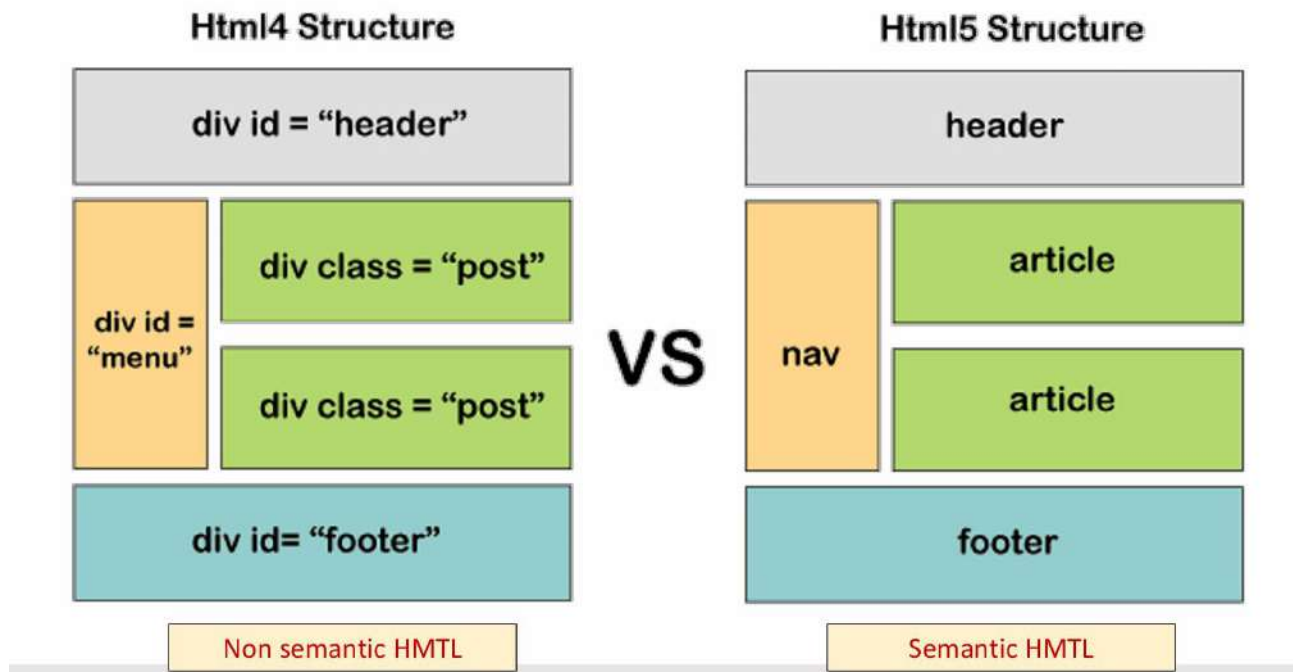
Quelle appena citate sono le differenze più ovvie tra le due versioni. La lista completa è sostanzialmente più lunga, ma la cosa fondamentale da ricordare è che XHTML è stato sviluppato per sopperire alle carenze di HTML tramite l'introduzione di alcune feature di XML.

La maggior parte delle differenze tra HTML e HTML5 si applicano anche a quest'ultimo. Tuttavia, c'è qualche ulteriore variazione tra i due:

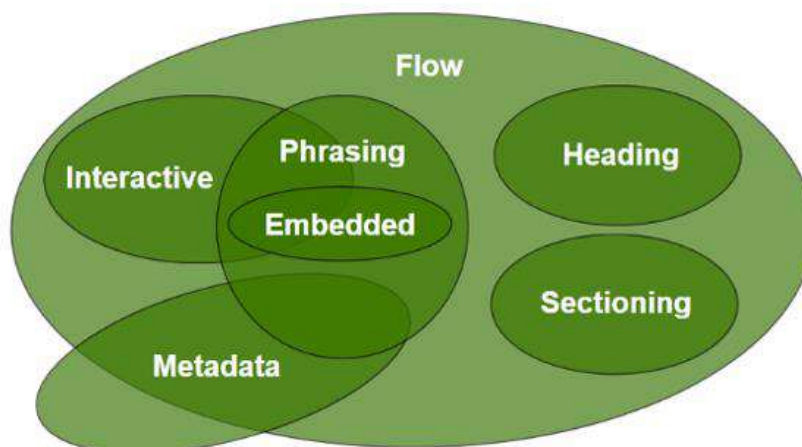
- XHTML è case sensitive (come HTML), mentre HTML5 non lo è.

- Sia XHTML che HTML hanno un doctype più complesso rispetto ad HTML5. Il doctype si occupa di dire ai browser come interpretare i dati.
- HTML5 è compatibile con tutti i browser mentre XHTML no.
- HTML5, come successore di HTML, è molto più flessibile di XHTML.
- XHTML è più compatibile con computer desktop, mentre HTML5 è più compatibile con dispositivi mobile (smartphone, tablet).

La seguente immagine confronta la struttura di una pagina HTML4 con quella di una pagina HTML5, evidenziando la differenza tra **HTML non semantico** e **HTML semantico**:



La seguente immagine illustra i vari tipi di contenuto HTML e come questi si relazionano tra loro, evidenziando diverse categorie di elementi HTML utilizzati nella strutturazione e presentazione dei documenti web:



1. Sectioning Content: definisce intestazioni e piè di pagina. Gli elementi di sectioning dividono il contenuto di una pagina in sezioni logiche. Esempi includono elementi come `<section>`, `<article>`, `<nav>`, e `<aside>`. Questi elementi aiutano a strutturare il contenuto in blocchi, ciascuno con il proprio scopo o contesto.

2. Heading Content: definisce le intestazioni di una sezione. Gli elementi di intestazione come `<h1>`, `<h2>`, etc., rappresentano i titoli di ciascuna sezione, fornendo gerarchia e struttura al contenuto.

3. Phrasing Content: è il testo del documento, così come gli elementi che marcano il testo a livello intra-paragrafo. Gli elementi di phrasing includono quelli utilizzati per formattare il testo come `<span>`, `<a>`, `<strong>`, `<em>`, ecc. Il phrasing content rappresenta parti del testo che si trovano all'interno dei paragrafi.

4. Flow Content: include la maggior parte degli elementi HTML. Il contenuto di flusso rappresenta la combinazione di tutti gli altri tipi di contenuto, come interactive, metadata, phrasing, embedded, heading, e sectioning. È il contenuto che fluisce naturalmente nella pagina e viene visualizzato in modo sequenziale.

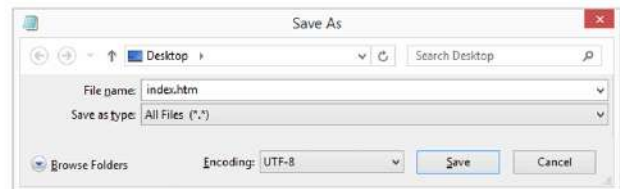
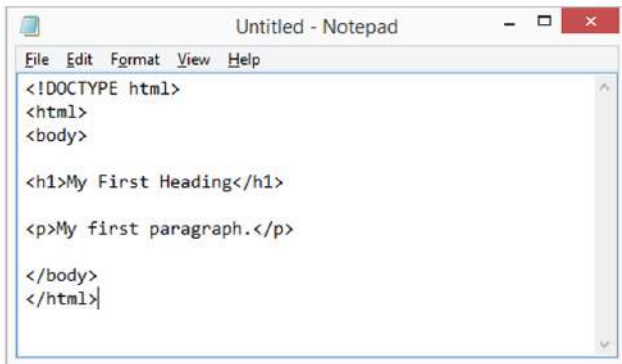
5. Interactive Content: comprende elementi con cui gli utenti possono interagire, come form, link o bottoni. Esempi includono `<button>`, `<a>`, `<input>`, ecc.

6. Embedded Content: include contenuti che vengono inseriti in un documento, come immagini, video o iframe. Elementi di esempio sono `<img>`, `<video>`, `<audio>`, e `<iframe>`.

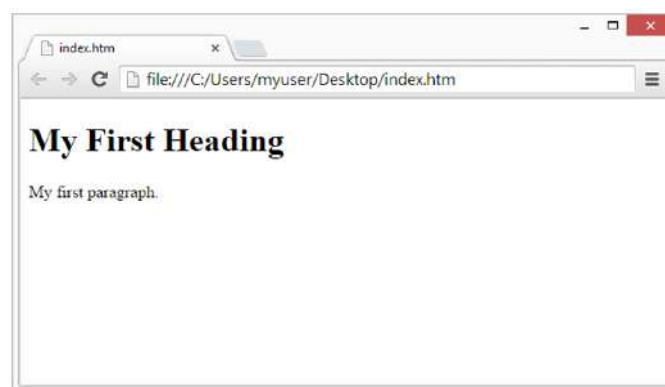
7. Metadata Content: fornisce informazioni aggiuntive sulla pagina o il documento, come informazioni di collegamento o meta-informazioni. Gli elementi di metadati includono `<meta>`, `<link>`, `<title>`, ecc.

9.3. Creare pagine HTML

Le pagine web possono essere create e modificate utilizzando degli editor HTML professionali. Ad ogni modo, per imparare HTML è sufficiente utilizzare un semplice editor di testo, come ad esempio Notepad. Una volta aperta un'istanza di Notepad e scritto del codice HTML, dobbiamo salvare il file con l'estensione .html o .htm:



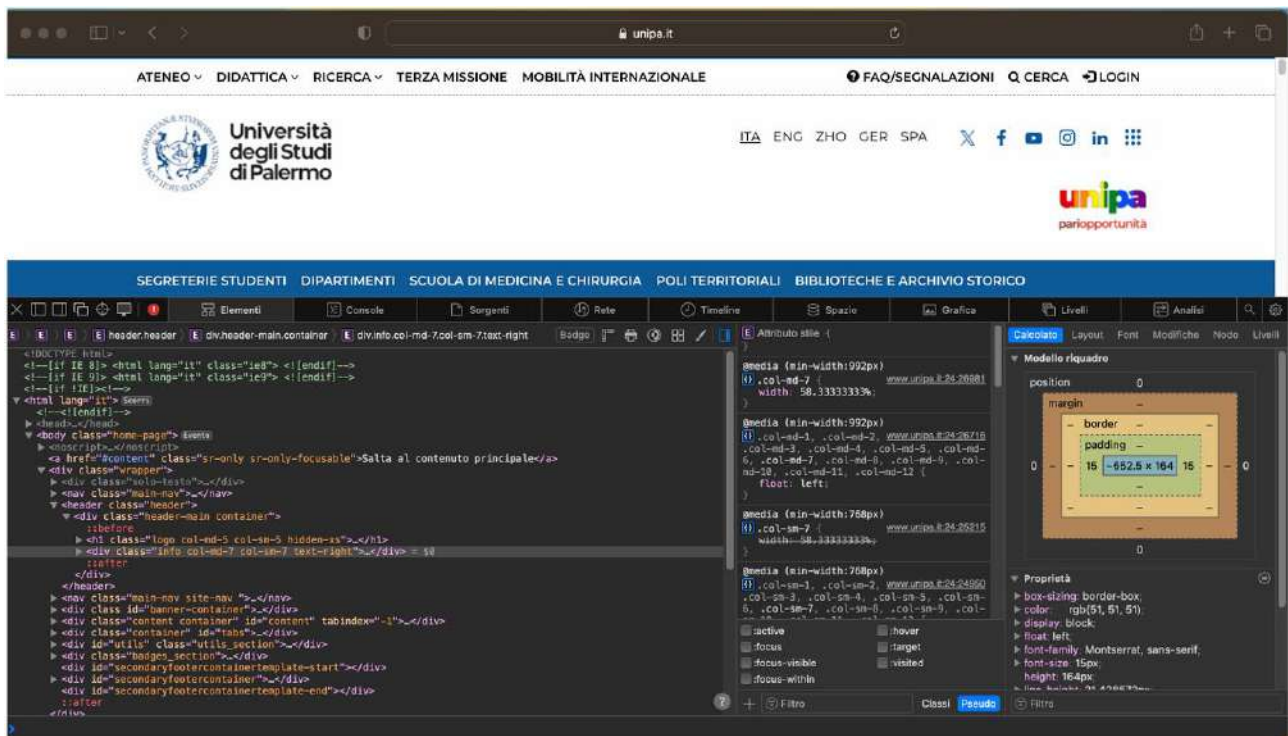
Aprendo il file salvato, verrà mostrato il seguente output nel nostro browser predefinito:



Scegliendo una pagina web qualsiasi (ad esempio quella di Unipa), possiamo fare un click destro e scegliere la voce "Ispeziona elemento":



Ci verrà mostrata una sezione chiamata **DevTools** (strumenti per sviluppatori), solitamente nella parte inferiore o laterale della finestra del browser. Questo pannello consente di esplorare il DOM (Document Object Model), il codice CSS, le risorse di rete e molto altro:



9.4. Basi di HTML

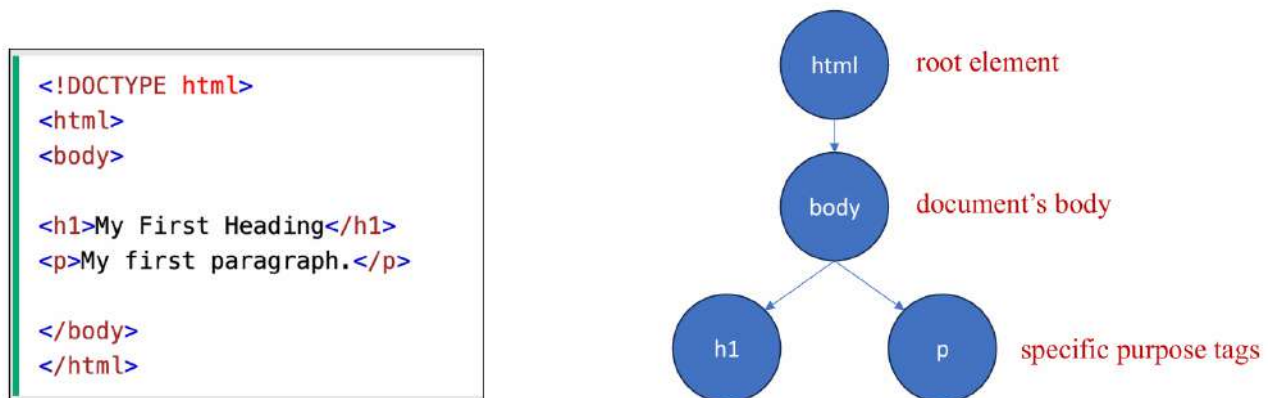
HTML si basa su tre regole principali:

- Tutti i documenti HTML devono iniziare con una dichiarazione del tipo di documento:
`<!DOCTYPE html>`
- Il documento HTML stesso inizia con `<html>` e finisce con `</html>`.
- La parte visibile del documento si trova tra `<body>` e `</body>`.

Quelli che abbiamo menzionato sono tutti elementi HTML, ma come sono definiti propriamente gli elementi HTML? Un elemento HTML è tutto quello che va dal tag di inizio al tag di fine:

`<tagname>Content goes here...</tagname>`

Gli elementi HTML possono essere annidati e creano una struttura ad albero che descrive il documento. Vediamo un esempio:



Gli elementi HTML senza contenuto sono chiamati **elementi vuoti**. I tag HTML non sono case sensitive, ad esempio `<P>` ha lo stesso significato di `<p>`.

```
<p>This is a <br> paragraph with a line break.</p>
```

Gli elementi HTML sono usati per definire il **significato** di una porzione di documento (gruppo semantico). Tale significato sarà renderizzato graficamente attraverso CSS secondo i fogli di stile (Style Sheets), i quali sono responsabili della presentazione del documento: colori, dimensioni, layout, font e altri dettagli visivi sono gestiti separatamente dal contenuto e dalla sua struttura semantica. Secondo i moderni standard web, infatti, l'HTML non dovrebbe essere usato per gestire l'aspetto grafico del documento.

Ogni elemento ha un valore di visualizzazione di default:

`display: block;`

- Occupa tutta la larghezza disponibile.
- Comincia su una nuova riga.
- Disposizione dall'alto verso il basso.

display:inline:

- Non comincia su una nuova riga.
- Occupa solo lo spazio necessario.
- Disposizione da sinistra verso destra.

Questo fa capire che questi valori influenzano il modo in cui gli elementi vengono visualizzati nella pagina web. Alcuni comuni elementi di base HTML sono:

- Il tag **<p>** definisce un paragrafo, che è un tipo di elemento a blocco. I browser aggiungono automaticamente una linea vuota prima e dopo ogni elemento **<p>**:

```
<p>This is some text in a paragraph.</p>
```

- Il tag **
** inserisce una singola interruzione di linea (elemento di tipo inline). Questo tag è di tipo inline (significa che non interrompe il flusso del testo e occupa solo lo spazio necessario per il suo contenuto) ed è vuoto, il che significa che non ha un tag di chiusura:

```
<p>To force<br> line breaks<br> in a text,<br> use the br<br> element.</p>
```

- Il tag **<a>** di tipo inline definisce un ipercollegamento, il quale è usato per collegare una pagina ad un'altra. L'attributo più importante di **<a>** è **href**, che indica la destinazione del link:

```
<a href="https://www.w3schools.com">Visit W3Schools.com!</a>
```

- Il tag **<div>** di tipo a blocco definisce una divisione o sezione in un documento HTML. È usato come contenitore per elementi HTML (di qualsiasi tipo) ed è stilizzato con CSS o manipolato con JavaScript. Questo tag è facilmente stilizzabile usando l'attributo **class** o **id**:

```
<html>
<head>
</head>
<body>

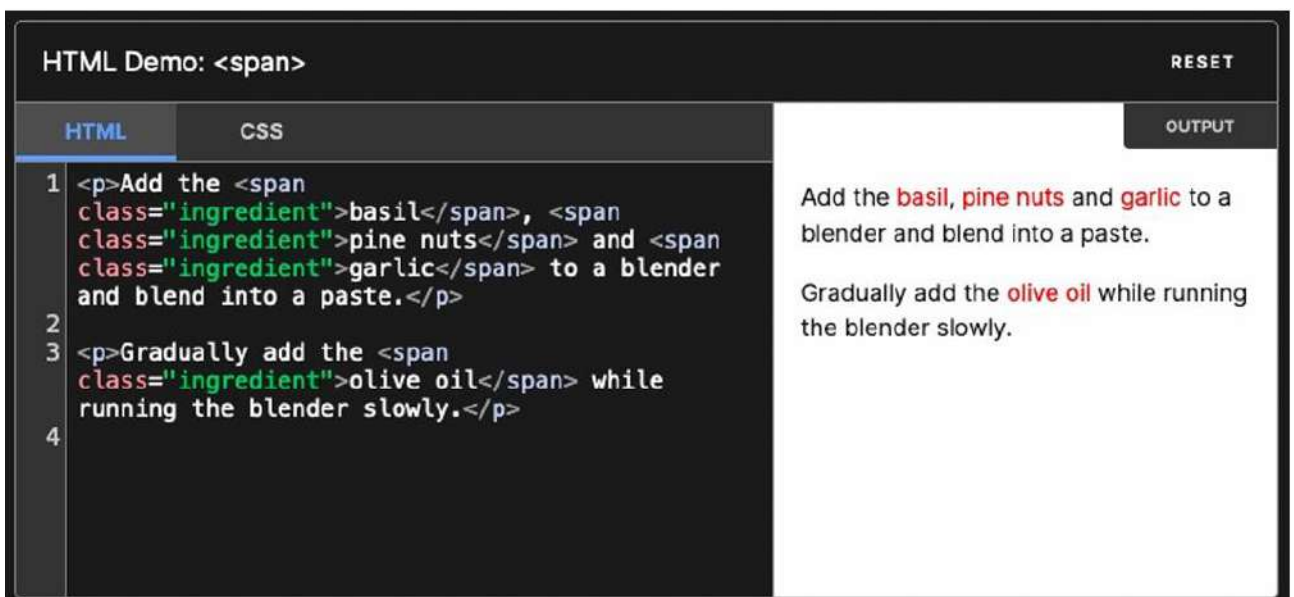
  <div class="myDiv">
    <h2>This is a heading in a div element</h2>
    <p>This is some text in a div element.</p>
  </div>

</body>
</html>
```

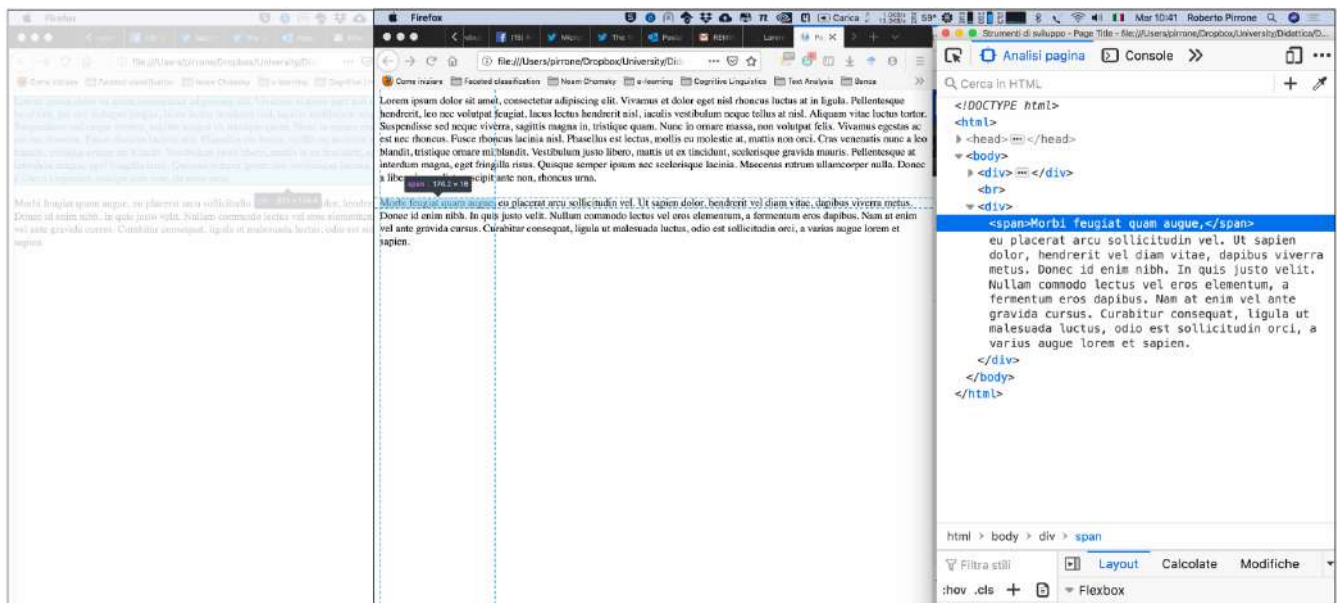
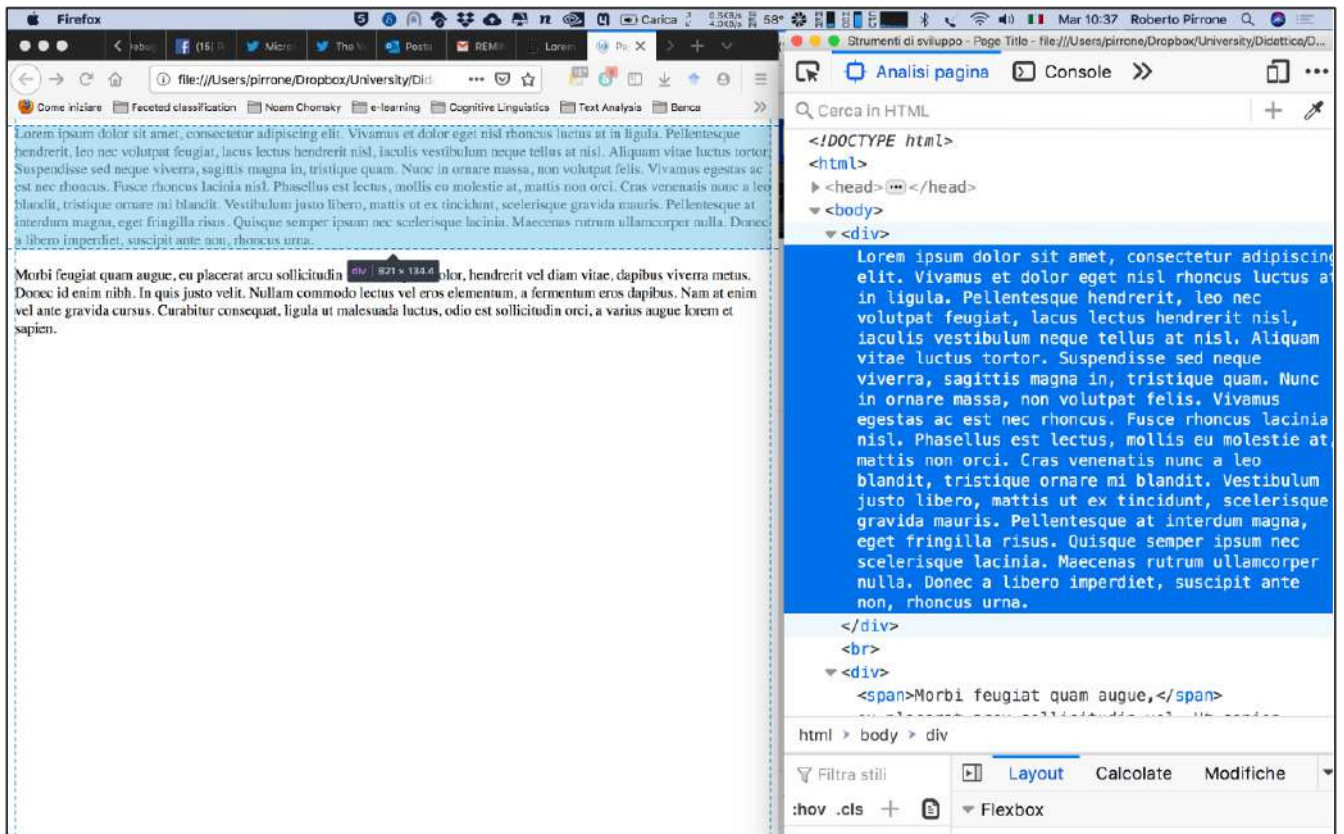
- Il tag **** di tipo inline è usato per incorporare un'immagine in una pagina HTML. Richiede due attributi, ovvero **src** (specifica il percorso dell'immagine) e **alt** (specifica un testo alternativo per l'immagine se questa non può essere mostrata per qualche motivo):



- Il tag **** di tipo inline è un contenitore usato per evidenziare una parte del testo o del documento. È facilmente stilizzabile con CSS o manipolabile tramite JavaScript usando gli attributi **class** o **id**:



I tag **** e **<div>** sono entrambi contenitori, quindi qual è la differenza?



- I tag `<ol>` e `<ul>` di tipo a blocco definiscono rispettivamente una lista ordinata (che può essere numerica o alfabetica) e una non ordinata. Il tag `<li>` è usato per definire ogni elemento della lista:

<pre> <!DOCTYPE html> <html> <body> <h1>The ol element</h1> Coffee Tea Milk <ol start="50"> Coffee Tea Milk </body> </html> </pre>	The ol element 1. Coffee 2. Tea 3. Milk 50. Coffee 51. Tea 52. Milk
<pre> <!DOCTYPE html> <html> <body> <h1>The ul element</h1> Coffee Tea Milk </body> </html> </pre>	The ul element • Coffee • Tea • Milk

- Il tag `<b>` di tipo inline viene utilizzato per applicare il grassetto al testo:

HTML Demo: <code></code>		RESET
HTML	CSS	OUTPUT
<pre> 1 <p>The two most popular science courses offered by the school are <b class="term">chemistry (the study of chemicals and the composition of substances) and <b class="term">physics (the study of the nature and properties of matter and energy).</p> 2 </pre>		The two most popular science courses offered by the school are chemistry (the study of chemicals and the composition of substances) and physics (the study of the nature and properties of matter and energy).

- Il tag `<em>` di tipo inline viene utilizzato per enfatizzare una parte del testo:

HTML Demo: `<em>`

RESET

HTML

CSS

OUTPUT

```

1 <p>Get out of bed <em>now</em>!/</p>
2
3 <p>We <em>had</em> to do something about it.</p>
4
5 <p>This is <em>not</em> a drill!</p>
6

```

Get out of bed *now*!

We *had* to do something about it.

This is *not* a drill!

- Il tag `<i>` di tipo inline viene utilizzato per rendere il testo in corsivo (*italic*). A differenza del tag `<em>`, il tag `<i>` non conferisce alcun significato semantico al testo, ma serve solo per applicare uno stile visivo:

HTML Demo: `<i>`

RESET

HTML

CSS

OUTPUT

```

1 <p>I looked at it and thought <i>This can't be
  real!</i></p>
2
3 <p><i>Musa</i> is one of two or three genera in
  the family <i>Musaceae</i>; it includes bananas
  and plantains.</p>
4
5 <p>The term <i>bandwidth</i> describes the
  measure of how much information can pass
  through a data connection in a given amount of
  time.</p>
6

```

I looked at it and thought *This can't be real!*

Musa is one of two or three genera in the family *Musaceae*; it includes bananas and plantains.

The term *bandwidth* describes the measure of how much information can pass through a data connection in a given amount of time.

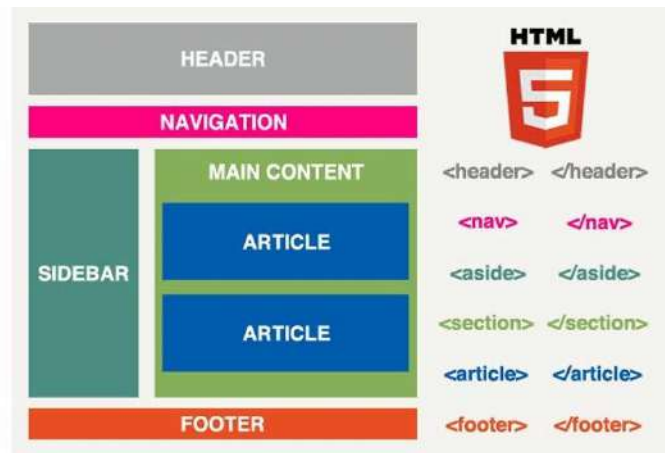
La seguente immagine fornisce una panoramica sugli elementi semantici introdotti con HTML5, organizzati in due categorie principali: **Sectioning content** e **Heading content**.

Sectioning content

- article
- aside
- nav
- section
- header
- footer

Heading content

- h1
- h2
- h3
- h4
- h5
- h6



Gli elementi di "sectioning content" sono utilizzati per definire sezioni diverse e semantiche della pagina web. Essi aiutano a strutturare meglio il contenuto in modo logico:

- **<article>**: rappresenta un contenuto autonomo che potrebbe essere distribuito indipendentemente, come articoli, post di blog o commenti.

```
<article>
  
  <p>My fave Masif tee so far!</p>
  <footer>Posted 2 days ago</footer>
</article>
<article>
  
  <p>Happy 2nd birthday Masif Saturdays!!!</p>
  <footer>Posted 3 weeks ago</footer>
</article>
```

HTML Demo: <article>

RESET

HTML	CSS	OUTPUT
<pre> 1 <article class="forecast"> 2 <h1>Weather forecast for Seattle</h1> 3 <article class="day-forecast"> 4 <h2>03 March 2018</h2> 5 <p>Rain.</p> 6 </article> 7 <article class="day-forecast"> 8 <h2>04 March 2018</h2> 9 <p>Periods of rain.</p> 10 </article> 11 <article class="day-forecast"> 12 <h2>05 March 2018</h2> 13 <p>Heavy rain.</p> 14 </article> 15 </article> 16 </pre>		Weather forecast for Seattle 03 March 2018 Rain. 04 March 2018 Periods of rain. 05 March 2018 Heavy rain.

- **<aside>**: utilizzato per contenuti secondari o marginali, come barre laterali (sidebar) o contenuti correlati.

```
<h1>Music</h1>
<p>As any burner can tell you, the event has a lot of trance.</p>
<aside>You can buy the music we played at our <a
href="buy.html">playlist page</a>.</aside>
<p>This year we played a kind of trance that originated in Belgium,
Germany, and the Netherlands in the mid-90s.</p>
```

HTML Demo: <aside>

RESET

HTML	CSS	OUTPUT
<pre> 1 <p>Salamanders are a group of amphibians with a lizard-like appearance, including short legs and a tail in both larval and adult forms.</p> 2 3 <aside> 4 <p>The Rough-skinned Newt defends itself with a deadly neurotoxin.</p> 5 </aside> 6 7 <p>Several species of salamander inhabit the temperate rainforest of the Pacific Northwest, including the Ensatina, the Northwestern Salamander and the Rough-skinned Newt. Most salamanders are nocturnal, and hunt for insects, worms and other small creatures.</p> 8 </pre>		Salamanders are a group of amphibians with a lizard-like appearance, including short legs and a tail in both larval and adult forms. Several species of salamander inhabit the temperate rainforest of the Pacific Northwest, including the Ensatina, the Northwestern Salamander and the Rough-skinned Newt. Most salamanders are nocturnal, <i>The Rough-skinned Newt defends itself with a deadly neurotoxin.</i>

- **<nav>**: definisce un insieme di link di navigazione che portano a diverse sezioni della stessa pagina o ad altre pagine.

```
<nav>
<p><a href="/">Home</a>
<p><a href="/biog.html">Bio</a>
<p><a href="/discog.html">Discog</a>
</nav>
```

HTML Demo: <nav>

RESET

HTML	CSS	OUTPUT
<pre> 1 <nav class="crumbs"> 2 3 <li class="crumb"><a 4 href="#">Bikes 5 <li class="crumb"><a 6 href="#">BMX 7 <li class="crumb">Jump Bike 3000 8 9 </nav> 10 <h1>Jump Bike 3000</h1> 11 <p>This BMX bike is a solid step into the pro world. It looks as legit as it rides and is built to polish your skills.</p> </pre>		Bikes > BMX > Jump Bike 3000 Jump Bike 3000 This BMX bike is a solid step into the pro world. It looks as legit as it rides and is built to polish your skills.

- **<section>**: definisce una sezione tematica del documento, utilizzata per raggruppare contenuti correlati.

```
<h1>Biography</h1>
<section>
  <h1>The facts</h1>
  <p>1500+ shows, 14+ countries</p>
</section>
<section>
  <h1>2010/2011 figures per year</h1>
  <p>100+ shows, 8+ countries</p>
</section>
```

HTML Demo: <section> RESET

HTML CSS OUTPUT

```

1 <h1>Choosing an Apple</h1>
2 <section>
3   <h2>Introduction</h2>
4   <p>This document provides a guide to help
   with the important task of choosing the correct
   Apple.</p>
5 </section>
6
7 <section>
8   <h2>Criteria</h2>
9   <p>There are many different criteria to be
   considered when choosing an Apple – size,
   color, firmness, sweetness, tartness...</p>
10 </section>
11

```

Choosing an Apple

Introduction

This document provides a guide to help with the important task of choosing the correct Apple.

Criteria

There are many different criteria to be considered when choosing an Apple — size, color, firmness, sweetness, tartness

- **<header>**: rappresenta l'intestazione di una sezione o di una pagina. Solitamente contiene il titolo, logo, o menu di navigazione.

```
<article>
  <header>
    <h1>Hard Trance is My Life</h1>
    <p>By DJ Steve Hill and Technikal</p>
  </header>
  <p>The album with the amusing punctuation has red artwork.</p>
</article>
```

HTML Demo: <header> RESET

HTML CSS OUTPUT

```

1 <header>
2   <a class="logo" href="#">Cute Puppies
   Express!</a>
3 </header>
4
5 <article>
6   <header>
7     <h1>Beagles</h1>
8     <time>08.12.2014</time>
9   </header>
10  <p>I love beagles <em>so</em> much! Like,
   really, a lot. They're adorable and their ears
   are so, so snuggly soft!</p>
11 </article>
12

```



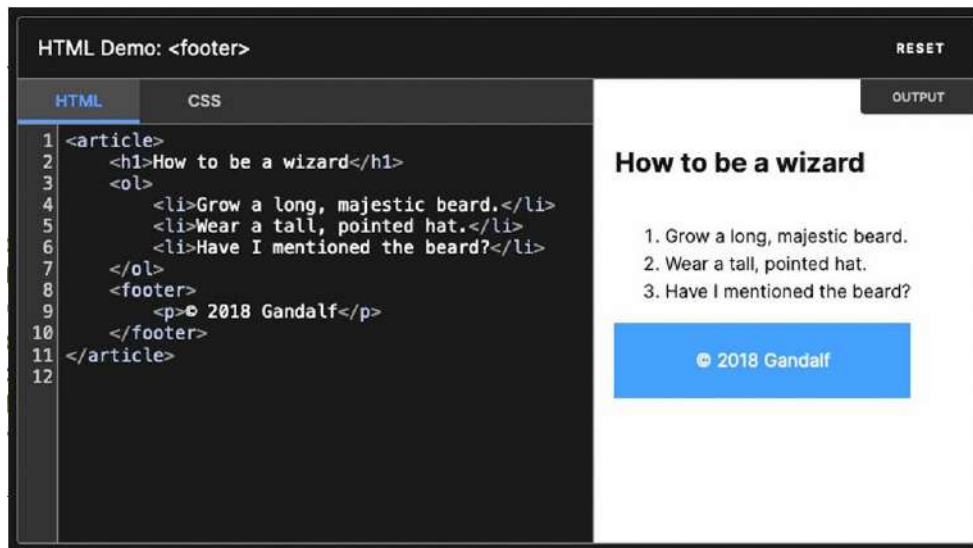
Beagles

08.12.2014

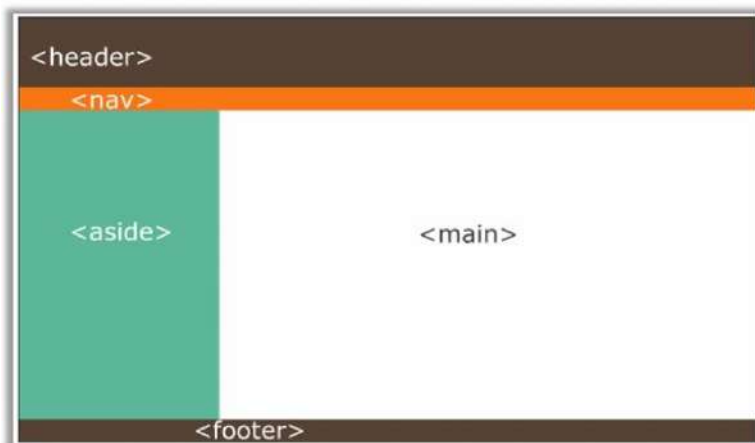
I love beagles so much! Like, really, a lot. They're adorable and their ears are so, so snuggly soft!

- **<footer>**: definisce il piè di pagina di una sezione o di una pagina. Solitamente contiene informazioni sul copyright, link o contatti.

```
<article>
  <h1>Hard Trance is My Life</h1>
  <p>The album with the amusing punctuation has red artwork.</p>
  <footer>
    <p>Artists: DJ Steve Hill and Technikal</p>
  </footer>
</article>
```



In generale vale questo tipo di gerarchia:



```
1 <body>
2   <header>
3     <nav>
4
5     </nav>
6   </header>
7   <aside>
8
9   </aside>
10  <main>
11
12  </main>
13  <footer>
14
15  </footer>
16 </body>
```

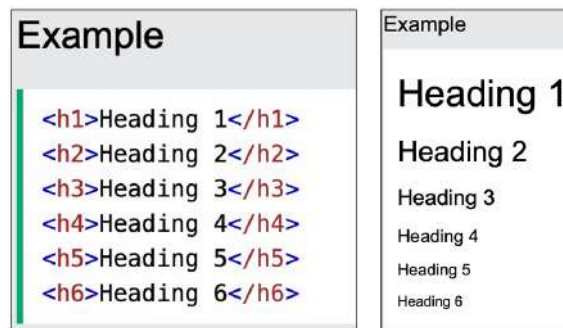
Tornando un attimo indietro, qual è la differenza tra <article> e <section>?

- **<section>**: Rappresenta una **parte di qualcosa di più grande**. È utilizzata per suddividere il contenuto di un documento in sezioni tematiche, che possono far parte di un insieme più ampio di contenuti.
- **<article>**: Rappresenta un **contenuto indipendente** che ha senso anche da solo. Un articolo può essere letto separatamente dal resto del contenuto e mantenere il suo significato.

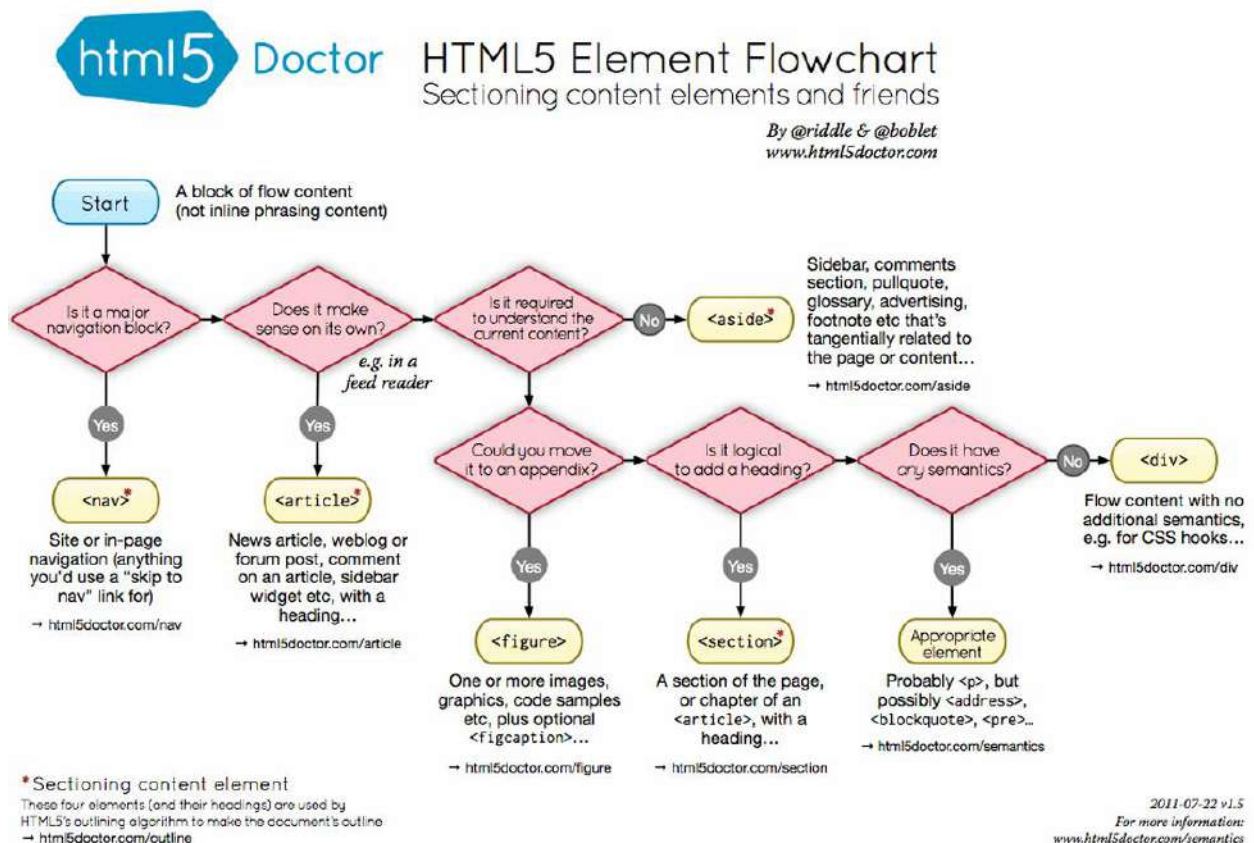
Immaginiamo un libro con un capitolo chiamato "Granny Smith" che discute di come le mele Granny Smith siano perfette per fare torte di mele. Questo capitolo sarebbe una `<section>` del libro perché fa parte di un contenuto più ampio, probabilmente composto da altri capitoli che parlano di altre varietà di mele. Al contrario, se troviamo un post su Twitter, un commento su Reddit, o un annuncio di giornale che dice semplicemente "Granny Smith. Queste succose mele verdi sono perfette per fare torte di mele.", questo sarebbe un `<article>`, poiché è un pezzo di contenuto autonomo che ha senso da solo.

Gli elementi di "heading content" definiscono i vari livelli di titoli per organizzare gerarchicamente il contenuto:

- `<h1>`: titolo principale, usato una sola volta per identificare il titolo più importante della pagina.
- `<h2>` - `<h6>`: titoli secondari e di sottolivello, con `<h2>` che rappresenta un sottotitolo immediatamente sotto `<h1>`, e così via fino a `<h6>`.



La seguente immagine mostra un diagramma di flusso che aiuta a decidere quale elemento HTML5 semantico utilizzare per il contenuto di una pagina web. Ecco come funziona il processo:



9.5. Attributi HTML

Tutti gli elementi HTML possono avere attributi. Tali attributi forniscono informazioni aggiuntive sugli elementi e sono sempre specificati nel tag di inizio. Solitamente si trovano nella forma coppia nome/valore come ad esempio name = "value". Vediamone qualcuno:

- L'attributo **href** specifica l'URL della pagina a cui porta un link:

```
<!DOCTYPE html>
<html>
<body>

<h2>The href Attribute</h2>

<p>HTML links are defined with the a tag. The link address is specified in the href attribute:</p>

<a href="https://www.w3schools.com">Visit W3Schools</a>

</body>
</html>
```

The href Attribute

HTML links are defined with the a tag. The link address is specified in the href attribute:

[Visit W3Schools](https://www.w3schools.com)

Di default, i link si presentano nel seguente modo in tutti i browser:

- Un link **non visitato** è sottolineato e blu.
- Un link **visitato** è sottolineato e viola.
- Un link **attivo** è sottolineato e rosso.

Sempre di default, la pagina linkata sarà mostrata nella finestra del browser corrente. Per modificare questa cosa, l'attributo **target** specifica dove aprire il documento linkato, e può avere i seguenti parametri:

- **_self** (di default): apre il documento nella stessa finestra/scheda in cui è stato cliccato.
- **_blank**: apre il documento in una nuova finestra/scheda.
- **_parent**: apre il documento nel frame genitore.
- **_top**: apre il documento nel corpo della finestra.

```
<!DOCTYPE html>
<html>
<body>

<h2>The target Attribute</h2>

<a href="https://www.w3schools.com/" target="_blank">Visit W3Schools!</a>

<p>If target="_blank", the link will open in a new browser window or tab.
</p>

</body>
</html>
```

The target Attribute

[Visit W3Schools!](https://www.w3schools.com/)

If target="_blank", the link will open in a new browser window or tab.



- L'attributo **title** specifica informazioni extra riguardanti un elemento. Tali informazioni sono spesso mostrate come testo **tooltip** quando il mouse si muove sopra l'elemento interessato.

```

<!DOCTYPE html>
<html lang="en-US">
<body>

<h2>Link Titles</h2>
<p>The title attribute specifies extra information about an element. The
information is most often shown as a tooltip text when the mouse moves over
the element.</p>
<a href="https://www.w3schools.com/html/" title="Go to W3Schools HTML
section">Visit our HTML Tutorial</a>

</body>
</html>

```

Link Titles

The title attribute specifies extra information about an element. The information is most often shown as a tooltip text when the mouse moves over the element.

[Visit our HTML Tutorial](https://www.w3schools.com/html/ "Go to W3Schools HTML section")

Esistono tantissimi altri attributi che si possono sfruttare, ma due dei più importanti sono **class** e **id**:

- L'attributo **class** è usato per specificare una classe per un elemento HTML. Diversi elementi HTML possono condividere la stessa classe. In CSS e JavaScript, le classi vengono spesso utilizzate per **stilizzare** e **manipolare** specifiche porzioni di contenuto. Un esempio di uso dell'attributo class è:

```
<div class="aa bb cc"></div>
```

Il precedente esempio mostra che un elemento può avere più classi, separate da spazi.

```

<!DOCTYPE html>
<html>
<head>
<style>
.city {
  background-color: tomato;
  color: white;
  border: 2px solid black;
  margin: 20px;
  padding: 20px;
}
</style>
</head>
<body>

<div class="city">
  <h2>London</h2>
  <p>London is the capital of England.</p>
</div>

<div class="city">
  <h2>Paris</h2>
  <p>Paris is the capital of France.</p>
</div>

<div class="city">
  <h2>Tokyo</h2>
  <p>Tokyo is the capital of Japan.</p>
</div>

</body>
</html>

```

London

London is the capital of England.

Paris

Paris is the capital of France.

Tokyo

Tokyo is the capital of Japan.

- L'attributo **id** specifica un id univoco per un elemento HTML. Il valore di tale id deve essere unico in tutto il documento HTML. Viene ampiamente utilizzato in CSS e JavaScript per **trovare** e **modificare** un elemento specifico nel DOM. Un esempio di uso dell'attributo id è:

```

<!DOCTYPE html>
<html>
<head>
<style>
#myHeader {
  background-color: lightblue;
  color: black;
  padding: 40px;
  text-align: center;
}
</style>
</head>
<body>

<h1 id="myHeader">My Header</h1>

</body>
</html>

```

The id Attribute

Use CSS to style an element with the id "myHeader":

My Header

```

<style>
/* Style the element with the id "myHeader" */
#myHeader {
  background-color: lightblue;
  color: black;
  padding: 40px;
  text-align: center;
}

/* Style all elements with the class name "city" */
.city {
  background-color: tomato;
  color: white;
  padding: 10px;
}
</style>

<!-- An element with a unique id -->
<h1 id="myHeader">My Cities</h1>

<!-- Multiple elements with same class -->
<h2 class="city">London</h2>
<p>London is the capital of England.</p>

<h2 class="city">Paris</h2>
<p>Paris is the capital of France.</p>

<h2 class="city">Tokyo</h2>
<p>Tokyo is the capital of Japan.</p>

```

Difference Between Class and ID

A class name can be used by multiple HTML elements, while an id name must only be used by one HTML element within the page:

9.6. Elementi HTML – Form

L'elemento HTML **<form>** è usato per acquisire l'input utente. Tale input è spesso inviato ad un server per essere processato:

```

<form>
  •
  form elements
  •
</form>

```

L'elemento **<form>** è un contenitore per differenti tipi di elementi di input, come ad esempio:

- Text fields
- Checkboxes
- Radio buttons
- ...

L'elemento HTML **<input>** è l'elemento form più usato e può essere mostrato in molti modi a seconda del tipo dell'attributo usato. Tra i più comuni abbiamo:

Type	Description
<code><input type="text"></code>	Displays a single-line text input field
<code><input type="radio"></code>	Displays a radio button (for selecting one of many choices)
<code><input type="checkbox"></code>	Displays a checkbox (for selecting zero or more of many choices)
<code><input type="submit"></code>	Displays a submit button (for submitting the form)
<code><input type="button"></code>	Displays a clickable button

Per quanto riguarda il **primo tipo di attributo** della tabella, vediamo un esempio:

```
<form>
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname">
</form>
```

First name:

Last name:

Attenzioniamo il tag **<label>**, il quale definisce un'etichetta per molti elementi form. Il tag **<label>** è particolarmente utile per l'accessibilità. Quando un utente utilizza uno screen reader, questo leggerà ad alta voce il testo del **<label>** quando l'utente si concentra sull'elemento associato, migliorando l'esperienza di navigazione per persone con disabilità visive. Il tag aiuta inoltre gli utenti che hanno difficoltà a cliccare su piccole aree, come le caselle di spunta o i pulsanti di opzione. Se un utente fa clic sul testo all'interno dell'elemento **<label>**, esso attiva anche l'elemento del modulo associato (come una casella di controllo o un radio button), rendendo l'interazione più semplice. Infine, è bene chiarire che l'attributo **for** del tag **<label>** deve corrispondere all'attributo **id** dell'elemento di input a cui è collegato. Questo stabilisce un collegamento tra l'etichetta e l'elemento di input, consentendo che l'etichetta attivi l'input quando viene cliccata.

Vediamo un esempio del **secondo tipo di attributo**:

<p>Choose your favorite Web language:</p>

```
<form>
  <input type="radio" id="html" name="fav_language" value="HTML">
  <label for="html">HTML</label><br>
  <input type="radio" id="css" name="fav_language" value="CSS">
  <label for="css">CSS</label><br>
  <input type="radio" id="javascript" name="fav_language" value="JavaScript">
  <label for="javascript">JavaScript</label>
</form>
```

Choose your favorite Web language:

- ☐ HTML
- ☐ CSS
- ☐ JavaScript

Vediamo un esempio del **terzo tipo di attributo**:

```
<form>
  <input type="checkbox" id="vehicle1" name="vehicle1" value="Bike">
  <label for="vehicle1"> I have a bike</label><br>
  <input type="checkbox" id="vehicle2" name="vehicle2" value="Car">
  <label for="vehicle2"> I have a car</label><br>
  <input type="checkbox" id="vehicle3" name="vehicle3" value="Boat">
  <label for="vehicle3"> I have a boat</label>
</form>
```

- ☐ I have a bike
- ☐ I have a car
- ☐ I have a boat

Vediamo un esempio del **quarto tipo di attributo**:

```
<form action="/action_page.php">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
  <input type="submit" value="Submit">
</form>
```

First name:

John

Last name:

Doe

Submit

Attenzioniamo l'attributo **<action>**, il quale specifica dove inviare i dati del modulo. In questo caso, il modulo invierà i dati a **action_page.php**, che è il file che si occuperà di gestire i dati inviati. Questo significa che, quando l'utente preme il pulsante "Submit", i dati inseriti nel modulo verranno inviati al file `action_page.php`. Se l'attributo `action` è omissso, i dati verranno inviati alla pagina corrente, ovvero quella in cui si trova il modulo. Il **form-handler** è lo script o file che processa i dati inviati, ed è proprio l'attributo `action` a specificare quale sia. In questo caso, si fa riferimento a `action_page.php` come form-handler. Questo file riceve i dati inviati e li elabora (ad esempio, li salva in un database o esegue altre operazioni con essi). Il form-handler è tipicamente un file sul server con uno script che gestisce e processa i dati di input.

```
<form action="/action_page.php">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
  <input type="submit" value="Submit">
</form>
```

First name:

Last name:

Chiariamo che **name** è un attributo obbligatorio per gli elementi di input all'interno di un modulo. Quando un modulo viene inviato, i dati raccolti nei campi di input vengono inviati al server come coppie chiave-valore. L'attributo `name` rappresenta la chiave, mentre il valore inserito dall'utente nel campo è il valore associato a quella chiave. Senza l'attributo `name`, il valore dell'input non verrà inviato affatto al server.

Il **quinto tipo di attributo** si riferisce a un button generico che deve essere gestito con del codice lato client.

Ovviamente possiamo usare tutti gli elementi usati nel form anche fuori dal form stesso:

HTML Demo: <button>

HTML
 CSS
 OUTPUT

```

1 <button class="favorite styled"
2   type="button">
3   Add to favorites
4 </button>
5

```

Add to favorites

HTML Demo: <input type="text">

HTML
 CSS
 OUTPUT

```

1 <label for="name">Name (4 to 8 characters):
2 </label>
3 <input type="text" id="name" name="name"
4   required
5   minlength="4" maxlength="8" size="18">

```

Name (4 to 8 characters):

HTML Demo: <select>

HTML
 CSS
 OUTPUT

```

1 <label for="pet-select">Choose a pet:</label>
2
3 <select name="pets" id="pet-select">
4   <option value="">--Please choose an option--
5 </option>
6   <option value="dog">Dog</option>
7   <option value="cat">Cat</option>
8   <option value="hamster">Hamster</option>
9   <option value="parrot">Parrot</option>
10  <option value="goldfish">Goldfish</option>
11 </select>
12

```

Choose a pet:

--Please choose an option--

HTML Demo: <textarea>

HTML
 CSS
 OUTPUT

```

1 <label for="story">Tell us your story:</label>
2
3 <textarea id="story" name="story"
4   rows="5" cols="33">
5   It was a dark and stormy night...
6 </textarea>
7

```

Tell us your story:

It was a dark and stormy night..

Questi elementi vengono generalmente chiamati **elementi interattivi** (di tipo inline).

Oltre all'attributo `action`, di cui abbiamo già parlato, esistono diversi altri attributi utili per l'elemento `<form>`:

Attribute	Description
<u>accept-charset</u>	Specifies the character encodings used for form submission
<u>action</u>	Specifies where to send the form-data when a form is submitted
<u>autocomplete</u>	Specifies whether a form should have autocomplete on or off
<u>enctype</u>	Specifies how the form-data should be encoded when submitting it to the server (only for <code>method="post"</code>)
<u>method</u>	Specifies the HTTP method to use when sending form-data
<u>name</u>	Specifies the name of the form
<u>novalidate</u>	Specifies that the form should not be validated when submitted
<u>rel</u>	Specifies the relationship between a linked resource and the current document
<u>target</u>	Specifies where to display the response that is received after submitting the form

Vediamo nello specifico l'attributo **target**, il quale specifica dove mostrare la risposta ricevuta dopo aver inviato il form. L'attributo `target` può avere uno dei seguenti valori:

Value	Description
<code>_blank</code>	The response is displayed in a new window or tab
<code>_self</code>	The response is displayed in the current window
<code>_parent</code>	The response is displayed in the parent frame
<code>_top</code>	The response is displayed in the full body of the window
<code>iframe</code>	The response is displayed in a named iframe

The screenshot shows a web browser window with the address bar displaying 'w3schools.com'. Below the browser window, there is a code editor and a live preview of the form. The code editor shows the following HTML code:

```
<!DOCTYPE html>
<html>
<body>

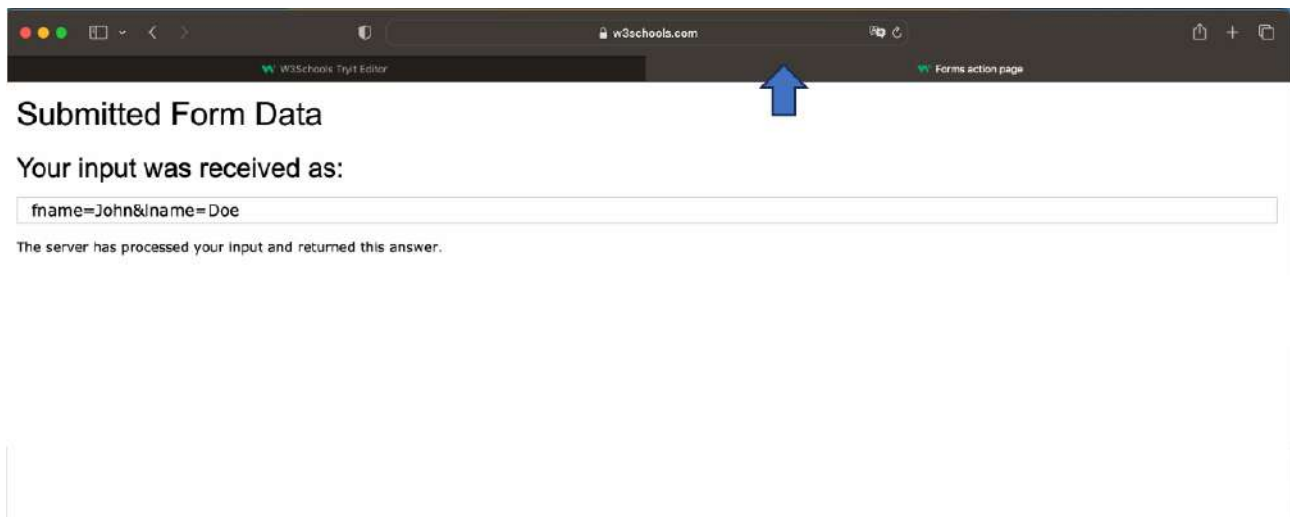
<h2>The form target attribute</h2>

<p>When submitting this form, the result will be opened in a new browser
tab:</p>

<form action="/action_page.php" target="_blank">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
  <input type="submit" value="Submit">
</form>

</body>
</html>
```

The live preview on the right shows the rendered form with the title 'The form target attribute'. It includes a paragraph: 'When submitting this form, the result will be opened in a new browser tab:'. Below this are two text input fields labeled 'First name:' and 'Last name:', with the values 'John' and 'Doe' respectively. A 'Submit' button is at the bottom, with a blue arrow pointing to it.



9.7. Elementi HTML – Table

Le tabelle HTML permettono agli sviluppatori web di organizzare i dati in righe e colonne.

Example

Company	Contact	Country
Alfreds Futterkiste	Maria Anders	Germany
Centro comercial Moctezuma	Francisco Chang	Mexico
Ernst Handel	Roland Mendel	Austria
Island Trading	Helen Bennett	UK
Laughing Bacchus Winecellars	Yoshi Tannamuri	Canada
Magazzini Alimentari Riuniti	Giovanni Rovelli	Italy

Una tabella in HTML è fatta da celle di tabella all'interno di righe e colonne.

```
<table>
  <tr>
    <th>Company</th>
    <th>Contact</th>
    <th>Country</th>
  </tr>
  <tr>
    <td>Alfreds Futterkiste</td>
    <td>Maria Anders</td>
    <td>Germany</td>
  </tr>
  <tr>
    <td>Centro comercial Moctezuma</td>
    <td>Francisco Chang</td>
    <td>Mexico</td>
  </tr>
</table>
```


- I tag **<th>** e **</th>** (**table header**) servono a contenere una cella di intestazione della tabella.
- I tag **<td>** e **</td>** (**table data**) definiscono una cella della tabella. Tutto ciò che si trova tra **<td>** e **</td>** è il contenuto della cella della tabella.
- I tag **<tr>** e **</tr>** (**table row**) definiscono una riga della tabella.

Le tabelle HTML possono avere celle che si estendono su più righe e/o colonne:

NAME	

APRIL		

2022		
FIESTA		

Per fare in modo che una cella si estenda su più colonne, possiamo usare l'attributo **colspan**:

```
<table>
  <tr>
    <th colspan="2">Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Jill</td>
    <td>Smith</td>
    <td>43</td>
  </tr>
  <tr>
    <td>Eve</td>
    <td>Jackson</td>
    <td>57</td>
  </tr>
</table>
```

Cell that spans two columns

To make a cell span more than one column, use the colspan attribute.

Name		Age
Jill	Smith	43
Eve	Jackson	57

Per fare in modo che una cella si estenda su più righe, possiamo usare l'attributo **rowspan**:

```
<table>
  <tr>
    <th>Name</th>
    <td>Jill</td>
  </tr>
  <tr>
    <th rowspan="2">Phone</th>
    <td>555-1234</td>
  </tr>
  <tr>
    <td>555-8745</td>
  </tr>
</table>
```

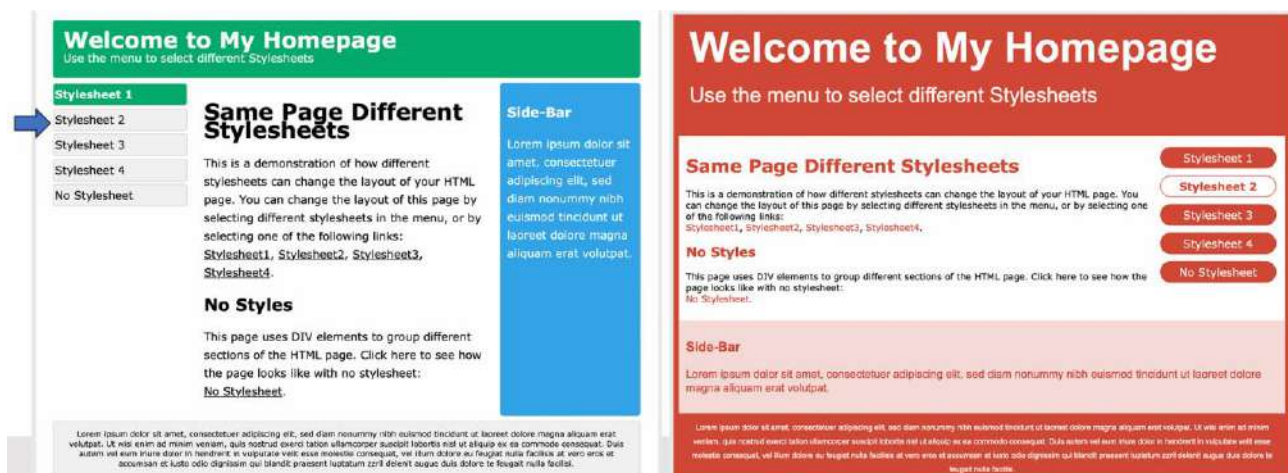
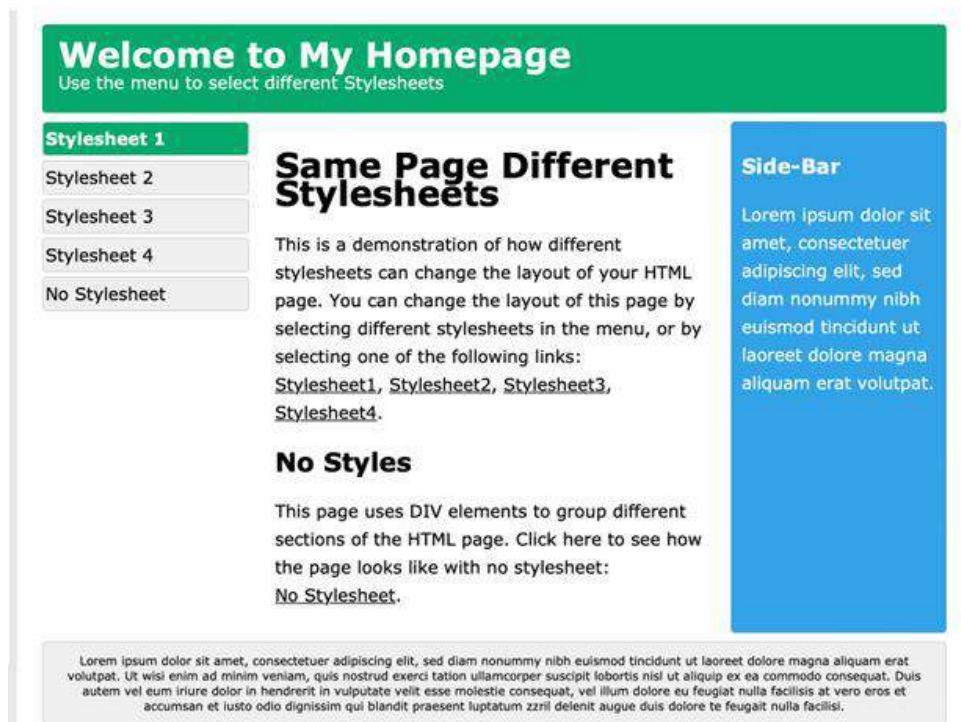
Cell that spans two rows

To make a cell span more than one row, use the rowspan attribute.

Name	Jill
Phone	555-1234
	555-8745

10. CSS

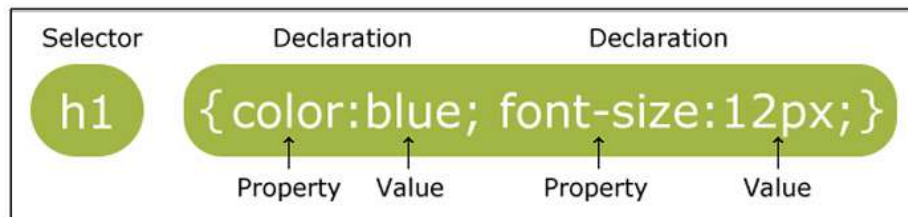
CSS sta per **Cascading Style Sheets** e rappresenta il metodo usato per dividere il contenuto dalla presentazione nelle pagine web. Gli stili definiscono come mostrare gli elementi HTML:



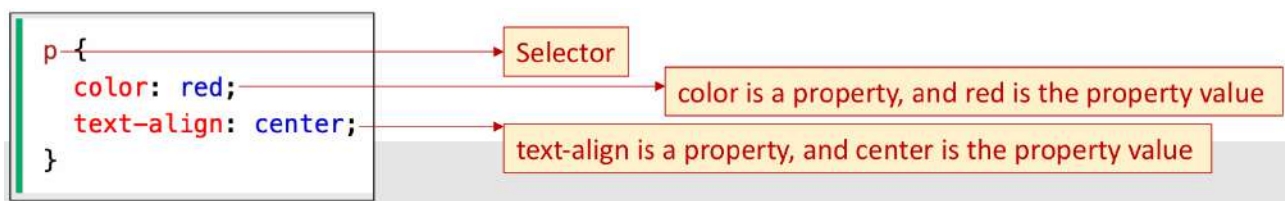
Una pagina HTML può avere quindi diversi stili.

Gli stili furono aggiunti ad HTML 4.0 per risolvere un problema. HTML, infatti, non è mai stato pensato per avere tag di formattazione, ma solo per definire il contenuto di un documento. Quando tag come `<font>` e gli attributi per i colori furono aggiunti ad HTML 3.2, lo sviluppo di grandi siti web, dove i font e le informazioni sul colore vennero aggiunte ad ogni singola pagina, divenne un processo lungo e dispendioso. Per risolvere questo problema il W3C (World Wide Web Consortium) ha creato CSS. Oggi tutti i browser supportano CSS. Parlando in modo generale CSS ci risparmia un sacco di lavoro, infatti le definizioni degli stili sono solitamente salvate in file .css esterni e, grazie a questa feature, possiamo cambiare il look di un intero sito web solamente cambiando un file.

Passando alla sintassi di CSS, una **regola CSS** consiste in un **selettore** e in un **blocco di dichiarazione**:



Il selettore punta all'elemento HTML che vogliamo stilizzare. Il blocco di dichiarazione contiene una o più dichiarazioni separate da punto e virgola. Ogni dichiarazione include il nome di una proprietà CSS e un valore, separati dai due punti. Dichiarazioni CSS multiple sono separate da punto e virgola e i blocchi di dichiarazione sono circondati da parentesi graffe.



Tutti gli elementi <p> saranno allineati centralmente con il colore del testo rosso.

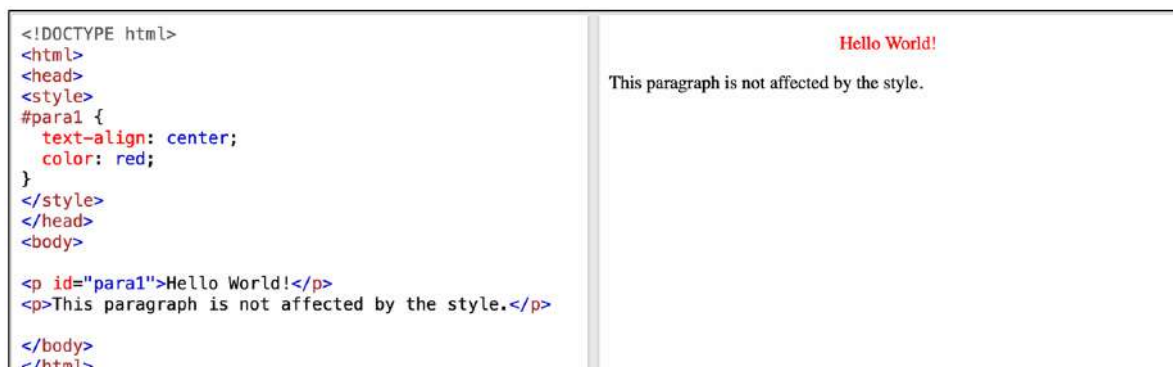
10.1. Selettori CSS

I selettori CSS sono usati per “trovare” (o selezionare) gli elementi HTML che vogliamo stilizzare. I selettori CSS possono essere divisi in cinque categorie:

- **Selettori semplici:** selezionano gli elementi in base a nome, id e classe.
- **Selettori combinatori:** selezionano gli elementi in base a una specifica relazione tra di loro.
- **Selettori di pseudo-classe:** selezionano gli elementi in base a un certo stato.
- **Selettori di pseudo-elementi:** selezionano e stilizzano una parte di un elemento.
- **Selettori di attributo:** selezionano gli elementi in base a un attributo o al valore dell'attributo.

10.1.1. Selettori semplici

Abbiamo visto nell'ultimo esempio la selezione in base al nome, adesso vediamo quella basata su id e classe. Il **selettore dell'id** usa l'attributo **id** di un elemento HTML per selezionare uno specifico elemento. Poiché, come già detto, l'id di un elemento è unico all'interno di una pagina, il selettore dell'id è usato per selezionare un unico elemento. Per selezionare un elemento con un id specifico, dobbiamo scrivere un carattere hash (#) seguito dall'id dell'elemento:



Il **selettore di classe** seleziona gli elementi HTML con uno specifico attributo di classe. Per selezionare gli elementi con una specifica classe, dobbiamo scrivere un carattere punto (.) seguito dal nome della classe:

<pre><!DOCTYPE html> <html> <head> <style> .center { text-align: center; color: red; } </style> </head> <body> <h1 class="center">Red and center-aligned heading</h1> <p class="center">Red and center-aligned paragraph. </p> </body> </html></pre>	Red and center-aligned heading Red and center-aligned paragraph.
--	---

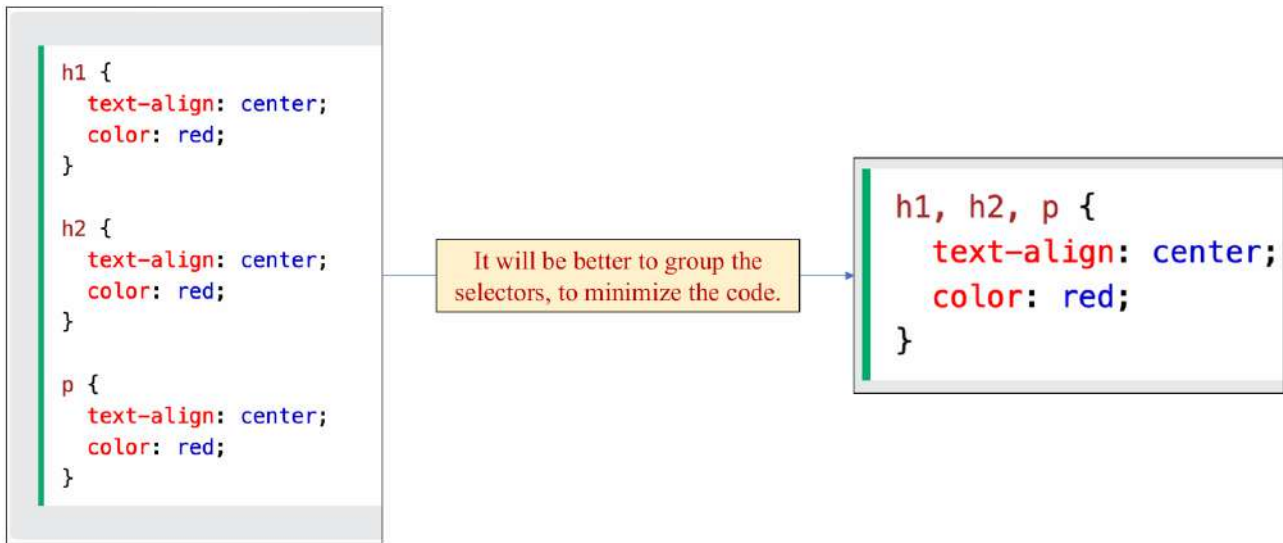
È possibile anche specificare che solo degli specifici elementi HTML dovrebbero essere interessati da una classe:

<pre><!DOCTYPE html> <html> <head> <style> p.center { text-align: center; color: red; } </style> </head> <body> <h1 class="center">This heading will not be affected</h1> <p class="center">This paragraph will be red and center-aligned.</p> </body> </html></pre>	This heading will not be affected This paragraph will be red and center-aligned.
--	---

Gli elementi HTML possono fare riferimento anche a più di una classe:

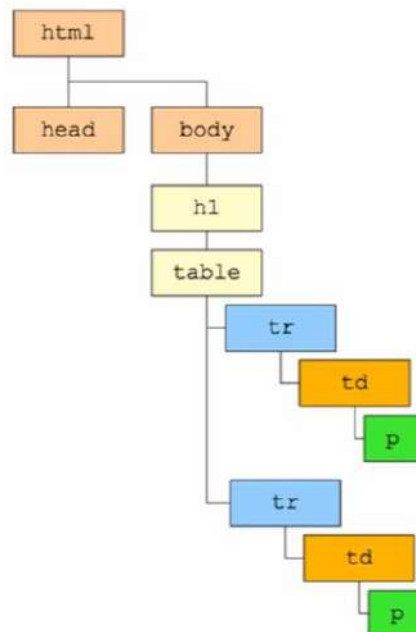
<pre><!DOCTYPE html> <html> <head> <style> p.center { text-align: center; color: red; } p.large { font-size: 300%; } </style> </head> <body> <h1 class="center">This heading will not be affected</h1> <p class="center">This paragraph will be red and center-aligned.</p> <p class="center large">This paragraph will be red, center-aligned, and in a large font-size.</p> </body> </html></pre>	This heading will not be affected This paragraph will be red and center-aligned. This paragraph will be red, center-aligned, and in a large font-size.
--	---

Esistono anche dei **selettori speciali** chiamati **selettori di gruppo**:



10.1.2. Selettori combinatori

Un combinator è qualcosa che spiega la relazione tra i selettori.



Nell'immagine:

- Testa (head) e corpo (body) sono **fratelli**.
- Testa (head) e corpo (body) sono **discendenti** rispetto ad html.
- ...

Ci sono quattro differenti combinatori in CSS:

- Selettore di discendenza (spazio)
- Selettore di figlio diretto (`>`)
- Selettore di fratello adiacente (`+`)
- Selettore di fratello generale (`~`)

Il **selettore di discendenza** viene utilizzato per selezionare tutti gli elementi che sono discendenti (figlio, nipote, ...) di un elemento specificato:

<pre><!DOCTYPE html> <html> <head> <style> div p { background-color: yellow; } </style> </head> <body> <h2>Descendant Selector</h2> <p>The descendant selector matches all elements that are descendants of a specified element.</p> <div> <p>Paragraph 1 in the div.</p> <p>Paragraph 2 in the div.</p> <section><p>Paragraph 3 in the div.</p></section> </div> <p>Paragraph 4. Not in a div.</p> <p>Paragraph 5. Not in a div.</p> </body> </html></pre>	Descendant Selector The descendant selector matches all elements that are descendants of a specified element. Paragraph 1 in the div. Paragraph 2 in the div. Paragraph 3 in the div. Paragraph 4. Not in a div. Paragraph 5. Not in a div.
--	--

Il **selettore di figlio diretto** viene utilizzato per selezionare tutti gli elementi che sono figli di un elemento specificato:

<pre><!DOCTYPE html> <html> <head> <style> div > p { background-color: yellow; } </style> </head> <body> <h2>Child Selector</h2> <p>The child selector (>) selects all elements that are the children of a specified element.</p> <div> <p>Paragraph 1 in the div.</p> <p>Paragraph 2 in the div.</p> <section> <!-- not Child but Descendant --> <p>Paragraph 3 in the div (inside a section element).</p> </section> <p>Paragraph 4 in the div.</p> </div> <p>Paragraph 5. Not in a div.</p> <p>Paragraph 6. Not in a div.</p> </body> </html></pre>	Child Selector The child selector (>) selects all elements that are the children of a specified element. Paragraph 1 in the div. Paragraph 2 in the div. Paragraph 3 in the div (inside a section element). Paragraph 4 in the div. Paragraph 5. Not in a div. Paragraph 6. Not in a div.
---	--

Il **selettore di fratello adiacente** viene utilizzato per selezionare un elemento che segue immediatamente un altro specifico elemento. Gli elementi figli devono avere lo stesso elemento padre:

<pre><!DOCTYPE html> <html> <head> <style> div + p { background-color: yellow; } </style> </head> <body> <h2>Adjacent Sibling Selector</h2> <p>The + selector is used to select an element that is directly after another specific element.</p> <p>The following example selects the first p element that are placed immediately after div elements:</p> <div> <p>Paragraph 1 in the div.</p> <p>Paragraph 2 in the div.</p> </div> <p>Paragraph 3. After a div.</p> <p>Paragraph 4. After a div.</p> <div> <p>Paragraph 5 in the div.</p> <p>Paragraph 6 in the div.</p> </div> <p>Paragraph 7. After a div.</p> <p>Paragraph 8. After a div.</p> </body> </html></pre>	Adjacent Sibling Selector The + selector is used to select an element that is directly after another specific element. The following example selects the first p element that are placed immediately after div elements: Paragraph 1 in the div. Paragraph 2 in the div. Paragraph 3. After a div. Paragraph 4. After a div. Paragraph 5 in the div. Paragraph 6 in the div. Paragraph 7. After a div. Paragraph 8. After a div.
---	---

Il **selettore di fratello generale** viene utilizzato per selezione tutti gli elementi che sono fratelli successivi di uno specifico elemento:

<pre><!DOCTYPE html> <html> <head> <style> div ~ p { background-color: yellow; } </style> </head> <body> <h2>General Sibling Selector</h2> <p>The general sibling selector (~) selects all elements that are next siblings of a specified element.</p> <p>Paragraph 1.</p> <div> <p>Paragraph 2.</p> </div> <p>Paragraph 3.</p> <code>Some code.</code> <p>Paragraph 4.</p> </body> </html></pre>	General Sibling Selector The general sibling selector (~) selects all elements that are next siblings of a specified element. Paragraph 1. Paragraph 2. Paragraph 3. Some code. Paragraph 4.
---	---

10.1.3. Selettori di pseudo-classe

Una **pseudo-classe** è usata per definire uno stato speciale di un elemento. Per esempio, può essere usata per:

- Stilizzare un elemento quando un utente gli passa il mouse di sopra.
- Stilizzare link visitati e non visitati in modo differente.
- Stilizzare un elemento quando ottiene il focus.



La sintassi delle pseudo-classi è:

```
selector:pseudo-class {  
    property: value;  
}
```

Un esempio diretto di pseudo-classe è dato dal tag <a>. I link in questo caso possono essere mostrati in diversi modi, ad esempio un link non ancora visitato è di colore rosso:

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
/* unvisited link */  
a:link {  
    color: red;  
}  
  
/* visited link */  
a:visited {  
    color: green;  
}  
  
/* mouse over link */  
a:hover {  
    color: hotpink;  
}  
  
/* selected link */  
a:active {  
    color: blue;  
}  
</style>  
</head>  
<body>  
  
<h2>Styling a link depending on state</h2>  
  
<p><b><a href="default.asp" target="_blank">This is a  
link</a></b></p>  
<p><b>Note:</b> a:hover MUST come after a:link and  
a:visited in the CSS definition in order to be  
effective.</p>  
<p><b>Note:</b> a:active MUST come after a:hover in  
the CSS definition in order to be effective.</p>  
  
</body>  
</html>
```

Styling a link depending on state

This is a link

Note: a:hover MUST come after a:link and a:visited in the CSS definition in order to be effective.

Note: a:active MUST come after a:hover in the CSS definition in order to be effective.

Invece un link che è stato visitato è di colore verde:

```

<!DOCTYPE html>
<html>
<head>
<style>
/* unvisited link */
a:link {
  color: red;
}

/* visited link */
a:visited {
  color: green;
}

/* mouse over link */
a:hover {
  color: hotpink;
}

/* selected link */
a:active {
  color: blue;
}
</style>
</head>
<body>

<h2>Styling a link depending on state</h2>

<p><b><a href="default.asp" target="_blank">This is a
link</a></b></p>
<p><b>Note:</b> a:hover MUST come after a:link and
a:visited in the CSS definition in order to be
effective.</p>
<p><b>Note:</b> a:active MUST come after a:hover in
the CSS definition in order to be effective.</p>

</body>
</html>

```

Styling a link depending on state

[This is a link](#)

Note: a:hover MUST come after a:link and a:visited in the CSS definition in order to be effective.

Note: a:active MUST come after a:hover in the CSS definition in order to be effective.

Se clicchiamo sul link diventerà temporaneamente blu:

```

<!DOCTYPE html>
<html>
<head>
<style>
/* unvisited link */
a:link {
  color: red;
}

/* visited link */
a:visited {
  color: green;
}

/* mouse over link */
a:hover {
  color: hotpink;
}

/* selected link */
a:active {
  color: blue;
}
</style>
</head>
<body>

<h2>Styling a link depending on state</h2>

<p><b><a href="default.asp" target="_blank">This is a
link</a></b></p>
<p><b>Note:</b> a:hover MUST come after a:link and
a:visited in the CSS definition in order to be
effective.</p>
<p><b>Note:</b> a:active MUST come after a:hover in
the CSS definition in order to be effective.</p>

</body>
</html>

```

Styling a link depending on state

[This is a link](#)

Note: a:hover MUST come after a:link and a:visited in the CSS definition in order to be effective.

Note: a:active MUST come after a:hover in the CSS definition in order to be effective.

Dopo aver rilasciato il click, il colore sarà impostato a verde.

Ci sono innumerevoli pseudo-classi:

Selector	Example	Example description			
:active	a:active	Selects the active link	:lang(<i>language</i>)	p:lang(it)	Selects every <p> element with a lang attribute value starting with "it"
:checked	input:checked	Selects every checked <input> element	:last-child	p:last-child	Selects every <p> elements that is the last child of its parent
:disabled	input:disabled	Selects every disabled <input> element	:last-of-type	p:last-of-type	Selects every <p> element that is the last <p> element of its parent
:empty	p:empty	Selects every <p> element that has no children	:link	a:link	Selects all unvisited links
:enabled	input:enabled	Selects every enabled <input> element	:not(selector)	:not(p)	Selects every element that is not a <p> element
:first-child	p:first-child	Selects every <p> elements that is the first child of its parent	:nth-child(n)	p:nth-child(2)	Selects every <p> element that is the second child of its parent
:first-of-type	p:first-of-type	Selects every <p> element that is the first <p> element of its parent	:nth-last-child(n)	p:nth-last-child(2)	Selects every <p> element that is the second child of its parent, counting from the last child
:focus	input:focus	Selects the <input> element that has focus	:nth-last-of-type(n)	p:nth-last-of-type(2)	Selects every <p> element that is the second <p> element of its parent, counting from the last child
:hover	a:hover	Selects links on mouse over	:nth-of-type(n)	p:nth-of-type(2)	Selects every <p> element that is the second <p> element of its parent
:in-range	input:in-range	Selects <input> elements with a value within a specified range	:only-of-type	p:only-of-type	Selects every <p> element that is the only <p> element of its parent
:invalid	input:invalid	Selects all <input> elements with an invalid value			

Vediamo in particolare la pseudo-classe **:first-child**, usata per selezionare uno specifico elemento che è il primo figlio di un altro elemento:

<pre><!DOCTYPE html> <html> <head> <style> p:first-child { color: blue; } </style> </head> <body> <p>This is some text.</p> <p>This is some text.</p> <div> <p>This is some text.</p> <p>This is some text.</p> </div> </body> </html></pre>	This is some text. This is some text. This is some text. This is some text.
---	---

Il selettore seleziona qualsiasi elemento <p> che è primo figlio di un qualsiasi elemento.

Vediamo altri due esempi:

```
p i:first-child {  
  color: blue;  
}
```

Il selettore seleziona il primo elemento `<i>` in tutti gli elementi `<p>`.

```
p:first-child i {  
  color: blue;  
}
```

Il selettore seleziona tutti gli elementi `<i>` negli elementi `<p>` che sono primi figli di un altro elemento.

10.1.4. Selettori di pseudo-elementi

Uno **pseudo-elemento CSS** è usato per stilizzare specifiche parti di un elemento. Per esempio, può essere usato per:

- Stilizzare la prima lettera o linea di un elemento.
- Inserire il contenuto prima o dopo il contenuto di un elemento.

La sintassi di uno pseudo-elemento è:

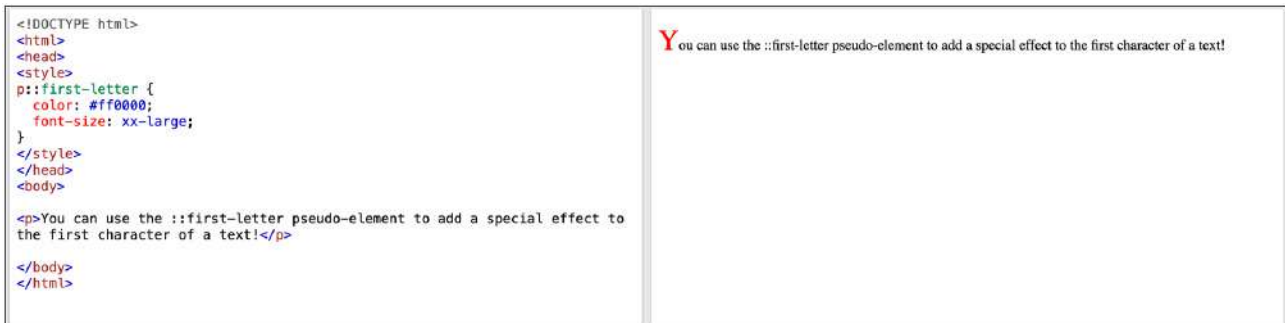
```
selector::pseudo-element {  
  property: value;  
}
```

Un primo esempio di pseudo-elemento è **::first-line**, usato per aggiungere uno stile speciale alla prima linea di un testo:

<pre><!DOCTYPE html> <html> <head> <style> p::first-line { color: #ff0000; font-variant: small-caps; } </style> </head> <body> <p>You can use the ::first-line pseudo-element to add a special effect to the first line of a text. Some more text. And even more, and more, and more, and more, and more, and more, and more, and more, and more, and more, and more, and more.</p> </body> </html></pre>	YOU CAN USE THE ::FIRST-LINE PSEUDO-ELEMENT TO ADD A SPECIAL EFFECT TO THE FIRST LINE OF A TEXT. SOME MORE text. And even more, and more, and more, and more, and more, and more, and more, and more, and more, and more, and more, and more.
---	--

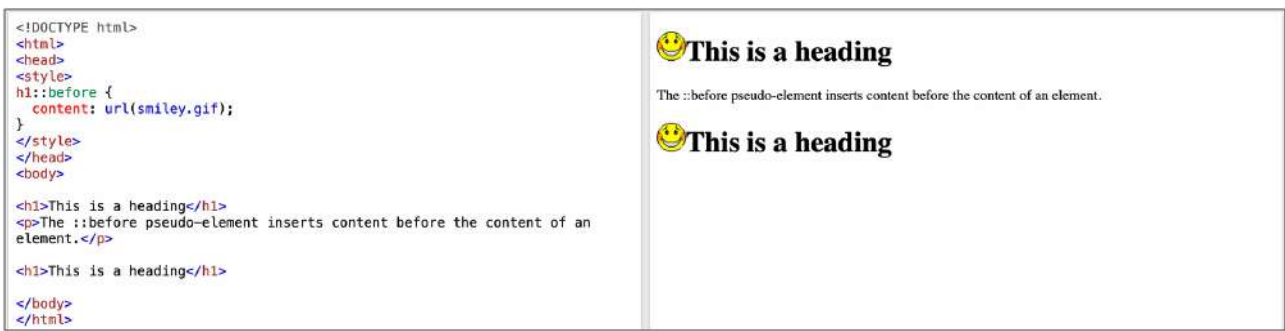
Questo pseudo-elemento può essere applicato solo ad elementi di tipo a blocco.

Un altro pseudo-elemento è **::first-letter**, usato per aggiungere uno stile speciale alla prima lettera di un testo:



Questo pseudo-elemento può essere applicato solo ad elementi di tipo a blocco.

Gli pseudo-elementi **::before** e **::after** possono essere usati rispettivamente per inserire del contenuto prima e dopo il contenuto di un elemento:



Ecco una lista di tutti gli pseudo-elementi CSS:

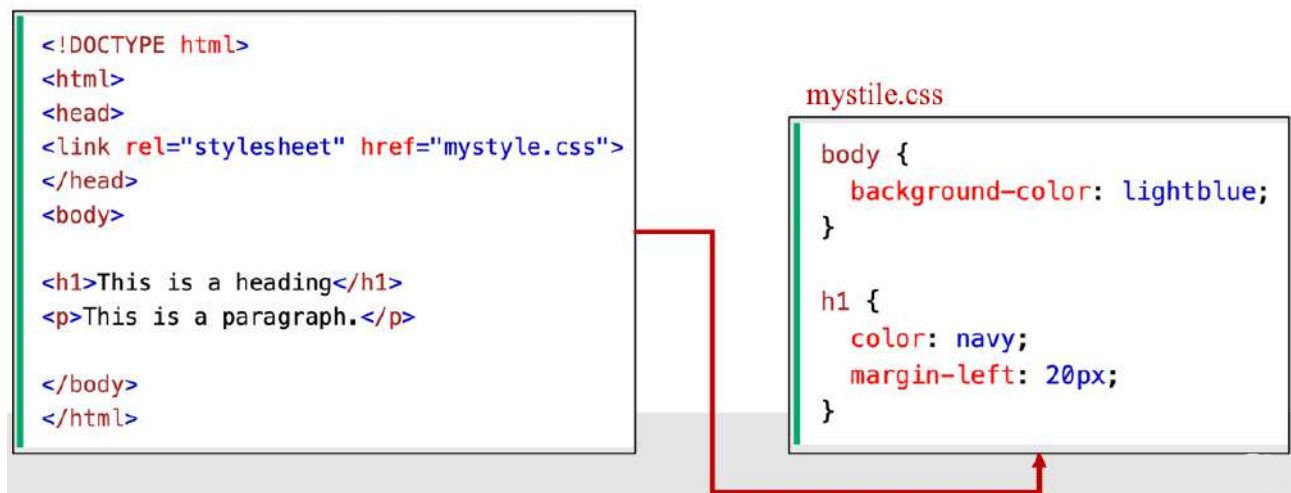
Selector	Example	Example description
<u>::after</u>	p::after	Insert something after the content of each <p> element
<u>::before</u>	p::before	Insert something before the content of each <p> element
<u>::first-letter</u>	p::first-letter	Selects the first letter of each <p> element
<u>::first-line</u>	p::first-line	Selects the first line of each <p> element
<u>::marker</u>	::marker	Selects the markers of list items
<u>::selection</u>	p::selection	Selects the portion of an element that is selected by a user

10.2. Includere CSS in un progetto

Quando un browser legge uno style sheet, formatterà il documento HTML in base alle informazioni contenute nello style sheet. Ci sono tre modi per inserire uno style sheet:

- CSS esterni
- CSS interni
- CSS in linea

I **CSS esterni** sono ideali quando lo stile è applicato a molte pagine. In questo modo è possibile cambiare il look di un intero sito web cambiando solamente un file. Essi sono definiti all'interno dell'elemento **<link>**, dentro la sezione **<head>** di una pagina HTML.



I **CSS interni** sono usati quando un singolo documento ha uno stile unico. Essi sono definiti all'interno dell'elemento **<style>**, dentro la sezione **<head>** di una pagina HTML.

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-color: linen;
}

h1 {
  color: maroon;
  margin-left: 40px;
}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

I **CSS in linea** perdono molti dei vantaggi degli style sheet mischiando il contenuto con la presentazione. Per utilizzarli, bisogna aggiungere l'attributo **style** all'elemento rilevante. L'attributo style può contenere una qualsiasi proprietà CSS.

```
<!DOCTYPE html>
<html>
<body>

<h1 style="color:blue;text-align:center;">This is a heading</h1>
<p style="color:red;">This is a paragraph.</p>

</body>
</html>
```

Chiaramente, dichiarando molteplici style sheets si incorre in un'eccezione. Se alcune proprietà sono state definite per lo stesso selettore (elemento) in diversi style sheets, sarà usato il valore dell'ultimo style sheet letto. Per esempio, assumiamo che uno **style sheet esterno** abbia il seguente stile per l'elemento <h1>:

```
h1 {
  color: navy;
}
```

In aggiunta, assumiamo che uno **style sheet interno** abbia a sua volta il seguente stile per l'elemento <h1>:

```
h1 {
  color: orange;
}
```

Allora:

CASO 1: se lo stile interno è definito **dopo** il link allo style sheet esterno, gli elementi <h1> saranno di colore arancione.

```
<head>
<link rel="stylesheet" type="text/css" href="mystyle.css">
<style>
h1 {
  color: orange;
}
</style>
</head>
```

CASO 2: se lo stile interno è definito **prima** del link allo style sheet esterno, gli elementi <h1> saranno di colore navy.

```
<head>
<style>
h1 {
  color: orange;
}
</style>
<link rel="stylesheet" type="text/css" href="mystyle.css">
</head>
```

Quando esiste più di uno stile specificato per un certo elemento HTML, tutti gli stili in una pagina si riverseranno a cascata (**CSS cascade**) in un nuovo style sheet “virtuale” secondo le seguenti regole (da quella con maggiore priorità a quella con priorità più bassa):

1. **Stili in linea:** gli stili definiti direttamente dentro un **elemento HTML** con l'attributo style hanno la **priorità più alta**.
2. **Stili esterni e interni:** gli stili definiti negli style sheet esterni (collegati con <link>) o interni (definiti con il tag <style>) nella sezione <head> hanno la **priorità media**. Gli stili esterni sovrascrivono i valori predefiniti del browser, ma saranno sovrascritti da stili in linea.
3. **Stili predefiniti del browser:** se non è stato specificato alcun stile, il browser utilizza i suoi valori predefiniti. Questi stili hanno la **priorità più bassa** e vengono applicati solo se non ci sono stili definiti nel documento.

Chiediamoci quindi, ad esempio, quale sarà il colore dello sfondo di questa pagina?

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" type="text/css" href="mystyle.css">
<style>
body {background-color: linen;}
</style>
</head>
<body style="background-color: lavender">

<h1>Multiple Styles Will Cascade into One</h1>
<p>Here, the background color of the page is set with inline CSS, and also
with an internal CSS, and also with an external CSS.</p>
<p>Try experimenting by removing styles to see how the cascading stylesheets
work (try removing the inline CSS first, then the internal, then the
external).</p>

</body>
</html>
```

Lo stile in linea ha la priorità più alta, quindi:

Multiple Styles Will Cascade into One

Here, the background color of the page is set with inline CSS, and also with an internal CSS, and also with an external CSS.

Try experimenting by removing styles to see how the cascading stylesheets work (try removing the inline CSS first, then the internal, then the external).

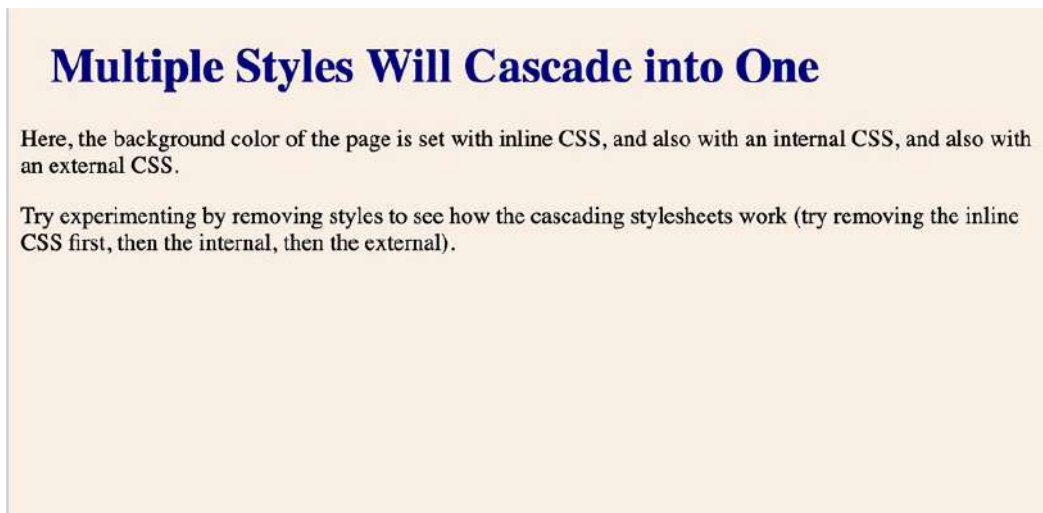
Quale sarà invece il colore dello sfondo di questa pagina?

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" type="text/css" href="mystyle.css">
<style>
body {background-color: linen;}
</style>
</head>
<body>

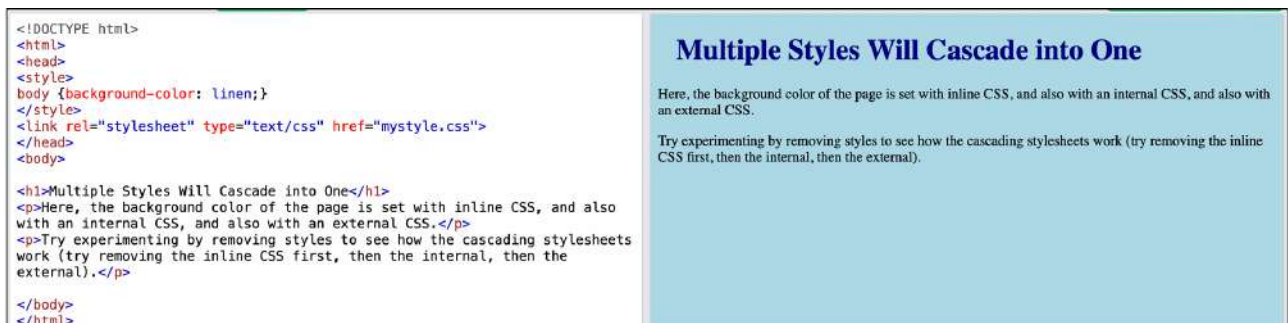
<h1>Multiple Styles Will Cascade into One</h1>
<p>Here, the background color of the page is set with inline CSS, and also
with an internal CSS, and also with an external CSS.</p>
<p>Try experimenting by removing styles to see how the cascading stylesheets
work (try removing the inline CSS first, then the internal, then the
external).</p>

</body>
</html>
```

L'ultimo stile definito (quello interno) ha la priorità più alta, quindi:



Se cambiamo ordine, l'ultimo stile definito sarà quello esterno, quindi:

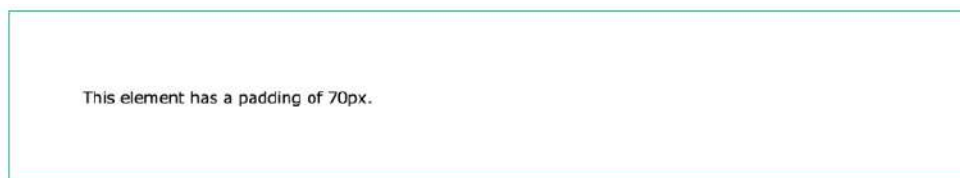


10.3. Il modello a scatola

Tutti gli elementi HTML possono essere considerati come scatole. In CSS, il termine “modello a scatola” si usa quando si parla di **design** e di **layout**. Il modello a scatola di CSS è essenzialmente una scatola che racchiude ogni elemento HTML, e consiste (dall'interno verso l'esterno) in: **content** (contenuto), **padding** (imbottitura), **borders** (bordi) e **margins** (margini).



Il **padding** è usato per creare spazio intorno al contenuto di un elemento, entro dei confini ben precisi:



CSS ha proprietà per specificare il padding per ogni lato di un elemento:

- padding-top
- padding-right
- padding-bottom
- padding-left

Tutte le proprietà di padding possono avere i seguenti valori:

- **length**: specifica un padding in px, pt, cm, ecc.
- **%**: specifica un padding in percentuale della larghezza dell'elemento contenitore.
- **inherit**: specifica che il padding dovrebbe essere ereditato dall'elemento genitore.

Vediamo esempi di utilizzo di padding:

Four parameters:

- top padding is 25px
- right padding is 50px
- bottom padding is 75px
- left padding is 100px

```
div {  
  padding: 25px 50px 75px 100px;  
}
```

Three parameters:

- top padding is 25px
- right and left paddings are 50px
- bottom padding is 75px

```
div {  
  padding: 25px 50px 75px;  
}
```

Two parameters:

- top and bottom paddings are 25px
- right and left paddings are 50px

```
div {  
  padding: 25px 50px;  
}
```

One parameter:

- all four paddings are 25px

```
div {  
  padding: 25px;  
}
```

I **margins** sono usati per creare spazio attorno agli elementi, fuori da qualsiasi confine definito:

This element has a margin of 70px.

CSS ha proprietà per specificare il margin per ogni lato di un elemento:

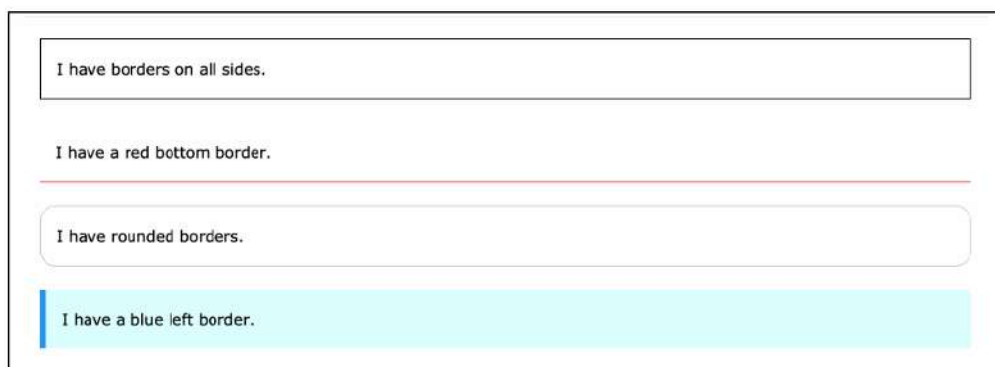
- margin-top
- margin-right
- margin-bottom
- margin-left

Tutte le proprietà di margin possono avere i seguenti valori:

- **auto**: il browser calcola in autonomia il margin da applicare.
- **length**: specifica un margin in px, pt, cm, ecc.
- **%**: specifica un margin in percentuale della larghezza dell'elemento contenitore.
- **inherit**: specifica che il margin dovrebbe essere ereditato dall'elemento genitore.

Gli esempi visti per il padding valgono in modo identico per il margin.

Le proprietà di **border** CSS ci permettono di specificare lo stile, la larghezza e il colore del border di un elemento:



La proprietà **border-style** specifica che tipo di border mostrare:

- Punteggiato (dotted)
- Tratteggiato (dashed)
- Solido (solid)
- Doppio (double)
- ...


```
<!DOCTYPE html>
<html>
<head>
<style>
p.one {
  border-style: solid;
  border-width: 5px;
}

p.two {
  border-style: solid;
  border-width: medium;
}

p.three {
  border-style: dotted;
  border-width: 2px;
}

p.four {
  border-style: dotted;
  border-width: thick;
}

p.five {
  border-style: double;
  border-width: 15px;
}

p.six {
  border-style: double;
  border-width: thick;
}
</style>
```

The border-width Property

This property specifies the width of the four borders:

Some text.

Some text.

Some text.

Some text.

Some text.

Some text.

Note: The "border-width" property does not work if it is used alone. Always specify the "border-style" property to set the borders first.

La proprietà `border-style` può avere da uno a quattro valori: `top border`, `right border`, `bottom border`, `left border`.

La proprietà **`border-width`** può avere da uno a quattro valori: `top border`, `right border`, `bottom border`, `left border`. La larghezza è settata in pixel ma si possono utilizzare anche i tre valori predefiniti: **`thin`** (sottile), **`medium`** (medio), **`thick`** (spesso).

La proprietà **`border-color`** permette di settare il colore del bordo in base a: nome, HEX, RGB, HSL, trasparente. Anche questa proprietà può avere da uno a quattro valori: `top border`, `right border`, `bottom border`, `left border`.

10.4. Posizionamento

La proprietà di **posizione** specifica il tipo di metodo di posizionamento per un elemento e può assumere i valori:

- static
- relative
- fixed
- absolute
- sticky

Gli elementi HTML sono posizionati static di default. Un elemento con **position: static;** non è posizionato in modo particolare. È posizionato in base al normale flusso della pagina:

<pre><!DOCTYPE html> <html> <head> <style> div.static { position: static; border: 3px solid #73AD21; } </style> </head> <body> <h2>position: static;</h2> <p>An element with position: static; is not positioned in any special way; it is always positioned according to the normal flow of the page:</p> <div class="static"> This div element has position: static; </div> </body> </html></pre>	position: static; An element with position: static; is not positioned in any special way; it is always positioned according to the normal flow of the page: This div element has position: static;
---	--

Un elemento con **position: relative;** è posizionato in relazione alla sua normale posizione:

<pre><!DOCTYPE html> <html> <head> <style> div.relative { left: 30px; border: 3px solid #73AD21; } </style> </head> <body> <h2>position: relative;</h2> <p>An element with position: relative; is positioned relative to its normal position:</p> <div class="relative"> This div element has position: relative; </div> </body> </html></pre>	position: relative; An element with position: relative; is positioned relative to its normal position: This div element has position: relative;
--	---

Senza alcuna indicazione, la normale posizione di `<div>` è quella mostrata nell'immagine. In questo caso `left 30 px` non influenza la posizione perché `<div>` è un elemento di tipo a blocco.

<pre><!DOCTYPE html> <html> <head> <style> div.relative { position: relative; left: 30px; border: 3px solid #73AD21; } </style> </head> <body> <h2>position: relative;</h2> <p>An element with position: relative; is positioned relative to its normal position:</p> <div class="relative"> This div element has position: relative; </div> </body> </html></pre>	position: relative; An element with position: relative; is positioned relative to its normal position: This div element has position: relative; Relative to its normal position, the <code><div></code> is moved on the left by 30px
--	---

Un elemento con **position: fixed;** è posizionato in modo relativo alla **viewport**, ovvero rimane sempre nella stessa posizione anche se la pagina viene scrollata.

```
<!DOCTYPE html>
<html>
<head>
<style>
div.fixed {
  position: fixed;
  bottom: 0;
  right: 0;
  width: 300px;
  border: 3px solid #73AD21;
}
</style>
</head>
<body>

<h2>position: fixed;</h2>

<p>An element with position: fixed; is positioned relative to the
viewport, which means it always stays in the same place even if the page
is scrolled:</p>

<div class="fixed">
This div element has position: fixed;
</div>

</body>
</html>
```

position: fixed;

An element with position: fixed; is positioned relative to the viewport, which means it always stays in the same place even if the page is scrolled:

This div element has position: fixed;

Un elemento con **position: absolute;** è posizionato in modo relativo al più vicino antenato posizionato. Se non ci sono antenati, viene usato il corpo del documento:

```
<!DOCTYPE html>
<html>
<head>
<style>
div.relative {
  position: relative;
  width: 400px;
  height: 200px;
  border: 3px solid #73AD21;
}
div.absolute {
  position: absolute;
  top: 80px;
  right: 0;
  width: 200px;
  height: 100px;
  border: 3px solid #73AD21;
}
</style>
</head>
<body>

<h2>position: absolute;</h2>

<p>An element with position: absolute; is positioned relative to the
nearest positioned ancestor (instead of positioned relative to the
viewport, like fixed):</p>

<div class="relative">This div element has position: relative;
  <div class="absolute">This div element has position: absolute;</div>
</div>

</body>
</html>
```

position: absolute;

An element with position: absolute; is positioned relative to the nearest positioned ancestor (instead of positioned relative to the viewport, like fixed):

This div element has position: relative;

This div element has position: absolute;

Gli elementi posizionati in modo assoluto sono rimossi dal normale flusso e possono sovrapporsi con altri elementi.

Un elemento con **position: sticky;** è posizionato in base alla posizione dello scroll dell'utente. La particolarità di un elemento sticky è che alterna due diversi comportamenti di posizionamento:

1. **Relative:** fino a quando l'utente non ha scansionato una certa posizione della pagina, l'elemento si comporta come se fosse posizionato in modo relative.
2. **Fixed:** quando l'elemento raggiunge un determinato offset durante lo scorrimento, si "incolla" (stick) nella posizione e si comporta come se fosse fixed, rimanendo visibile nella finestra.



In this moment, the element is relative to its normal position.

When scrolling...

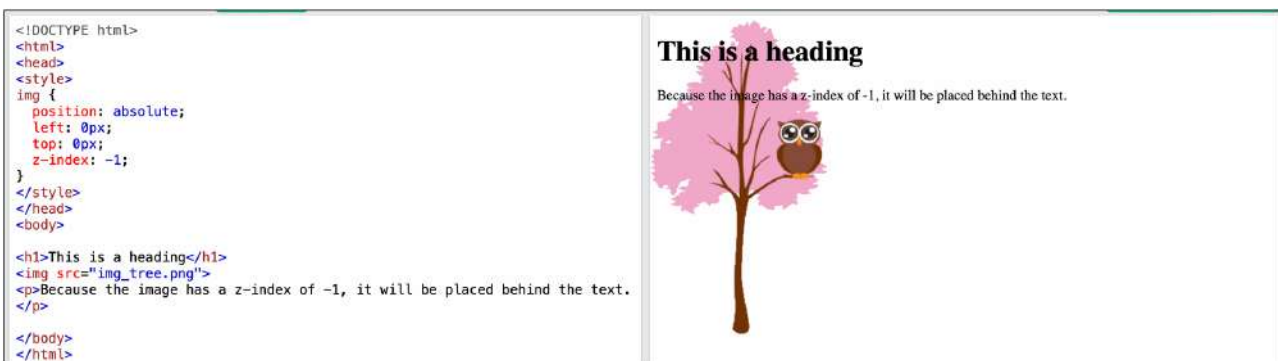


In this moment, the element is relative to its normal position.

When scrolling, the position sticks to the top (in this case).

Internet Explorer non supporta il posizionamento sticky. Safari richiede un prefisso -webkit-. Inoltre per far funzionare il posizionamento sticky bisogna specificare almeno uno tra top, right, bottom e left.

Per quanto riguarda gli elementi posizionati in modo assoluto, relativo o sticky è anche possibile utilizzare la proprietà **z-index**, la quale specifica l'ordine di stack di un elemento, ovvero quale elemento dovrebbe essere posizionato di fronte o dietro gli altri.



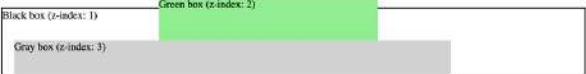
```
<!DOCTYPE html>
<html>
<head>
<style>
.container {
  position: relative;
}
.black-box {
  position: relative;
  z-index: 1;
  border: 2px solid black;
  height: 100px;
  margin: 30px;
}
.gray-box {
  position: absolute;
  z-index: 3; /* gray box will be above both green and black box */
  background: lightgray;
  height: 60px;
  width: 70%;
  left: 50px;
  top: 50px;
}
.green-box {
  position: absolute;
  z-index: 2; /* green box will be above black box */
  background: lightgreen;
  width: 35%;
  left: 270px;
  top: -15px;
  height: 100px;
}
</style>
</head>
<body>

<!--Z-index Example-->
<p>An element with greater stack order is always above an element with a lower stack order.</p>

<div class="container">
  <div class="black-box">Black box (z-index: 1)</div>
  <div class="gray-box">Gray box (z-index: 3)</div>
  <div class="green-box">Green box (z-index: 2)</div>
</div>
</body>
</html>
```

Z-index Example

An element with greater stack order is always above an element with a lower stack order.



La proprietà **float** serve a rimuovere un elemento dal normale flusso del documento e spostarlo verso uno dei lati (sinistro o destro) dell'elemento contenitore. La proprietà float può avere i seguenti valori:

- **left**: l'elemento fluttua verso la sinistra del proprio contenitore.
- **right**: l'elemento fluttua verso la destra del proprio contenitore.
- **none** (default): l'elemento non fluttua ma viene mostrato esattamente dove occorre nel testo.
- **inherit**: l'elemento eredita il valore float del suo genitore.

La sua più semplice utilità è quella di avvolgere testo intorno alle immagini:

```
<!DOCTYPE html>
<html>
<head>
<style>
img {
  float: none;
}
</style>
</head>
<body>

<h2>Float None</h2>

<p>In this example, the image will be displayed just where it occurs in the text (float: none;).</p>

<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio. Praesent convallis urna a lacus interdum ut hendrerit risus congue. Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac. In at libero sed nunc venenatis imperdiet sed ornare turpis. Donec vitae dui eget tellus gravida venenatis. Integer fringilla congue eros non fermentum. Sed dapibus pulvinar nibh tempor porta. Cras ac leo purus. Mauris quis diam velit.</p>

</body>
</html>
```

Float None

In this example, the image will be displayed just where it occurs in the text (float: none;).



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio. Praesent convallis urna a lacus interdum ut hendrerit risus congue. Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac. In at libero sed nunc venenatis imperdiet sed ornare turpis. Donec vitae dui eget tellus gravida venenatis. Integer fringilla congue eros non fermentum. Sed dapibus pulvinar nibh tempor porta. Cras ac leo purus. Mauris quis diam velit.


```

<!DOCTYPE html>
<html>
<head>
<style>
img {
  float: right;
}
</style>
</head>
<body>

<h2>Float Right</h2>

<p>In this example, the image will float to the right in the paragraph, and the text in the paragraph will wrap around the image.</p>

<p>
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio. Praesent convallis urna a lacus interdum ut hendrerit risus congue. Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac. In at libero sed nunc venenatis imperdiet sed ornare turpis. Donec vitae dui eget tellus gravida venenatis. Integer fringilla congue eros non fermentum. Sed dapibus pulvinar nibh tempor porta. Cras ac leo purus. Mauris quis diam velit.</p>

</body>
</html>

```

Float Right

In this example, the image will float to the right in the paragraph, and the text in the paragraph will wrap around the image.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio. Praesent convallis urna a lacus interdum ut hendrerit risus congue. Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac. In at libero sed nunc venenatis imperdiet sed ornare turpis. Donec vitae dui eget tellus gravida venenatis. Integer fringilla congue eros non fermentum. Sed dapibus pulvinar nibh tempor porta. Cras ac leo purus. Mauris quis diam velit.



```

<!DOCTYPE html>
<html>
<head>
<style>
img {
  float: left;
}
</style>
</head>
<body>

<h2>Float Left</h2>

<p>In this example, the image will float to the left in the paragraph, and the text in the paragraph will wrap around the image.</p>

<p>
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio. Praesent convallis urna a lacus interdum ut hendrerit risus congue. Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac. In at libero sed nunc venenatis imperdiet sed ornare turpis. Donec vitae dui eget tellus gravida venenatis. Integer fringilla congue eros non fermentum. Sed dapibus pulvinar nibh tempor porta. Cras ac leo purus. Mauris quis diam velit.</p>

</body>
</html>

```

Float Left

In this example, the image will float to the left in the paragraph, and the text in the paragraph will wrap around the image.



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio. Praesent convallis urna a lacus interdum ut hendrerit risus congue. Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac. In at libero sed nunc venenatis imperdiet sed ornare turpis. Donec vitae dui eget tellus gravida venenatis. Integer fringilla congue eros non fermentum. Sed dapibus pulvinar nibh tempor porta. Cras ac leo purus. Mauris quis diam velit.

Un comportamento più complesso si ottiene quando gli elementi fluttuano gli uni accanto agli altri. Normalmente gli elementi div sono mostrati uno sopra l'altro. Se però utilizziamo float: left possiamo fare in modo che fluttuino l'uno accanto all'altro:

```

<!DOCTYPE html>
<html>
<head>
<style>
div {
  float: left;
  padding: 15px;
}
.div1 {
  background: red;
}
.div2 {
  background: yellow;
}
.div3 {
  background: green;
}
</style>
</head>
<body>

<h2>Float Next To Each Other</h2>

<p>In this example, the three divs will float next to each other.</p>

<div class="div1">Div 1</div>
<div class="div2">Div 2</div>
<div class="div3">Div 3</div>

</body>
</html>

```

Float Next To Each Other

In this example, the three divs will float next to each other.

Div 1

Div 2

Div 3

Quando usiamo la proprietà `float` e vogliamo che il prossimo elemento sia sotto (non a destra o sinistra), dovremo utilizzare la proprietà **clear**. La proprietà `clear` specifica cosa dovrebbe accadere con l'elemento che si trova vicino a un elemento flottante. La proprietà `clear` può avere uno dei seguenti valori:

- **none** (default): l'elemento non viene spinto sotto gli elementi flottanti a sinistra o a destra.
- **left**: l'elemento viene spostato sotto gli elementi flottanti a sinistra.
- **right**: l'elemento viene spostato sotto gli elementi flottanti a destra.
- **both**: l'elemento viene spostato sotto gli elementi flottanti sia a sinistra che a destra.
- **inherit**: l'elemento eredita il valore `clear` del suo genitore.

```
<!DOCTYPE html>
<html>
<head>
<style>
div1 {
  float: left;
  padding: 10px;
  border: 3px solid #73AD21;
}
div2 {
  padding: 10px;
  border: 3px solid red;
}
div3 {
  float: left;
  padding: 10px;
  border: 3px solid #73AD21;
}
div4 {
  padding: 10px;
  border: 3px solid red;
  clear: left;
}
</style>
</head>
<body>
<h2>Without clear</h2>
<div class="div1">div1</div>
<div class="div2">div2 - Notice that div2 is after div1 in the HTML code. However, since div1 floats to the left, the text in div2 flows around div1.</div>
<br><br>
<h2>With clear</h2>
<div class="div3">div3</div>
<div class="div4">div4 - Here, clear: left; moves div4 down below the floating div3. The value "left" clears elements floated to the left. You can also clear "right" and "both".</div>
</div>
</body>
</html>
```

Without clear

div1 div2 - Notice that div2 is after div1 in the HTML code. However, since div1 floats to the left, the text in div2 flows around div1.

With clear

div3

div4 - Here, clear: left; moves div4 down below the floating div3. The value "left" clears elements floated to the left. You can also clear "right" and "both".

Se un elemento flottante è più alto dell'elemento contenitore, andrà in “overflow” fuori dal suo contenitore. Per risolvere questo problema possiamo aggiungere il **clearfix**:

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  border: 3px solid #4CAF50;
  padding: 5px;
}
img1 {
  float: right;
}
img2 {
  float: right;
}
.clearfix {
  overflow: auto;
}
</style>
</head>
<body>
<h2>Without Clearfix</h2>
<p>This image is floated to the right. It is also taller than the element containing it, so it overflows outside of its container:</p>
<div>
  
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet...
</div>
<h2 style="clear:right">With Clearfix</h2>
<p>We can fix this by adding a clearfix class with overflow: auto; to the containing element:</p>
<div class="clearfix">
  
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet...
</div>
</body>
</html>
```

Without Clearfix

This image is floated to the right. It is also taller than the element containing it, so it overflows outside of its container:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet...

With Clearfix

We can fix this by adding a clearfix class with overflow: auto; to the containing element:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet...

Come si può evincere da ciò che abbiamo visto, float si presta a differenti tipi di contenuti:

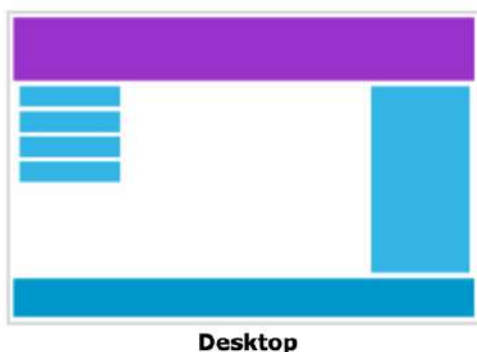


...and more



10.5. Overview sul design web reattivo

Il design web reattivo rende la nostra pagina accattivante su tutti i dispositivi. Utilizza solamente HTML e CSS, non si tratta di un programma o di un JavaScript. Le pagine web possono essere visualizzate tramite vari dispositivi: pc desktop, tablet e telefoni. La nostra pagina dovrebbe sempre avere un look accattivante ed essere facile da usare, indipendentemente dal dispositivo.



Desktop

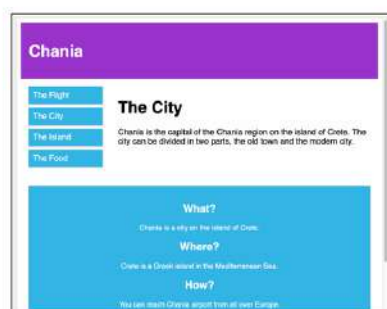


Tablet



Phone

Design web reattivo significa usare CSS e HTML per ridimensionare, nascondere, ridurre, allargare o spostare il contenuto per fare in modo che sia accattivante su qualsiasi dispositivo.



Tutto dipende dalla già citata **Viewport**, ovvero l'area della pagina web visibile all'utente. La viewport varia da dispositivo a dispositivo, e sarà ovviamente più piccola su un dispositivo come uno smartphone rispetto allo schermo di un computer. Prima dei tablet e dei telefonini, le pagine web erano progettate solo per gli schermi dei computer, ed era comune per esse avere un design statico e una grandezza fixata. Dopo l'avvento degli altri dispositivi, questo tipo di pagine web non era più in grado di adattarsi alla viewport. Per risolvere questo problema, i browser in quei dispositivi hanno ridimensionato intere pagine web per farle entrare nello schermo, ma questa era una risoluzione temporanea, non certo ideale. HTML5 ha introdotto un metodo per fare in modo che i designer web prendessero il controllo della viewport tramite il tag **<meta>**:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

La dicitura **width=device-width** setta la larghezza della pagina in base alla larghezza dello schermo del dispositivo corrente. La dicitura **initial-scale=1.0** setta il livello di zoom iniziale quando la pagina è caricata per la prima volta dal browser.



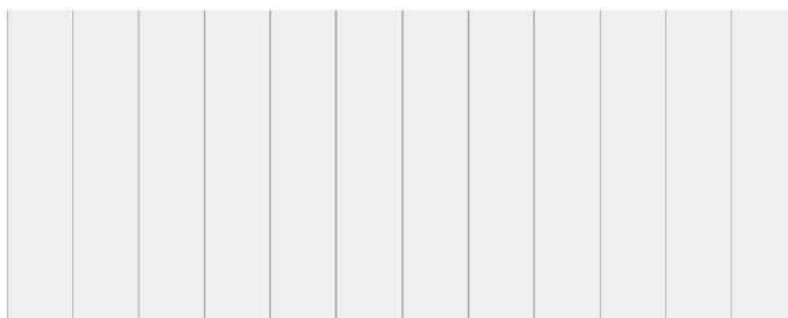
Without the viewport meta tag



With the viewport meta tag

La domanda è: come fare ad ottenere un layout reattivo?

Molte pagine web sono basate su una **grid-view**, il che significa che la pagina è divisa in colonne. Ciò aiuta molto a piazzare gli elementi sulla pagina:





Una grid-view reattiva ha spesso 12 colonne, una larghezza totale del 100% e si rimpicciolirà o espanderà quando ridimensioniamo la finestra del browser.

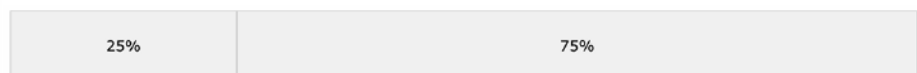
La domanda faticosa è: come ottenere una grid-view reattiva? Per prima cosa dobbiamo assicurarci che tutti gli elementi HTML abbiano l'attributo **box-sizing: border-box;**. Ciò fa in modo che padding e border siano inclusi nella larghezza e altezza totali degli elementi:

```
* {  
  box-sizing: border-box;  
}
```

In seguito, dobbiamo applicare la percentuale desiderata per ogni colonna della grid-view. Per esempio, avendo due colonne:

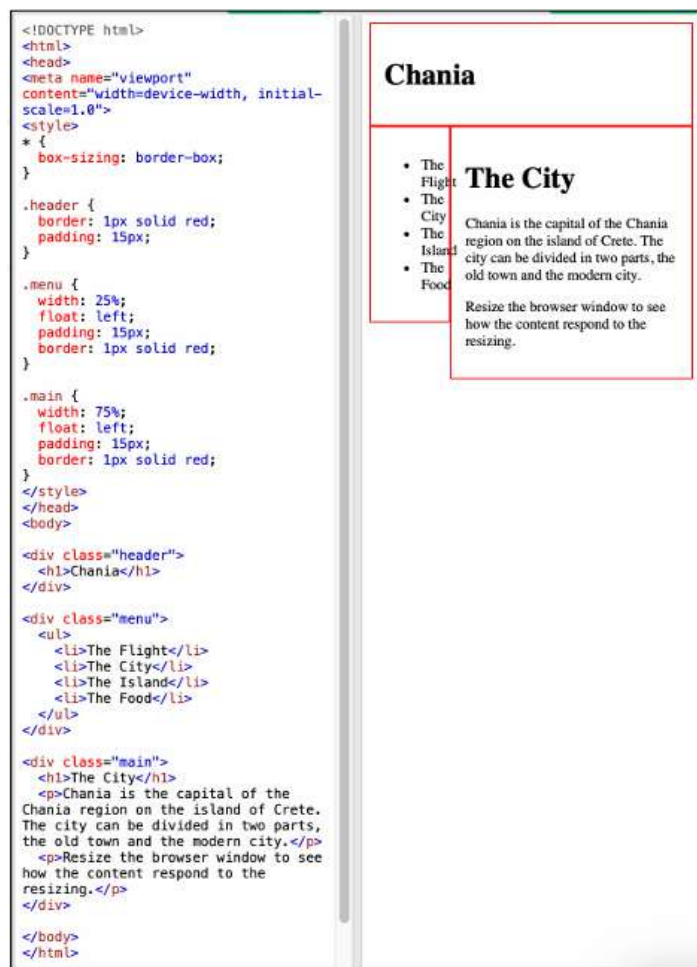
- La prima colonna prende il 25% del viewport.
- La seconda colonna prende il 75% del viewport.

```
.menu {  
  width: 25%;  
  float: left;  
}  
.main {  
  width: 75%;  
  float: left;  
}
```





Se la viewport è ristretta, i rapporti percentuali sono gestiti in base al browser. In ogni caso questo esempio va bene se la pagina web contiene solamente due colonne, non riguarda un caso reale.



Per una grid-view reattiva di 12 colonne: $100\% / 12 \text{ colonne} = 8.33\%$ a colonna.

1. Realizziamo una classe per ognuna delle 12 colonne, **class="col-"** e un numero che definisce quante colonne deve mostrare la sezione.
2. Tutte queste colonne dovrebbero fluttuare a sinistra e avere un padding di 15 px.
3. Ogni riga dovrebbe trovarsi in un contenitore `<div>` e il numero di colonne all'interno di una riga deve essere sempre pari a 12.
4. Le colonne all'interno di una riga sono tutte fluttuanti a sinistra e sono quindi sottratte al flusso della pagina. Gli altri elementi saranno posizionati come se le colonne non esistessero. Per evitare questo, aggiungeremo uno stile che pulisce il flusso.
5. Vogliamo anche aggiungere alcuni stili e colori per farla sembrare più bella.

1

```
.col-1 {width: 8.33%;}
.col-2 {width: 16.66%;}
.col-3 {width: 25%;}
.col-4 {width: 33.33%;}
.col-5 {width: 41.66%;}
.col-6 {width: 50%;}
.col-7 {width: 58.33%;}
.col-8 {width: 66.66%;}
.col-9 {width: 75%;}
.col-10 {width: 83.33%;}
.col-11 {width: 91.66%;}
.col-12 {width: 100%;}
```

2

```
[class*=col-] {
  float: left;
  padding: 15px;
  border: 1px solid red;
}
```

3

```
<div class="row">
  <div class="col-3">...</div> <!-- 25% -->
  <div class="col-9">...</div> <!-- 75% -->
</div>
```

4

```
.row:after {
  content: "";
  clear: both;
  display: table;
}
```

5

```
html {
  font-family: "Lucide Sans", sans-serif;
}

.header {
  background-color: #9933cc;
  color: #ffffff;
  padding: 15px;
}

.menu ul {
  list-style-type: none;
  margin: 0;
  padding: 0;
}

.menu li {
  padding: 8px;
  margin-bottom: 7px;
  background-color: #e333e5;
  color: #ffffff;
  box-shadow: 0 1px 3px rgba(0,0,0,0.12), 0 1px 2px rgba(0,0,0,0.24);
}

.menu li:hover {
  background-color: #8859cc;
}
```

Così facendo otterremo lo stesso stile di pagina ottenuto precedentemente:



Fare tutto questo manualmente potrebbe rivelarsi più tedioso del dovuto. Esistono vari Framework CSS che offrono design reattivi. Ad esempio, **Bootstrap** è un framework front-end gratuito pensato per uno sviluppo web più rapido e semplice. Bootstrap include modelli di design basati su HTML e CSS per tipografia, moduli, pulsanti, tabelle, navigazione, modali, caroselli di immagini e molti altri, nonché plugin JavaScript opzionali. Bootstrap ci dà anche la possibilità di creare agilmente e in modo semplice dei design reattivi.

